



Program Studi Teknologi Informasi
Fakultas Informatika dan Pariwisata

**Febri Damatraseta Fairuz
Septian Cahyadi
Suci Sutjipto
Isnani Mulia
Risma Handayani**

INSTITUT BISNIS & INFORMATIKA KESATUAN
Jl. Rangka Gading No.1, RT.02/RW.09, Gudang,
Kecamatan Bogor Tengah, Kota Bogor, Jawa Barat 16123

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Kuasa, yang telah memberikan rahmat-Nya sehingga Buku Pemrograman Website untuk mahasiswa/i Program Studi Teknologi Informasi Fakultas Informatika dan Pariwisata Institut Bisnis dan Informatika Kesatuan Bogor ini dapat diselesaikan dengan sebaik-baiknya.

Buku Pembelajaran ini dibuat sebagai pedoman dalam melakukan kegiatan Pembelajaran Pemrograman Website yang merupakan kegiatan penunjang mata kuliah Pemrograman Website pada Program Studi Teknologi Informasi Fakultas Informatika dan Pariwisata Institut Bisnis dan Informatika Kesatuan Bogor. Buku Pembelajaran ini diharapkan dapat membantu mahasiswa/i dalam mempersiapkan dan melaksanakan Pembelajaran baik teori maupun praktikum dengan lebih baik, terarah, dan terencana. Pada setiap topik telah ditetapkan tujuan pelaksanaan Pembelajaran dan semua kegiatan yang harus dilakukan oleh mahasiswa/i serta teori-teori untuk memperdalam pemahaman mahasiswa/i mengenai materi yang dibahas.

Dalam pembuatan Buku Pembelajaran Pemrograman Website ini, penyusun menyadari masih jauh dari sempurna. Oleh karena itu penyusun mengharapkan kritik dan saran yang membangun guna penyempurnaan modul Pembelajaran ini dimasa yang akan datang. Akhir kata, penyusun mengucapkan banyak terima kasih kepada semua pihak yang telah terlibat baik secara langsung maupun tidak langsung dalam pembuatan Buku ini.

Bogor, Mei 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR GAMBAR.....	v
DAFTAR TABEL.....	viii
BAB I MARKUP LANGUAGE MEMBANGUN KERANGKA HALAMAN WEB	1
1.1. Tujuan Pembelajaran.....	1
1.2. Dasar Teori.....	1
1.1.1 Pengertian Markup Language	1
1.1.2 Pengertian HTML	2
1.1.3 Struktur Dasar Dokumen HTML	4
1.1.4 Elemen Dasar HTML.....	7
1.1.5 Konsep Tag, Elemen, dan Atribut.....	20
1.3. Latihan Pembelajaran.....	21
1.4. Tugas Pembelajaran	26
BAB II PENGENALAN CASCADING STYLE SHEETS (CSS).....	27
2.1 Tujuan Pembelajaran.....	27
2.2 Dasar Teori.....	27
2.2.1 Cascading Style Sheet.....	27
2.2.2 Cara Penggunaan CSS.....	28
2.2.3 CSS Selector.....	29
2.2.4 CSS Inline	33
2.2.5 CSS Internal	34
2.2.6 CSS External	35
2.2.7 Component Properties.....	36
2.3 Latihan Pembelajaran.....	44
2.4 Tugas Pembelajaran	48
BAB III PENGENALAN JAVASCRIPT.....	49
3.1 Tujuan Pembelajaran.....	49
3.2 Dasar Teori.....	49
3.2.1 Pengertian Javascript (JS)	49
3.2.2 Cara Penggunaan JS ke HTML.....	50

3.2.3	Variable.....	52
3.2.4	Tipe Data.....	53
3.2.5	Operator.....	53
3.2.6	Document Object Model (DOM) HTML.....	58
5.2	Latihan Pembelajaran.....	65
5.3	Tugas Pembelajaran	66
BAB IV PENGENALAN MERN STACK		67
4.1	Tujuan Pembelajaran.....	67
4.2	Dasar Teori.....	67
4.2.1	Konsep Full-Stack Development (MERN STACK).....	67
4.2.2	MySQL.....	68
4.2.3	Express JS	68
4.2.4	React JS.....	69
4.2.5	Node JS	70
4.2.6	REST API dan HTTP Method	70
4.3	Latihan Pembelajaran.....	70
4.4	Tugas Pembelajaran	85
BAB V FRAMEWORK UI REACT JS: PART 1		86
5.1	Tujuan Pembelajaran.....	86
5.2	Dasar Teori.....	86
5.2.1	Pengertian React.....	86
5.2.2	Dasar Javascript untuk React	87
5.2.3	Konsep Component-Based Architecture.....	102
5.2.4	Struktur Project	103
5.3	Latihan Pembelajaran.....	105
5.4	Tugas Pembelajaran	125
BAB VI FRAMEWORK UI REACT JS: PART 2		126
6.1	Tujuan Pembelajaran.....	126
6.2	Dasar Teori.....	126
6.2.1	State.....	127
6.2.2	Props.....	129
6.2.3	Hooks	132
6.3	Latihan Pembelajaran.....	138
6.4	Tugas Pembelajaran	174

BAB VII MEMBANGUN REST API DENGAN EXPRESS.JS	175
7.1 Tujuan Pembelajaran.....	175
7.2 Dasar Teori.....	175
7.2.1 REST API.....	175
7.2.2 Sequelize	176
7.2.3 Mongoose.....	181
7.3 Latihan Pembelajaran.....	184
7.4 Tugas Pembelajaran	217
BAB VIII AUTHENTICATION AND INTEGRATING	219
8.1 Tujuan Pembelajaran.....	219
8.2 Dasar Teori.....	219
8.2.1 Authentication vs Authorization	219
8.2.2 JSON Web Token (JWT).....	221
8.2.3 Hasing Password	223
8.2.4 Middleware pada Backend.....	224
8.2.5 REST API Integration	225
8.2.6 State Management Authentication	227
8.2.7 Protected Route	231
8.3 Latihan Pembelajaran.....	232
8.4 Tugas Pembelajaran	279
DAFTAR PUSTAKA	280

DAFTAR GAMBAR

Gambar 1.1 Struktur HTML	5
Gambar 1.2 Simple Document HTML.....	6
Gambar 1.3 HTML Documents dalam text editor	7
Gambar 1.4 Hasil Preview dari HTML Documents.....	7
Gambar 1.5 Hasil Preview Kode Input Types	19
Gambar 2.1 Struktur Dasar CSS	29
Gambar 2.2 CSS Colors	36
Gambar 2.3 CSS Box Model.....	38
Gambar 3.1 Output Latihan JS (JS External).....	51
Gambar 3.2 Output Latihan JS (Operasi Aritmatika)	57
Gambar 3.3 Output Latihan JS (Memasukkan Bilangan)	58
Gambar 3.4 Output Latihan Objek Text	60
Gambar 3.5 Output Latihan Objek Radio	61
Gambar 3.6 Output Latihan Objek Checkbox.....	62
Gambar 3.7 Output Latihan Objek TextArea.....	63
Gambar 3.8 Output Latihan Objek Select	64
Gambar 4.1 Arsitektur Umum MERN Stack	68
Gambar 4.2 Halaman Website Download Node.js	71
Gambar 4.3 Download Node.js.....	72
Gambar 4.4 Instalasi Package Node JS.....	72
Gambar 4.5 Cek Versi Node.js dan npm.....	72
Gambar 4.6 Output Perintah npm init -y.....	73
Gambar 4.7 Hasil Instalasi Dependency pada Project Node.js	74
Gambar 4.8 Tampilan Terminal Running Server Backend Node.js	74
Gambar 4.9 Halaman Akses Server Backend localhost:3000.....	75
Gambar 4.10 Koneksi server gagal ke database.....	76
Gambar 4.11 Gambar Koneksi serve berhasil ke database	77
Gambar 4.12 Running Project React.....	79

Gambar 4.13 Halaman Utama Project Frontend menggunakan React Framework	79
Gambar 4.14 Kode Ubah Port Backend.....	81
Gambar 4.15 Halaman Akses localhost:3001/api/info.....	82
Gambar 4.16 Hasil Testing Rest API Berhasil.....	84
Gambar 4.17 Hasil Testing Rest API Gagal	84
Gambar 4.18 Hasil Testing Rest API Server Mati.....	85
Gambar 5.1 Contoh Variable Javascript dalam Kode	87
Gambar 5.2 Contoh Penggunaan Tipe Data dalam Kode	87
Gambar 5.3 Contoh Operasi Data dalam Kode.....	88
Gambar 5.4 Tampilan Output Kode navbar.jsx	112
Gambar 5.5 Tampilan Output Kode herosection.jsx.....	114
Gambar 5.6 Tampilan Output Feature Book Section.....	116
Gambar 5.7 Tampilan Output Category Section.....	118
Gambar 5.8 Tampilan Output Testimonial Section	120
Gambar 5.9 Tampilan Output Footer Section	123
Gambar 5.10 Tampilan Output Landing Page Section	125
Gambar 6.1 Struktur Penggunaan useState pada React	127
Gambar 6.2 Tampilan Output Kombinasi State dan Props	131
Gambar 6.3 UI Web Apps CMS Book Management.....	139
Gambar 6.4 Varian Button Komponen	145
Gambar 6.5 Tampilan Sidebar Komponen.....	149
Gambar 6.6 Tampilan Komponen Alert.....	150
Gambar 6.7 Tampilan Komponen TextInput dan TextArea	153
Gambar 6.8 Tampilan Penggunaan komponen Card	155
Gambar 6.9 Tampilan Modal Pop Up	159
Gambar 6.10 Halaman Books Management	172
Gambar 7.1 Logo Sequelize.....	177
Gambar 7.2 Logo Mongoose.....	182
Gambar 7.3 Koneksi Sequelize telah berhasil.....	190
Gambar 7.4 Hasil Eksekusi Create Database Menggunakan Sequelize CLI	195
Gambar 7.5 Hasil Eksekusi Migrasi Database Menggunakan Sequelize CLI	197

Gambar 7.6 Daftar Tabel Database pw_praktikum_db.....	198
Gambar 7.7 Spesifikasi Tipe Data Tabel books.....	198
Gambar 7.8 Pengujian API Create Data Books Menggunakan Postman.....	210
Gambar 7.9 Response API Create Data Books pada Postman.....	211
Gambar 7.10 Hasil pencarian filename return dari key 'image'.....	212
Gambar 8.1 Output Uji Coba /api/users/login tanpa Username dan Password.....	245
Gambar 8.2 Output Output Uji Coba /api/users/login dengan Password Salah.....	245
Gambar 8.3 Output Uji Coba Endpoint /api/users/login Akun terverifikasi.....	246
Gambar 8.4 Konfigurasi Autentikasi Bearer Token pada Postman	247
Gambar 8.5 Hasil Pengujian Endpoint Books tanpa Access Token	248
Gambar 8.6 Hasil Pengujian Endpoint Books dengan Access Token	248
Gambar 8.7 Hasil Pengujian Endpoint Books dengan Acces Token Expired	249
Gambar 8.8 Tampilan Halaman Sign-in	272
Gambar 8.9 Tampilan Halaman utama frontend.....	277
Gambar 8.10 Halaman Pengisian Sign In (kondisi valid dan invalid akun)	278
Gambar 8.11 Halaman setelah melakukan sign-in.....	278

DAFTAR TABEL

Tabel 1.1 Elemen Formatting HTML	10
Tabel 1.2 Elemen Quotations HTML.....	11
Tabel 1.3 Elemen List HTML.....	11
Tabel 1.4 Elemen Links HTML	13
Tabel 1.5 Elemen Tag Table	14
Tabel 1.6 Elemen Attribute Form	16
Tabel 1.7 Attributes Elemen <input>.....	19
Tabel 2.1 Selector Pseudo-class.....	31
Tabel 2.2 Selector Pseudo-Elements.....	33
Tabel 2.3 Font Properties	42
Tabel 3.1 Operator Aritmatika	53
Tabel 3.2 Operator Pemberian Nilai	54
Tabel 3.3 Operator Manipulasi Bit.....	54
Tabel 3.4 Operator Pembanding	55
Tabel 3.5 Operator Logika	55
Tabel 4.1 Method Rest API.....	70
Tabel 5.1 Tipe Data React.....	87
Tabel 5.2 Contoh Kode RFC Umum dan ES6	96
Tabel 5.3 Mouse Events dan Fungsinya	100
Tabel 5.4 Form Event dan Fungsinya	100
Tabel 5.5 Keyboard Events dan Fungsinya.....	100
Tabel 5.6 Clipboard Events dan Fungsinya	101
Tabel 5.7 Page Layout Landing Page	108
Tabel 6.1 Perbandingan Format Non Destructuring dan Destructuring	128
Tabel 6.2 Perilaku useEffect Berdasarkan Dependency Array	134
Tabel 6.3 Perbandingan Fungsi Hooks	138
Tabel 6.4 Deskripsi Kolom Tabel Headers	169
Tabel 7.1 Routing Rest API Book.....	209

Tabel 8.1 Perbandingan Authentication dan Authorization	220
Tabel 8.2 Keuntungan dan Kekurangan JWT	223
Tabel 8.3 Perbandingan Plaintext vs Hash.....	224
Tabel 8.4 Method REST API.....	225
Tabel 8.5 Deskripsi Penjelasan Response Body	246

BAB I

MARKUP LANGUAGE: MEMBANGUN KERANGKA HALAMAN WEB

1.1. Tujuan Pembelajaran

1. Memahami konsep dasar markup language dan perbedaannya dengan bahasa pemrograman.
2. Menjelaskan fungsi dan struktur dasar dari HTML (*HyperText Markup Language*).
3. Membuat struktur dasar dokumen HTML dengan benar.
4. Menggunakan tag dasar HTML seperti:
 - a. `<html>`
 - b. `<head>`
 - c. `<title>`
 - d. `<body>`
 - e. *Heading* (`<h1>` - `<h6>`)
 - f. *Paragraph* (`<p>`)
 - g. *List* (`<ul>`, `<ol>`, `<li>`)
5. Memahami konsep elemen, tag, dan atribut.
6. Menjalankan file HTML menggunakan browser.
7. Membuat kerangka halaman web sederhana sesuai standar struktur HTML5.

1.2. Dasar Teori

1.1.1 Pengertian Markup Language

Markup Language adalah bahasa yang digunakan untuk memberi tanda (*mark*) pada dokumen agar dapat dibaca dan ditampilkan dengan struktur tertentu oleh sistem komputer.

Contoh markup language yang digunakan untuk web adalah:

- HTML
- XML

Berbeda dengan bahasa pemrograman seperti JavaScript atau Python, HTML tidak digunakan untuk melakukan logika program, melainkan untuk menyusun struktur halaman web.

1.1.2 Pengertian HTML

HTML merupakan standar umum yang digunakan untuk membangun sebuah halaman website. HTML merupakan kepanjangan dari *Hyper Text Markup Language*, merupakan:

- a. Menggambarkan struktur halaman website.
- b. Terdiri dari serangkaian element, dimana element HTML akan memberitahukan browser untuk menampilkan konten dari berbagai platform komputer manapun.
- c. Mengontrol tampilan dari halaman website dan kontennya. Seperti memberikan label potongan konten dengan menggunakan element HTML seperti “bagian judul”, “bagian paragraf”, “bagian tautan” dan lain sebagainya.

Pada tahun 1980 seorang ahli fisika, Tim Berners-Lee, dan juga seorang kontraktor di CERN (Organisasi Eropa untuk Riset Nuklir) mengusulkan dan menyusun ENQUIRE, sebuah sistem untuk ilmuwan CERN dalam membagi dokumen. Sembilan tahun kemudian, Berners-Lee mengusulkan adanya sistem markah berbasis internet. Berners-Lee menspesifikasikan HTML dan menulis jaringan beserta perangkat lunaknya di akhir 1990. Pada tahun yang sama, Berners-Lee dan Robert Cailliau, insinyur sistem data CERN berkolaborasi dalam sebuah permintaan untuk pendanaan, namun tidak diterima secara resmi oleh CERN. Di catatan pribadinya sejak 1990 dia mendaftar "beberapa dari banyak daerah yang menggunakan hypertext" dan pertama-tama menempatkan sebuah ensiklopedia.

Berikut ini adalah sekilas mengenai perjalanan versi dari HTML yang telah diakui sampai saat ini:

- a. HTML 1.0

Penjelasan pertama kali HTML dipublikasikan sebagai bentuk bahasa pemrograman untuk sistem markah berbasis internet ialah sebuah dokumen

yang disebut "Tanda HTML", pertama kali disebutkan di Internet oleh Tim Berners-Lee pada akhir 1991. Tanda ini menggambarkan 18 elemen awal mula, versi sederhana dari HTML. Kecuali untuk tag hyperlink, yang sangat dipengaruhi oleh SGMLguid, in-house Standard Generalized Markup Language (SGML) berbasis format dokumen di CERN.

b. HTML 2.0

Dikembangkan oleh Kelompok Kerja HTML IETF, yang ditutup pada tahun 1996. Ini menetapkan standar untuk fitur inti HTML berdasarkan praktik saat ini pada tahun 1994.

c. HTML 3.0

Rekomendasi pertama W3C untuk HTML yang mewakili konsensus tentang fitur HTML untuk tahun 1996. HTML 3.2 menambahkan fitur yang digunakan secara luas seperti tabel, applet, aliran teks di sekitar gambar, skrip super, dan subskrip, sekaligus memberikan kompatibilitas mundur dengan standar HTML 2.0 yang ada.

d. HTML 4.01

Versi ini dikenalkan pada tahun 1999 dengan menambahkan dukungan untuk lebih banyak opsi multimedia, bahasa skrip, styles sheets, fasilitas pencetakan yang lebih baik, dan dokumen yang lebih mudah diakses oleh pengguna penyandang disabilitas.

e. XHTML 1.0

Pada tahun 2000 dengan banyak fitur baru, XHTML 1.0 merupakan formulasi ulang HTML 4.01 dalam XHTML, dan menggabungkan kekuatan HTML 4 dengan kekuatan XML.

f. HTML 5.0

dibuat untuk menggantikan HTML 3.2, HTML 4. dan XHTML 1.x, HTML 5 memiliki banyak fitur baru dibandingkan pendahulunya. ini termasuk dukungan penyimpanan media offline, elemen konten yang lebih spesifik (seperti footer, header, navigasi, dll), doctype inline yang lebih sederhana, dukungan penyematan audio, dan video. Pembaharuan ini diperkenalkan pada tahun 2015.

g. HTML 5.1

Selang satu tahun dari peluncuran HTML 5.0, bahasa pemrograman standar berbasis internet ini sudah memiliki pembaharuannya seperti kemampuan yang lebih baik terkait pengalaman video, formulir web, aksesibilitas gambar, dan pemeriksaan ejaan dan tata bahasa.

h. HTML 5.2

Peningkatan kebijakan keamanan konten, API permintaan pembayaran untuk situs web eCommerce, aksesibilitas aplikasi rich internet untuk penyandang disabilitas, dan perubahan pada elemen `<main>` untuk mendukung desain responsif. Versi HTML 5.2 dikenalkan pada tahun 2017 dan masih dipergunakan hingga saat ini sebagai bahasa pemrograman standar berbasis internet yang direkomendasikan oleh W3C.

HTML adalah bahasa standar yang digunakan untuk membuat struktur halaman web. HTML terdiri dari:

- Tag
- Elemen
- Atribut

Contoh struktur dasar HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>Judul Halaman</title>
</head>
<body>
  <h1>Selamat Datang</h1>
  <p>Ini adalah halaman web pertama saya.</p>
</body>
</html>
```

1.1.3 Struktur Dasar Dokumen HTML

Sifat dari HTML ialah clientscript dimana dokumen tersebut dapat dibuka didalam komputer secara stand alone yang tidak membutuhkan sebuah server untuk dapat menampilkannya di dalam browser. Dokumen HTML biasanya berekstensi *.html* atau *.htm* dimana bahasa HTML tersebut tersusun atas tag atau syntax yang berformat `<tagname>...</tagname>`. Aturan dalam penulisan HTML sebagai berikut:

- Dalam penulisan tag HTML diapit dengan menggunakan dua kurung siku (angle bracket) “<” dan “>”
- Tag HTML selalu berpasangan, dimana tag awal dalam suatu syntax disebut dengan tag awal dan tag kedua sebagai tag terakhir. contoh: <p>...</p>
- Penulisan tag HTML tidak case sensitive dimana tag dengan huruf kecil sama dengan tag huruf besar, contoh: <P>...</P> sama dengan <p>...</p> Namun berdasarkan aturan pada W3C (HTML5.0 dan XHTML) penulisan tag HTML sebaiknya menggunakan huruf kecil.
- Jika memiliki tag nested atau terdapat suatu tag berisi tag lagi, maka penulisan tag akhir tidak boleh bersilang dan harus berurutan. Contoh: <p>Ini contoh Huruf Tebal dan <i>Huruf cetak miring</i> </p>
- Penulisan syntax dokumen HTML selalu diawali dengan tag <html> dan diakhiri dengan tag </html>



Gambar 1.1 Struktur HTML

Semua dokumen HTML harus dimulai dengan deklarasi tipe dokumen: <!DOCTYPE html>. Dokumen HTML itu sendiri diawali dengan <html> dan diakhiri dengan </html>. Bagian yang terlihat dari dokumen HTML adalah antara <body> dan </body>.

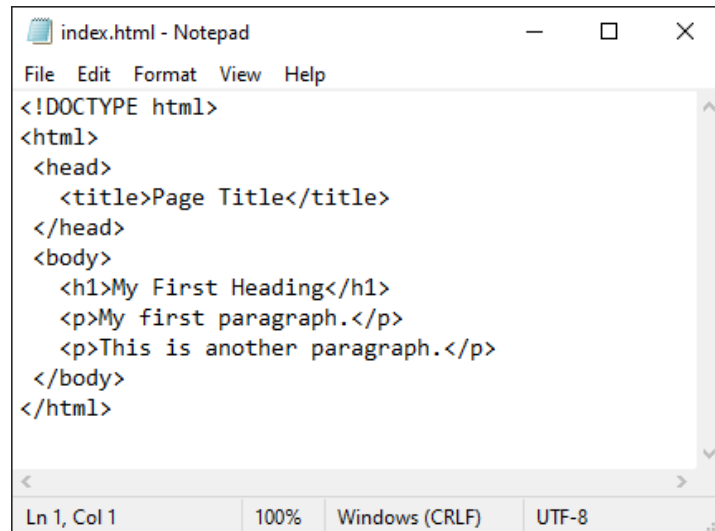
```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
    <p>This is another paragraph.</p>
  </body>
</html>
```

Gambar 1.2 Simple Document HTML

Dari contoh penulisan dokumen HTML diatas dapat diartikan sebagai berikut:

- a. Deklarasi `<!DOCTYPE html>` mendefinisikan bahwa dokumen ini adalah dokumen HTML5.
- b. Elemen `<html>` adalah elemen root dari halaman HTML.
- c. Elemen `<head>` berisi informasi meta tentang halaman HTML.
- d. Elemen `<title>` menentukan judul untuk halaman HTML (yang ditampilkan di bilah judul browser atau di tab halaman).
- e. Elemen `<body>` mendefinisikan tubuh dokumen, dan merupakan wadah untuk semua konten yang terlihat, seperti judul, paragraf, gambar, hyperlink, tabel, daftar, dll.
- f. Elemen `<h1>` mendefinisikan heading besar.
- g. Elemen `<p>` mendefinisikan paragraf.

Dalam membangun sebuah halaman website dengan menggunakan HTML, text editor yang dapat digunakan cukup sederhana dimana aplikasi text editor ini terdapat di semua operating sistem pada komputer. Jika menggunakan Windows OS dapat menggunakan aplikasi Notepad, sedangkan jika menggunakan UNIX OS dapat menggunakan aplikasi TextEdit.



Gambar 1.3 HTML Documents dalam text editor

Untuk membuat sebuah dokumen HTML tahap pertama ialah buka text editor yang dimiliki seperti Notepad dan masukan script dibawah ini, lalu simpan dengan nama *index.html* atau *index.htm*. Setelah berhasil tersimpan dengan format extension *.html* atau *.htm*, maka file tersebut akan membentuk sebuah file dengan format icon browser. Jika dibuka file tersebut maka akan membuka sebuah aplikasi browser dan menampilkan sebuah tampilan berdasarkan tag HTML.

My First Heading

My first paragraph.

This is another paragraph.

Gambar 1.4 Hasil Preview dari HTML Documents

1.1.4 Elemen Dasar HTML

A. HTML Element Head

Berikut adalah tag atau elemen yang dapat digunakan di bagian `<head>...</head>` pada sebuah dokumen HTML:

1. Title

Elemen <title>, menentukan judul untuk halaman HTML (yang ditampilkan di bilah judul browser atau di tab halaman)

```
<title>Page Title</title>
```

2. Favicon

Elemen ini diperuntukan untuk menentukan icon pada halaman HTML (yang ditampilkan di bilah judul browser atau di tab halaman). Untuk menggunakan favicon, dikhususkan memiliki file gambar dengan extension .ico, .svg atau .png dan size yang digunakan ialah 16x16 pixel atau 32x32 pixel.

```
<link rel="icon" type="image/x-icon"
href="/path/to/icons/favicon.ico">
<link rel="icon" type="image/png"
href="/path/to/icons/favicon.png">
<link rel="icon" type="image/svg"
href="/path/to/icons/favicon.svg">
```

3. Meta

Elemen <meta> biasanya digunakan untuk menentukan set karakter, deskripsi halaman, kata kunci, penulis dokumen, dan pengaturan *viewport*. Metadata tidak akan ditampilkan di halaman, tetapi digunakan oleh browser (cara menampilkan konten atau memuat ulang halaman), oleh mesin pencari (kata kunci), dan layanan web lainnya.

- Menentukan set karakter

```
<meta charset="UTF-8">
```

- Menentukan kata kunci untuk search engine

```
<meta name="keywords" content="HTML, CSS,
JavaScript">
```

- Menentukan deskripsi halaman web

```
<meta name="description" content="Website IBIK -
Pemrograman Web">
```

- Menentukan penulis halaman

```
<meta name="author" content="Febry D Fairuz">
```

- Merefresh halaman setiap 30 detik

```
<meta http-equiv="refresh" content="30">
```

- Mengatur *viewport* agar situs website terlihat bagus di semua perangkat

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

4. Javascript

JavaScript membuat halaman HTML lebih dinamis dan interaktif. Tag HTML `<script>` digunakan untuk mendefinisikan skrip sisi klien (*JavaScript*). Elemen `<script>` berisi pernyataan skrip, atau menunjuk ke file skrip eksternal melalui atribut `src`. Penggunaan umum untuk *JavaScript* adalah manipulasi gambar, validasi formulir, dan perubahan konten yang dinamis. Untuk memilih elemen HTML, *JavaScript* paling sering menggunakan metode `document.getElementById()`. Contoh *JavaScript* ini menulis "Halo *JavaScript*!" menjadi elemen HTML dengan `id="demo"`:

```
<p id="demo"></p>

<script>
  document.getElementById("demo").innerHTML = "Hello
  JavaScript!";
</script>
```

5. Styles CSS

Cascading Style Sheets (CSS) digunakan untuk memformat tata letak halaman web. Dengan CSS, dapat mengontrol warna, font, ukuran teks, jarak antar elemen, bagaimana elemen diposisikan dan ditata, gambar latar belakang atau warna latar apa yang akan digunakan, tampilan berbeda untuk perangkat dan ukuran layar yang berbeda, dan lebih banyak!

```
<style>
  body {background-color: powderblue;}
  h1   {color: blue;}
  p    {color: red;}
</style>
```

B. HTML Element Body

Sedangkan berikut ini adalah tag atau elemen yang bisa digunakan di bagian `<body>...</body>` pada sebuah dokumen HTML:

1. Headings

Headings adalah judul atau subjudul yang ingin ditampilkan di halaman web. *Headings* didefinisikan dengan tag <h1> hingga <h6>. <h1> mendefinisikan heading yang paling penting. <h6> mendefinisikan heading yang paling tidak penting.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

2. Paragraphs

Sebuah paragraf selalu dimulai pada baris baru, dan biasanya berupa blok teks. Elemen HTML <p> mendefinisikan sebuah paragraf. Sebuah paragraf selalu dimulai pada baris baru, dan browser secara otomatis menambahkan beberapa spasi (margin) sebelum dan sesudah paragraf.

```
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
```

3. Formatting

HTML berisi beberapa elemen untuk mendefinisikan teks dengan arti khusus. Elemen pemformatan dirancang untuk menampilkan jenis teks khusus:

Tabel 1.1 Elemen Formatting HTML

Element	Description
	Bold
	Important
<i>	Italic
	Emphasized
<mark>	Marked
<small>	Smaller
	Deleted text
<sub>	Subscript text
<sup>	Superscript text

4. Quotations

Bagian ini akan membahas elemen HTML `<blockquote>`, `<q>`, `<abbr>`, `<address>`, `<cite>`, dan `<bdo>`.

Tabel 1.2 Elemen Quotations HTML

Element	Description
<code><abbr></code>	singkatan atau akronim
<code><address></code>	informasi kontak untuk penulis/pemilik dokumen
<code><bdo></code>	arah teks
<code><blockquote></code>	bagian yang dikutip dari sumber lain
<code><cite></code>	judul sebuah karya
<code><q></code>	kutipan inline pendek

5. Comments

Untuk menambahkan komentar ke sumber HTML Anda dengan menggunakan sintaks berikut:

```
<!-- Write your comments here -->
```

6. List

List HTML memungkinkan programmer untuk mengelompokkan satu set item terkait dalam bentuk list. Daftar tidak berurutan dimulai dengan tag `<ul>` atau `<ol>`. Setiap item list dimulai dengan tag `<li>`.

Tabel 1.3 Elemen List HTML

Syntax	Display
<pre> Coffee Tea Milk </pre>	• Coffee • Tea • Milk
<pre> Coffee Tea Milk </pre>	1. Coffee 2. Tea 3. Milk

7. Block and Inline

Setiap elemen HTML memiliki nilai tampilan default, tergantung pada jenis elemennya. Ada dua nilai tampilan: block dan inline.

- Block Element

Elemen level blok selalu dimulai pada baris baru. Elemen level blok selalu menggunakan lebar penuh yang tersedia (membentang ke kiri dan kanan sejauh mungkin). Elemen level blok memiliki margin atas dan bawah, sedangkan elemen inline tidak.

```
<div>Hello World</div>
```

- Inline Element

Elemen Inline tidak dimulai pada baris baru. Elemen Inline hanya membutuhkan lebar sebanyak yang diperlukan. Ini adalah elemen `<span>` di dalam paragraf.

```
<span>Hello World</span>
```

8. Links

Links ditemukan di hampir semua halaman web. Links memungkinkan pengguna mengklik jalan mereka dari halaman ke halaman lain. Links HTML disebut dengan *hyperlink*. Untuk dapat mengklik tautan dan melompat ke halaman lain. Saat Anda menggerakkan mouse di atas tautan, panah mouse akan berubah menjadi tangan kecil.

Atribut terpenting dari elemen `<a>` adalah atribut `href`, yang menunjukkan tujuan tautan. Teks tautan adalah bagian yang akan terlihat oleh pembaca. Mengklik teks tautan, akan mengirim pembaca ke alamat URL yang ditentukan.

```
<a href="https://www.ibik.ac.id/"> Visit IBIK</a>
```

HTML Links - The target Attribute

Secara default, halaman tertaut akan ditampilkan di jendela browser saat ini. Untuk mengubahnya, Anda harus menentukan target lain untuk tautan tersebut. Atribut target dapat memiliki salah satu dari nilai berikut:

Tabel 1.4 Elemen Links HTML

Target	Description
<code><_self></code>	Membuka windows atau tab dalam satu halaman yang sama
<code><_blank></code>	Membuka windows atau tab baru
<code><_parent></code>	Membuka windows atau tab di dalam elemen induk
<code><_top></code>	Membuka windows atau tab secara keseluruhan pada body

Create Bookmarks

Link HTML dapat digunakan untuk membuat bookmark, sehingga pembaca dapat melompat ke bagian tertentu dari halaman web.

```
<h2 id="C4">Chapter 4</h2>
<a href="#C4">Jump to Chapter 4</a>
```

9. Images

Gambar dapat meningkatkan desain dan tampilan halaman web. Tag HTML `<img>` digunakan untuk menyematkan gambar di halaman web. Gambar secara teknis tidak dimasukkan ke dalam halaman web; gambar ditautkan ke halaman web. Tag `<img>` membuat ruang penyimpanan untuk gambar yang direferensikan. Tag `<img>` kosong, hanya berisi atribut, dan tidak memiliki tag penutup. Tag `<img>` memiliki dua atribut yang diperlukan:

- `src` = Menentukan jalur ke gambar
- `alt` = Menentukan teks alternatif untuk gambar.
- `width` = Menentukan panjang
- `height` = Menentukan tinggi

```

```

10. Tables

Tabel HTML memungkinkan pengembang web untuk mengatur data ke dalam baris dan kolom.

Table Kolom

```
<table>
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
</table>
```

Each table cell is defined by a `<td>` and a `</td>` tag. Segala sesuatu di antara `<td>` dan `</td>` adalah konten sel tabel.

Table Baris

```
<table>
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>
```

Setiap baris tabel dimulai dengan `<tr>` dan diakhiri dengan tag `</tr>`. Berikut ini adalah tag elemen tabel yang dapat dipergunakan:

Tabel 1.5 Elemen Tag Table

Element	Description
<code><table></code>	Mendefinisikan tabel
<code><th></code>	Mendefinisikan sel header dalam tabel
<code><tr></code>	Mendefinisikan baris dalam tabel
<code><td></code>	Mendefinisikan sel dalam tabel
<code><caption></code>	Mendefinisikan judul tabel

<code><colgroup></code>	Menentukan grup dari satu atau lebih kolom dalam tabel untuk pemformatan
<code><col></code>	Menentukan properti kolom untuk setiap kolom dalam elemen <code><colgroup></code>
<code><thead></code>	Mengelompokkan konten header dalam sebuah tabel
<code><tbody></code>	Mengelompokkan konten tubuh dalam sebuah tabel
<code><tfoot></code>	Mengelompokkan konten footer dalam tabel

11. Iframe

Tag HTML `<iframe>` menentukan bingkai sebaris. Bingkai sebaris digunakan untuk menyematkan dokumen lain dalam dokumen HTML saat ini.

```
<iframe src="url" title="description" width="100px"
      height="50px"></iframe>
```

12. Audio/Video

HTML5 memiliki metode, properti, dan peristiwa untuk elemen `<audio>` dan `<video>`. Atribut kontrol menambahkan kontrol audio, seperti putar, jeda, dan volume. Elemen `<source>` memungkinkan Anda menentukan file audio alternatif yang dapat dipilih oleh browser. Browser akan menggunakan format pertama yang dikenali. Teks diantara tag `<audio>` dan `</audio>` hanya akan ditampilkan di browser yang tidak mendukung elemen `<audio>`.

```
<audio controls>
  <source src="lagu-1.ogg" type="audio/ogg">
  <source src="lagu-2.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Elemen HTML `<video>` digunakan untuk menampilkan video di halaman web.

```
<video width="320" height="240" controls>
  <source src="vidio-1.mp4" type="video/mp4">
  <source src="video-2.ogg" type="video/ogg">
```

Your browser does not support the video tag.
</video>

Atribut kontrol menambahkan kontrol video, seperti putar, jeda, dan volume. Sebaiknya selalu sertakan atribut lebar dan tinggi. Jika tinggi dan lebar tidak disetel, halaman mungkin berkedip saat video dimuat. Elemen <source> memungkinkan Anda menentukan file video alternatif yang dapat dipilih oleh browser. Browser akan menggunakan format pertama yang dikenali. Teks antara tag <video> dan </video> hanya akan ditampilkan di browser yang tidak mendukung elemen <video>.

C. HTML Element Content Form

Form HTML digunakan untuk mengumpulkan input data. Input data paling sering dikirim ke server untuk diproses.

Elemen HTML <form> digunakan untuk membuat formulir HTML untuk input pengguna:

First name:	<form>	<label for="fname">First	name:</label>
Febry	<input type="text" id="fname" name="fname"> 	<label for="lname">Last	name:</label>
Last name:	<input type="text" id="lname" name="lname">		
Fairuz	</form>		
Submit			

1. Form Attributes

List atribut <form> yang dapat digunakan:

Tabel 1.6 Elemen Attribute Form

Attribute	Description
accept-charset	Menentukan pengkodean karakter yang digunakan untuk pengiriman formulir
action	Menentukan kemana harus mengirim formulir-data saat formulir dikirimkan
autocomplete	Menentukan apakah formulir harus memiliki pelengkapan otomatis aktif atau nonaktif

enctype	Menentukan bagaimana formulir-data harus dikodekan saat mengirimkannya ke server (hanya untuk metode = "posting")
method	Menentukan metode HTTP yang akan digunakan saat mengirim data formulir
name	Menentukan nama formulir
novalidate	Menentukan bahwa formulir tidak boleh divalidasi saat dikirimkan
rel	Menentukan hubungan antara sumber daya yang ditautkan dan dokumen saat ini
target	Menentukan tempat untuk menampilkan respons yang diterima setelah mengirimkan formulir

2. Form Elements

Elemen `<form>` HTML dapat berisi satu atau lebih elemen form sebagai berikut:

- Elemen `<input>`

Salah satu elemen form yang paling sering digunakan adalah elemen `<input>`. Elemen `<input>` dapat ditampilkan dalam beberapa cara, tergantung pada atribut `type`.

```
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname">
```

- Elemen `<label>`

Elemen `<label>` mendefinisikan label untuk beberapa elemen form. Elemen `<label>` berguna untuk pengguna pembaca layar, karena pembaca layar akan membacakan label dengan keras saat pengguna fokus pada elemen input.

Elemen `<label>` juga membantu pengguna yang mengalami kesulitan mengklik wilayah yang sangat kecil (seperti tombol radio atau kotak centang) - karena ketika pengguna mengklik teks di dalam elemen `<label>`, itu akan mengaktifkan tombol radio/kotak centang. Atribut `for` dari tag `<label>` harus sama dengan atribut `id` dari elemen `<input>` untuk mengikatnya bersama.

- Elemen `<textarea>`

Elemen `<textarea>` mendefinisikan bidang input multi-baris (area teks):

```
<textarea name="message" rows="10" cols="30">
Type something in here
</textarea>
```

Atribut *rows* menentukan jumlah baris yang terlihat di area teks. Atribut *cols* menentukan lebar yang terlihat dari area teks.

- Elemen <button>

Elemen <button> mendefinisikan tombol yang dapat diklik:

```
<button type="button">Click Me! Button</button>
<button type="submit">Click Me! Submit</button>
<button type="reset">Click Me! Reset</button>
```

- Elemen <select>

Elemen <select> mendefinisikan daftar drop-down:

```
<label for="cars">Choose a car:</label>
<select id="cars" name="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

The <option> elements defines an option that can be selected. By default, the first item in the drop-down list is selected. To define a pre-selected option, add the selected attribute to the option:

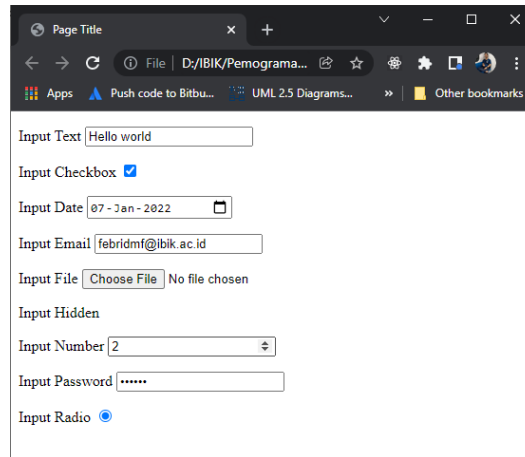
```
<option value="fiat" selected>Fiat</option>
```

3. Input Types

Berikut adalah berbagai jenis input yang dapat digunakan dalam HTML:

<input type="text">	<input type="hidden">
<input type="checkbox">	<input type="number">
<input type="date">	<input type="password">
<input type="email">	<input type="radio">
<input type="file">	<input type="range">

Display:

A screenshot of a web browser window displaying a collection of HTML input elements. The browser's address bar shows a local file path. The page contains the following inputs: a text input with 'Hello world', a checked checkbox, a date input showing '07-Jan-2022', an email input with 'febridm@ibik.ac.id', a file input with a 'Choose File' button, a hidden input, a number input with the value '2', a password input with masked characters, and a selected radio button.

Gambar 1.5 Hasil Preview Kode Input Types

4. Input Attributes

Bagian ini menjelaskan atribut yang berbeda untuk elemen `<input>` HTML.

Tabel 1.7 Attributes Elemen `<input>`

Attribute	Description
value	Menentukan nilai awal untuk bidang input
readonly	Menentukan bahwa bidang input hanya-baca
disabled	Menentukan bahwa bidang input harus dinonaktifkan
size	Menentukan lebar yang terlihat, dalam karakter, dari bidang input
maxlength	Menentukan jumlah karakter maksimum yang diizinkan dalam bidang input
min & mun	Menentukan nilai minimum dan maksimum untuk bidang input
pattern	Menentukan ekspresi reguler yang memeriksa nilai bidang input, saat formulir dikirimkan.
placeholder	Menentukan petunjuk singkat yang menjelaskan nilai yang diharapkan dari bidang input (nilai sampel atau deskripsi singkat tentang format yang diharapkan)
required	Menentukan bahwa bidang input harus diisi sebelum mengirimkan formulir

1.1.5 Konsep Tag, Elemen, dan Atribut

Tag, elemen, dan atribut adalah tiga komponen fundamental dalam HTML yang membentuk struktur halaman web. Tag adalah penanda kurung sudut (misal: `<p>`) yang menunjukkan awal/akhir elemen, elemen adalah kesatuan komponen dari tag pembuka hingga penutup beserta isinya, dan atribut adalah modifier yang memberikan informasi tambahan di dalam tag pembuka. Berikut adalah rincian konsep ketiganya:

1. Tag HTML

Tag adalah penanda yang memberi instruksi kepada browser tentang format atau struktur teks. Tag ditulis berpasangan (pembuka `<...>` dan penutup `</...>` dengan garis miring), namun ada juga tag tunggal/kosong (seperti `<img>` atau `<br>`).

- Contoh: `<p>` (tag pembuka paragraf), `</p>` (tag penutup paragraf).

2. Elemen HTML

Elemen adalah komponen utuh yang menyusun dokumen HTML. Elemen terdiri dari tag pembuka, isi/konten, dan tag penutup.

- Contoh: `<p>Ini adalah paragraf</p>`
 - `<p>`: Tag pembuka
 - Ini adalah paragraf: Isi
 - `</p>`: Tag penutup
- Catatan: Beberapa elemen tidak memiliki konten (elemen kosong), seperti `<br>` (baris baru).

3. Atribut HTML

Atribut adalah informasi tambahan atau karakteristik yang ditambahkan pada tag pembuka untuk mengatur perilaku atau tampilan elemen tersebut. Atribut selalu ditempatkan pada tag pembuka, biasanya berpasangan nama="nilai".

- Contoh: `<a href="https://google.com">Google</a>`
 - href: Nama atribut (menunjukkan tujuan link).
 - "https://google.com": Nilai atribut.
- Contoh Lain: `` (src dan alt adalah atribut).

Ringkasan Perbedaan:

- *Tag*: Penanda (`<h1, </h1>`)
- *Elemen*: Isi+Tag (`<h1>Judul</h1>`)

- *Atribut*: Pengatur/Informasi Tambahan (<p class="text">)

1.3. Latihan Pembelajaran

Kerjakan dan unggahlah jawaban ke repository GitHub Anda!

1. Latihan *Typography*

Buatlah dokumen HTML bernama *Latihan-1*.

Buatlah sebuah artikel yang berisi *Heading*, *Paragraph* dan *Formatting*. Minimum penggunaan paragraf sebanyak 4 buah.

2. Latihan *Image*

Buatlah dokumen HTML bernama *Latihan-2*.

Dari dokumen HTML *Latihan-1*, tambahkanlah elemen *Image*. Penggunaan image minimum 2 buah. Dan tambahkan atribut gambar untuk mengatur panjang dan lebar dari gambar tersebut.

3. Latihan *Table*

Buatlah dokumen HTML bernama *Latihan-3*.

Silakan implementasikan bentuk dibawah ini kedalam scripting HTML:

Oksigen mempunyai beberapa SIFAT FISIKA, diantaranya yang terdapat dalam tabel berikut:

Sifat Fisika	Oksigen
Massa atom <i>relative</i>	15,994
Nomor Atom	8
Konfigurasi <i>electron</i>	2s ² 2p ⁴
Jari-jari atom (nm)	0,074
Jari-jari X ²⁻ (nm)	0.140
Keelectronegatifan	3,5
Energy ionisasi I (kJ/mol)	1316
Energy ionisasi II (kJ/mol)	3396
Kerapatan (g/cm ³)	1.27 (padatan)
Titik leleh (°C)	+183
Titik beku (°C)	-219

Potensial elektroda (V)	+0.401	
$X_{2(g)} + 2e^+_{(aq)} \rightleftharpoons 2X^-_{(aq)}$	-	

4. Latihan List

Buatlah dokumen bernama Latihan-4.

Ubahlah bentuk dibawah ini kedalam scripting HTML:

The 10 Most Popular Programming Languages to Learn in 2022

There's no question that software programming is a hot career right now. The [U.S. Bureau of Labor Statistics](#) projects 21 percent growth for programming jobs from 2018 to 2028, which is more than four times the average for all occupations. What's more, the median annual pay for a software programmer is about \$106,000, which is nearly three times the median pay for all U.S. workers.

Not all programming jobs are the same, however. Different roles, companies, and types of software require knowing and understanding different programming languages—and it's often beneficial to know multiple languages. Trying to break into the field of software programming can be a daunting experience, especially for professionals with no prior programming experience.

The [Master of Science in Computer Science Align program](#) at Northeastern University is specifically designed for students who want to transition into computer science from another field of study.

“Our aim is to transport students from a variety of different backgrounds and have them come out as software engineers,” says Ian Gorton, PhD and director of the graduate computer science programs at Northeastern University—Seattle. *“We focus on math, programming, and a variety of computer science and engineering concepts.”*

Whether you're new to programming or looking to brush up on your skills, it helps to know which languages are in high demand. Here are 10 of the most popular programming languages of 2020 based on the number of job postings listed on job search sites Indeed, the average annual salary for those jobs, and factors such as ease of use and potential for growth.

Top 10 Most Popular Programming Languages

1. Python
Average annual salary: \$120.000
2. JavaScript
Average annual salary: \$118.000
3. Java
Average annual salary: \$104.000
4. C#
Average annual salary: \$97.000
5. C
Average annual salary: \$97.000
6. C++
Average annual salary: \$97.000
7. Go
Average annual salary: \$93.000
8. R
Average annual salary: \$93.000
9. Swift
Average annual salary: \$93.000
10. PHP
Average annual salary: \$81.000

7 Other Programming Languages to Consider

- **Web-based** startups are more likely to be programming in **Python** and **JavaScript**.
- Larger companies tend to develop their internal software applications using **C#** or **Java** and their **Web applications** using **PHP**.
- Programs for data *analytics* typically use the **R** and **MATLAB** programming languages.
- Embedded devices, such as those in the automotive and healthcare industries, run software written in **C**, **C++**, or **Rust**.
- Applications that run on the cloud are increasingly written in **Go** or **Scala**.

Mobile applications are increasingly written in Swift or Kotlin

5. Latihan Iframe, Block dan Inline

Buatlah dokumen HTML bernama Latihan-5.

- a. Ubahlah bentuk dibawah ini menjadi script HTML:

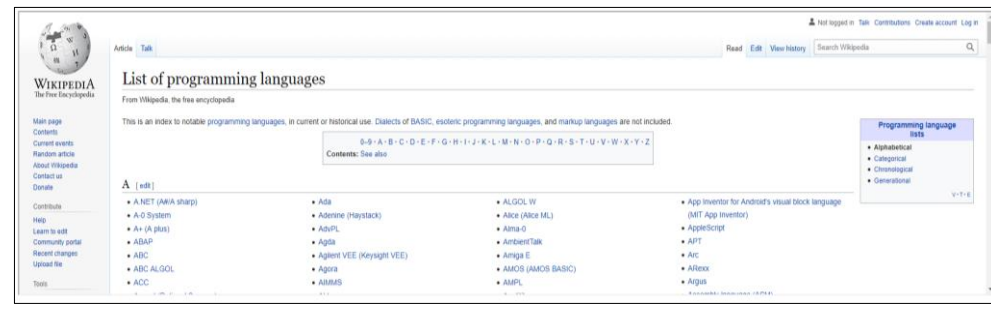
Programming Language Base on Career

Front-end Web Development	Back-end Web Development	Mobile Development
 JavaScript	 JavaScript	 JavaScript
 Elm	 Scala	 Java
 TypeScript	 Go	 Objective C
	 Ruby	 Swift
	 Python	

Top 10 Programming Language in 2022

...Iframe to Latihan-4...

List of programming languages from Wikipedia



- Buatlah 1 buah iframe yang memanggil file dokumen Latihan-4
- Buatlah 1 buah iframe yang memanggil sebuah url

6. Latihan Link

Buatlah dokumen HTML bernama Latihan-6.

- Implementasikan bentuk dibawah ini kedalam script HTML:

Top 10 Daftar Bahasa Pemrograman

Daftar isi:

- [Python](#)
- [JavaScript](#)
- [Java](#)
- [C#](#)

- [C](#)
- [C++](#)
- [Go](#)
- [R](#)
- [Swift](#)
- [PHP](#)

Python

Python adalah ...

JavaScript

JavaScript adalah ...

...dan seterusnya...

Buatlah *hyperlink* pada elemen **Daftar Isi**, dimana link tersebut merujuk ke *diverent area*, ketika mengklik salah satu *link* tersebut maka akan merujuk ke area yang telah dituju. Contoh: jika mengklik *link Java*, maka akan dirujuk ke area *Java*.

b. Buatlah tiga buah link baru dimana:

- 1) Link “Go to Latihan-4” menuju ke dokumen HTML Latihan-4 dengan menggunakan target **_self**
- 2) Link “Go to Latihan-5” menuju ke dokumen HTML Latihan-5 dengan menggunakan target **_blank**
- 3) Link “Go to URL” menuju URL dengan menggunakan target **_parent**

7. Latihan Form

Buatlah dokumen HTML bernama *Latihan-7*.

Buatlah form yang berisi isian field sebagai berikut:

- NPM (input text)
- Fullname (input text)
- Email (input email)
- Password (input password)
- Place Birth (input text)

- Birthdate (input date)
- Gender (radio button, *Male and Female*)
- Address (textarea)

Dari form diatas memiliki dua buah button. Button submit dengan nama Save dan button reset dengan nama Clear.

8. Latihan Layout

Buatlah folder project dengan nama *my-portfolio*. Di Dalam folder tersebut terdapat dua buah folder yaitu folder *css* dan *images*. Dan terdapat satu buah file bernama *index.html*. Dimana file tersebut berisi script HTML yang akan menampilkan bentuk portofolio atau curriculum vitae pada *browser*. Silakan membuat bentuk portofolio dengan bentuk bebas namun mengimplementasikan bentuk elemen HTML yang telah diberikan minimal 5 buah elemen.

1.4. Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-Praktikum-1-NPM"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB II

PENGENALAN CASCADING STYLE SHEETS (CSS)

2.1 Tujuan Pembelajaran

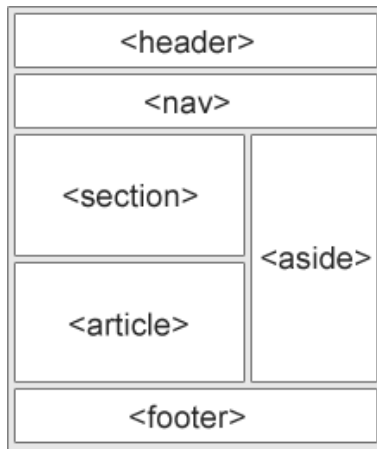
1. Memahami fungsi dan peran CSS dalam pengembangan website.
2. Menjelaskan hubungan antara HTML dan CSS.
3. Menggunakan tiga cara penulisan CSS:
 - Inline CSS
 - Internal CSS
 - External CSS
4. Memahami dan menggunakan selector dasar CSS.
5. Mengatur warna, font, ukuran teks, dan background.
6. Mengatur margin, padding, dan border.
7. Menerapkan CSS untuk memperindah tampilan halaman HTML sederhana.

2.2 Dasar Teori

2.2.1 Cascading Style Sheet

CSS atau Cascading Style Sheets adalah sebuah dokumen yang berisi aturan yang digunakan untuk memisahkan isi dengan layout dalam halaman-halaman web yang dibuat. CSS memperkenalkan “template” yang berupa style untuk dibuat dan mengijinkan penulisan kode yang lebih mudah dari halaman-halaman web yang dirancang. CSS mampu menciptakan halaman yang tampak sama pada resolusi layar yang berbeda-beda tanpa memerlukan penggunaan tabel seperti pada html klasik.

Umumnya HTML memiliki beberapa elemen semantik yang mendefinisikan berbagai bagian halaman web seperti gambar dibawah ini:



- `<header>`: Mendefinisikan header untuk dokumen atau bagian.
- `<nav>`: Mendefinisikan sekumpulan link navigasi.
- `<section>`: Mendefinisikan bagian dalam dokumen.
- `<article>`: Mendefinisikan konten mandiri yang mandiri.
- `<aside>`: Mendefinisikan konten selain dari konten (seperti sidebar).
- `<footer>`: Mendefinisikan footer untuk dokumen atau bagian.

Gambar diatas menggambarkan sebuah Layouting dari sebuah template website. Untuk mempercantik tampilan dari layout tersebut dapat menggunakan CSS sebagai template.

CSS dapat ditambahkan ke dokumen HTML dengan 3 cara:

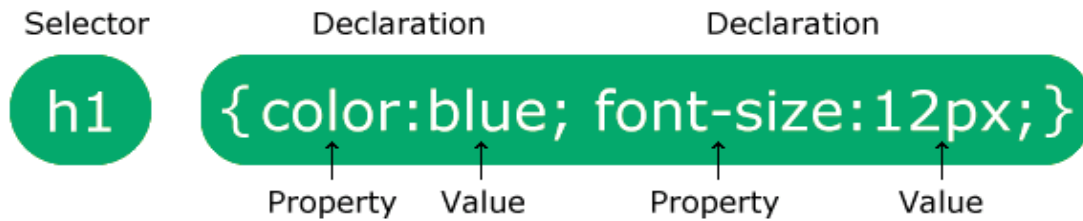
- Inline - dengan menggunakan atribut style di dalam elemen HTML
- Internal - dengan menggunakan elemen `<style>` di bagian `<head>`
- Eksternal - dengan menggunakan elemen `<link>` untuk menautkan ke file CSS eksternal

Cara paling umum untuk menambahkan CSS, adalah dengan menyimpan style di file CSS eksternal.

2.2.2 Cara Penggunaan CSS

Aturan CSS terdiri dari selector dan blok deklarasi:

- a. Selector menunjuk ke elemen HTML yang ingin diberikan style. Blok deklarasi berisi satu atau lebih deklarasi yang dipisahkan oleh titik koma (;).
- b. Setiap deklarasi menyertakan nama properti CSS dan nilai, dipisahkan oleh titik dua(:).
- c. Beberapa deklarasi CSS dipisahkan dengan titik koma, dan blok deklarasi dikelilingi oleh kurung kurawal.



Gambar 2.1 Struktur Dasar CSS

Contoh pemberian style pada element HTML paragraph <p> :

```
p {
  color: red;
  text-align: center;
}
```

Penjelasan:

- p adalah pemilih di CSS (ini menunjuk ke elemen HTML yang ingin diberikan style: <p>).
- color adalah *properti*, dan red adalah *nilai properti*
- text-align adalah *properti*, dan center adalah *nilai properti*

2.2.3 CSS Selector

Selector CSS digunakan untuk "menemukan" (atau memilih) elemen HTML yang ingin diberikan style. Pemilih selector CSS dibagi menjadi lima kategori, yaitu:

- Selektor sederhana (pilih elemen berdasarkan nama, id, kelas)
- Selektor kombinator (memilih elemen berdasarkan hubungan spesifik di antara mereka)
- Selector Pseudo-class (memilih elemen berdasarkan status tertentu)
- Selector Pseudo-element (memilih dan menata bagian elemen)
- Selector atribut (memilih elemen berdasarkan atribut atau nilai atribut)

Contoh syntax selector:

- Selektor Sederhana

p { text-align: right; color: red; }	#para1 { text-align: center; color: blue; }	p.description { text-align: left; color: green; }
---	--	--

```

<p>Every paragraph will be affected by the style.</p>
<p id="para1">Me too!</p>
<p class="description">And me!</p>

```

- Selektor kombinator

```

div p {
  background-color: yellow;
}

<h1>Descendant Selector</h1>
<div>
  <p>Every paragraph will be affected by the style.</p>
</div>

```

- Selector *Pseudo-class*

Pseudo-class digunakan untuk mendefinisikan keadaan khusus suatu elemen.

Misalnya, dapat digunakan untuk:

- style elemen saat pengguna mengarahkan mouse ke atasnya
- style tautan yang dikunjungi dan yang belum dikunjungi secara berbeda
- style elemen saat mendapat fokus

Syntax *pseudo-class*

```

selector:pseudo-
class {
  property: value;
}

```

Contoh:

```

.para1:hover {
  background-color: tomato;
  font-weight: bold;
}

<h1>Descendant Selector</h1>
<div id="content">
  <p class="para1">Every paragraph will be affected by the
style.</p>
  <p>Every paragraph will be affected by the style.</p>
</div>

```

Berikut ini adalah beberapa pseudo-class yang dapat digunakan:

Tabel 2.1 Selector Pseudo-class

Selector	Example	Example description
<u>:active</u>	a:active	Selects the active link
<u>:checked</u>	input:checked	Selects every checked <input> element
<u>:disabled</u>	input:disabled	Selects every disabled <input> element
<u>:empty</u>	p:empty	Selects every <p> element that has no children
<u>:enabled</u>	input:enabled	Selects every enabled <input> element
<u>:first-child</u>	p:first-child	Selects every <p> elements that is the first child of its parent
<u>:first-of-type</u>	p:first-of-type	Selects every <p> element that is the first <p> element of its parent
<u>:focus</u>	input:focus	Selects the <input> element that has focus
<u>:hover</u>	a:hover	Selects links on mouse over
<u>:in-range</u>	input:in-range	Selects <input> elements with a value within a specified range
<u>:invalid</u>	input:invalid	Selects all <input> elements with an invalid value
<u>:lang(language)</u>	p:lang(it)	Selects every <p> element with a lang attribute value starting with "it"
<u>:last-child</u>	p:last-child	Selects every <p> elements that is the last child of its parent
<u>:last-of-type</u>	p:last-of-type	Selects every <p> element that is the last <p> element of its parent
<u>:link</u>	a:link	Selects all unvisited links
<u>:not(selector)</u>	:not(p)	Selects every element that is not a <p> element
<u>:nth-child(n)</u>	p:nth-child(2)	Selects every <p> element that is the second child of its parent
<u>:nth-last-child(n)</u>	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child
<u>:nth-last-of-type(n)</u>	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child
<u>:nth-of-type(n)</u>	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent

:only-of-type	p:only-of-type	Selects every <p> element that is the only <p> element of its parent
:only-child	p:only-child	Selects every <p> element that is the only child of its parent
:optional	input:optional	Selects <input> elements with no "required" attribute
:out-of-range	input:out-of-range	Selects <input> elements with a value outside a specified range
:read-only	input:read-only	Selects <input> elements with a "readonly" attribute specified
:read-write	input:read-write	Selects <input> elements with no "readonly" attribute
:required	input:required	Selects <input> elements with a "required" attribute specified
:root	root	Selects the document's root element
:target	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)
:valid	input:valid	Selects all <input> elements with a valid value
:visited	a:visited	Selects all visited links

- Selector Pseudo-element

Elemen pseudo CSS digunakan untuk menata bagian tertentu dari suatu elemen.

Misalnya, dapat digunakan untuk:

- Gaya huruf pertama, atau baris, dari suatu elemen
- Sisipkan konten sebelum, atau setelah, konten elemen

Syntax pseudo-elements

```
selector::pseudo-element {
  property: value;
}
```

Contoh:

```
p::first-line {
  color: #ff0000;
  font-variant: small-caps;
}
```

[illegible]

Berikut ini adalah beberapa pseudo-elements yang dapat digunakan:

Tabel 2.2 Selector Pseudo-Elements

Selector	Example	Example description
<u>::after</u>	p::after	Insert something after the content of each <p> element
<u>::before</u>	p::before	Insert something before the content of each <p> element
<u>::first-letter</u>	p::first-letter	Selects the first letter of each <p> element
<u>::first-line</u>	p::first-line	Selects the first line of each <p> element
<u>::marker</u>	::marker	Selects the markers of list items
<u>::selection</u>	p::selection	Selects the portion of an element that is selected by a user

- Selector atribut

Selector [attribute] digunakan untuk memilih elemen dengan atribut tertentu. Dimungkinkan untuk menata elemen HTML yang memiliki atribut atau nilai atribut tertentu. Contoh berikut memilih semua <a> elemen dengan atribut target:

Syntax atribut

<pre>a[target] { background-color: yellow; }</pre>	<pre>a[target="_blank"] { background-color: purple; }</pre>
<h2>CSS [attribute="value"] Selector</h2> CNN Indonesia Detik.com Kompas.com	

2.2.4 CSS Inline

Atribut style digunakan untuk menambahkan gaya ke elemen, seperti warna, font, ukuran, dan lainnya. Pengaturan style pada suatu elemen, dapat dilakukan dengan atribut style. Atribut style HTML memiliki sintaks berikut:

```
<tagname style="property:value;">
```

Properti adalah properti CSS. *Value* adalah nilai CSS.

Contoh penggunaan *css inline*:

```
<h1 style="text-align:center;">Centered Heading</h1>
<p style="text-align:center;">Centered paragraph.</p>

<h1 style="font-size:300%;">This is a heading</h1>
<p style="font-size:160%;">This is a paragraph.</p>

<h1 style="font-family:verdana;">This is a heading</h1>
<p style="font-family:courier;">This is a paragraph.</p>

<h1 style="color:blue;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

<h1 style="background-color:blue;">This is a heading</h1>
<p style="background-color:tomato;">This is a
paragraph.</p>
```

2.2.5 CSS Internal

Kode CSS internal diletakkan di dalam bagian `<head>` pada halaman. *Class* dan *ID* bisa digunakan untuk merujuk pada kode CSS, namun hanya akan aktif pada halaman tersebut. Style CSS yang dipasang dengan metode ini akan di-download setiap kali halaman dipanggil, jadi ini akan meningkatkan kecepatan akses.

Namun, ada beberapa kasus dimana penggunaan internal *stylesheet* justru berguna. Salah satu contohnya adalah untuk mengirimkan template halaman ke seseorang – karena semuanya bisa terlihat dalam 1 halaman, maka akan lebih mudah untuk melihat previewnya. CSS internal diletakkan di dalam tag `<style></style>`. Contohnya:

```

<head>
<style>
  body {background-color: blue;}
  h1   {color: blue; font-family:verdana; font-family:courier; }
  p    {color: red;}
  .center {text-align: "center";}
  #card {border: 2px solid blue; margin: 50px;}
  #card > h1 {color: tomato}
  #card > p {background-color: pink}
</style>
</head>
<body>
<h1>Centered Heading</h1>
<p>Centered paragraph.</p>

<h1 class="center">This is a heading</h1>
<p class="center">This is a paragraph.</p>

<div id="card">
  <h1>This is a heading</h1>
  <p>This is a paragraph.</p>
</div>
</body>

```

2.2.6 CSS External

Salah satu cara yang paling nyaman untuk menambahkan CSS ke website adalah dengan menghubungkannya ke file .CSS eksternal. Dengan cara tersebut, perubahan apapun yang Anda buat pada file CSS akan tampil pada website Anda secara keseluruhan. File CSS eksternal biasanya diletakkan setelah bagian <head> pada halaman:

```

<head>
  <link rel="stylesheet" href="styles.css">
</head>

```

File styles.css

```

body {background-color: blue;}
h1   {color: blue; font-family:verdana; font-family:courier; }
p    {color: red;}
.center {text-align: "center";}
#card {border: 2px solid blue; margin: 50px;}
#card > h1 {color: tomato}
#card > p {background-color: pink}}

```

2.2.7 Component Properties

1. Comments

Komentar digunakan untuk menjelaskan kode, dan dapat membantu saat Anda mengedit kode sumber di kemudian hari. Komentar diabaikan oleh browser. Komentar CSS ditempatkan di dalam elemen `<style>`, dan dimulai dengan `/*` dan diakhiri dengan `*/`:

```
/* Set text paragraph color to red */  
p {  
    color: red;  
}
```

2. Colors

Color ditentukan menggunakan nama warna yang telah ditentukan sebelumnya, atau nilai RGB, HEX, HSL, RGBA, HSLA. Di CSS, warna dapat ditentukan dengan menggunakan nama warna yang telah ditentukan:



Gambar 2.2 CSS Colors

Mengatur warna latar belakang untuk elemen HTML:

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>  
<p style="background-color: #4CAF50;">Lorem ipsum...</p>
```

Mengatur set color pada text:

```
<h1 style="color:Tomato;">Hello World</h1>  
<p style="color: #1e90ff;">Lorem ipsum...</p>  
<p style="color: rgb(30 255 203);">Ut wisi enim...</p>
```

Mengatur set color pada border:

```
<h1 style="border:1px dashed Tomato;">Hello World</h1>
<p style="border:2px solid #1e90ff;">Lorem ipsum...</p>
<p style="border:3px dotted rgb(30 255 203);">Ut wisi enim...</p>
```

3. Backgrounds

Property *background* CSS digunakan untuk menambahkan efek latar belakang untuk elemen. Dalam bagian ini, berikut ini beberapa property background CSS yang dapat digunakan:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position
- background

4. Borders

Property *border* CSS memungkinkan untuk menentukan style, lebar, dan warna pada setiap elemen. Property border-style menentukan jenis border apa yang akan ditampilkan.

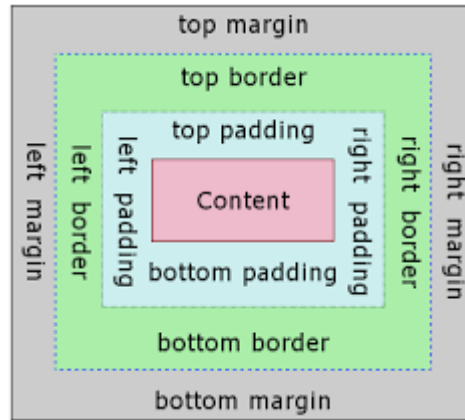
```
p{border-style: value;}
```

Berikut ini beberapa jenis value border yang dapat digunakan:

- dotted
- dashed
- solid
- double
- groove
- ridge
- inset
- outset
- none
- hidden

5. Margins dan Paddings

Margin dan padding merupakan properties CSS yang diperuntukan untuk menentukan batas setiap element. Margin diperuntukan untuk menentukan batas luar sedangkan padding diperuntukan untuk batas dalam pada suatu elemen.



Gambar 2.3 CSS Box Model

- **Margin**

Margin digunakan untuk membuat ruang di sekitar elemen, di luar batas yang ditentukan. Properti margin CSS digunakan untuk membuat ruang di sekitar elemen, batas diluar elemen. Dengan CSS, memiliki kontrol penuh atas margin. Ada properti untuk mengatur margin untuk setiap sisi elemen (top, right, bottom, dan left).

CSS memiliki properti untuk menentukan margin untuk setiap sisi elemen:

- margin-top
- margin-right
- margin-bottom
- margin-left
- margin

Semua properti margin dapat memiliki nilai sebagai berikut:

- auto : browser menghitung margin
- length : menentukan margin dalam px, pt, cm, dll.
- % : menentukan margin dalam % dari lebar elemen yang mengandung

- inherit : menentukan bahwa margin harus diwarisi dari elemen induk

Contoh

```
p {  
  margin-top: 100px;  
  margin-bottom: 100px;  
  margin-right: 150px;  
  margin-left: 80px;  
}  
h1{ margin: 25px 50px 75px 100px; }
```

- **Padding**

Jika margin diperuntukan untuk menentukan batas luar pada elemen, berbeda dengan padding. Padding diperuntukan untuk menentukan batas dalam pada element.

Properti padding CSS digunakan untuk menghasilkan ruang di sekitar konten elemen, di dalam batas yang ditentukan. Dengan CSS, memiliki kontrol penuh atas padding. Ada properti untuk mengatur padding untuk setiap sisi elemen (atas, kanan, bawah, dan kiri):

- padding-top
- padding-right
- padding-bottom
- padding-left
- padding

Semua properti margin dapat memiliki nilai sebagai berikut:

- length : menentukan margin dalam px, pt, cm, dll.
- % : menentukan margin dalam % dari lebar elemen yang mengandung
- inherit : menentukan bahwa margin harus diwarisi dari elemen induk

Contoh:

```
div {  
  padding-top: 50px;  
  padding-right: 30px;  
  padding-bottom: 50px;  
  padding-left: 80px;  
}
```

```
.card {  
  padding: 25px 50px 75px 100px;  
}
```

6. Height dan Width

Properti Height dan Width digunakan untuk mengatur tinggi dan lebar suatu elemen. Properti Height dan Width tidak termasuk padding, border, atau margin. Ini mengatur tinggi/lebar area di dalam padding, border, dan margin elemen.

Properti height dan width mungkin memiliki nilai berikut:

- auto : Ini adalah default. Browser menghitung tinggi dan lebar
- length : Mendefinisikan tinggi/lebar dalam px, cm dll.
- % : Mendefinisikan tinggi/lebar dalam persen dari blok yang berisi
- initial : Mengatur tinggi/lebar ke nilai default
- inherit : Tinggi/lebar akan diwarisi dari nilai induknya

Contoh:

```
div {  
  height: 200px;  
  width: 50%;  
  background-color: powderblue;  
}
```

7. Text

- Text alignment

Properti text-align digunakan untuk mengatur perataan *horizontal* teks. Sebuah teks dapat diratakan kiri atau kanan, di tengah, atau diratakan. Contoh berikut menunjukkan rata tengah, dan teks rata kiri dan kanan (perataan kiri adalah default jika arah teks dari kiri ke kanan, dan perataan kanan adalah default jika arah teks dari kanan ke kiri):

```
h1 {text-align: center; }  
h2 {text-align: left; }  
h3 {text-align: right;}  
p {text-align: justify;}
```

- Text Decoration

Properti text-decoration digunakan untuk mengatur atau menghapus dekorasi dari teks. Nilai text-decoration: none; sering digunakan untuk menghapus garis bawah dari tautan:

```
a {text-decoration: none;}
```

Contoh:

```
h2 {text-decoration: overline;}  
h3 {text-decoration: line-through;}  
h4 {text-decoration: underline;}
```

- Text Transformation

Properti text-transform digunakan untuk menentukan huruf besar dan huruf kecil dalam sebuah teks. Ini dapat digunakan untuk mengubah semuanya menjadi huruf besar atau kecil, atau menggunakan huruf besar untuk huruf pertama setiap kata:

```
.title {text-transform: uppercase;}  
span {text-transform: lowercase;}  
p {text-transform: capitalize;}
```

- Text Spacing

Text spacing merupakan properti yang diperuntukan untuk memberikan jarak tertentu pada suatu text word. Berikut ini ada beberapa properti yang dapat digunakan untuk menentukan text spacing:

```
p {text-indent: 50px;}  
h1 {letter-spacing: 5px;}  
h2 {letter-spacing: -2px;}  
p.big {line-height: 1.8;}  
.two {word-spacing: -2px;}  
p {white-space: nowrap;}
```

- Text Shadow

Property text-shadow menambahkan bayangan ke teks. Dalam penggunaannya yang paling sederhana, Anda hanya menentukan bayangan horizontal (2px), bayangan vertikal (2px), efek blur (5px) dan warna red untuk bayangannya:

```
h1 {text-shadow: 2px 2px 5px red; }
```

8. Font

Memilih font yang tepat memiliki dampak besar pada bagaimana pembaca mengalami situs web. Font yang tepat dapat menciptakan identitas yang kuat untuk merek Anda. Menggunakan font yang mudah dibaca adalah penting. Font menambah nilai pada teks Anda. Penting juga untuk memilih warna dan ukuran teks yang benar untuk font.

Tabel 2.3 Font Properties

Property	Description
font	Sets all the font properties in one declaration
font-family	Specifies the font family for text
font-size	Specifies the font size of text
font-style	Specifies the font style for text
font-variant	Specifies whether or not a text should be displayed in a small-caps font
font-weight	Specifies the weight of a font

Contoh

```
.p1 {
  font-family: "Times New Roman", Times, serif;
  font-size: 40px;
}
.normal {
  font-style: normal;
  font-variant: normal;
  font-weight: normal;
  font-style: normal;
}
.italic {font-style: italic;}
.oblique {font-style: oblique;}
.thick {font-weight: bold;}
.small {font-variant: small-caps;}
```

9. List

Dalam HTML, ada dua jenis daftar utama:

- List unordered () ? item daftar ditandai dengan peluru atau bulat

- List ordered () 2 item daftar ditandai dengan angka atau huruf

Properti daftar CSS memungkinkan Anda untuk:

- Tetapkan penanda item daftar yang berbeda untuk daftar yang dipesan
- Tetapkan penanda item daftar yang berbeda untuk daftar yang tidak berurutan
- Tetapkan gambar sebagai penanda item daftar
- Tambahkan warna latar belakang ke daftar dan daftar item

Property list-style-type menentukan tipe penanda item daftar. Contoh berikut menunjukkan beberapa penanda item daftar yang tersedia:

```
ul.a {list-style-type: circle;}
ul.b {list-style-type: square;}
ol.c {list-style-type: upper-roman;}
ol.d {list-style-type: lower-alpha;}
```

10. Display

Properti display menentukan perilaku tampilan (tipe kotak rendering) dari suatu elemen. Dalam HTML, nilai properti display default value diambil dari spesifikasi HTML atau dari style css default browser/pengguna. Nilai default dalam XML adalah inline, termasuk elemen SVG.

```
p.ex1 {display: none;}
p.ex2 {display: inline;}
p.ex3 {display: block;}
p.ex4 {display: inline-block;}
```

11. Float

Properti float digunakan untuk memposisikan dan memformat konten, misalkan. biarkan gambar melayang ke kiri ke teks dalam wadah. Properti float dapat memiliki salah satu dari nilai berikut:

- left : Elemen mengapung di sebelah kiri wadahnya
- right : Elemen mengapung di sebelah kanan wadahnya
- none : Elemen tidak mengapung (akan ditampilkan tepat di tempat kemunculannya dalam teks). Ini adalah default
- inherit : Elemen mewarisi nilai float dari induknya

Dalam penggunaannya yang paling sederhana, properti float dapat digunakan untuk membungkus teks di sekitar gambar.

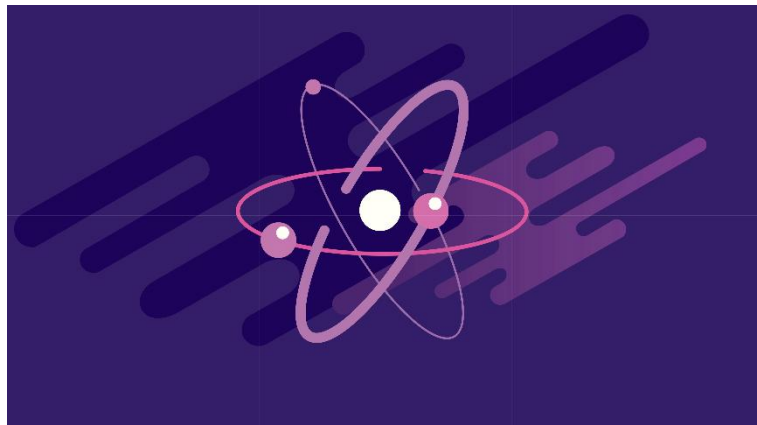
```
img {float: right;}
```

2.3 Latihan Pembelajaran

1. Buatlah file dokumen HTML bernama Latihan-1, ubahlah contoh dibawah ini kedalam scripting HTML dan menggunakan CSS sebagai variasi style pada elemen HTML:

Mengenal Partikel dan Notasi Atom

Nama anda, 20 Feb 2022 – 08:55 am



Perhatikan sekeliling kalian, matahari terbit dari timur di pagi hari, bulan muncul pada malam hari, bumi mengelilingi matahari dalam dua belas bulan, dan banyak lagi keteraturan di alam semesta ini. Hebat ya *Sang Pencipta* kita mengatur alam semesta ini dengan rapi. Bahkan, sampai tingkat paling kecil pun, elektron-elektron di alam semesta ini telah diatur dengan rapi menurut bilangan kuantumnya! **Wow apa tuh bilangan kuantum?**

Elektron-elektron tersebar di sekeliling atom dengan teratur berdasarkan tingkat energinya. Nah, tingkat energi inilah yang digambarkan dengan bilangan kuantum. Artinya, dari bilangan kuantum, lokasi-lokasi penyebaran elektron dapat digambarkan. Sedetail itu loh Sang Pencipta kita mengaturnya. Bayangkan kalau elektron, penyusun segala sesuatu di alam semesta ini, tidak teratur. Alam semesta ini tidak stabil dong. Mana bisa kita hidup di dunia seperti itu. Keren kan?

Salah satu contoh atom di alam semesta ini adalah atom karbon. Atom karbon adalah penyusun dari berbagai benda yang sangat berguna. Mulai dari bensin, plastik, berlian, bahkan tubuh kita pun tersusun dari karbon! Nah, karbon (*biasa dilambangkan dengan huruf C*) punya 6 elektron. Bagaimana bilangan kuantum dari elektron terakhirnya? Tinggal ikuti deh langkah-langkahnya.

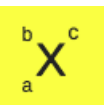
Partikel Dasar Penyusun Atom dan Lambang Atom

Partikel dasar penyusun atom ada tiga yaitu proton (*p*), neutron (*n*) dan elektron (*e*). Jadi, massa atom = (massa *p* + massa *n*) + massa *e*. Massa elektron jauh lebih kecil daripada massa proton dan massa neutron, maka massa elektron dapat diabaikan. Dengan demikian: massa atom = massa *p* + massa *n*.

Partikel	Lambang	Massa(g)	Muatan	
			Satuan	Coulomb
proton	p	1.673×10^{-24}	+1	1.6×10^{-9}
neutron	n	1.673×10^{-24}	0	0
elektron	e	9.109×10^{-28}	-1	1.6×10^{-19}

Lambang Atom

X Simbol dari unsur.



a nomor atom merupakan jumlah proton. Saat netral (tidak bermuatan) akan sama dengan jumlah elektron.

b nomor massa melambangkan jumlah proton ditambah jumlah neutron atau disebut juga jumlah nukleon.

c Muatan/bilangan oksidasi (biloks) terdiri dari melepas elektron (positif) dan menangkap elektron atau bertambah (negatif).

Bagikan artikel ini



- 2 Buatlah file dokumen HTML bernama Latihan-2, ubahlah contoh dibawah ini kedalam scripting HTML dan menggunakan CSS sebagai variasi style pada elemen HTML:

Daftar Hewan Dilindungi



Orang Utan

Orang utan dicirikan oleh rambut di seluruh badannya yang berwarna kemerahan. Satwa ini merupakan mamalia arboreal terbesar yang menghabiskan hampir seluruh waktunya di pepohonan. Lengannya yang panjang dan kuat serta tangan dan kakinya yang dapat mencengkeram erat.



Kera Emas

Monyet yang unik ini juga dikenal sebagai hewan yang memiliki adaptasi yang sangat baik dengan lingkungan yang bersuhu sangat dingin. Umumnya, monyet hidung pesek rambut emas tinggal di hutan pegunungan.

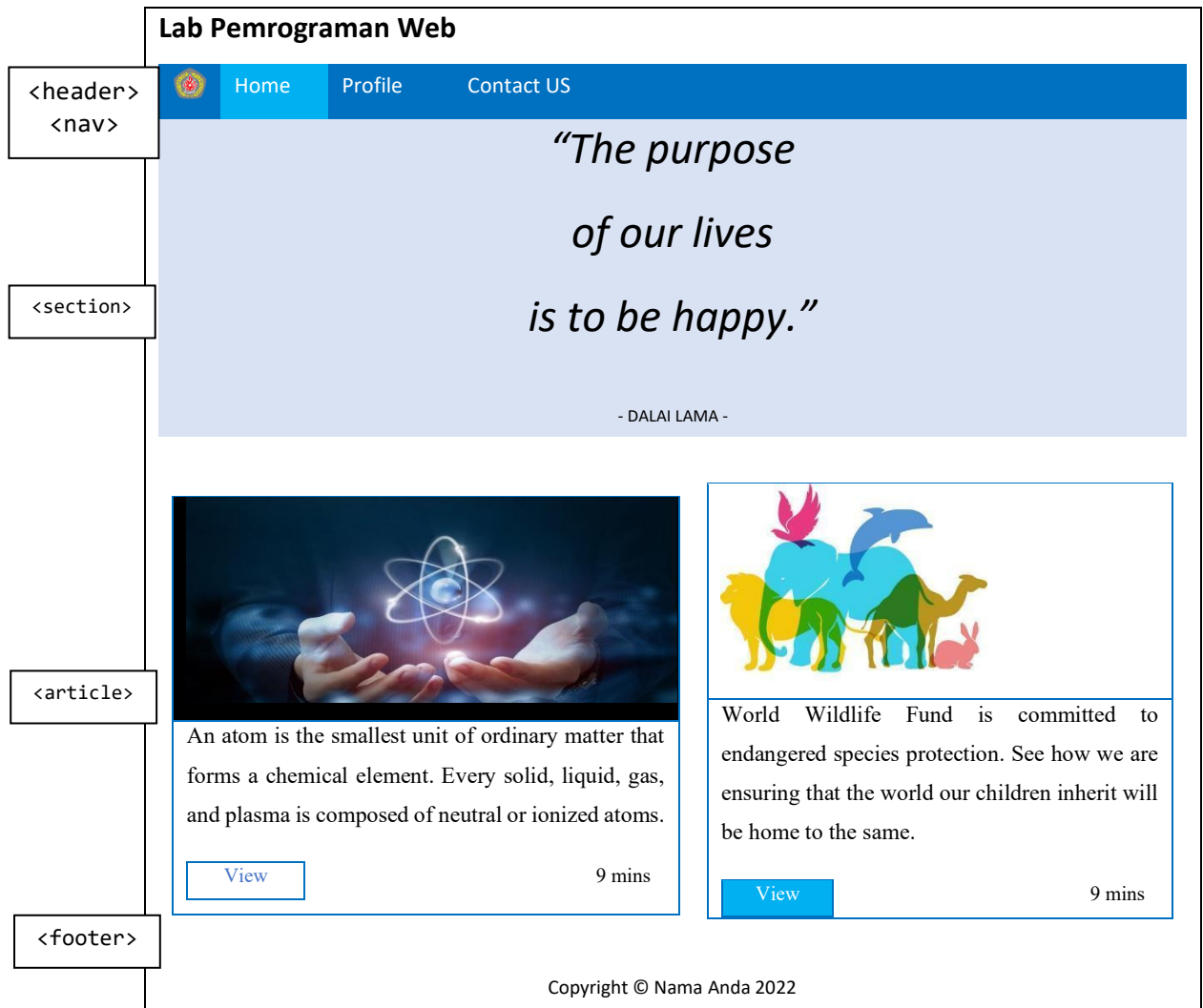


Burung Cenderawasih

Burung cenderawasih termasuk ke dalam hewan langka dan hanya bisa ditemukan di Papua. Burung cenderawasih punya tampilan yang cantik yang membuat burung ini sering menjadi perburuan liar untuk didagangkan.

Copyright © Nama Anda 2022

- 3 Buatlah file dokumen HTML bernama Latihan-3, ubahlah contoh dibawah ini kedalam scripting HTML dan menggunakan CSS sebagai variasi style pada elemen HTML:



2.4 Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-Praktikum-2-NPM"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB III

PENGENALAN JAVASCRIPT

3.1 Tujuan Pembelajaran

1. Memahami peran JavaScript dalam pengembangan website.
2. Menjelaskan perbedaan fungsi antara:
 - HTML (struktur)
 - CSS (tampilan)
 - JavaScript (interaktivitas)
3. Menuliskan sintaks dasar JavaScript.
4. Menggunakan variabel, tipe data, dan operator.
5. Membuat percabangan dan perulangan sederhana.
6. Membuat dan memanggil fungsi.
7. Menghubungkan JavaScript dengan elemen HTML.
8. Membuat interaksi sederhana pada halaman web (misalnya klik tombol)

3.2 Dasar Teori

3.2.1 Pengertian Javascript (JS)

Javascript diperkenalkan pertama kali oleh Netscape pada tahun 1995. Pada awalnya bahasa ini dinamakan “LiveScript” yang berfungsi sebagai bahasa sederhana untuk browser Netscape Navigator 2. Pada masa itu bahasa ini banyak dikritik karena kurang aman, pengembangannya yang terkesan buru buru dan tidak ada pesan kesalahan yang ditampilkan setiap kali membuat kesalahan pada saat menyusun suatu program. Kemudian sejalan dengan sedang giatnya kerjasama antara Netscape dan Sun (pengembang bahasa pemrograman “Java”) pada masa itu, maka Netscape memberikan nama “JavaScript” kepada bahasa tersebut pada tanggal 4 desember 1995. Pada saat yang bersamaan Microsoft sendiri mencoba untuk mengadaptasikan teknologi ini yang mereka sebut sebagai “Jscript” di browser Internet Explorer 3.

Javascript adalah bahasa yang berbentuk kumpulan skrip yang pada fungsinya berjalan pada suatu dokumen HTML, sepanjang sejarah internet bahasa ini adalah bahasa skrip pertama untuk web. Bahasa ini adalah bahasa pemrograman untuk memberikan

kemampuan tambahan terhadap bahasa HTML dengan mengizinkan pengeksekusian perintah perintah di sisi user, yang artinya di sisi browser bukan di sisi server web.

Javascript bergantung kepada browser(navigator) yang memanggil halaman web yang berisi skrip skrip dari Javascript dan tentu saja terselip di dalam dokumen HTML. Javascript juga tidak memerlukan kompilator atau penterjemah khusus untuk menjalankannya (pada kenyataannya kompilator Javascript sendiri sudah termasuk di dalam browser tersebut). Lain halnya dengan bahasa “Java” (dengan mana JavaScript selalu di banding bandingkan) yang memerlukan kompilator khusus untuk menerjemahkannya di sisi user/klien. Untuk mempelajari pemrograman Java Script, ada dua piranti yang diperlukan, yaitu :

- Teks Editor

Digunakan untuk menuliskan kode-kode Java Script, teks editor yang dapat digunakan antara lain notepad dan ultraedit.

- Web Browser

Digunakan untuk menampilkan halaman web yang mengandung kode-kode Java Script. Web browser yang digunakan harus mendukung Javascript. Browser yang dapat digunakan adalah internet explorer dan Netscape Navigator.

3.2.2 Cara Penggunaan JS ke HTML

Kode Java Script dituliskan pada file HTML. Terdapat dua cara untuk menuliskan kode-kode Java Script agar dapat ditampilkan pada halaman HTML, yaitu :

a. JS Internal

Untuk penulisan dengan cara ini, perintah yang digunakan adalah

```
<SCRIPT LANGUAGE ="JavaScript" >code javascript disini</SCRIPT>
```

Perintah tersebut biasanya diletakkan diantara Tag <body>...</body>

Contoh Penulisan :

```
<!DOCTYPE html>
<html>
<head>
  <title>Judul Halaman</title>
  <SCRIPT LANGUAGE ="JavaScript" >code javascript disini</SCRIPT>
</head>
<body>
  <h1>html</h1>
  <p>disini</p>
```

```
</body>
</html>
```

b. JS External

Kode Javascript bisa juga buat dalam file terpisah dengan tujuan agar dokumen HTML isinya tidak terlalu panjang. Atribut yang digunakan adalah

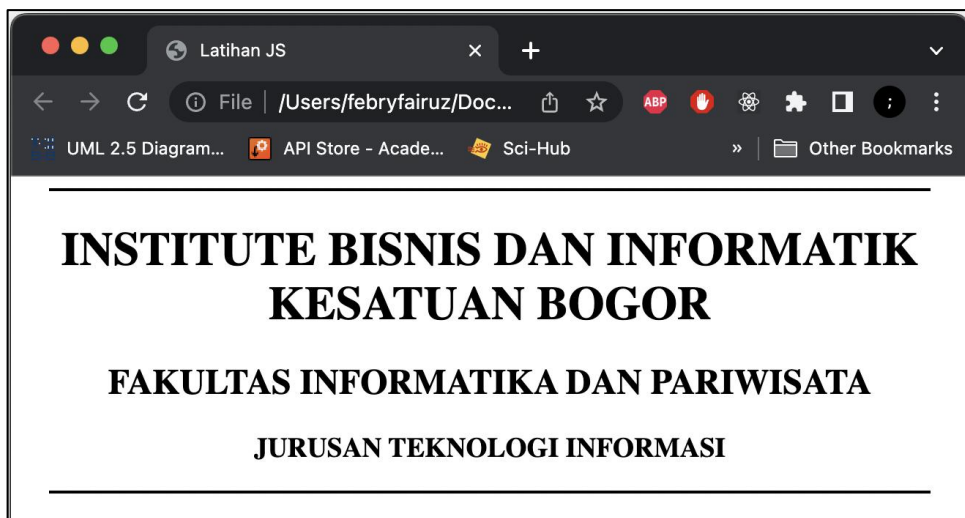
```
<SCRIPT SRC="namafile.js">...</SCRIPT>
```

Diantara tag <SCRIPT...> dan </SCRIPT> tidak diperlukan lagi kode Javascriptnya karena sudah dibuat dalam file terpisah. File yang mengandung kode Javascript berekstensi .js Pada bagian ini akan membuat program dengan menggunakan Javascript. Program ini akan menampilkan teks “ *Belajar Pemrograman Javascript*”

```
<!DOCTYPE html>
<html>
<head>
  <title>Judul Halaman</title>
  <SCRIPT LANGUAGE =”JavaScript” SRC=”sample.js” ></SCRIPT>
</head>
<body>
</body>
</html>
```

File *sample.js*

```
document.writeln("<CENTER>");
document.writeln("<HR WIDTH=600 COLOR=Black>");
document.writeln("<H1>INSTITUTE BISNIS DAN INFORMATIK KESATUAN
BOGOR</H1>");
document.writeln("<H2>FAKULTAS INFORMATIKA DAN PARIWISATA<H2>");
document.writeln("<H3>JURUSAN TEKNOLOGI INFORMASI<H3>");
document.writeln("<HR WIDTH=600 COLOR=Black>");
document.writeln("</CENTER>");
```



Gambar 3.1 Output Latihan JS (JS External)

Sama seperti bahasa pemrograman lain. Javascript juga menyediakan fasilitas untuk menuliskan komentar, komentar ini berguna bila nantinya anda atau orang lain membaca program. Pemberian komentar dalam Javascript dapat dilakukan dengan dua cara yaitu dengan menuliskan komentar setelah tanda garis miring dua kali, contoh :

```
//ini komentar
atau
/*ini juga komentar */
```

3.2.3 Variable

Variabel adalah tempat dimana menyimpan nilai-nilai atau informasi-informasi pada JavaScript. Variabel yang dideklarasikan dapat diisi dengan nilai apa saja. Dalam JavaScript pendeklarasian sebuah variabel sifatnya opsional, artinya anda boleh mendeklarasikan atau tidak hal tersebut tidak menjadi masalah. Jika anda memberi nilai pada variabel, maka dalam JavaScript dianggap bahwa anda telah mendeklarasikan variabel tersebut.

Aturan penamaan variabel :

- Harus diawali dengan karakter (huruf atau baris bawah)
- Tidak boleh menggunakan spasi
- Huruf Kapital dan kecil memiliki arti yang berbeda
- Tidak boleh menggunakan kata-kata yang merupakan perintah dalam JavaScript.

Deklarasi Variabel

```
nama_variabel = nilai
atau
nama_variabel = nilai
```

Contoh :

```
let nama;      Nama = "Bunga Lestari"
const nama = " Zaskia Mecca"      X = 1990;
let X = 1998; Y = 08170223513 var Y;
```

3.2.4 Tipe Data

Tidak seperti bahasa pemrograman lainnya, JavaScript tidak memiliki tipe data secara eksplisit. Hal ini dapat dilihat dari beberapa contoh variabel diatas. Anda mendeklarasikan variabel tapi tidak menentukan tipenya. Meskipun JavaScript tidak memiliki tipe data secara eksplisit. JavaScript mempunyai tipe data implisit. Terdapat empat macam tipe data implisit yang dimiliki oleh JavaScript yaitu:

- Numerik : seperti 0222532531, 1000, 45, 3.146789 dsb
- String : seperti : seperti “Hallo”, “April”, “Jl. Setiabudi No 17A”, “Cece Kirani” dsb
- Boolean : bernilai true atau false
- Null : variabel yang tidak diinisialisasi

3.2.5 Operator

Operator pada JavaScript terbagi menjadi enam, yaitu :

a. Operator Aritmatika

Digunakan untuk operan bertipe numerik. Ada dua macam operator aritmatik, yaitu operator numerik tunggal dan operator aritmatik biner. Perbedaan kedua operator terletak pada jumlah operan yang harus dioperasikan.

Tabel 3.1 Operator Aritmatika

Operator	Tunggal/Biner	Keterangan
+	Biner	Penjumlahan
-	Biner	Pengurangan
*	Biner	Perkalian
/	Biner	Pembagian
%	Biner	Modulus
-	Tunggal	Negasi
++	Tunggal	Penambahan dengan satu
--	Tunggal	Pengurangan dengan satu

b. Operator Pemberian Nilai

Digunakan untuk memberikan nilai ke suatu operan atau mengubah nilai suatu operan.

Tabel 3.2 Operator Pemberian Nilai

Operator	keterangan	Contoh	Ekuivalen
=	Sama dengan	X=Y	
+=	Ditambah dengan	X+=Y	X=X+Y
-=	Dikurangi dengan	X-=Y	X=X-Y
=	Dikali dengan	X=Y	X=X*Y
/=	Dibagi dengan	X/=Y	X=X/Y
%=	Modulus dengan	X%=Y	X=X%Y
&=	Bit AND dengan	X&Y	X=X&Y
=	Bit OR	X =Y	X=X Y

c. Operator Manipulasi Bit

Operasi ini berhubungan dengan pemanipulasian bit pada operan bertipe bilangan bulat.

Tabel 3.3 Operator Manipulasi Bit

Operator	Keterangan
&	Bit AND
	Bit OR
^	Bit XOR
~	Bit NOT
<<	Geser ke kiri
>>	Geser ke Kanan
>>>	Geser ke kanan dengan diisi nol

Contoh :

```
var A = 12;    // A = 1100b
```

```
var B = 10;    // B = 1010b
```

```
var C = A & B
```

maka akan dihasilkan bilangan seperti berikut :

1100b

1010b AND

1000b

```
var A = 12;
```

```
var C = A << 2
```

```
var D = A >> 1
```

Maka variabel C akan bernilai 48(0011 0000b) variabel D akan bernilai 6 (0110b)

d. Operator Pembandingan

Digunakan untuk membandingkan dua buah operand. Operan yang dikenal operator ini dapat bertipe string, numerik, maupun ekspresi lain.

Tabel 3.4 Operator Pembandingan

Operator	Keterangan
==	Sama dengan
!=	Tidak sama dengan
>	Lebih besar
<	Lebih kecil
>=	Lebih besar atau sama dengan
<=	Lebih kecil atau sama dengan

e. Operator Logika

Digunakan untuk mengoperasikan operan yang bertipe boolean.

Tabel 3.5 Operator Logika

Operator	Keterangan
&&	Operator logika AND
	Operator Logika OR
!	Operator logika NOT

Contoh :

```
var A = true; var B = false;
var C = A && B; //menghasilkan false
var D = A || B; // false
var E = !A; //false
```

f. Operator String

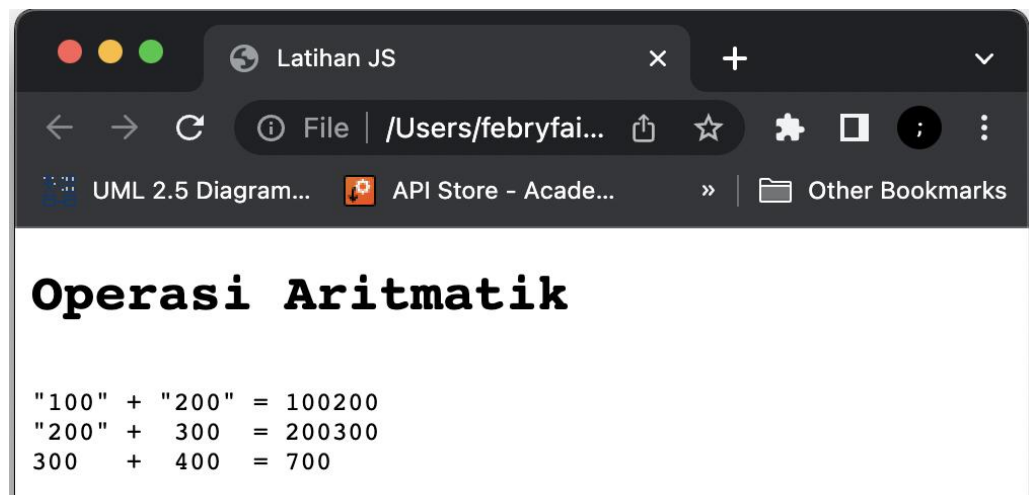
Selain operator pembandingan, operator string pada JavaScript juga mengenal satu operator lagi yang bernama PENGGABUNGAN. Operator ini digunakan untuk menggabungkan beberapa string menjadi sebuah string yang lebih panjang.

Contoh : nama = "Java" + "Script";

akan menghasilkan "JavaScript" pada variabel nama

Contoh Program JavaScript :

```
<HTML>
<HEAD><TITLE>Operasi Aritmatika</TITLE></HEAD>
<BODY>
<P><SCRIPT language="JavaScript">
<!--
document.writeln("<PRE>");
document.writeln("<H1>Operasi Aritmatika</H1>"); var A = "100";
var B = "200"; var C = 300; var D = 400; var E = A + B;
document.writeln('"100" + "200" = ' + E); E = B + C;
document.writeln('"200" + 300 = ' + E); E = C + D;
document.writeln('300 + 400 = ' + E);
document.writeln("<PRE>");
//-->
</SCRIPT></P>
</BODY>
</HTML>
```

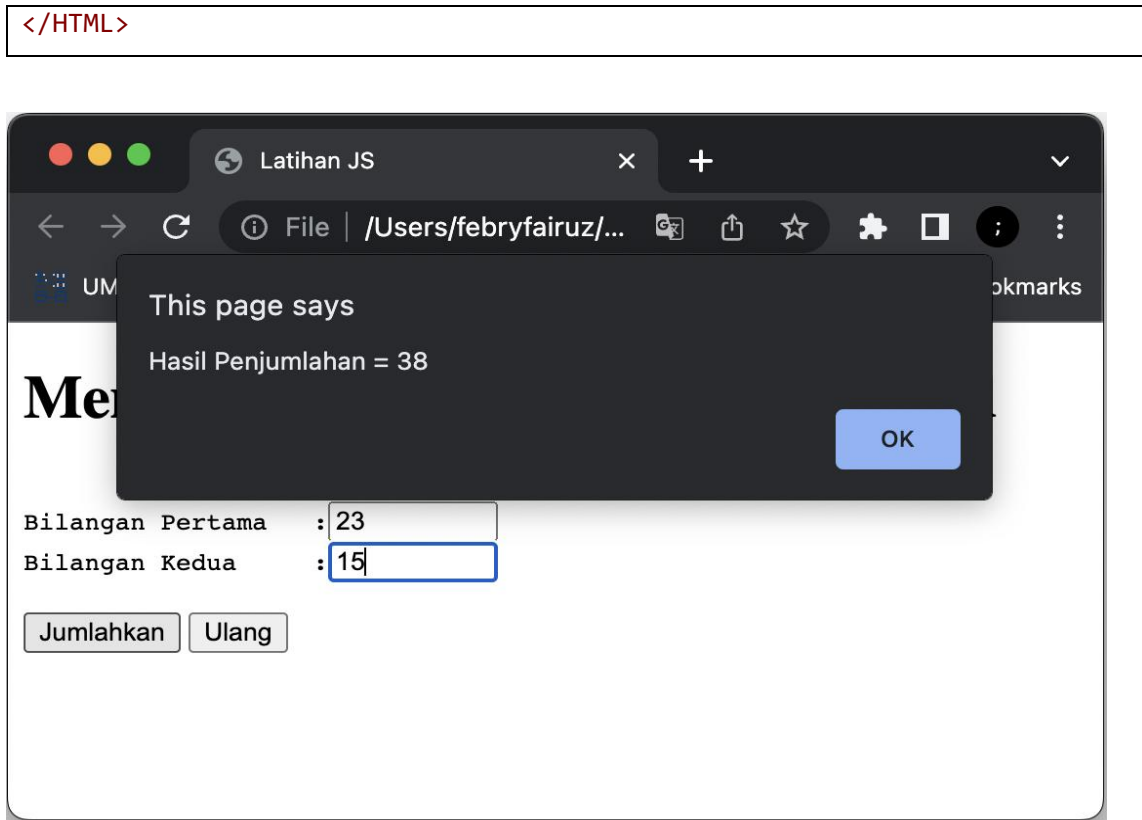


Gambar 3.2 Output Latihan JS (Operasi Aritmatika)

Untuk memasukkan data dari keyboard dapat dilakukan dengan menggunakan perintah **input**.

Contoh Program JavaScript :

```
<HTML>
<HEAD>
  <TITLE>Memasukkan Bilangan</TITLE>
</HEAD>
<BODY>
  <P>
    <SCRIPT language="JavaScript">
      function jumlah()
      {
        var bil1 = parseFloat(document.fform.bilangan1.value); if
(isNaN (bil1))    bil1=0.0;
        var bil2 = parseFloat(document.fform.bilangan2.value); if
(isNaN (bil2))    bil2=0.0;    var hasil = bil1 + bil2;
        alert ("Hasil Penjumlahan = " + hasil);
      }
    </P>
    <FORM NAME="fform">
      <H1><BR>Memasukkan Data Lewat Keyboard</H1>
      <PRE>
Bilangan Pertama    :<input type="text" size="11" name="bilangan1">
Bilangan Kedua      :<input type="text" size="11" name="bilangan2">
</PRE>
      <P>
        <INPUT TYPE="button" value="Jumlahkan" onclick="jumlah()">
        <INPUT TYPE="reset" value="Ulang">
      </P>
    </FORM>
  </BODY>
```



Gambar 3.3 Output Latihan JS (Memasukkan Bilangan)

3.2.6 Document Object Model (DOM) HTML

DOM (*Document Object Model*) adalah antarmuka pemrograman (interface) yang merepresentasikan dokumen HTML sebagai struktur pohon (tree) objek. Ini memungkinkan bahasa skrip seperti JavaScript untuk mengakses, mengubah, menambah, atau menghapus elemen, gaya, dan konten HTML secara dinamis tanpa memuat ulang halaman, menjadikan website interaktif.

Berikut adalah poin-poin penting mengenai DOM:

- **Struktur Pohon (Tree Structure):** DOM mengatur dokumen HTML menjadi hierarki node (pohon), di mana tag `<html>` adalah akar, diikuti cabang seperti `<head>` dan `<body>`, serta anak-anaknya (elemen, teks, atribut).
- **Jembatan JavaScript:** DOM berfungsi sebagai penghubung antara kode JavaScript dan dokumen HTML. Saat browser memuat halaman, ia membangun DOM agar JavaScript bisa berinteraksi dengannya.

- Fungsi Utama: Digunakan untuk mengubah teks, mengubah gaya (CSS), menambah atau menghapus elemen HTML, serta merespons event pengguna seperti klik tombol.
- Perbedaan dengan HTML: HTML adalah kode sumber yang ditulis (.html), sedangkan DOM adalah representasi struktur tersebut di memori saat halaman sedang berjalan (live).

Contoh penggunaan DOM: `document.getElementById("demo").innerHTML = "Hello World!"`; akan mengubah konten elemen dengan ID "demo" secara langsung

- **DOM untuk memasukan data**

Terdapat beberapa objek yang dapat digunakan untuk memasukkan data. Objek Objek tersebut biasanya terdapat dalam suatu form. Adapun objek-objek tersebut meliputi Objek Text, Objek Radio, Objek Checkbox, Objek Textarea, dan Objek Select.

1. Objek Text

Untuk menginputkan data dapat menggunakan komponen/objek text. Contoh penggunaannya dapat lihat pada contoh berikut :

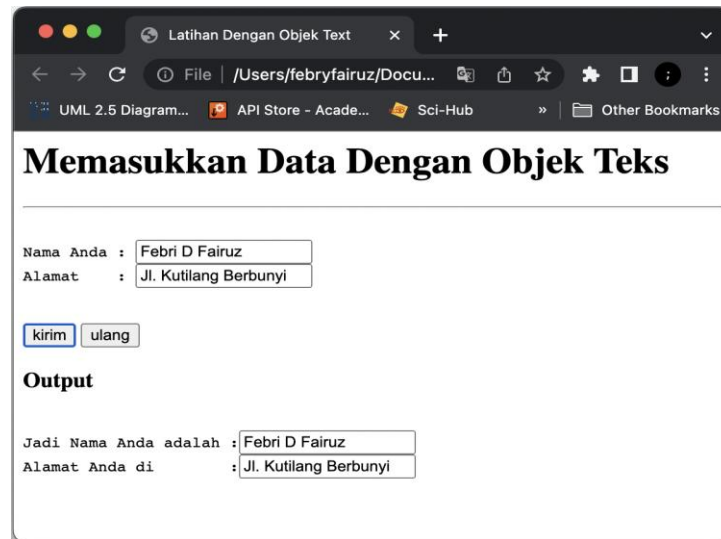
```
<html>
<head>
  <title>Latihan Dengan Objek Text</title>
</head>

<body>
  <script language="JavaScript">
    <!--
      function tekan()
      {
        var namastr = (document.fform.nama.value); var alamatstr =
        (document.fform.alamat.value); document.fform.onama.value =
        namastr; document.fform.oalamat.value = alamatstr;
      }
    //-->
  </script>
  <form name="fform">
    <H1> Memasukkan Data Dengan Objek Teks</H1>
    <hr>
    <PRE>
    Nama Anda : <input type="text" size="11" name="nama">
    Alamat   : <input type="text" size="25" name="alamat">
  </PRE>
    <BR>
    <input type="button" value="kirim" onclick="tekan()">
```

```


<H3>Output</H3>
<PRE>
Jadi Nama Anda adalah :<input type="text" size="11" name="onama">
Alamat Anda di          :<input type="text" size="25" name="oalamat">
</form>
</body>
</html>

```



Gambar 3.4 Output Latihan Objek Text

2. Objek Radio

Objek radio adalah komponen yang digunakan untuk melakukan suatu pemilihan data. Karena selalu berupa Array, untuk mengakses satu tombol radio digunakan `radio[indeks]`. Disamping itu objek radio juga mempunyai nilai `True` jika dipilih dan `False` jika tidak. Untuk memilih suatu objek radio menggunakan properti `Checked`.

```

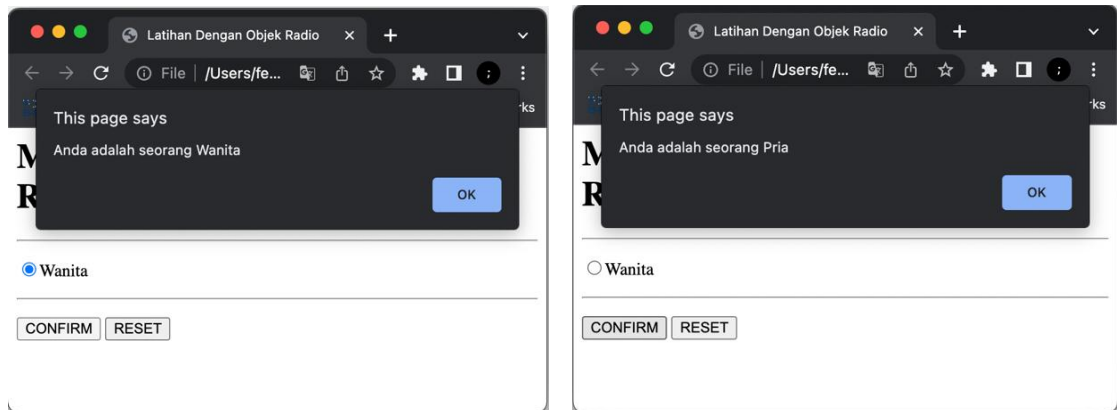
<html>
<head>
  <title>Latihan Dengan Objek Radio</title>
</head>
<body>
  <script language="JavaScript">
    <!--
    function radio_box(form)
    { var ket = "";
      if (form.wanita.checked == true)
      {
        ket = "Wanita";
      } else {

```

```

        ket = "Pria";
    }
    alert('Anda adalah seorang ' +ket);
}
//-->
</script>
<form>
    <H1> Memasukkan Data Dengan Objek Radio</H1>
    <hr>
    <p><input type="radio" value="wanita" name="wanita">Wanita</p>
    <hr>
    <p><input type="button" value="CONFIRM"
onclick="radio_box(this.form)"> <input type="reset" value="RESET"></p>
</form>
</body>
</html>

```



Gambar 3.5 Output Latihan Objek Radio

3. Objek Checkbox

Objek checkbox menyimpan informasi tentang elemen form yang berupa kotak cek. Penggunaannya hampir sama seperti objek radio.

```

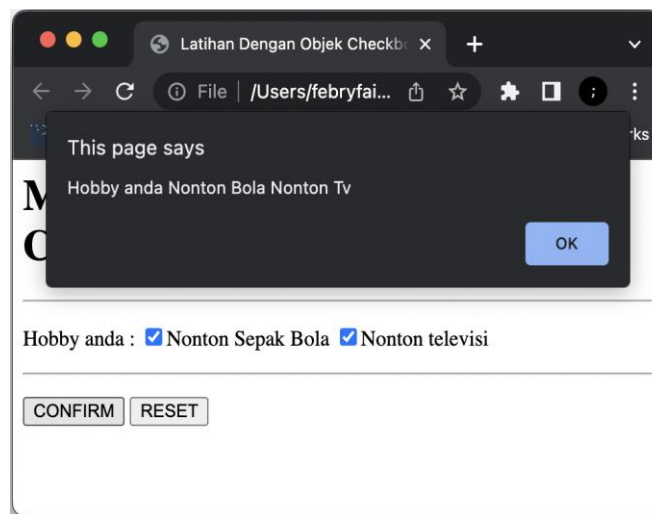
<html>
<head>
    <title>Latihan Dengan Objek Checkbox</title>
</head>
<body>
    <script language="JavaScript">
        function radio_box(form)
        { var ket = ""; var ket1 = "";
          if (form.bola.checked == true)
          {
              ket = "Nonton Bola";
          }
          if (form.tv.checked == true)
          {

```

```

        ket1 = " Nonton Tv";
    }
    alert('Hobby anda ' +ket+''+ket1);
}
</script>
<form>
    <H1> Memasukkan Data Dengan Objek Checkbox</H1>
    <hr>
    <p>Hobby anda :
        <input type="checkbox" value="ON" name="bola">Nonton Sepak
Bola
        <input type="checkbox" value="ON" name="tv">Nonton televisi
    </p>
    <hr>
    <p><input type="button" value="CONFIRM"
onclick="radio_box(this.form)"> <input type="reset" value="RESET"></p>
</form>
</body>
</html>

```



Gambar 3.6 Output Latihan Objek Checkbox

4. Objek TextArea

Objek textarea menyimpan informasi tentang elemen form yang berupa kotak teks dengan banyak baris.

```

<html>
<head>
    <title>Latihan Dengan Objek TextArea</title>
</head>

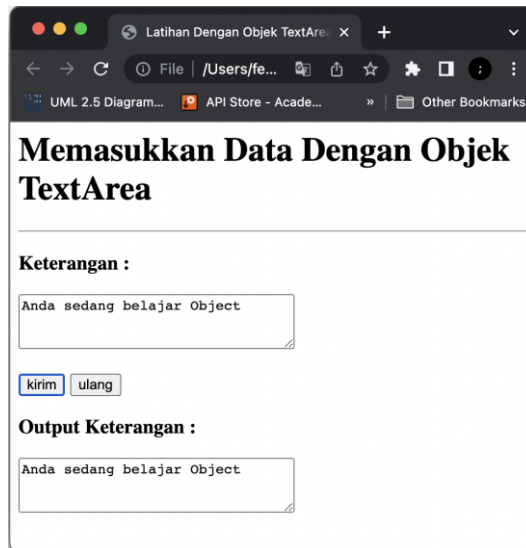
<body>
    <script language="JavaScript">
        <!--

```

```

        function tekan()
        {
            var ketstr =
(document.fform.Ket.value); document.fform.Oket.value = ketstr;
        }
        //-->
    </script>
    <form name="fform">
        <H1> Memasukkan Data Dengan Objek TextArea</H1>
        <hr>
        <h3>Keterangan :<h3><br>
            <textarea name="Ket" rows="3" cols="30"></textarea>
            <BR><BR>
            <input type="button" value="kirim" onclick="tekan()">
            <input type="reset" value="ulang">
            <H3>Output Keterangan :</H3>
            <textarea name="Oket" rows="3" cols="30"></textarea>
    </form>
</body>
</html>

```



Gambar 3.7 Output Latihan Objek TextArea

5. Objek Select

Objek Select menyimpan informasi tentang elemen form yang berupa kotak daftar. Objek select berguna apabila di dalam form terdapat banyak pilihan yang telah mempunyai nilai tertentu.

```

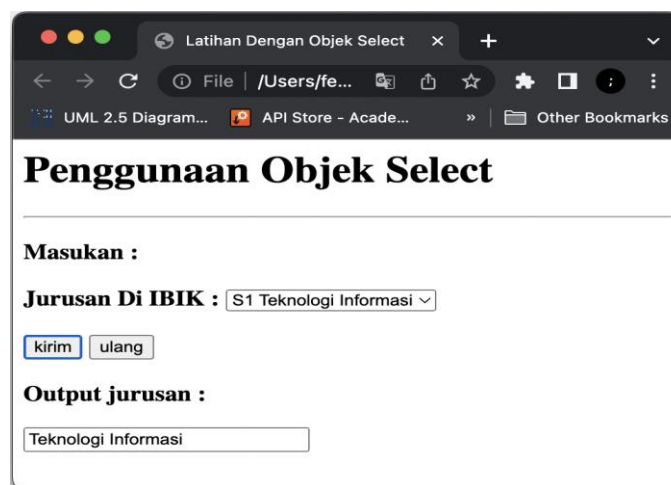
<html>
<head>
    <title>Latihan Dengan Objek Select</title>
</head>

```

```

<body>
  <script language="JavaScript">
    <!--
      function tekan()
      {
        var jurusanstr =
(document.fform.Jurusan.value); document.fform.Ojurusan.value =
jurusanstr;
      }
    <!-->
  </script>
  <form name="fform">
    <H1> Penggunaan Objek Select</H1>
    <hr>
    <h3>Masukan :</h3>
      Jurusan Di IBIK :
      <select name="Jurusan" Size="1">
        <option value="Teknologi Informasi"> S1 Teknologi
Informasi </option>
        <option value="Pariwisata"> S1 Pariwisata</option>
        <option value="Teknik Industri"> Teknik Industri
</option>
      </select>
      <p><input type="button" value="kirim"
onclick="tekan()">
        <input type="reset" value="ulang">
      <H3>Output jurusan :</H3>
      <input type="text" name="Ojurusan" size="30">
    </form>
  </body>
</html>

```



Gambar 3.8 Output Latihan Objek Select

5.2 Latihan Pembelajaran

1. Buatlah halaman seperti berikut, yang mencakup semua materi yang terdapat pada buku ini.

- Input berupa : NIM, NAMA, JENIS KELAMIN, AGAMA, STATUS, JURUSAN, KOMENTAR. (SESUAIKAN OBJEK YANG DIPAKAI)
- Proses terjadi di tombol KIRIM
- Output berupa isian yang sama dengan soal nomor 1.
- Tugas individu, dikumpulkan minggu ke 5, dalam bentuk print out KODE HTMLnya serta Tampilannya

2. Buatlah sebuah halaman form inputan, yang mencakup semua materi yang terdapat pada buku ini.

Perusahaan Travel IBIK Abadi memiliki armada dengan tujuan Jakarta, Solo dan Surabaya. Setiap tujuan memiliki kelas Eksekutif, Bisnis dan Ekonomi.

	Eksekutif	Bisnis	Ekonomi
Jakarta	70000	40000	10000
Solo	80000	50000	20000
Surabaya	90000	60000	30000

Diskon 10% diberikan apabila pemesan tiket merupakan anggota Travel Bintang Abadi.

Input : Nama Pemesan, Tujuan, Kelas, Banyak Tiket dan Status Member/Bukan

Output : Harga Tiket, Subtotal, Diskon dan Total Bayar

3. Buat program untuk menentukan faktorial dan jumlah deret sampai ke N.

PERHITUNGAN FAKTORIAL dan DERET			
N			
Hasil			
Faktorial		Submit	Reset
Jumlah			

4. Buat program untuk menghitung saldo akhir dari suatu tabungan dengan bunga dan jangka waktu tertentu :

Input:

Saldo : 100000

Bunga : 10

Waktu : 3

Output:

Ketika user menekan tombol Hitung maka akan muncul hasil seperti berikut :

Saldo Bulan 1 = Rp. 110000

Saldo Bulan 2 = Rp. 121000

Saldo Bulan 3 = Rp. 133100

5.3 Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-Praktikum-3-NPM"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB IV

PENGENALAN MERN STACK

4.1 Tujuan Pembelajaran

1. Memahami konsep arsitektur aplikasi web full-stack.
2. Menjelaskan peran masing-masing teknologi dalam MERN Stack.
3. Memahami fungsi:
 - Node.js sebagai runtime server-side
 - Express.js sebagai backend framework
 - React sebagai frontend library
 - MySQL sebagai sistem manajemen basis data
4. Membuat REST API sederhana menggunakan Express.
5. Menghubungkan backend dengan database MySQL.
6. Menghubungkan frontend React dengan backend melalui HTTP request.
7. Memahami alur data client-server

4.2 Dasar Teori

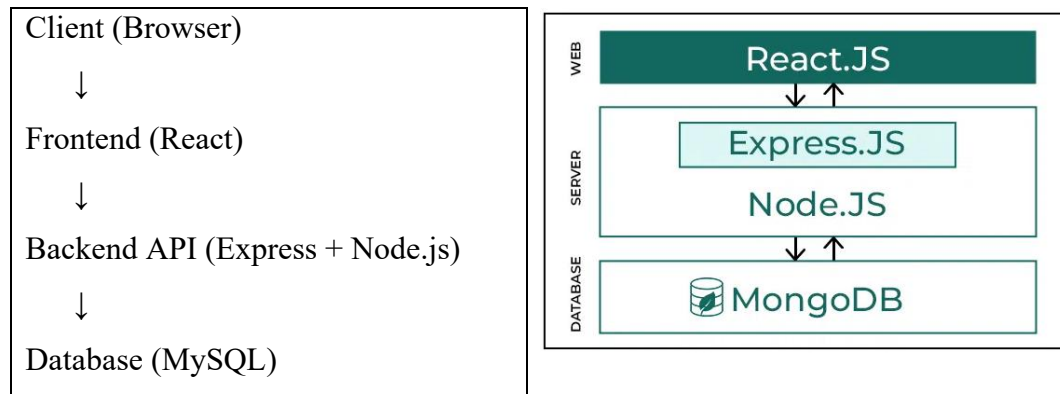
4.2.1 Konsep Full-Stack Development (MERN STACK)

MERN Stack adalah salah satu kombinasi teknologi (*tech stack*) paling populer di dunia pengembangan web untuk membangun aplikasi *Fullstack* yang modern, dinamis, dan skalabel. Istilah MERN merupakan akronim dari empat teknologi utama berbasis JavaScript yang digunakan di dalamnya, yaitu **M**ongodb, **E**xpress, **R**ead dan **N**ode. Namun dalam media pembelajaran kali ini penggunaan *MongoDB akan digantikan dengan MySQL* sebagai media penyimpanan data.

Full-stack development adalah proses pengembangan aplikasi yang mencakup:

- ✓ Frontend → Antarmuka pengguna
- ✓ Backend → Logika dan pengolahan data
- ✓ Database → Penyimpanan data

Arsitektur umum:



Gambar 4.1 Arsitektur Umum MERN Stack

Mengapa MERN Stack Sangat Populer?

- Satu Bahasa (JavaScript Everywhere): Pengembang tidak perlu belajar banyak bahasa. Cukup kuasai JavaScript, bisa menjadi Fullstack Developer.
- Kecepatan Pengembangan: Banyaknya library pendukung membuat proses pembuatan aplikasi menjadi sangat cepat.
- Performa Tinggi: Aplikasi MERN dikenal sangat responsif (seperti aplikasi mobile tapi di web).

4.2.2 MySQL

MySQL adalah sistem manajemen basis data relasional (RDBMS). Dengan Karakteristik:

- Menggunakan SQL
- Tabel, relasi, *primary key*, *foreign key*
- Cocok untuk sistem terstruktur

Contoh Query:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

4.2.3 Express JS

Express.js adalah framework minimalis untuk Node.js yang mempermudah pembuatan server dan API.

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Fungsi utama:

- Routing
- Middleware
- REST API
- Handling request & response

4.2.4 React JS

React adalah library JavaScript untuk membangun antarmuka pengguna berbasis komponen. Karakteristik:

- Component-based
- Virtual DOM
- Reusable
- Reactive state

Contoh komponen sederhana:

```
import React from 'react'

function Wellcome() {
  return (
    <div>
      <h1>Hello world!!</h1>
    </div>
  )
}

export default Wellcome;
```

React berfungsi sebagai:

- Frontend UI
- Pengelola state
- Pengirim request ke backend (axios)

4.2.5 Node JS

Node.js adalah runtime JavaScript yang memungkinkan JavaScript dijalankan di sisi server.

Fungsi:

- Menjalankan server
- Mengelola request & response
- Menghubungkan ke database
- Mengelola API

Keunggulan:

- Non-blocking (asynchronous)
- Cepat dan ringan
- Satu bahasa untuk frontend & backend

4.2.6 REST API dan HTTP Method

REST API adalah antarmuka komunikasi antara frontend dan backend.

Tabel 4.1 Method Rest API

Method	Fungsi
GET	Mengambil data
POST	Menambahkan data
PUT	Mengupdate data
DELETE	Menghapus data

Contoh endpoint:

GET	/users
POST	/users
PUT	/users/:id
DELETE	/users/:id

4.3 Latihan Pembelajaran

Pada latihan pembelajaran BAB ini, akan memulai proses pembangunan aplikasi web berbasis *Full-Stack JavaScript* menggunakan pendekatan MERN Stack (*MySQL, Express, React, Node*).

Latihan ini dirancang secara bertahap, dimulai dari tahap instalasi perangkat lunak yang dibutuhkan, konfigurasi lingkungan pengembangan (*development environment*), hingga implementasi aplikasi sederhana yang terintegrasi antara *frontend*, *backend*, dan database.

- Node.js digunakan untuk menjalankan server.
- Express.js digunakan untuk membangun REST API.
- MySQL digunakan untuk menyimpan dan mengelola data.
- React digunakan untuk membangun antarmuka pengguna (UI).

Langkah-1: Instalasi dan Persiapan Lingkungan

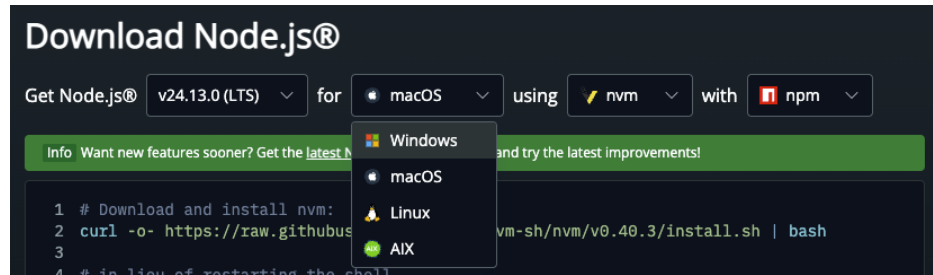
1. Buka halaman resmi Node.js (LTS version).

(<https://nodejs.org/en/download>)



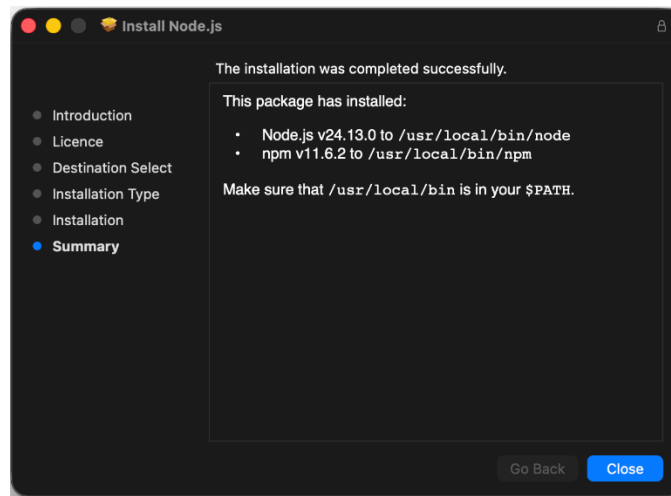
Gambar 4.2 Halaman Website Download Node.js

2. Download dan install sesuai sistem operasi.



Gambar 4.3 Download Node.js

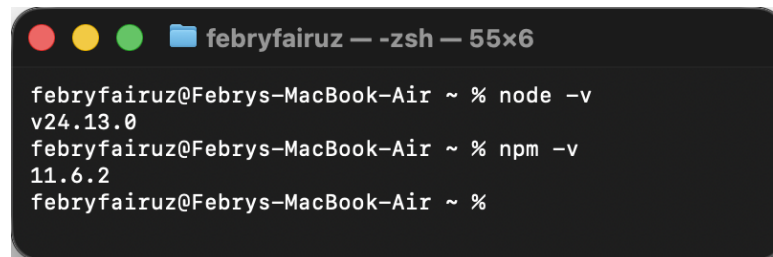
Setelah mendownload sesuai dengan Sistem Operasi maka silakan install package tersebut.



Gambar 4.4 Instalasi Package Node JS

3. Setelah selesai, buka Terminal / Command Prompt:

Buka terminal atau command prompt pada komputer dan masukan syntax dibawah ini: `node -v` dan `npm -v` secara terpisah untuk mengetahui bahwa telah berhasil dan menggunakan versi yang sesuai.



Gambar 4.5 Cek Versi Node.js dan npm

Note:

Updated version of node @ Feb 2026

Langkah-2: Membangun Backend Server

2. Buatlah folder bernama **MERN-STACK** lalu didalamnya buatlah folder *'backend'* seperti sturktur dibawah ini:



3. Buka terminal dan masukan syntax ini secara bergantian pada directory backend:

```
npm init -y  
npm install express mysql2 cors
```

Syntax pertama akan membuat *package.json* sebagai bagian konfigurasi server node bernama backend:

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>npm init -y  
Wrote to D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend\package.json:  
  
{  
  "name": "backend",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Gambar 4.6 Output Perintah npm init -y

Sedangkan syntax yang kedua akan menginstall tiga buah library package yaitu *express*, *mysql2* dan *cors*:

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>npm install express mysql2 cors
added 76 packages, and audited 77 packages in 5s

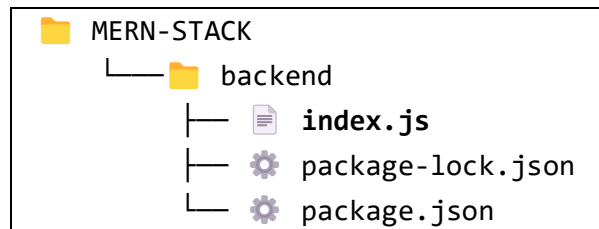
25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>
```

Gambar 4.7 Hasil Instalasi Dependency pada Project Node.js

4. Buatlah file index.js untuk membuat script server express js



File index.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Cara menjalankannya buka terminal masuk kedalam directory path backend dan masukan syntax **node index** :

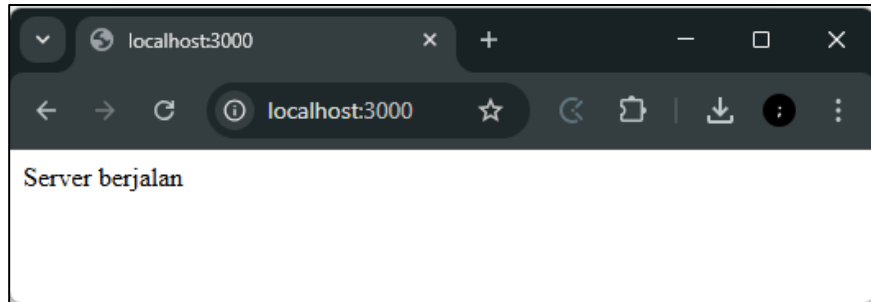
```

D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>node index
Server running on port 3000

```

Gambar 4.8 Tampilan Terminal Running Server Backend Node.js

Jika menampilkan informasi ‘Server running on port 3000’ maka backend server anda telah berhasil dibuat. Jika anda masukan **localhost:3000** pada browser maka akan tampil seperti dibawah ini:



Gambar 4.9 Halaman Akses Server Backend localhost:3000

Untuk mematikan serve tekan keyboard Ctrl+C, maka server akan mati. Perlu diketahui, pada environment express setiap kali melakukan perubahan code, diharuskan untuk merestart server. Backend ini tidak dapat mengupdate perubahan secara real-time.

Jika ingin server berjalan secara real-time, dapat menginstall **library nodemon**. Namun ini hanya bekerja pada OS Windows dan tidak bisa bekerja pada ranah production. Disarankan digunakan pada saat development. Cara menjalankannya cukup masukan syntax ``nodemon index``.

Langkah-3: Integrasi Database

1. Buatlah database MySQL bernama ``pw_praktikum_db``
2. Ubahlah file index.js dengan menambahkan pengecekan koneksi ke MySQL

```

const express = require("express");
const mysql = require("mysql2");

const app = express();

app.use(express.json());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "pw_praktikum_db"
});

db.connect((err) => {
  if (err) {
    console.error("Koneksi ke database gagal:", err.message);
    process.exit(1);
  }
  console.log("Koneksi ke database MySQL berhasil!");
});

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});

```

Penjelasan:

- Import *mysql2* untuk koneksi ke MySQL
 - Membuat konfigurasi koneksi dengan *createConnection()*
 - Menggunakan method *connect()* untuk mengecek koneksi
 - Jika berhasil, akan muncul pesan "*Koneksi ke database MySQL berhasil!*"
 - Jika gagal, akan muncul pesan error dan aplikasi akan berhenti
3. Setelah mengupdate, restart kembali servernya lalu buatlah beberapa kondisi seperti, koneksi ke MySQL nyala, matikan atau nama database tidak ditemukan.

```

D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>node index
Server running on port 3000
Koneksi ke database gagal:

```

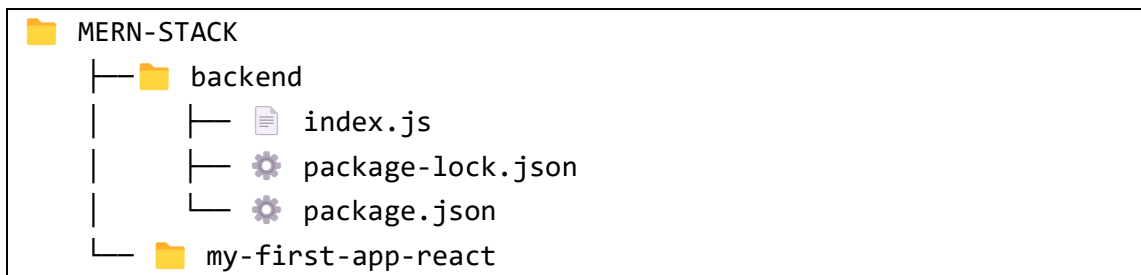
Gambar 4.10 Koneksi server gagal ke database

```
D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend>node index
Server running on port 3000
Koneksi ke database MySQL berhasil!
```

Gambar 4.11 Gambar Koneksi serve berhasil ke database

Langkah-4: Membangun Frontend dengan React

Selanjutnya membuat project web client menggunakan framework React, dimana project yang akan dibuat sejajar atau masih berada dalam folder MERN-STACK seperti ilustrasi dibawah ini.



1. Buka terminal dan arahkan ke direktori path MERN-STACK lalu masukan syntax untuk membuat project react :

```
npx create-next-app@latest
```

Pada terminal anda akan ditanya beberapa instruksi seperti membuat nama project hingga template project. Isilah pertanyaan tersebut sesuai dengan instruksi dibawah ini:

```
D:\25-26 Genap\Pem Web\workspaces\MERN-STACK>npx create-next-app@latest
Need to install the following packages:
create-next-app@16.1.6
Ok to proceed? (y) y
✓ What is your project named? ... my-first-app-react
✓ Would you like to use the recommended Next.js defaults? » No, customize settings
✓ Would you like to use TypeScript? ... No / Yes
✓ Which linter would you like to use? » ESLint
✓ Would you like to use React Compiler? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
✓ What import alias would you like configured? ... @/*
Creating a new Next.js app in D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\my-first-app-react.
```

Masukan nama project: **my-first-app-react**

Pilih No

Jika sudah mengisi selanjutnya akan menginstalasi kebutuhan dari project tersebut dan akan menampilkan informasi **“Success! Created”** pada terminal anda, menandakan project react telah berhasil dibuat.

```
Generating route types...
✓ Types generated successfully

Initialized a git repository.

Success! Created my-first-app-react at D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\my-first-app-react
```

2. Bukalah project dengan IDE dan masuk ke dalam file `page.tsx` yang berada pada `src/app/` lalu modifikasi file tersebut dengan menggantinya menjadi kode sederhana React untuk menampilkan “Hello World”.



File `page.tsx`:

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "My First Project"
};

export default function Home() {
  return (
    <div>
      <h1>Hello World!!</h1>
    </div>
  );
}
```

3. Selanjutnya menjalankan project react tersebut dengan memasukan syntax `npm run dev` pada terminal project `'my-first-app-react'`.

```

D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\my-first-app-react>npm run dev

> my-first-app-react@0.1.0 dev
> next dev

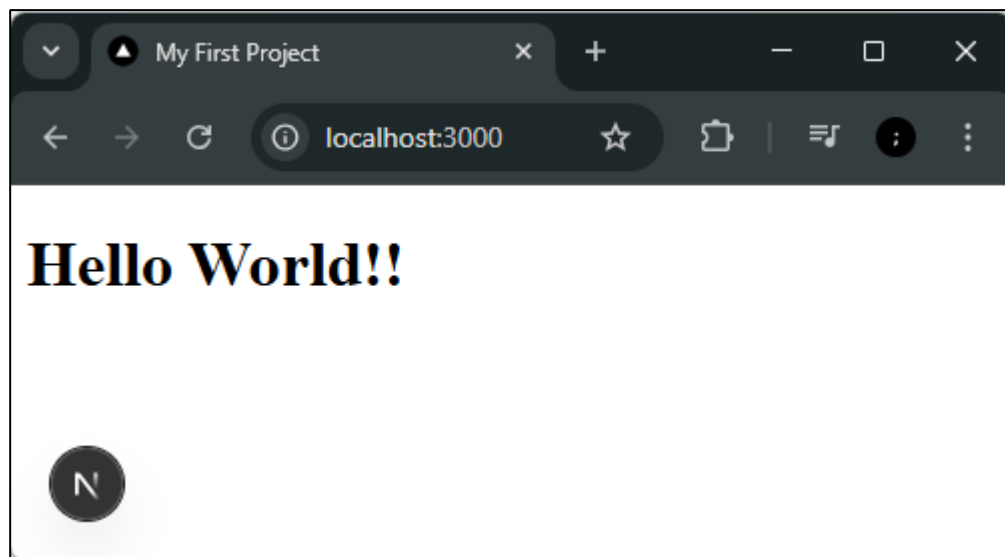
▲ Next.js 16.1.6 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.1.11:3000

✓ Starting...
✓ Ready in 1789ms
GET / 200 in 1572ms (compile: 839ms, render: 733ms)

```

Gambar 4.12 Running Project React

Buka browser dan masukan local network sesuai yang disampaikan oleh terminal, yaitu <http://localhost:3000> atau menggunakan ip public <http://192.168.1.11:3000>



Gambar 4.13 Halaman Utama Project Frontend menggunakan React Framework

Untuk penjelasan lebih lanjut mengenai ruang lingkup kebutuhan Framework React akan dijelaskan pada bab selanjutnya!

Langkah-5: Integrasi Frontend dan Backend

Pada Langkah 3, telah berhasil membangun backend sederhana menggunakan Express.js yang berjalan di atas lingkungan runtime Node.js. Backend tersebut berfungsi sebagai

penyedia layanan (service) atau REST API yang menangani request dan response, serta mengelola komunikasi dengan database.

Selanjutnya, pada Langkah 4, telah membangun frontend menggunakan React sebagai antarmuka pengguna (user interface). Frontend ini bertugas menampilkan data kepada pengguna serta mengirimkan request ke backend melalui HTTP.

Sebelum melakukan proses integrasi antara frontend dan backend, terdapat hal penting yang perlu dipahami, yaitu terkait penggunaan port pada local environment.

Dalam arsitektur *client-server*, setiap aplikasi yang berjalan pada komputer lokal (*localhost*) menggunakan nomor port tertentu sebagai jalur komunikasi. Port berfungsi sebagai “alamat pintu” yang membedakan satu layanan dengan layanan lainnya dalam satu sistem yang sama.

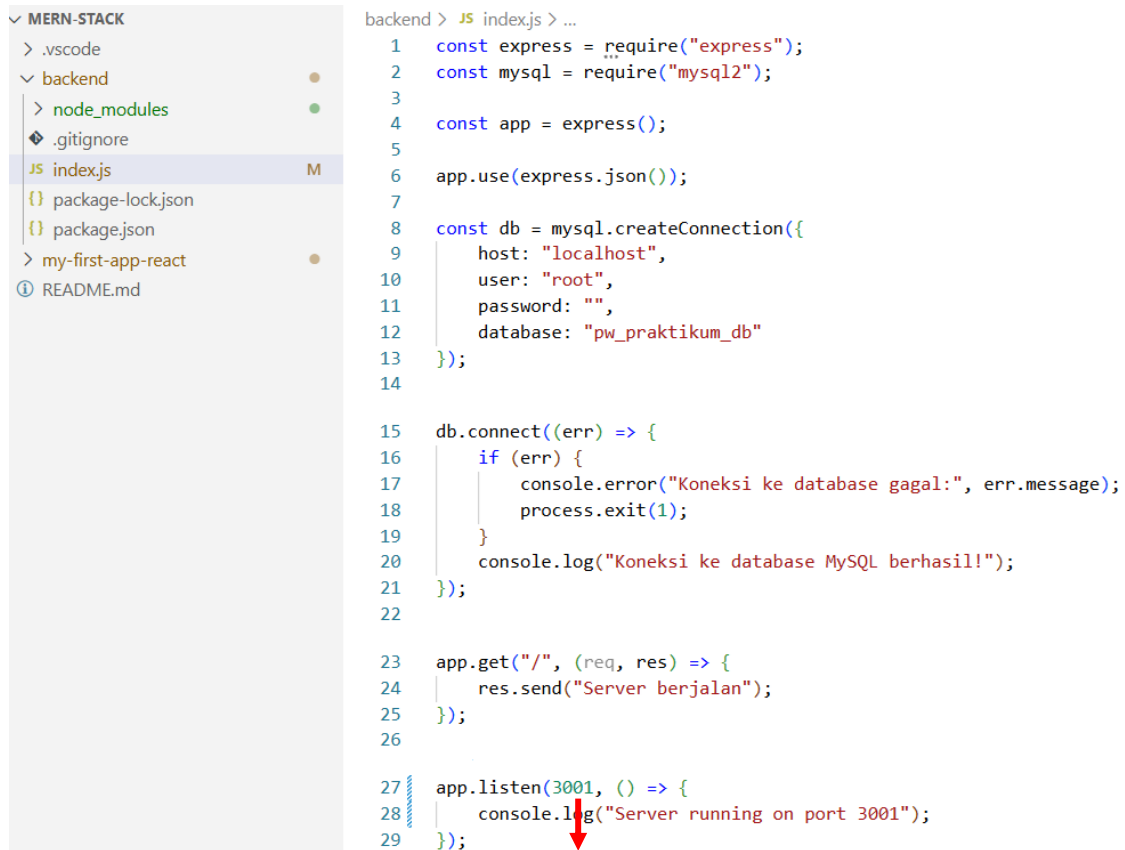
Karena backend dan frontend dijalankan secara bersamaan pada satu mesin (*local development*), maka keduanya **tidak boleh** menggunakan nomor port yang sama. Jika dua aplikasi mencoba berjalan pada port yang identik (misalnya sama-sama menggunakan port 3000), maka akan terjadi konflik dan salah satu aplikasi tidak dapat dijalankan.

Sebagai contoh:

- Backend (Express) dapat dijalankan pada <http://localhost:3001>
- Frontend (React) dapat dijalankan pada <http://localhost:3000>

Pemahaman mengenai konfigurasi port ini sangat penting sebelum proses integrasi dilakukan, karena menjadi dasar komunikasi antara frontend dan backend dalam pengembangan aplikasi *full-stack*.

1. Bukalah project MERN-STACK pada IDE seperti VS Code, lalu ubah port backend menjadi 3001



```
backend > JS index.js > ...
1  const express = require("express");
2  const mysql = require("mysql2");
3
4  const app = express();
5
6  app.use(express.json());
7
8  const db = mysql.createConnection({
9    host: "localhost",
10   user: "root",
11   password: "",
12   database: "pw_praktikum_db"
13 });
14
15 db.connect((err) => {
16   if (err) {
17     console.error("Koneksi ke database gagal:", err.message);
18     process.exit(1);
19   }
20   console.log("Koneksi ke database MySQL berhasil!");
21 });
22
23 app.get("/", (req, res) => {
24   res.send("Server berjalan");
25 });
26
27 app.listen(3001, () => {
28   console.log("Server running on port 3001");
29 });
```

Gambar 4.14 Kode Ubah Port Backend

Ubahlah port backend pada `app.listen` dengan parameter 3001, agar tidak sama dengan default port frontend yaitu 3000.

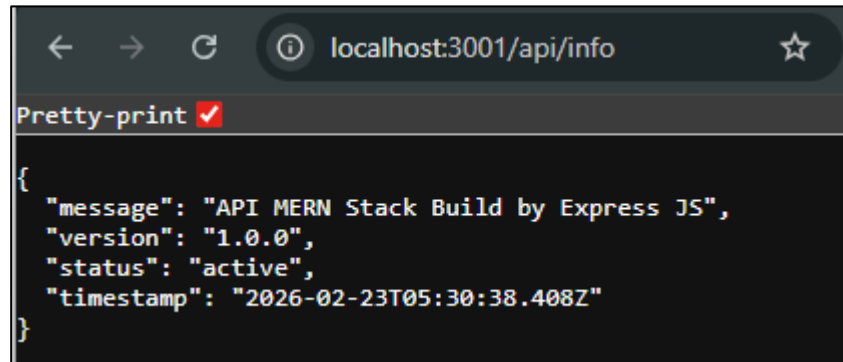
1. Membuat route sederhana untuk menampilkan Rest API

Tahap ini akan membuat sebuah url Rest API sederhana dengan address <http://localhost:3001/api/info> dimana API tersebut menampilkan informasi mengenai version dari backend.

Pada file *index.js* tambahkan code route dibawah ini:

```
app.get("/api/info", (req, res) => {
  res.json({
    message: "API MERN Stack Build by Express JS",
    version: "1.0.0",
    status: "active",
    timestamp: new Date()
  });
});
```

Selanjutnya jalankan kembali server backend expressnya untuk mengecek apakah url tersebut sudah berhasil dibuat. Contoh menggunakan browser:



Gambar 4.15 Halaman Akses localhost:3001/api/info

Penjelasan lebih lanjut mengenai ruang lingkup kebutuhan express akan dibahas pada bab selanjutnya!

2. Memberikan akses backend agar frontend dapat berkomunikasi

Masih pada file index.js milik backend, tambahkan code CORS untuk memberikan akses local network frontend, dimana akses local network frontend adalah <http://localhost:3000>

```
const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");
const app = express();

app.use(cors({
  origin: "http://localhost:3000",
  credentials: true
}));

app.use(express.json());

//rest of code
```

3. Mengintegrasikan frontend React dengan server Express

Pada **project react** yang telah dibuat tambahkan sebuah library package bernama axios dengan syntax:

```
npm install axios
```

Axios bertindak sebagai klien HTTP yang memudahkan komunikasi antara aplikasi web dan API, didukung fitur canggih seperti konversi JSON otomatis, intersektor, dan penanganan kesalahan yang lebih baik. Pengenalan lebih lanjut perihal AXIOS akan dibahas pada bab selanjutnya.

4. Memanggil Rest API untuk mengecek versioning backend

Untuk memanggil url API <http://localhost:3001/api/info> pada file `page.tsx` tambahkanlah code dibawah ini:

```
"use client";

import axios from "axios";
import { useEffect, useState } from "react";

export default function Home() {
  const [info, setInfo] = useState<any>(null);

  const GetAPIInfo = async () => {
    try {
      const response = await axios.get(
        "http://localhost:3001/api/info"
      );
      setInfo(response.data);
    } catch (error) {
      console.error('Error fetching API data:', error);
      setInfo(error?.message);
    }
  }

  useEffect(() => {
    GetAPIInfo();
  }, []);

  return (
    <div>
      <h1>Hello World!!</h1>
      {info && (
        <pre style={{
          backgroundColor: "#f4f4f4",
          padding: "15px",
          borderRadius: "8px",
          border: "1px solid #ddd",
          overflow: "auto"
        }}>
          {JSON.stringify(info, null, 2)}
        </pre>
      )}
    </div>
  );
}
```

Penjelasan:

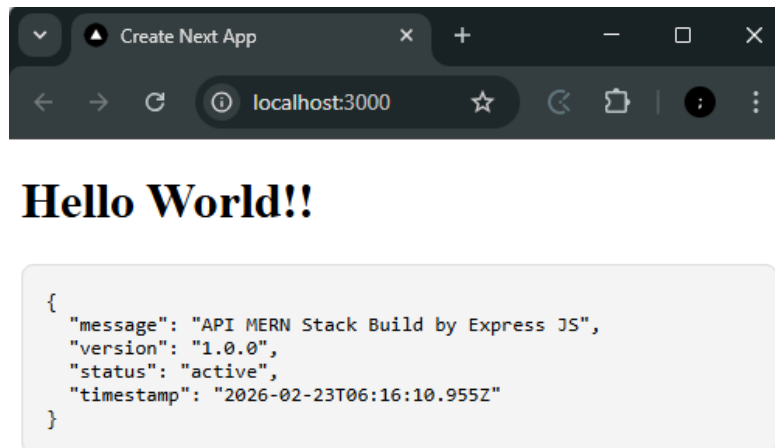
- a. `use client` : Menandai komponen ini sebagai *Client Component*.
- b. `axios` : Library untuk melakukan HTTP request ke backend

- c. `const [info, setInfo] = useState<any>(null)`: Membuat state info dengan nilai awal null
- d. `GetAPIInfo()` : Fungsi yang berjalan secara asynchronous
- e. `axios.get` : Melakukan HTTP GET request ke backend Express.js
- f. `useEffect` Hook : Menjalankan `GetAPIInfo()` sekali saat komponen pertama kali dimuat

5. Hasil testing Rest API pada frontend

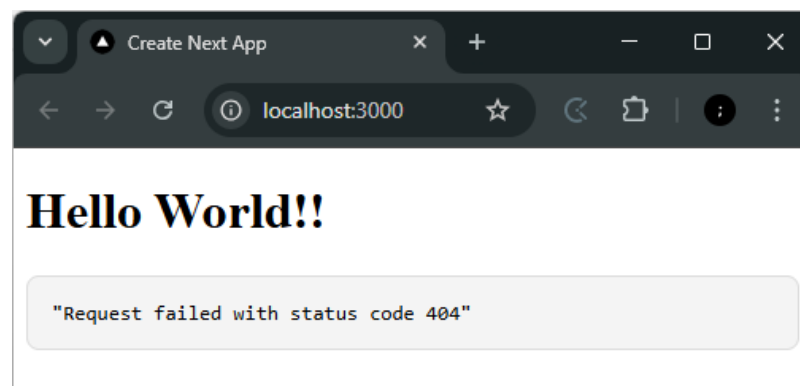
Berikut ini adalah beberapa kondisi hasil dari integrasi Frontend dengan Backend:

- Jika koneksi berhasil ke address <http://localhost:3001/api/info>



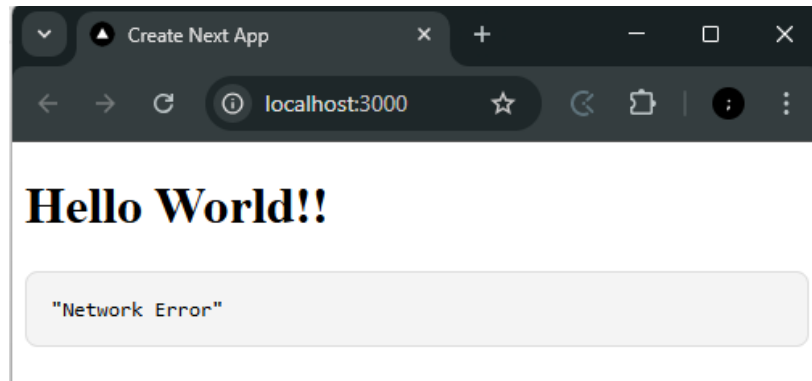
Gambar 4.16 Hasil Testing Rest API Berhasil

- Jika koneksi gagal ke backend dimana url path salah



Gambar 4.17 Hasil Testing Rest API Gagal

- Jika koneksi server mati



Gambar 4.18 Hasil Testing Rest API Server Mati

4.4 Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-MERN-STACK-NPM"** dengan nama commit **"Praktikum 4"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB V

FRAMEWORK UI REACT JS: PART 1

5.1 Tujuan Pembelajaran

1. Memahami konsep dasar framework/library frontend modern.
2. Menjelaskan peran React dalam pengembangan aplikasi web modern.
3. Memahami konsep:
 - Component-based architecture
 - Virtual DOM
 - State dan Props
4. Membuat project React menggunakan tools modern.
5. Membuat dan mengelola komponen fungsional (functional component).
6. Menggunakan hook dasar seperti useState dan useEffect.
7. Mengelola event dan interaksi pengguna.
8. Membuat aplikasi React sederhana berbasis komponen.

5.2 Dasar Teori

5.2.1 Pengertian React

React adalah library JavaScript yang digunakan untuk membangun antarmuka pengguna (*User Interface*) berbasis komponen. Meskipun sering disebut sebagai framework, React lebih tepat disebut library karena hanya mengurus lapisan tampilan (*view*) aplikasi. React menggunakan JSX, Virtual DOM, dan pendekatan modular, membuatnya efisien dan populer untuk web modern.

React dikembangkan oleh Meta (Facebook) dan banyak digunakan dalam pengembangan aplikasi modern karena:

- Reusable component
- Performa tinggi dengan Virtual DOM
- Struktur kode lebih terorganisir
- Mudah dikembangkan menjadi aplikasi skala besar

React memfasilitasi pembuatan aplikasi web satu halaman (*Single Page Applications* / SPA) yang kompleks dengan performa tinggi. Banyak perusahaan besar seperti Facebook, Instagram, WhatsApp, Netflix, dan Airbnb menggunakan teknologi ini.

Secara ringkas, React adalah alat yang efisien untuk membangun antarmuka pengguna, sering dipasangkan dengan pustaka lain untuk menjadi solusi lengkap seperti framework.

5.2.2 Dasar Javascript untuk React

A. Variable

JavaScript menyediakan tiga cara utama untuk mendeklarasikan variabel, yaitu menggunakan *let* dan *const*.

- `let` : untuk data yang dapat berubah.
- `const` : untuk data yang tidak berubah.
- `var` : tipe data ini sudah tidak direkomendasikan untuk digunakan pada ES6.

```
var nama = "Febry";  
let nim = 3120200907;  
const kelas = "TI-24-KA";
```

Gambar 5.1 Contoh Variable Javascript dalam Kode

B. Tipe Data

Beberapa tipe data yang sering digunakan dalam React:

Tabel 5.1 Tipe Data React

Tipe Data	Contoh
String	"Mobile Programming"
Number	10, 3.14
Boolean	true, false
Object	{ nama: "Budi" }
Array	["Android", "iOS"]

Contoh penggunaannya seperti dibawah ini:

```
1  const class_name = 'Praktikum Pemograman Perangkat Bergerak';  
2  const total_student = 28;  
3  const lecturer = {name: 'Febry Damatraseta Fairuz', nim: '3120200907'};  
4  const status_class = true;  
5  const intake = [2023, 2024];  
6  const students = [  
7    {name: 'Anton', nim: '3120200901'},  
8    {name: 'Isnan', nim: '3120200902'},  
9    {name: 'Edi', nim: '3120200903'},  
10 ]
```

Gambar 5.2 Contoh Penggunaan Tipe Data dalam Kode

Penjelasan:

- variable `class_name` : menyimpan nilai String.
- variable `total_student` : menyimpan nilai number.
- variable `lecturer` : menyimpan nilai object.
- variable `status_class` : menyimpan nilai boolean.
- variable `intake` : menyimpan nilai array.
- variable `students` : menyimpan nilai array object.

C. Operator

Operator digunakan untuk melakukan operasi pada data, seperti dibawah ini:

```
1 let a = 10;  
2 let b = 5;  
3 let hasil_penjumlahan = a + b;  
4 let hasil_pengurangan = a - b;  
5 let hasil_perkalian = a * b;  
6 let hasil_pembagian = a / b;  
7 let hasil_modulus = a % b;
```

Gambar 5.3 Contoh Operasi Data dalam Kode

D. Fungsi

Fungsi digunakan untuk mengelompokkan logika program.

```
function greetings(){  
  return `Welcome to ${class_name} class with ${total_student} students.`;  
}  
  
function penjumlahan(a, b) {  
  return a + b;  
}
```

Atau menggunakan **arrow function** (lebih sering dipakai di React):

```
const greetings = () =>{  
  return `Welcome to ${class_name} class with ${total_student} students.`;  
}  
  
const penjumlahan = (a, b) => {  
  return a + b;  
};
```

E. Array dan Object

Dalam JavaScript, **array** dan **object** digunakan untuk menyimpan data yang lebih kompleks dibandingkan variabel tunggal. Keduanya memiliki karakteristik dan tujuan penggunaan yang berbeda.

1. Array

a) Data Disimpan dalam Urutan (Index)

- Setiap elemen memiliki index
- Index dimulai dari 0

```
const intake = [2023, 2024];
```

Variable intake memiliki 2 buah value number. Dimana index **ke-0** adalah **2023** dan **index ke-1** adalah **2024**.

b) Tidak Menggunakan Key Bernama

Akses data menggunakan index angka. Jika ingin menampilkan value dari variable array, cukup menginisialisasi index yang diinginkan misal, intake[0] akan menampilkan output 2023.

c) Baik digunakan untuk data yang sejenis

```
const intake = [2023, 2024];  
const courses = ['Matematika', 'Kalkulus'];
```

Dalam menggunakan data yang sejenis, tidak boleh memiliki nilai yang berbeda-beda, seperti dalam 1 array terdapat number, string ataupun object.

d) Jumlah data dapat berubah

Bentuk array memiliki sebuah function yang diperuntukan untuk menambah data, yaitu function push().

```
courses.push('Fisika');
```

2. Object

Object adalah struktur data yang digunakan untuk menyimpan data dengan pasangan **key** dan **value**.

a) Harus memiliki Key dan Value

- key: nama properti
- value: isi data

```
const lecturer = {  
  name: 'Febry Damatraseta Fairuz'  
};
```

Untuk memiliki nilai dalam bentuk object selain harus memiliki key dan value, harus menggunakan simbol **kurung kurawal** buka dan tutup, dan simbol titik dua (:) sebagai penanda antara key dan value, seperti { **key** : **value** }.

Pada contoh variable lecturer, disana memiliki 1 bentuk object yang terdiri dari:

- key → name
- value → 'Febry Damatraseta Fairuz'

Jika ingin menambahkan data lebih dari 1 key maka gunakan simbol koma (,) sebagai pemisah, seperti dibawah ini:

```
const lecturer = {  
  name: 'Febry Damatraseta Fairuz',  
  nim: '3120200907'  
};
```

b) Key bersifat unik

Tidak boleh ada key yang sama dalam satu object. Seperti dibawah ini:

```
{  
  name: 'Febry Damatraseta Fairuz',  
  name: 'Anton Sukanto'  
}
```

- c) Value bisa berbagai tipe data

```
{  
  name: 'Febry Damatraseta Fairuz',  
  nim: '3120200907',  
  is_active: true,  
  degrees: ['S1 Teknik Informatika', 'S2 Teknologi Rekayasa Terapan']  
}
```

Value pada object bisa memiliki bentuk tipe data apapun dalam 1 key

- d) Akses data menggunakan key

Dalam mengakses sebuah informasi dari object, cara yang dilakukan ialah dengan memiliki format bentuk yaitu [variable.key] dimana diawali dengan nama variabelnya lalu tambahkan simbol titik (.) dan diakhiri dengan nama key yang akan dipanggil.

```
lecturer.name;  
lecturer.nim;
```

Kapan menggunakan Array atau Object?

Gunakan array jika data yang dimiliki berupa daftar, memiliki urutan, data lebih dari 1. Sedangkan menggunakan Object jika dalam kondisi datanya memiliki atribut, dan setiap data memiliki identitas jelas.

F. Selection (Percabangan)

Percabangan adalah struktur kontrol dalam JavaScript yang digunakan untuk menjalankan kode berdasarkan kondisi tertentu. Dengan percabangan, program dapat mengambil keputusan yang berbeda tergantung pada nilai atau keadaan yang diberikan.

- Jenis-Jenis Percabangan dalam JavaScript:

1. IF Statement

IF digunakan untuk menjalankan kode jika suatu kondisi bernilai benar (true). Sedangkan jika kondisi salah kode if tidak dijalankan.

```
const score = 80;
if (score >= 70) {
  console.log("Congratulations! You passed the exam.");
}
```

2. IF ELSE Statement

IF ELSE digunakan ketika terdapat dua kemungkinan kondisi, yaitu benar dan salah.

```
const score = 80;
if (score >= 70) {
  console.log("Congratulations! You passed the exam.");
} else {
  console.log("Sorry, you failed the exam.");
}
```

3. Nested IF

Nested IF adalah percabangan di dalam percabangan, digunakan ketika terdapat lebih dari dua kondisi.

```
const score = 80;
if(score >= 70){
  if(score >= 90){
    alert("Congratulations! You passed the exam with a grade A");
  }else{
    alert("Congratulations! You passed the exam with a grade B");
  }
}else{
  alert("Sorry, you failed the exam");
}
```

4. Ternary Operator

Ternary operator digunakan untuk percabangan singkat dalam satu baris kode.

Syntax Short IF: kondisi ? nilaiJikaTrue : nilaiJikaFalse;

```
const score = 80;
const results = score >= 70 ? 'Passed' : 'Failed';
console.log(`Your score: ${score}, Result: ${results}`);
```

Jika ternary operator digunakan dalam JSX maka dapat diimplementasikan seperti dibawah ini:

```
export default function LatihanTernary() {  
  const score = 80;  
  return (  
    <div>  
      <h1>Welocome</h1>  
      <h2>Praktikum Pemograman Website</h2>  
      <p>Grade: {score >= 70 ? "Passed":"Failed"}</p>  
    </div>  
  )  
}
```

5. Switch Case

Switch Case digunakan untuk memilih satu blok kode dari banyak kemungkinan kondisi, berdasarkan nilai tertentu. Percabangan ini cocok digunakan ketika terdapat banyak kondisi yang membandingkan satu variabel yang sama.

```
const hari = 3;  
switch (hari) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  case 3:  
    console.log("Wednesday");  
    break;  
  default:  
    console.log("Day is not a valid day.");  
}
```

G. Repetition (Perulangan)

Perulangan adalah struktur kontrol dalam pemrograman yang digunakan untuk menjalankan satu atau beberapa baris kode secara berulang selama suatu kondisi tertentu masih terpenuhi. Dalam pengembangan aplikasi React, looping sering digunakan untuk:

- Menampilkan daftar data (list)
- Memproses array atau object
- Menghasilkan komponen secara dinamis

- Jenis-Jenis Looping dalam JavaScript:

1. For Loop

Digunakan ketika jumlah perulangan **sudah diketahui**.

```
for (let i = 1; i <= 5; i++) {
  console.log(`Looping iteration ${i}`);
}
```

2. While Loop

Digunakan ketika perulangan dilakukan selama kondisi bernilai true.

```
let i = 1;
while (i <= 3) {
  console.log("Value i:", i);
  i++;
}
```

3. Do While Loop

Mirip dengan while, tetapi kode dijalankan minimal satu kali, meskipun kondisi bernilai false.

```
let i = 5;
do {
  console.log("Nilai i:", i);
  i++;
} while (i < 5);
```

4. For ...of Loop

Digunakan untuk mengakses nilai dari array atau string.

```
const professors = ["Anton", "Isnan", "Septian"];
for (const item of professors) {
  console.log(item);
}
```

5. For ...in Loop

Digunakan untuk mengakses key (property) dari object.

```
const lecturer = {
  name: "Jemy",
  age: 70,
  degree: "S2"
};
for (const key in lecturer) {
  console.log(key + ": " + lecturer[key]);
}
```

6. Looping Array

map() digunakan untuk:

- Mengolah array

```
const data = ["TI", "SI", "MP"];
data.map((item, index) => {
  console.log(index, item);
});
```

- Menghasilkan komponen JSX

```
const LatihanMap = () => {
  const data = ["TI", "SI", "MP"]
  return (
    <div>
      <h3>Concentration</h3>
      {data.map((item, index)=>{
        <span key={index}>{item}</span>
      })}
    </div>
  )
}
```

H. React Function Component (RFC) dan Class Component (RCC)

Dalam pengembangan library React, antarmuka aplikasi dibangun dari unit-unit kode yang disebut komponen (components). Ada dua pendekatan utama untuk membuat komponen: *Functional Components* dan *Class Components*. Pendekatan ini sering disingkat dalam literatur sebagai RFC dan RCC.

1. Function Components (RFC)

Function Components (RFC) adalah komponen yang ditulis sebagai fungsi JavaScript biasa yang menerima props (input) dan mengembalikan JavaScript XML (JSX) sebagai representasi antarmuka yang akan ditampilkan. Komponen ini tidak memerlukan kelas atau metode render, sehingga sintaksnya lebih singkat dan sederhana. Seiring perkembangan React, terutama sejak versi 16.8, RFC semakin direkomendasikan karena dapat menggunakan Hooks untuk mengelola state dan lifecycle, yang sebelumnya hanya dimungkinkan pada komponen kelas [4].

Contoh RFC:

Tabel 5.2 Contoh Kode RFC Umum dan ES6

RFC dengan function umum	RFC dengan format ES6
<pre>export default function Greeting() { return <Text>Welcome.. }</pre>	<pre>const Greeting = () =>{ return <Text>Welcome.. } export {Greeting}</pre>

2. Class Components (RCC)

Class Components (RCC) adalah komponen yang ditulis menggunakan kelas ES6 yang mewarisi dari *React.Component*. Komponen kelas ini memiliki metode *render()* yang mengembalikan JSX. Secara tradisional, RCC diperlukan ketika komponen harus mengelola state internal atau memanfaatkan *lifecycle methods*. Walaupun kini fungsionalitas ini telah tersedia melalui Hooks, RCC masih didukung oleh React dan dapat ditemukan dalam kode *legacy* atau pada kasus tertentu [5].

Contoh RCC:

```
import React, { Component } from 'react'  
  
export default class page extends Component {  
  render() {  
    return (  
      <div>  
        <h1>Welcome</h1>  
      </div>  
    )  
  }  
}
```

3. Perbedaan RFC dengan RCC

Berikut ringkasan perbedaan utama antara kedua pendekatan komponen:

A. Sintaks dan Struktur

- RFC ditulis sebagai fungsi JavaScript yang langsung mengembalikan JSX, tanpa metode *render()* atau pewarisan kelas [4].
- RCC adalah kelas yang memperluas `React.Component` dan memerlukan metode *render()* untuk mengembalikan JSX [5].

B. Pengelolaan State

- RFC menggunakan Hooks seperti *useState* untuk mengelola state dalam komponen[6].
- RCC menggunakan properti `this.state` dan metode *this.setState()* untuk mengubah state[7].

C. Lifecycle Methods

- RFC dapat menggunakan Hooks seperti *useEffect* untuk mensimulasikan perilaku *lifecycle* seperti *mount*, *update*, dan *unmount* [6].
- RCC menggunakan metode built-in seperti *componentDidMount()*, *componentDidUpdate()*, dan *componentWillUnmount()* [4].

D. Kompleksitas dan Kode

- RFC biasanya lebih ringkas dan mudah dibaca, cocok untuk komponen presentasional atau sederhana[4].
- RCC dapat memiliki kode lebih panjang karena struktur kelas dan penanganan state tradisional[5].

E. Rekomendasi Industri

- Seiring perkembangan React, RFC dengan Hooks makin menjadi *best practice* karena fleksibilitas dan kemudahan penggunaannya, sementara RCC masih didukung untuk kompatibilitas kode lama[6].

4. Kapan menggunakan RFC dan RCC

- RFC sangat cocok untuk komponen modern dan umumnya direkomendasikan karena sintaks yang sederhana dan dukungan Hook.

- RCC masih relevan untuk kode lama atau ketika diperlukan metode lifecycle tradisional tanpa Hook.

I. JSX (JavaScript XML)

JSX adalah sintaks khusus yang digunakan dalam React untuk menuliskan struktur tampilan (UI) yang menyerupai HTML langsung di dalam kode JavaScript. JSX bukan HTML, tetapi ekstensi sintaks JavaScript yang akan diterjemahkan (ditranspile) menjadi fungsi JavaScript biasa oleh tools seperti Babel.

Contoh JSX:

```
const element = <h1>Hello World</h1>;
```

Kode di atas sebenarnya akan diubah menjadi:

```
React.createElement("h1", null, "Hello World");
```

Artinya, JSX hanyalah cara yang lebih mudah dan lebih readable untuk menulis UI.

Sebelum JSX, penulisan UI dengan JavaScript cukup kompleks karena harus menggunakan *createElement()* secara manual.

Dengan JSX:

- Kode lebih mudah dibaca
- Struktur UI lebih jelas
- Logika dan tampilan bisa berada dalam satu file
- Lebih mudah membuat komponen reusable

Aturan Dasar JSX

1. Harus Memiliki Satu Parent Element

JSX tidak boleh mengembalikan dua elemen sejajar tanpa pembungkus.

Salah	Benar	
<pre>return (<h1>Judul</h1> <p>Paragraf</p>);</pre>	<pre>return (<div> <h1>Judul</h1> <p>Paragraf</p> </div>);</pre>	<pre>return (<> <h1>Judul</h1> <p>Paragraf</p> </>);</pre>

2. Menggunakan className, bukan class

Karena JSX adalah JavaScript, maka atribut HTML tertentu menyesuaikan dengan aturan JavaScript.

Salah	Benar
<code><h1 class="p-1">Judul</h1></code>	<code><h1 className="p-1">Judul</h1></code>

3. Menggunakan {} untuk JavaScript Expression

Untuk menyisipkan variabel atau ekspresi JavaScript di dalam JSX, gunakan {}.

```
const nama = "Febry";

return (
  <>
    <h1>Name {nama}</h1>
    <p>Hasil: {5 + 5}</p>
  </>
);
```

4. JSX dan JavaScript Expression

Yang bisa dimasukkan dalam {}:

- Variabel
- Operasi matematika
- Ternary operator
- Pemanggilan fungsi

Contoh:

```
const data = ["TI", "SI", "MP"];
const score = 80;
return (
  <div>
    <h3>Concentration</h3>
    {data.map((item, index)=>(
      <span key={index}>{item}</span>
    ))}
    <p>Grade: {score >= 70 ? "Passed" : "Failed"}</p>
  </div>
)
```

5. JSX dan Event Handling

Event dalam JSX menggunakan camelCase.

```
<button onClick={handleClick}>Klik</button>  
  
<button onClick={() => handleClick(id)}>Klik</button>
```

Berbeda dengan HTML biasa yang menggunakan onclick.

Selain *onClick*, pada React terdapat banyak jenis event handling yang dapat digunakan dalam JSX. Event di React ditulis dalam *camelCase* dan menggunakan kurung kurawal {} untuk memanggil fungsi.

A. Mouse Events

Tabel 5.3 Mouse Events dan Fungsinya

Event	Fungsi
onClick	Saat elemen diklik
onDoubleClick	Klik dua kali
onmouseenter	Mouse masuk ke elemen
onmouseleave	Mouse keluar dari elemen
onmouseover	Mouse di atas elemen
onmouseout	Mouse keluar dari area elemen
onmousedown	Tombol mouse ditekan
onmouseup	Tombol mouse dilepas
onContextMenu	Klik kanan

B. Form Events

Tabel 5.4 Form Event dan Fungsinya

Event	Fungsi
onChange	Saat nilai input berubah
onSubmit	Saat form disubmit
onFocus	Saat input mendapat fokus
onBlur	Saat input kehilangan fokus
onInput	Saat user mengetik
onInvalid	Validasi gagal

C. Keyboard Events

Tabel 5.5 Keyboard Events dan Fungsinya

Event	Fungsi
onKeyDown	Saat tombol ditekan
onKeyUp	Saat tombol dilepas
onKeyPress	(Deprecated, jarang digunakan)

D. Clipboard Events

Tabel 5.6 Clipboard Events dan Fungsinya

Event	Fungsi
onCopy	Saat teks disalin
onCut	Saat teks dipotong
onPaste	Saat teks ditempel

React menggunakan sistem bernama SyntheticEvent, yaitu wrapper dari event asli browser agar:

- Konsisten di semua browser
- Lebih efisien
- Mudah dikontrol

5. JSX dan Styling

Style dalam JSX ditulis dalam bentuk object JavaScript.

- Gunakan camelCase → fontSize
 - Nilai string harus dalam tanda kutip
- Inline Styling:

```
const style = {
  color: "red",
  fontSize: "20px"
};

return <h1 style={style}>Hello</h1>;
```

Atau

```
return <h1 style={{ color: "blue", fontSize: "18px" }}>Hello</h1>
```

- External Styling:

File external css disimpan dalam extension .css, contoh global.css:

```
h1 {
```

```
color: "blue", fontSize: 18px
}
```

- Memanggil external css pada RFC:

```
import React from 'react'
import './globals.css';

export default const LatihanCSS = () => {
  return <h1>Hello</h1>;
}
```

5.2.3 Konsep Component-Based Architecture

Component-Based Architecture adalah pendekatan dalam pengembangan aplikasi dimana antarmuka pengguna (UI) dibangun dari bagian-bagian kecil yang berdiri sendiri (komponen), yang dapat digunakan kembali (*reusable*) dan digabungkan membentuk aplikasi yang lebih besar.

Alih-alih membuat satu file besar yang berisi seluruh tampilan dan logika, React membagi UI menjadi potongan kecil seperti:

- Header
- Sidebar
- Card
- Button
- Form
- Footer

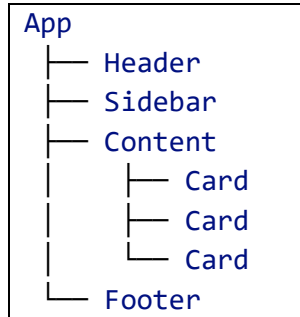
Setiap bagian tersebut disebut *Component*.

Pendekatan ini memiliki beberapa keunggulan utama:

1. *Reusability* (Dapat Digunakan Ulang)
2. *Maintainability* (Mudah Dipelihara)
3. *Scalability* (Mudah Dikembangkan)
4. *Separation of Concern*

Struktur Hirarki Komponen (Component Tree)

Dalam React, komponen umumnya berbentuk *Function Component* (RFC) dengan membentuk struktur pohon (tree).



- App → Sebagai Parent Component
- Card → Sebagai Child Component
- Child bisa memiliki child lagi (nested component)

Contoh Parent and Child Component:

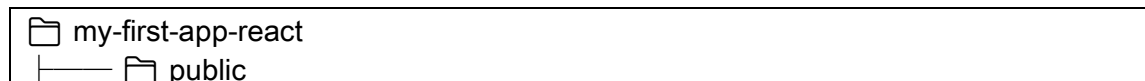
```
export default function App() {
  return (
    <div>
      <Header />
      <Content />
    </div>
  )
}

const Header = () => {
  return <h1>Header</h1>;
}

const Content = () => {
  return (
    <div>
      <h2>Content</h2>
      <p>This is a paragraph.</p>
    </div>
  );
}
```

5.2.4 Struktur Project

Berikut ini adalah bentuk struktur project react yang telah digenerate sebelumnya:





Ini adalah struktur App Router (Next 13+ sampai Next 14/15) yang memang digunakan bersama React 19.

- *public* : Berisi file-file statis seperti gambar, untuk mengaksesnya dapat langsung melalui URL.
- *src/app/* : Inti arsitektur App Router, semua routing berbasis struktur folder.
- *page.tsx* atau *.jsx*: File page adalah halaman utama dalam suatu folder.
 - Misal: *app/page.tsx* mengartikan route *http://localhost:3000/*
 - Jika memiliki struktur folder *app/latihan-2/page.jsx* maka diartikan sebagai route *http://localhost:3000/latihan-2*
- *layout.tsx* : File ini adalah layout global.

```
export default function RootLayout({ children }) {  
  return (  
    <html>  
      <body>  
        {children}  
      </body>  
    </html>  
  )  
}
```

Fungsinya:

- Membungkus semua page
- Template global
- Biasanya berisi Navbar, Footer

- Layout bisa nested (layout per folder).
- *globals.css* dan *page.module.css* : CSS module khusus untuk satu page.
- *package.json* : file konfigurasi utama project yang mencatat dependency, menyimpan script, mengontrol versi dan mengatur eksekusi aplikasi. Tanpa file ini, project JavaScript modern tidak bisa berjalan dengan benar.

5.3 Latihan Pembelajaran

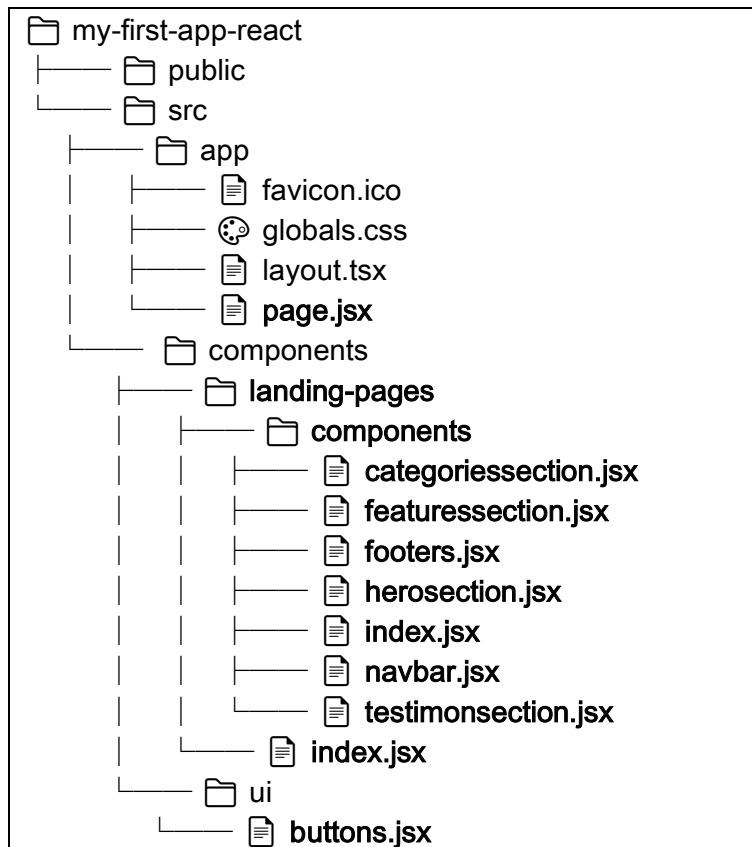
Pada latihan pembelajaran ini, mahasiswa akan membangun sebuah halaman landing page sederhana menggunakan pendekatan *Component-Based Architecture* pada React dengan framework Next.js (*App Router*).

Landing page yang akan dibuat terdiri dari beberapa bagian utama seperti:

- Hero Section
- Category Section
- Feature Section
- Testimonial Section
- Footer

Dalam latihan ini tidak menuliskan seluruh tampilan Landing Page dalam satu file *page.tsx*, melainkan wajib untuk membagi setiap bagian menjadi komponen terpisah agar struktur project lebih modular, reusable, dan scalable.

Structure Folder:



Dari sturktur folder diatas yang akan digunakan ialah file `page.jsx` sebagai halaman utama yang menampilkan Landing Page. Lalu folder *landing-pages* dan *ui* akan digunakan sebagai komponen modular yang akan dibangun.

Langkah 1: Instalasi Framework CSS

Bootstrap adalah framework CSS *open-source* yang digunakan untuk membangun tampilan website secara cepat, responsif, dan konsisten tanpa harus menulis CSS dari nol.

Bootstrap menyediakan:

- Sistem grid (layout)
- Komponen siap pakai (button, card, navbar, modal)
- Utility class
- Desain responsif (mobile-first)

Bootstrap pertama kali dikembangkan oleh tim di Twitter dan menjadi salah satu CSS framework paling populer di dunia.

Cara instalasi dalam project react, tambahkan dependency ini dalam project react dengan syntax:

```
npm install bootstrap bootstrap-icons react-bootstrap
```

Bootstrap dirancang dengan pendekatan:

- Tampilan mobile terlebih dahulu, kemudian menyesuaikan ke layar lebih besar.
- Bootstrap memiliki breakpoint seperti:
 - sm (small)
 - md (medium)
 - lg (large)
 - xl (extra large)

Untuk mengenal lebih lanjut mengenai class apa saja yang dapat dipakai silakan baca <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Bootstrap sangat cocok digunakan dalam latihan ini karena:

- Mudah dipelajari
- Hasil cepat terlihat
- Membantu memahami konsep layout responsif

Import Bootstrap pada layout.jsx seperti dibawah ini:

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import "bootstrap/dist/css/bootstrap.min.css";
import "bootstrap-icons/font/bootstrap-icons.css";

//rest of code
```

Langkah 2: Membangun komponen modular untuk Single Page Layout

A. Single Page Layout

Tabel 5.7 Page Layout Landing Page

Navbar (Fixed)
Hero Section (Welcome + main CTA)
Features Section (Why Choose Us – 3 Cards)
Categories Section (Book Categories Display)
Testimonial Section (Customer Reviews)
Footer (Links, Contact, Social)

➤ Prinsip yang Digunakan:

1. *Separation of Concerns*

- Setiap section adalah komponen terpisah
- Mudah di-maintain dan di-update secara independen

2. *Reusability*

- Komponen dapat digunakan kembali di halaman lain
- Modular dan scalable

3. *Single Responsibility*

- Setiap komponen memiliki satu tanggung jawab spesifik
- `LandingPageSection` hanya bertanggung jawab untuk layout/komposisi

4. *Declarative Approach*

- Menggunakan JSX untuk mendeklarasikan struktur UI
- Mudah dibaca dan dipahami

➤ Styling Approach

- Menggunakan Bootstrap untuk *styling*
- Bootstrap Icons untuk *iconography*
- Inline styles untuk *customization* tertentu
- Responsive design dengan Bootstrap grid system

B. Modular Components UI

Komponen Button adalah komponen reusable yang dibuat dengan React dan Bootstrap, dirancang untuk memberikan fleksibilitas tinggi dalam penggunaan tombol di seluruh aplikasi.

Komponen *ui/buttons.jsx*:

```
import React from "react";

const Button = ({
  children,
  variant = "primary",
  size = "md",
  outline = false,
  icon,
  iconPosition = "left",
  disabled = false,
  loading = false,
  fullWidth = false,
  className = "",
  onClick,
  type = "button",
  ...props
}) => {
  const variantClass = outline ? `btn-outline-${variant}` : `btn-${variant}`;
  const sizeClass = size !== "md" ? `btn-${size}` : "";
  const widthClass = fullWidth ? "w-100" : "";
  const disabledClass = disabled || loading ? "disabled" : "";

  const buttonClasses = `btn ${variantClass} ${sizeClass} ${widthClass} ${disabledClass} ${className}`.trim();

  return (
    <button
      type={type}
      className={buttonClasses}
      onClick={onClick}
      disabled={disabled || loading}
      {...props}
    >
      {loading ? (
        <>
```

```

        <span
          className="spinner-border spinner-border-sm me-2"
          role="status"
          aria-hidden="true"
        ></span>
        Loading...
      </>
    ) : (
      <>
        {icon && iconPosition === "left" && (
          <i className={`bi bi-${icon} me-2`} ></i>
        )}
        {children}
        {icon && iconPosition === "right" && (
          <i className={`bi bi-${icon} ms-2`} ></i>
        )}
      </>
    )}
  </button>
);
}

export {Button}

```

C. Modular Navbar

Navbar (Navigation Bar) adalah elemen antarmuka (UI) pada situs web, biasanya berupa bilah di bagian atas, yang berisi kumpulan tautan (*link*) atau menu untuk memudahkan navigasi antar halaman atau bagian situs. *Navbar* berfungsi sebagai pemandu utama bagi pengguna untuk menjelajahi konten, meningkatkan pengalaman pengguna (UX), dan sering kali dibuat responsif agar sesuai dengan berbagai ukuran perangkat.

 components/landing-pages/components/navbar.jsx

```

export function Navbar() {
  const menu_nav = [
    { id: 1, name: "Home", link: "#home" },
    { id: 2, name: "Books", link: "#books" },
    { id: 3, name: "Categories", link: "#categories" },
    { id: 4, name: "About", link: "#about" },
    { id: 5, name: "Contact", link: "#contact" },
  ];
}

```

```

];

return (
  <nav
    className={`navbar navbar-expand-lg fixed-top transition-navbar`}
    style={styles.navigationBar}
  >
    <div className="container">
      <a className="navbar-brand" href="#">
        <i className="bi bi-book me-2"></i>
        Readly <sup>+</sup>
      </a>
      <button
        className="navbar-toggler"
        type="button"
        data-bs-toggle="collapse"
        data-bs-target="#navbarNav"
      >
        <span className="navbar-toggler-icon"></span>
      </button>
      <div className="collapse navbar-collapse" id="navbarNav">
        <ul className="navbar-nav mx-auto">
          {menu_nav.map((menu, index) => (
            <li className="nav-item" key={index}>
              <a className="nav-link" href={menu.link}>
                {menu.name}
              </a>
            </li>
          ))}
        </ul>

        <div className="d-flex gap-2">
          <Button className="btn btn-light text-dark">
            <i className="bi bi-box-arrow-in-right me-2"></i>
            Sign In
          </Button>
          <Button className="btn btn-warning">
            <i className="bi bi-person-plus me-2"></i>
            Sign Up
          </Button>
        </div>
      </div>
    </nav>
  );

```

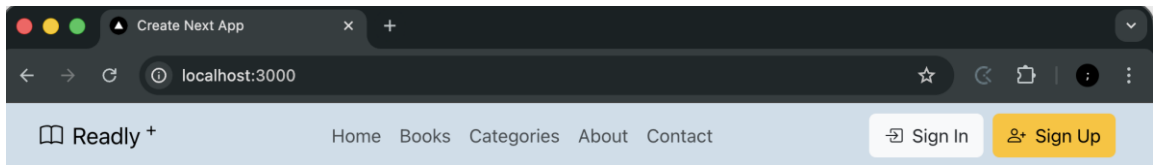
```

}

const styles = {
  navigationBar: {
    backgroundColor: "rgba(247, 248, 226, 0.8)",
    transition: "all 0.3s ease-in-out",
  },
};

```

Komponen Navbar ini akan membuat tampilan seperti dibawah ini:



Gambar 5.4 Tampilan Output Kode navbar.jsx

Dimana navigasi bar ini terdiri dari 3 item:

1. Judul aplikasi (pojok kiri)
2. Menu navigasi (tengah)
3. Tombol Auth (pojok kanan)

D. Modular Hero Section

Hero section adalah bagian pertama yang menonjol secara visual di atas homepage atau halaman web, yang bertujuan menyambut pengunjung, menjelaskan nilai produk, dan mengarahkan mereka ke tindakan utama. Ini adalah "*wajah*" situs web yang menggabungkan gambar atau video menarik, judul, dan tombol *Call-to-Action* (CTA).

components/landing-pages/components/herosection.jsx

```

import React from "react";
import { Button } from "../../ui/buttons";

export function HeroSection() {
  return (
    <section
      id="home"
      className="relative min-h-screen flex items-center overflow-
hidden bg-primary text-white"
      style={styles.section}
    >

```

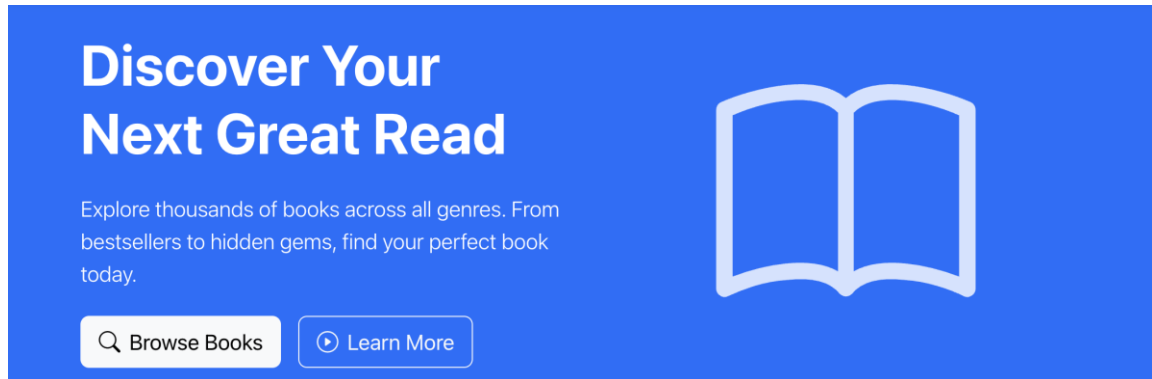
```

    >
    <div className="container">
      <div className="row align-items-center">
        <div className="col-lg-6">
          <h1 className="display-4 fw-bold mb-4">
            Discover Your Next Great Read
          </h1>
          <p className="lead mb-4">
            Explore thousands of books across all genres. From
bestsellers to
            hidden gems, find your perfect book today.
          </p>
          <div className="d-flex gap-3">
            <Button className="btn btn-light btn-lg">
              <i className="bi bi-search me-2"></i>
              Browse Books
            </Button>
            <Button className="btn btn-outline-light btn-lg">
              <i className="bi bi-play-circle me-2"></i>
              Learn More
            </Button>
          </div>
        </div>
        <div className="col-lg-6 text-center mt-4 mt-lg-0">
          <i
            className="bi bi-book"
            style={{ fontSize: "15rem", opacity: 0.8 }}
          ></i>
        </div>
      </div>
    </div>
  </section>
);
}

const styles = {
  section: {
    minHeight: "100vh",
    marginTop: "-70px",
    paddingTop: "70px",
    display: "flex",
    alignItems: "center",
  },
};

```

Maka tampilan Hero Section berdasarkan code tersebut akan menampilkan "wajah" sederhana yang memadu padankan text dengan icon.



Gambar 5.5 Tampilan Output Kode herosection.jsx

E. Modular Feature Section

Feature section adalah jenis tulisan jurnalistik khas yang menyajikan fakta dengan gaya bercerita (*storytelling*) yang kreatif, mendalam, dan menghibur, tidak terikat aktualitas ketat layaknya berita lempur (*hard news*).

Secara teknis *feature section* biasanya diisi dengan fungsionalitas atau keunggulan produk, dalam konteks *landing page*, daftar produk terlaris (*best-sellers*) bisa dianggap sebagai fitur utama yang ingin ditonjolkan kepada pengunjung.

Buatlah minimal 6 data array object dengan key isian seperti ini:

```
const ListBooks = [{
  id: number,
  title: string,
  img: 'filename',
  author: string,
  is_free: boolean,
  sinopsis: string,
  rating: number,
  views: number
}]
```

components/landing-pages/components/featuredbookssection.jsx

```

export function FeaturedBooksSection() {
  return (
    <section id="books" className="py-5">
      <div className="container">
        <div className="row mb-5">
          <div className="col">
            <h2 className="fw-bold text-center">Featured Books</h2>
            <p className="text-center text-muted">
              Handpicked selections just for you
            </p>
          </div>
        </div>
        <div className="row g-4">
          {ListBooks.map((book) => (
            <div key={book.id} className="col-md-6 col-lg-3">
              <BookCard book={book} />
            </div>
          ))}
        </div>
      </div>
    </section>
  );
}

```

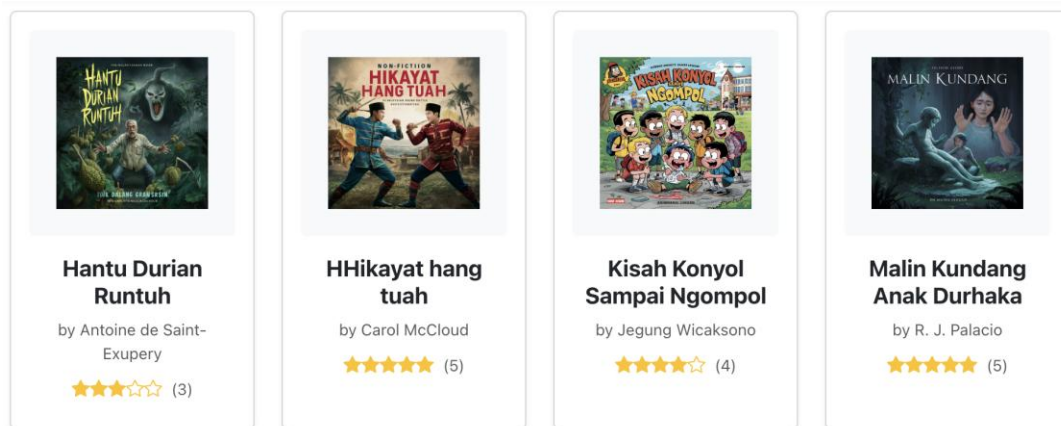
Komponen BookCard

```
const BookCard = ({ book }) => {
  const { title, author, is_free, rating, img } = book;

  const renderStars = (rating) => {
    return [...Array(5)].map((_, i) => (
      <i
        key={i}
        className={`bi bi-star${i < Math.floor(rating) ? "-fill" : ""} text-warning`}
      ></i>
    ));
  };

  return (
    <div className="card h-100 shadow-sm">
      <div className="card-body text-center">
        <div className="bg-light p-4 mb-3 rounded">
          {img ? (
            <img
              src={` /books/${img}`}
              alt={title}
              className="img-fluid"
              style={{ maxHeight: "150px", objectFit: "cover" }}
            />
          ) : (
            <i
              className="bi bi-book-half"
              style={{ fontSize: "4rem", color: "#6c757d" }}
            ></i>
          )}
        </div>

        <h5 className="card-title fw-bold">{title}</h5>
        <p className="text-muted small mb-2">by {author}</p>
        <div className="mb-2">
          {renderStars(rating)}
          <span className="ms-2 text-muted small">({rating})</span>
        </div>
      </div>
    </div>
  );
};
```



Gambar 5.6 Tampilan Output Feature Book Section

F. Modular Category Section

Category Section adalah struktur pengorganisasian konten yang membagi informasi ke dalam kelompok-kelompok spesifik (kategori) berdasarkan topik atau jenisnya. Ini berfungsi mempermudah navigasi, pencarian, dan pengelolaan konten di situs web atau manajemen inventaris, di mana kategori adalah subset dari bagian yang lebih besar

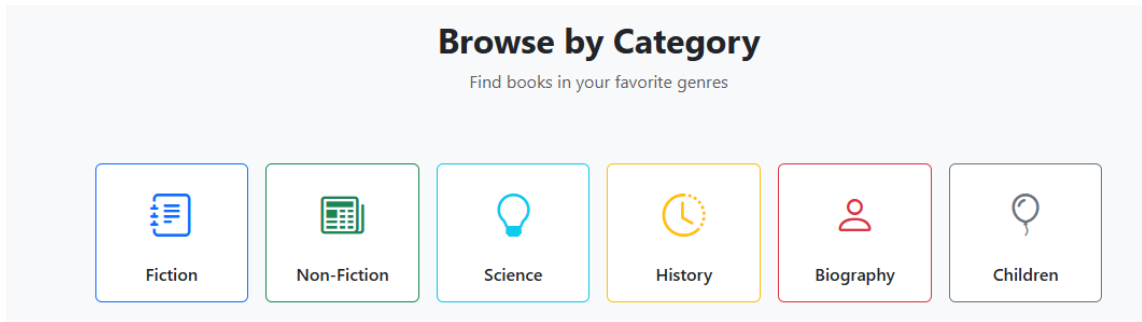
 components/landing-pages/components/categoriessection.jsx

```
import React from 'react'

export function CategoriesSection() {
  return (
    <section id="categories" className="py-5 bg-light">
      <div className="container">
        <div className="row mb-5">
          <div className="col">
            <h2 className="fw-bold text-center">Browse by Category</h2>
            <p className="text-center text-muted">
              Find books in your favorite genres
            </p>
          </div>
        </div>
        <div className="row g-3">
          {genre_data.map((category, index) => (
            <div key={index} className="col-md-4 col-lg-2">
              <div className={`card text-center border-${category.color} h-100`} >
                <div className="card-body">
                  <i
                    className={`bi ${category.icon} text-${category.color}`}
                    style={{ fontSize: "2.5rem" }}
                  ></i>
                  <h6 className="mt-3 mb-0">{category.name}</h6>
                </div>
              </div>
            </div>
          ))}
        </div>
      </div>
    </section>
  )
}
```

```
const genre_data = [
  { name: "Fiction", icon: "bi-journal-text", color: "primary" },
  { name: "Non-Fiction", icon: "bi-newspaper", color: "success" },
  { name: "Science", icon: "bi-lightbulb", color: "info" },
  { name: "History", icon: "bi-clock-history", color: "warning" },
  { name: "Biography", icon: "bi-person", color: "danger" },
  { name: "Children", icon: "bi-balloon", color: "secondary" },
];
```

Output dari Category Section:



Gambar 5.7 Tampilan Output Category Section

G. Modular Testimonials Category Section

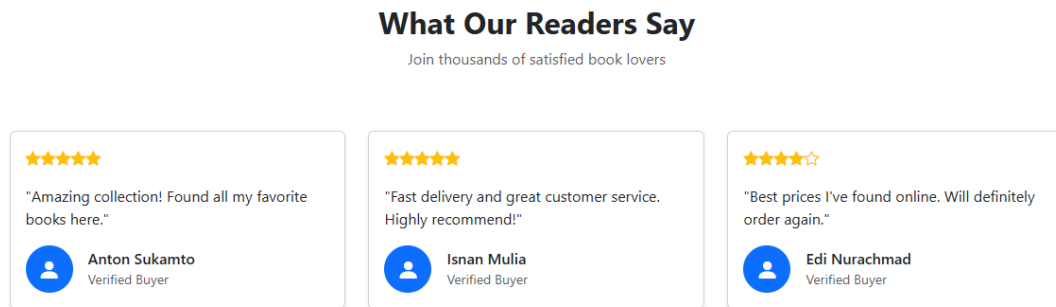
Testimonials category section adalah bagian khusus pada website, landing page, atau aplikasi yang didesain untuk menampilkan ulasan, pengalaman positif, dan umpan balik pelanggan secara terstruktur. Bagian ini krusial untuk membangun *social proof* (bukti sosial), kredibilitas merek, dan kepercayaan konsumen guna meningkatkan penjualan.

`components/landing-pages/components/testimonsection.jsx`

```
import React from "react";

export function TestimonSection() {
  return (
    <section className="py-5">
      <div className="container">
        <div className="row mb-5">
          <div className="col">
            <h2 className="fw-bold text-center">What Our Readers
Say</h2>
            <p className="text-center text-muted">
              Join thousands of satisfied book lovers
            </p>
          </div>
        </div>
        <div className="row g-4">
          {user_testimonials.map((testimonial, index) => (
            <div key={index} className="col-md-4">
              <div className="card h-100 shadow-sm">
                <div className="card-body">
                  <div className="mb-3">
                    {[...Array(5)].map((_, i) => (
                      <i
                        key={i}
                        className={`bi bi-star${i < testimonial.rating
? "-fill" : ""}` text-warning`}
```


Output Testimonial Section:



Gambar 5.8 Tampilan Output Testimonial Section

H. Modular Footer Section

Footer section adalah bagian paling bawah pada sebuah halaman website atau dokumen yang muncul secara konsisten di setiap halaman. Berfungsi sebagai area informasi sekunder, footer umumnya berisi hak cipta, kontak, tautan media sosial, navigasi tambahan, dan kebijakan privasi, yang membantu meningkatkan pengalaman pengguna (UX) dan navigasi situs.

components/landing-pages/components/footers.jsx

Constanta variable yang diperlukan:

```
const socialLinks = [  
  { icon: "facebook", url: "#", label: "Facebook" },  
  { icon: "twitter", url: "#", label: "Twitter" },  
  { icon: "instagram", url: "#", label: "Instagram" },  
  { icon: "linkedin", url: "#", label: "LinkedIn" },  
];  
  
const quickLinks = [  
  { text: "About Us", url: "#" },  
  { text: "Contact", url: "#" },  
  { text: "Privacy Policy", url: "#" },  
  { text: "Terms of Service", url: "#" },  
];  
  
const contactItems = [  
  { icon: "geo-alt", text: "123 Book Street, Reading City" },
```

```

    { icon: "telephone", text: "+1 234 567 8900" },
    { icon: "envelope", text: "info@bookstore.com" },
  ];

```

Component Child :

- RFC SocialMediaLinks:

```

const SocialMediaLinks = ({ links, iconSize = "1.5rem", className = ""
}) => {
  return (
    <div className={`d-flex gap-3 ${className}`}>
      {links.map((link, index) => (
        <a key={index} href={link.url} className="text-dark" aria-
label={link.label}>
          <i className={`bi bi-${link.icon}`} style={{ fontSize:
iconSize }}></i>
        </a>
      ))}
    </div>
  );
}

```

- RFC FooterSection

```

const FooterSection = ({ title, children, className = "" }) => {
  return (
    <div className={`col-md-4 mb-3 ${className}`}>
      <h6 className="fw-bold">{title}</h6>
      {children}
    </div>
  );
}

```

- RFC ContactInfo

```

const ContactInfo = ({ items, className = "" }) => {
  return (
    <ul className={`list-unstyled text-muted ${className}`}>
      {items.map((item, index) => (
        <li key={index}>
          <i className={`bi bi-${item.icon} me-2`} ></i>
          {item.text}
        </li>
      ))}
    </ul>
  );
}

```

```
);  
}
```

- RFC LinkList

```
const LinkList = ({ links, className = "" }) => {  
  return (  
    <ul className={`list-unstyled ${className}`}>  
      {links.map((link, index) => (  
        <li key={index}>  
          <a href={link.url} className="text-muted text-decoration-  
none">  
            {link.text}  
          </a>  
        </li>  
      ))}  
    </ul>  
  );  
}
```

Parent Component RFC Footers:

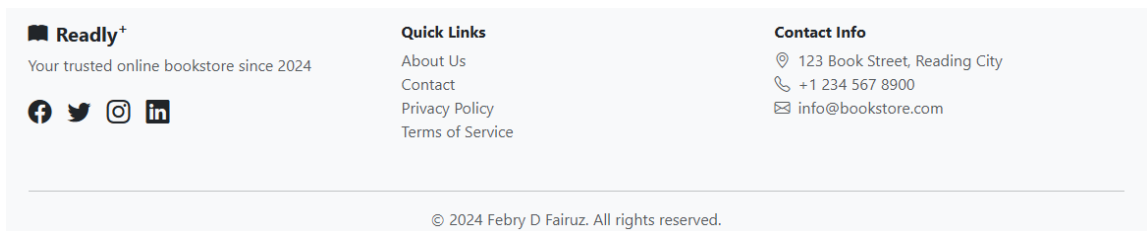
```
import React from "react";  
export function Footers() {  
  return (  
    <footer id="contact" className="bg-light py-4">  
      <div className="container">  
        <div className="row">  
          <div className="col-md-4 mb-3">  
            <h5 className="fw-bold">  
              <i className="bi bi-book-fill me-2"></i>  
              Readly<sup>+</sup>  
            </h5>  
            <p className="text-muted">  
              Your trusted online bookstore since 2024  
            </p>  
            <SocialMediaLinks links={socialLinks} />  
          </div>  
  
          <FooterSection title="Quick Links">  
            <LinkList links={quickLinks} />  
          </FooterSection>  
  
          <FooterSection title="Contact Info">  
            <ContactInfo items={contactItems} />  
          </FooterSection>  
        </div>  
      </div>  
    </div>  
  );  
}
```

```

        </FooterSection>
      </div>
      <hr />
      <div className="text-center text-muted">
        <p className="mb-0">&copy; 2024 Febry D Fairuz. All rights
reserved.</p>
      </div>
    </div>
  </footer>
);
}

```

Output untuk Footer Section seperti:



Gambar 5.9 Tampilan Output Footer Section

I. Menggabungkan komponen modular tersebut kedalam RFC Landing Page untuk dijadikan single page

components/landing-pages/index.jsx

LandingPageSection adalah komponen utama (*parent component*) yang berfungsi sebagai *container* untuk halaman *landing page* aplikasi *Ready+*. Komponen ini mengatur layout dan urutan tampilan dari berbagai section yang membentuk halaman landing.

```
import React from "react";
import {child component file here} from "../components";

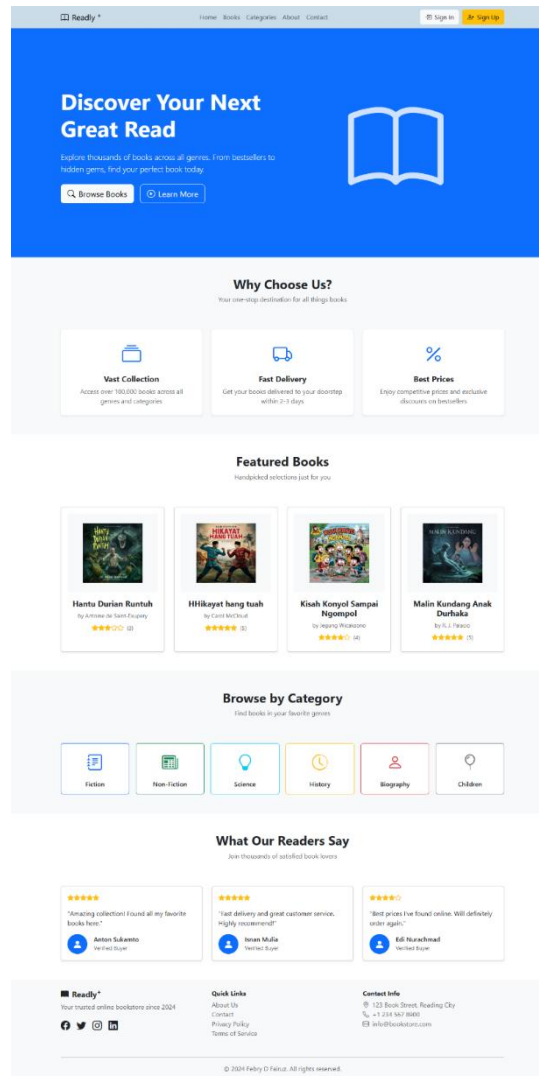
export default function LandingPageSection() {
  return (
    <div>
      <Navbar />
      <HeroSection />
      <FeaturedBooksSection />
      <CategoriesSection />
      <TestimonSection />
      <Subscribe />
      <Footers />
    </div>
  );
}
```

J. Memanggil RFC Landing Page pada root app

Pada file *app/page.jsx* panggilah RFC Landing Page dalam file tersebut:

```
export { default } from "@components/landing-pages/index";
```

Selanjutnya ialah pada browser maka tampilan halaman utama jika mengakses <http://localhost:3000/> akan seperti dibawah ini, yaitu menampilkan halaman Landing Page yang telah dibuat sebelumnya:



Gambar 5.10 Tampilan Output Landing Page Section

5.4 Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-MERN-STACK-NPM"** dengan nama commit **"Praktikum 5"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB VI

FRAMEWORK UI REACT JS: PART 2

6.1 Tujuan Pembelajaran

1. Memahami konsep state dan fungsinya dalam pengelolaan data dinamis pada aplikasi React.
2. Memahami konsep *props* sebagai mekanisme pengiriman data antar komponen.
3. Menjelaskan peran *Hooks*, khususnya *useState* dan *useEffect*, dalam pengembangan komponen berbasis fungsi.
4. Menerapkan *state*, *props*, dan *hooks* dalam studi kasus aplikasi React sederhana.
5. Mengembangkan komponen yang lebih interaktif dan responsif terhadap input pengguna.

6.2 Dasar Teori

Dalam pengembangan aplikasi menggunakan React modern berbasis Next.js (App Router), terdapat perbedaan penting antara Server Component dan Client Component. Secara default, semua file di dalam folder `app/` pada Next.js App Router adalah Server Component. Artinya, komponen tersebut dijalankan di sisi server dan tidak memiliki akses terhadap fitur interaktif berbasis browser seperti:

- State
- Hook
- Event handler (`onClick`, `onChange`, dll)
- Akses DOM
- Akses window atau document

Karena itu, ketika ingin menggunakan *state* atau *hook* React, wajib mendeklarasikan komponen tersebut sebagai *Client Component*.

Deklarasi ini dilakukan dengan menambahkan perintah berikut di baris paling atas file:

`"use client"`

6.2.1 State

State merupakan mekanisme untuk menyimpan dan mengelola data yang bersifat dinamis di dalam sebuah komponen. Data yang disimpan sebagai state dapat berubah akibat interaksi pengguna atau proses tertentu, dan setiap perubahan state akan secara otomatis memperbarui tampilan (*re-render*).

Pada React modern, state dikelola menggunakan *Hook* **useState**. Dalam mengimplementasi useState, ada dua tipe yang bisa digunakan, pertama ialah dalam bentuk Destructuring dan Non Destructuring.

A. Penggunaan Destructuring:

Format ini menggunakan **Array Destructuring** dari JavaScript ES6. Dengan bentuk struktur *useState()* seperti `[nilai_state, function_state]`.

Contoh:

```
const [count, setCount] = useState(0);
const [name, setName] = useState("");
const [isactive, setIsactive] = useState(false);
const [listBook, setListBook] = useState([]);
```

Dalam penggunaan useState pattern yang dipakai ialah:

- State variable: *namaVariable*
- Setter function: *setNamaVariable* (dengan prefix "set")

```
const [count, setCount] = useState(0)
```

The diagram illustrates the structure of the `useState` hook call. It shows the following components and their roles:

- `count`: State variable (untuk baca nilai)
- `setCount`: Setter function (untuk update)
- `useState`: Hook function
- `0`: Initial value

Gambar 6.1 Struktur Penggunaan useState pada React

B. Penggunaan Non Destructuring:

Untuk menggunakan Non Destructuring, dapat dilakukan dengan cara membuat sebuah variable yang mengembalikan nilai dalam bentuk array dan menginisialisasinya setiap elemen dengan index.

```
const countState = useState(0)
const count = countState[0]
const setCount = countState[1]
```

Dari code diatas menyatakan bahwa:

- variable countState memiliki nilai hook ialah 0 dalam bentuk array
- variable count digunakan untuk mengambil nilai index ke-0 dari countState
- variabel setCount akan mengambil nilai index ke-1 dari countState

C. Perbandingan antara Destructuring dengan Non Destructuring

Dari penjelasan sebelumnya dapat disimpulkan bahwa penggunaan *useState* *Destructuring* lebih baik digunakan sebagai media pengolahan data, kenapa lebih baik berikut table ringkasan dari kedua format *useState*:

Tabel 6.1 Perbandingan Format Non Destructuring dan Destructuring

Non Destructuring	Destructuring
3 baris per state	1 baris per state
Kurang readable	Mudah dibaca
Tidak idiomatis (tidak mengikuti best practice React)	Mengikuti best practice React, singkat dan jelas
Sulit maintain untuk banyak state	Lebih maintainable

Oleh karenanya selalu gunakan destructuring untuk *useState()* dan hooks lainnya!

Contoh penggunaan:

```
"use client"

import React, { useState } from 'react'
import { Button } from '@components/ui/buttons';

export default function Latihan6() {
  const [count, setCount] = useState(0);
  return (
    <div className='container bg-light border rounded mt-5 p-3 text-center'>
      <h1>{count}</h1>
      <div className="btn-group">
        <Button className='btn-primary' icon={"patch-plus"} onClick={()=>setCount(count + 1)}>
          Increment
        </Button>
        <Button className='btn-danger' icon={"patch-minus"} onClick={()=>setCount(count - 1)}>
          Decrement
        </Button>
      </div>
    </div>
  )
}
```

Penjelasan:

1. Client Component Declaration

"use client"

- Menandakan bahwa ini adalah Client Component di Next.js
- Diperlukan karena menggunakan interaktivitas (state dan event handlers)
- Komponen ini akan di-render di sisi client (browser)

2. State Management dengan useState

- `useState(0)`: Menginisialisasi state dengan nilai awal 0
- `count`: Variable yang menyimpan nilai state saat ini
- `setCount`: Function untuk mengupdate nilai count
- Setiap kali `setCount` dipanggil, komponen akan re-render dengan nilai baru

3. Increment Button

- Ketika ditekan, memanggil `setCount(count + 1)`
- Menambah nilai count sebesar 1

4. Decrement Button

- Ketika ditekan, memanggil `setCount(count - 1)`
- Mengurangi nilai count sebesar 1

6.2.2 Props

Props merupakan mekanisme untuk mengirim data dari komponen induk (*parent*) ke komponen anak (*child*). Data yang dikirim melalui props bersifat satu arah (*one-way data flow*) dan tidak dapat diubah oleh komponen penerima.

Agar penggunaan props lebih mudah dipahami, berikut format umum implementasi props dalam React.

Parent Component

```
export default function Latihan6() {
  const book = { title: 'Web Programming with Next JS',
author: "Edi Nurachman", year: 2025 };
  return (
    <div className='container bg-light p-3 text-center border
rounded>
      <h3>Popular Book</h3>
      <PopularBook book={book} />
    </div>
  )
}
```

```
}
```

Child Component

```
const PopularBook = ({ book }) => {  
  return (  
    <div className="text-start">  
      <p>Title: {book.title}</p>  
      <p>Author: {book.author}</p>  
      <p>Year: {book.year}</p>  
    </div>  
  )  
}
```

A. Function Declaration dan Data Object

- Nama RFC *Latihan6* menunjukkan ini adalah komponen induk (parent)
- Dan komponen ini tidak menerima props (parameter kosong)
- Membuat object literal bernama book
 - Berisi 3 properti:
 - title: String - judul buku
 - author: String - nama penulis
 - year: Number - tahun terbit
- Object tersebut akan di-pass sebagai props ke child component

B. Menghubungkan dengan child component

- Nama RFC *PopularBook* menunjukkan ini adalah komponen anak (child)
- Data komponen ini menerima props bernama 'book'
- Data yang diterima props ialah bertipe object

C. Cara kerja Props

1. Parent mengirimkan data:

```
PopularBook book={book} />  
  ↑   ↑  
prop name; prop value
```

2. Child menerima data:

```
const PopularBook = ({ book }) => {  
  ↑  
  destructuring props
```

3. Child menggunakan data:

```
<p>Title: {book.title}</p>
```

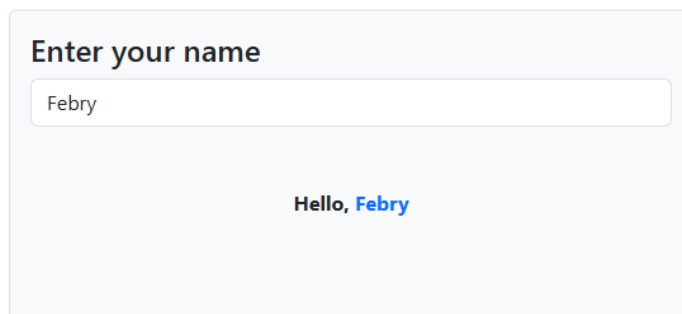
↑
akses properti dari props

Berikut adalah contoh penggunaan kombinasi antara state dan props:

```
"use client"
import React, { useState } from 'react'

export default function Latihan6() {
  const [name, setName] = useState("");
  return (
    <div className='container bg-light p-3 border rounded mt-5'>
      <div>
        <h3>Enter your name</h3>
        <input
          type="text"
          name="name"
          className='form-control'
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
      </div>
      {name && <ViewName name={name} />}
    </div>
  )
}

const ViewName = ({ name }) => {
  return (
    <div className="text-center py-5">
      <p className='fw-bold'>
        Hello,
        <span className="text-primary">
          {name}
        </span>
      </p>
    </div>
  )
}
```



Enter your name

Hello, Febry

Gambar 6.2 Tampilan Output Kombinasi State dan Props

Penjelasan:

- Deklarasi state management untuk mengahandle inputan dalam bentuk string yang bernilai kosong:

```
const [name, setName] = useState('');
```

- Controlled component pattern

```
<input
  type="text"
  name="name"
  className='form-control'
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

Cara kerja:

User ketik "F"

↓

onChangeText triggered

↓

setName("F") dipanggil

↓

State name berubah: "" → "F"

↓

Component re-render

↓

TextInput value={name} → value="F"

↓

UI update: TextInput menampilkan "F"

- Conditional Rendering

```
{name && <ViewName name={name} /> }
```

↑

↑

prop value

Ini adalah *Short-Circuit Evaluation* (Logical AND), mengecek kondisi HANYA JIKA state name memiliki nilai atau tidak kosong maka akan menampilkan RFC ViewName dengan mengirim properti bernama name.

6.2.3 Hooks

Hooks merupakan fitur pada React yang memungkinkan penggunaan *state*, *lifecycle*, dan optimasi performa di dalam komponen berbasis fungsi (*Functional*

Component). Dengan *Hooks*, pengembang tidak lagi bergantung pada *Class Component*, sehingga kode menjadi lebih ringkas, mudah dibaca, dan mudah dirawat.

Hooks bekerja berdasarkan prinsip:

- Hanya dapat digunakan di komponen fungsi (RFC)
- Dipanggil di level teratas komponen (bukan di dalam kondisi atau loop)
- Mengandalkan dependency array untuk mengontrol kapan sebuah proses dijalankan
- Tidak dipanggil didalam *Selections*, *Repetitions*, ataupun fungsi biasa
- Urutan pemanggilan *Hooks* harus konsisten

Hooks yang umum digunakan pada React:

- *useState*: Mengelola state
- *useEffect*: Menangani efek samping (lifecycle)
- *useMemo*: Optimasi performa dalam bentuk memoizing
- *useContext*: Berbagi data global (lanjutan)

useRef: Penyimpanan nilai yang tidak perlu dirender

A. Hook *useEffect*

Jika diibaratkan, React adalah sebuah panggung sandiwara, *useEffect* adalah kru di balik layar yang bertugas membereskan panggung setelah aktor selesai beraksi atau menyiapkan properti sebelum adegan dimulai.

Secara teknis, *useEffect* adalah *Hook* dalam React yang memungkinkan melakukan *side effects* (efek samping) di dalam function component.

Dalam pemrograman, side effect adalah segala sesuatu yang "keluar" dari alur murni render UI. Contohnya:

- Mengambil data dari API (*Fetching data*).
- Mengubah DOM secara manual (misalnya mengganti judul dokumen).
- Mengatur timer seperti *setTimeout* atau *setInterval*.
- Berlangganan (Subscription) ke sumber data eksternal.

Struktur dasar *useEffect* terdiri dari dua bagian utama: fungsi efek dan array ketergantungan (*dependency array*).

```
useEffect(() => {  
  // 1. Kode efek di sini (Logika yang ingin dijalankan)  
  
  return () => {  
    // 2. Cleanup function (Opsional: dijalankan sebelum komponen di-unmount)  
  };  
}, [dependency]); // 3. Dependency Array (Menentukan kapan efek berjalan)
```

Cara *useEffect* berjalan sangat bergantung pada apa yang masukkan ke dalam array ketergantungan di bagian akhir:

Tabel 6.2 Perilaku *useEffect* Berdasarkan Dependency Array

Dependency Array	Kapan Efek Berjalan?
Tanpa Array <code>useEffect(() => {...})</code>	Berjalan setiap kali komponen melakukan render (sangat jarang digunakan karena boros performa).
Array Kosong <code>useEffect(() => {...}, [])</code>	Hanya berjalan satu kali setelah render pertama (mirip <code>componentDidMount</code>). Cocok untuk ambil data API.
Ada Isinya <code>useEffect(() => {...}, [prop, state])</code>	Berjalan saat pertama kali render DAN setiap kali nilai prop atau state tersebut berubah.

Kegunaan Utama:

- Sinkronisasi dengan Data Luar: Menghubungkan aplikasi dengan API atau database.
- Manajemen Event Listener: Menambahkan event seperti scroll atau resize pada jendela browser.
- Pembersihan (Cleanup): Menghapus timer atau membatalkan langganan agar tidak terjadi kebocoran memori (memory leak).

B. Hook useMemo

Jika *useEffect* adalah kru di balik layar, maka *useMemo* adalah asisten pribadi yang memiliki ingatan sangat tajam.

Dalam React, *useMemo* adalah *Hook* yang digunakan untuk memoarisasi (menyimpan hasil perhitungan) agar tidak perlu dihitung ulang setiap kali komponen melakukan render, kecuali jika ada data pendukung yang berubah.

Secara default, React akan menjalankan ulang semua logika di dalam fungsi komponen setiap kali terjadi *re-render* (misalnya saat ada *state* atau *props* yang berubah). Jika memiliki perhitungan yang sangat berat (seperti memproses ribuan data atau algoritma matematika yang rumit), menjalankannya di setiap render akan membuat aplikasi terasa lambat atau "lag". *useMemo* hadir untuk mencegah hal tersebut.

```
const hasilMemo = useMemo(() => {  
  // 1. Logika perhitungan yang berat  
  return hitungSesuatuYangMahal(a, b);  
}, [a, b]); // 2. Dependency Array
```

- Fungsi *hasilMemo*: Fungsi yang mengembalikan nilai yang ingin disimpan.
- *Dependency Array*: Daftar variabel yang dipantau. Jika a atau b berubah, React akan menghitung ulang nilainya. Jika tidak, React akan langsung mengambil hasil simpanan sebelumnya.

Kapan Sebaiknya TIDAK menggunakan *useMemo*:

- Jangan gunakan *useMemo* untuk setiap variabel kecil
- *Overhead*: *useMemo* sendiri butuh memori untuk menyimpan nilai. Jika perhitungannya sangat ringan (seperti $1 + 1$), penggunaan *useMemo* justru malah membebani performa.
- Keterbacaan: Kode jadi lebih kompleks. Gunakan hanya jika benar-benar merasakan adanya penurunan performa atau jika nilai tersebut digunakan sebagai dependensi di Hook lain.

C. Hook `useContext`

Jika `useEffect` adalah kru panggung dan `useMemo` adalah ingatan asisten, maka `useContext` adalah sistem pengeras suara (*broadcast*) di seluruh gedung pertunjukan.

Dalam React, data biasanya dikirim dari atas ke bawah (*parent to child*) melalui props. Namun, jika komponennya sangat dalam, akan mengalami masalah yang disebut *Prop Drilling*, yaitu melewatkan data melalui komponen tengah yang sebenarnya tidak butuh data tersebut. `useContext` hadir untuk memintas jalur itu.

`useContext` memungkinkan komponen untuk "subscribe" ke sebuah data global tanpa harus menerima data tersebut lewat props.

Cara penggunaan:

1. Membuat context (`createContext`)

```
const LayoutContext = createContext();
```

Biasanya dibuat di file terpisah atau di bagian atas komponen induk.

2. Menyediakan context (`Provider`)

Bungkus komponen yang membutuhkan data dengan `.Provider`. Semua komponen di dalamnya (dan anak-anaknya) sekarang punya akses ke nilai tersebut.

```
<LayoutContext.Provider value={{ token, account }}>
  {children}
</LayoutContext.Provider>
```

3. Mengkonsumsi context (`useContext`)

```
useContext(LayoutContext);
```

Di dalam komponen anak, panggil Hook ini untuk mengambil nilainya.

Kapan Harus Menggunakan `useContext`:

Gunakan `useContext` untuk data yang bersifat global atau digunakan oleh banyak komponen di tingkat yang berbeda:

- Tema: Mode gelap/terang.
- Autentikasi: Data pengguna yang sedang login (username, token).
- Bahasa: Lokalisasi (Inggris, Indonesia, dll).
- Keranjang Belanja: Data produk yang dipilih di aplikasi e-commerce.

D. Hook `useRef`

Jika `useContext` adalah pengeras suara, maka `useRef` adalah sebuah kotak penyimpanan pribadi yang tidak terlihat oleh penonton.

Secara sederhana, `useRef` adalah *Hook* yang mengembalikan sebuah objek dengan satu properti saja: `{ current: ... }`. Keunikan utamanya adalah: mengubah nilai di dalam `useRef` tidak akan memicu render ulang (*re-render*) pada komponen.

Manfaat menggunakan `useRef`:

1. Mengakses DOM Secara Langsung

Ini adalah penggunaan yang paling umum. Di React, biasanya membiarkan React yang mengatur DOM. Tapi terkadang, perlu "menyentuh" elemen JSX secara manual, misalnya untuk:

- Fokus pada input secara otomatis.
- Mengukur dimensi sebuah elemen (tinggi/lebar).
- Mengintegrasikan library pihak ketiga yang bukan berbasis React (seperti Google Maps atau Chart.js).

2. Menyimpan Nilai yang "Tahan Render"

Jika menyimpan data di `useState`, setiap kali data berubah, layar akan di-update. Tapi jika Anda butuh menyimpan variabel yang nilainya tetap ada selama siklus hidup komponen tanpa ingin memicu render ulang, `useRef` adalah solusinya.

```
const myRef = useRef(initialValue);

// Cara akses:
console.log(myRef.current);

// Cara ubah:
myRef.current = newValue;
```

Kapan harus memilih atau menggunakan Hook:

- Gunakan `useState` jika datanya harus tampil dan berubah di layar.

- Gunakan *useEffect* jika ingin "keluar" dari ekosistem React (interaksi dengan browser, server, atau timer).
- Gunakan *useContext* hanya jika props drilling sudah terasa sangat mengganggu (lebih dari 3-4 lapis komponen).
- Gunakan *useMemo* hanya jika mendeteksi aplikasi melambat (optimasi ini seringkali tidak perlu).
- Gunakan *useRef* paling sering untuk `.focus()`, `.scroll()`, atau integrasi library grafik.

Perbandingan Hooks secara umum:

Tabel 6.3 Perbandingan Fungsi Hooks

Hook	Fungsi Utama	Memicu Re-render
<i>useState</i>	Data dinamis	Ya
<i>useEffect</i>	Efek samping	Tidak langsung
<i>useContext</i>	Data global	Ya
<i>UseRef</i>	Referensi / cache	Tidak
<i>useMemo</i>	Optimasi nilai	Tidak

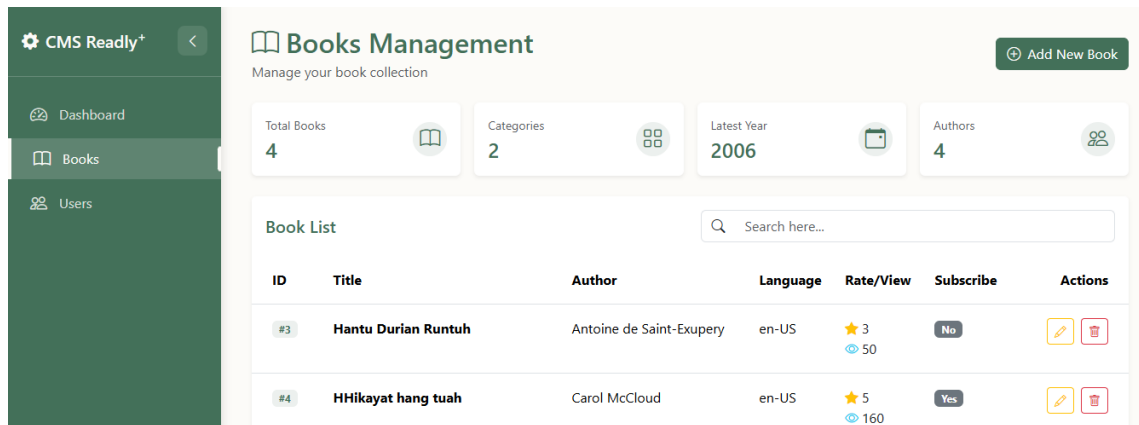
6.3 Latihan Pembelajaran

Pada latihan praktikum ini, mahasiswa akan mengembangkan sebuah *Web Application* berbasis CMS (*Content Management System*) sederhana menggunakan React (dengan pendekatan modern berbasis komponen).

Aplikasi yang dibangun akan berfungsi untuk:

- Mengelola data buku (CRUD)
- Mengelola data user
- Menggunakan *Sidebar Navigation* interaktif
- Menerapkan konsep Props, State, dan Hooks secara terintegrasi

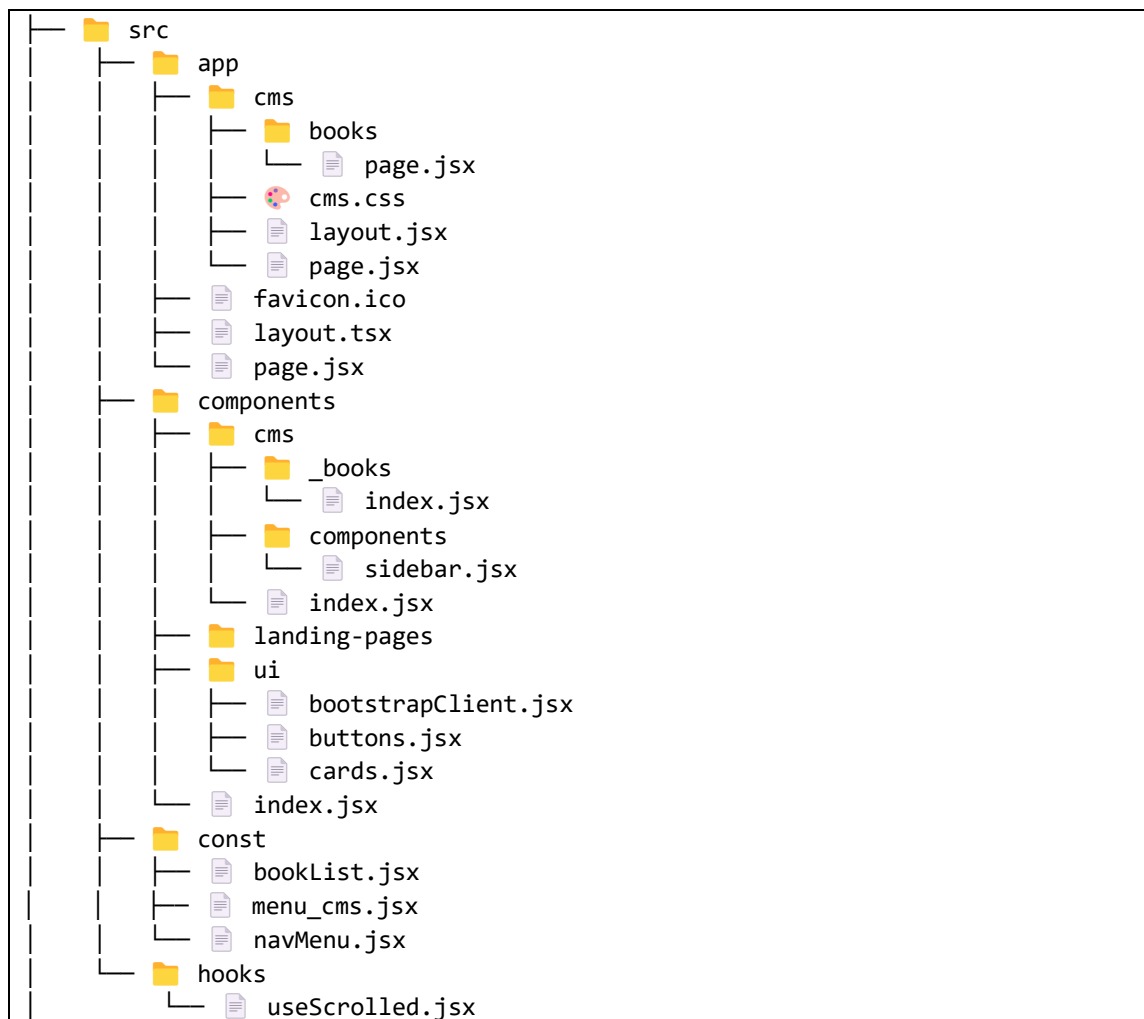
Latihan ini bertujuan agar mahasiswa tidak hanya memahami teori, tetapi mampu menerapkan konsep React dalam studi kasus nyata berbasis sistem manajemen.



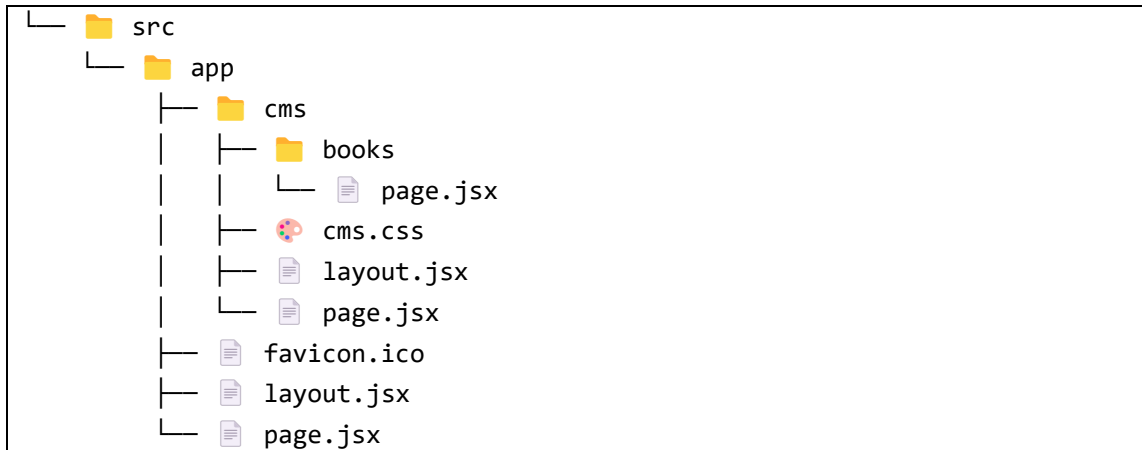
Gambar 6.3 UI Web Apps CMS Book Management

Langkah 1: Mempersiapkan struktuk folder

Buatlah struktur folder dan file seperti dibawah ini untuk menentukan root dan komponen JSX:



A. App Router



Struktur ini menggunakan konsep App Router yang berbasis *file-system routing* (routing otomatis berdasarkan folder).

Dalam App Router:

- Folder = Route (URL)
- page.jsx = Halaman
- layout.jsx = Layout pembungkus
- Layout bisa bersifat nested (bertingkat)

Semua routing dimulai dari folder *app/* sebagai root aplikasi.

➤ Root Level (app/)

📄 app/page.jsx -> Ini adalah halaman root dengan route: <http://localhost:3000/>

📄 app/layout.jsx -> Ini adalah Root Layout Global. Semua halaman di dalam folder *app/* akan dibungkus oleh layout ini.

➤ Folder *cms/*

📁 cms Folder ini membuat route baru <http://localhost:3000/cms>

📄 cms/page.jsx -> Halaman utama direktori CMS dengan route */cms*

📄 cms/layout.jsx -> Ini adalah Nested Layout khusus untuk route */cms*. Artinya, semua halaman di dalam */cms* akan dibungkus oleh layout ini.

Biasanya layout CMS berisi:

- Sidebar
- Navbar khusus admin
- Struktur dashboard

```
export default function CMSLayout({ children }) {  
  return (  
    <>  
      <div className="cms-container">  
        <Sidebar />  
        <main className="main-content p-4">{children}</main>  
      </div>  
      <Modals />  
    </>  
  );  
}
```

 `cms.css` -> CSS khusus untuk area CMS yang diimport pada `cms/layout.jsx`

➤ Folder Hierarki Rendering `/cms/books`

 `cms/books/page.jsx` -> Ini adalah halaman root dengan route:

<http://localhost:3000/cms/books>

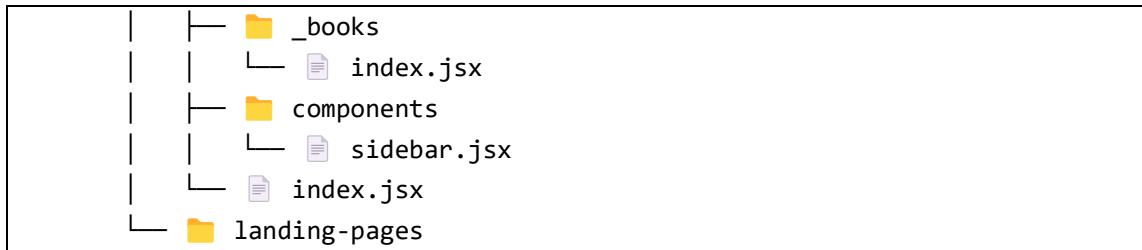
```
Root Layout  
├── Home Page  
└── CMS Layout  
    ├── CMS Page  
    └── Books Page
```

Struktur folder tersebut menunjukkan:

- *Root Layout* membungkus seluruh aplikasi.
- Folder *cms/* memiliki layout sendiri (*nested*).
- Folder *books/* mewarisi layout CMS.
- *Routing* otomatis berdasarkan folder.
- *Rendering* bertingkat dari *root* → *nested* → *page*.

B. Component Architecture Layer

```
└── src  
    └── components  
        └── cms
```

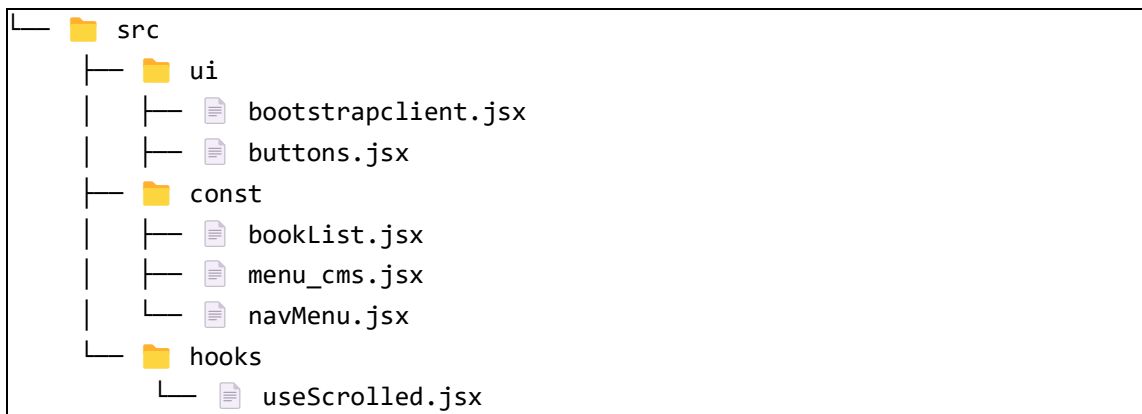


Dalam ekosistem Next.js dan React, folder *components/* biasanya berfungsi sebagai:

- Tempat menyimpan *reusable* UI *components* yang digunakan oleh berbagai halaman di dalam folder *app/*.

Struktur ini bukan *routing*, melainkan arsitektur komponen (*Component-Based Architecture*)

C. Layered Architecture



Struktur ini disebut:

- Layered Modular Frontend Architecture
- Dan menunjukkan bahwa project Anda sudah:
- Menggunakan separation of concerns
- Menggunakan custom hooks
- Menggunakan reusable UI system
- Menggunakan konfigurasi berbasis data

ui

Digunakan untuk *pure reusable UI* Berisi komponen UI generik yang reusable dan tidak terikat domain tertentu.

 bootstrapclient.jsx

```
'use client';
```

```
import { useEffect } from 'react';

export default function BootstrapProvider({ children }) {
  useEffect(() => {
    import('bootstrap/dist/js/bootstrap.bundle.min.js');
  }, []);

  return <>{children}</>;
}
```

 buttons.jsx

```
import React from "react";

const Button = ({
  children,
  variant = "primary",
  size = "md",
  outline = false,
  icon,
  iconPosition = "left",
  disabled = false,
  loading = false,
  fullWidth = false,
  className = "",
  type = "button",
  ...props
}) => {
  const variantClass = outline ? `btn-outline-${variant}` : `btn-${variant}`;
  const sizeClass = size !== "md" ? `btn-${size}` : "";
  const widthClass = fullWidth ? "w-100" : "";
  const disabledClass = disabled || loading ? "disabled" : "";

  const buttonClasses = `btn ${variantClass} ${sizeClass} ${widthClass} ${disabledClass} ${className}`.trim();

  return (
    <button
      type={type}
      className={buttonClasses}
      disabled={disabled || loading}
      {...props}
    >
      {loading ? (
        <>
          <span
            className="spinner-border spinner-border-sm me-2"
            role="status"
          />
        </>
      ) : children}
    </button>
  );
}
```

```

        aria-hidden="true"
      ></span>
      Loading...
    </>
  ) : (
    <>
      {icon && iconPosition === "left" && (
        <i className={`bi bi-${icon} me-2`} ></i>
      )}
      {children}
      {icon && iconPosition === "right" && (
        <i className={`bi bi-${icon} ms-2`} ></i>
      )}
    </>
  )}
</button>
);
}
export {Button}

```

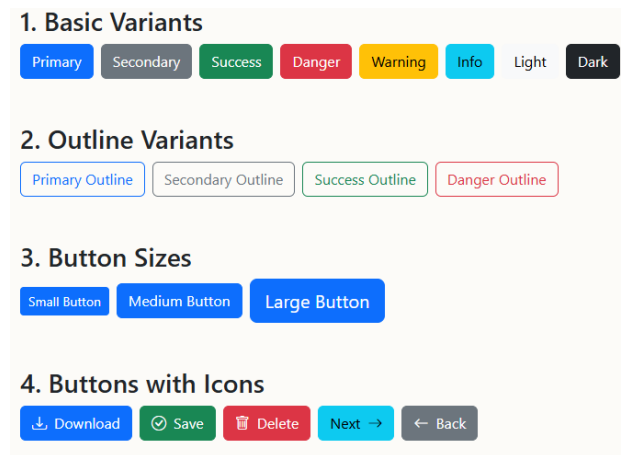
Komponen custom Button diatas merupakan komponen React yang dapat digunakan kembali (*reusable*) untuk membuat tombol dengan berbagai variasi styling dan fitur menggunakan Bootstrap classes. Seperti contoh dibawah ini, dengan cukup menggunakan komponen `<Button />` dengan menggunakan beberapa variasi property dapat menghasilkan bentuk button yang beragam.

Cara penggunaan dari komponen Button seperti dibawah ini:

```

import { Button } from "@components/ui/buttons";
...
return (
  <>
    <Button variant="primary">Primary</Button>
    <Button variant="primary" outline>Primary Outline</Button>
    <Button variant="primary" size="lg">Large Button</Button>
    <Button variant="success" icon="check-circle" iconPosition="left">
      Save
    </Button>
  </>
)

```



Gambar 6.4 Varian Button Komponen

const

Menyimpan data statis, konfigurasi, atau dummy data. Seperti halnya pada `menu_cms.jsx` menyimpan daftar menu statis.

```
const Menu_CMS = [
  {
    name: 'Dashboard',
    path: '/cms',
    icon: 'bi-speedometer2'
  },
  {
    name: 'Books',
    path: '/cms/books',
    icon: 'bi-book'
  },
  {
    name: 'Users',
    path: '/cms/users',
    icon: 'bi-people'
  }
];

export {Menu_CMS};
```

hooks

Menyimpan custom hook yang reusable. Seperti `useScrolled.jsx` berisi hook untuk mendeteksi scroll.

useScrolled.jsx

```
"use client";
import { useState, useEffect } from "react";
```

```

export default function useScrolled(threshold = 10) {
  const [scrolled, setScrolled] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      const isScrolled = window.scrollY > threshold;
      if (isScrolled !== scrolled) {
        setScrolled(isScrolled);
      }
    };

    window.addEventListener("scroll", handleScroll);

    return () => {
      window.removeEventListener("scroll", handleScroll);
    };
  }, [scrolled, threshold]);

  return scrolled;
}

```

useScrolled adalah custom *React Hook* yang digunakan untuk mendeteksi apakah pengguna telah melakukan scroll pada halaman web melewati ambang batas (*threshold*) tertentu.

- **threshold:** Nilai dalam pixel yang menentukan seberapa jauh pengguna harus scroll sebelum hook mengembalikan true
- **Default value:** 10 pixel
- Dapat disesuaikan saat memanggil hook, misalnya: `useScrolled(50)` untuk threshold 50 pixel

Cara kerja dari *useScrolled* ini:

- Navbar yang berubah saat scroll
 - Mengubah background navbar dari transparan ke solid
 - Mengurangi ukuran navbar saat scroll
- Tombol "Back to Top"

Menampilkan tombol hanya setelah user scroll ke bawah
- Animasi elemen

Trigger animasi berdasarkan posisi scroll
- Sticky header

Mengubah style header saat user mulai scroll

Langkah 2: Membangun komponen SideBar

Sidebar adalah komponen navigasi vertikal yang biasanya ditempatkan di sisi kiri atau kanan halaman untuk menampilkan menu dan struktur navigasi aplikasi. Dalam aplikasi berbasis React atau Next.js, sidebar sering dibuat sebagai reusable component yang menerima data menu melalui props agar dinamis. Sidebar membantu meningkatkan pengalaman pengguna dengan menyediakan akses cepat ke berbagai halaman atau fitur tanpa harus kembali ke halaman utama.

Sidebar.jsx

```
"use client"

import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { useState } from 'react';
import { Menu_CMS } from '@/const/menu_cms';

export default function Sidebar() {
  const pathname = usePathname();
  const [isCollapsed, setIsCollapsed] = useState(false);
  const [isHovered, setIsHovered] = useState(false);

  const toggleSidebar = () => {
    setIsCollapsed(!isCollapsed);
  };

  const handleMouseEnter = () => {
    if (isCollapsed) {
      setIsHovered(true);
    }
  };

  const handleMouseLeave = () => {
    setIsHovered(false);
  };

  const isExpanded = !isCollapsed || isHovered;

  return (
    <aside
      className={`sidebar ${isExpanded ? 'collapsed' : ''}`}
      onMouseEnter={handleMouseEnter}
      onMouseLeave={handleMouseLeave}
    >
      <div className="sidebar-header">
```

```

        {isExpanded && (
          <h4>
            <i className="bi bi-gear-fill me-2"></i>
            CMS Ready<sup>+</sup>
          </h4>
        )}
        <button
          className="toggle-btn"
          onClick={toggleSidebar}
          aria-label="Toggle Sidebar"
        >
          <i className={`bi ${isCollapsed ? 'bi-chevron-
right' : 'bi-chevron-left'}`}></i>
        </button>
      </div>
      <ul className="sidebar-menu">
        {Menu_CMS.map((item) => (
          <li key={item.path}>
            <Link
              href={item.path}
              className={pathname === item.path ?
'active' : ''}
              data-title={item.name}
            >
              <i className={item.icon}></i>
              <span className="menu-
text">{item.name}</span>
            </Link>
          </li>
        ))}
      </ul>
    </aside>
  );
}

```

Penjelasan:

1. “use client”

Baris ini menandakan bahwa komponen Sidebar adalah Client Component pada Next.js. Artinya, komponen ini dijalankan di browser karena menggunakan state dan interaksi pengguna (event handler), yang tidak bisa dijalankan di Server Component.

2. Manajement State

Komponen ini memiliki dua state:

- *isCollapsed*: menentukan apakah sidebar dalam kondisi:
 - Terbuka (false)

- Tertutup / mengecil (true)
- *isHovered*: digunakan untuk efek interaktif, jika sidebar dalam kondisi collapse lalu mouse diarahkan ke sana, sidebar bisa terbuka sementara.

3. Fungsi Interaksi

handleMouseEnter() dan *handleMouseLeave()*

Digunakan untuk mendeteksi hover:

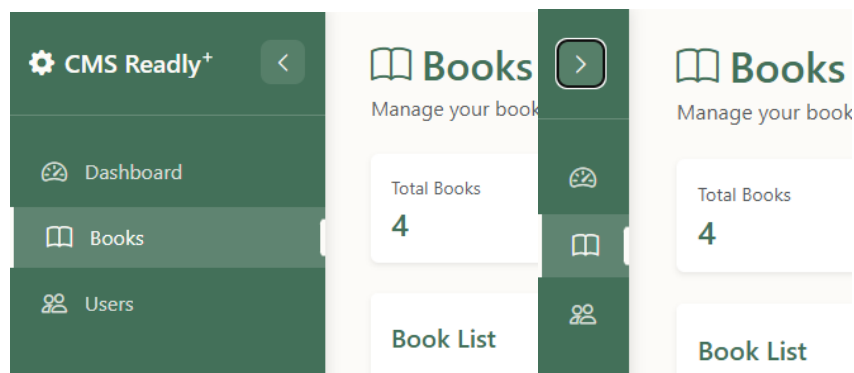
- Jika sidebar sedang *collapse* lalu mouse masuk → sementara diperbesar.
- Saat mouse keluar → kembali collapse.

4. Komponen Sidebar ini menunjukkan penerapan:

- *Client Component* pada Next.js
- Penggunaan *useState* untuk manajemen UI interaktif
- *Conditional rendering*
- *Dynamic rendering* dengan *.map()*
- *Active route detection* menggunakan *usePathname()*
- Pemisahan data dan UI (menu berasal dari file konstanta)

5. Secara konsep, komponen ini mengimplementasikan:

- *Component-Based Architecture*
- *Separation of Concerns*
- *State-driven UI*



Gambar 6.5 Tampilan Sidebar Komponen

Komponen Sidebar adalah navigasi samping (*side navigation*) yang dapat di-*collapse* atau *expand* dengan fitur hover untuk menampilkan menu saat dalam mode *collapsed*. Komponen ini digunakan untuk sistem CMS (*Content Management System*).

Langkah 3: Membuat komponen UI global

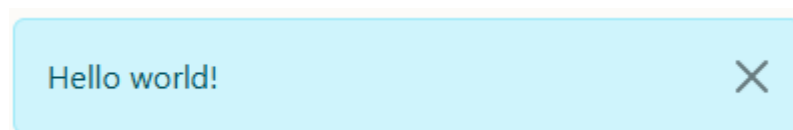
📁 ui/alert.jsx

```
const Alert = ({ message, variant = 'warning' }) => {
  return (
    <div
      className={`alert alert-${variant} alert-dismissible fade
show` }
      role="alert">
      {message}
      <button
        type="button"
        className="btn-close"
        data-bs-dismiss="alert"
        aria-label="Close">
      </button>
    </div>
  )
}

export { Alert }
```

Komponen Alert ini adalah komponen notifikasi/peringatan yang digunakan untuk menampilkan pesan kepada pengguna dengan berbagai tingkat kepentingan. Untuk menggunakan komponen Alert dapat dipanggil dengan cara:

```
import { Alert } from "@components/ui/alerts";
...
return (
  <Alert message={"Hello World"} variant="info" />
)
```



Gambar 6.6 Tampilan Komponen Alert

Komponen ini berfungsi untuk:

1. Menampilkan pesan informasi kepada pengguna (*success*, *error*, *warning*, *info*, *danger*)

2. Memberikan feedback visual terhadap aksi yang dilakukan pengguna
3. Dapat ditutup oleh pengguna melalui tombol close (×)

📁 ui/forms.jsx

```
const ALLOWED_PATTERN = /^[a-zA-Z0-9\s.,!@#%&*()\-_+=:;'"<>\[\]\{\}|\~`]*$/;

const validateInput = (value) => {
  return ALLOWED_PATTERN.test(value);
};

const sanitizeInput = (value) => {
  return value.replace(/^[a-zA-Z0-9\s.,!@#%&*()\-_+=:;'"<>\[\]\{\}|\~`]/g, '');
};

const LableTitle = ({ title, required }) => {
  return (
    <label
      className={`form-label fw-semibold ${required && 'required'}}`>
      {title}
    </label>
  )
}

const TextInput = ({ title, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title && (
        <LableTitle title={title} required={required} />
      )}
      <input
        type="text"
        required={required}
        className="form-control" {...props} />
    </div>
  )
}

const TextAreaInput = ({ title, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title && (
        <LableTitle title={title} required={required} />
      )}
      <textarea
        required={required}
        className="form-control"
        {...props}></textarea>
    </div>
  )
}
```

```

    </div>
  )
}

const InputCheckbox = ({ title, value, required, is_switch = false,
...props }) => {
  return (
    <div className="form-group">
      {title && (
        <LabelTitle title={title} required={required} />
      )}
      <div className={`"form-check ${is_switch ? 'form-switch' :
''}`}>
        <input
          type="checkbox"
          className="form-check-input"
          {...props}
        />
        <label className="form-check-label ms-1">
          {value}
        </label>
      </div>
    </div>
  )
}

const InputImage = ({ title, imagePreview, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title && (
        <LabelTitle title={title} required={required} />
      )}
      <input
        type="file"
        className="form-control"
        id="coverImage"
        name="coverImage"
        accept="image/*"
        required={required}
        {...props}
      />
      {imagePreview && (
        <div className="mt-3">
          <img
            src={imagePreview}
            alt="Cover Preview"
            style={{ maxWidth: '200px', maxHeight: '300px',
objectFit: 'cover' }}
            className="img-thumbnail"
          />
        </div>
      )}
    </div>
  )
}

```

```

    </div>
  )
}

export { validateInput, sanitizeInput, TextInput,
  TextAreaInput, InputCheckbox, InputImage }

```

File *forms.jsx* ini berisi komponen-komponen form reusable yang digunakan untuk membuat input form dengan styling Bootstrap yang konsisten di seluruh aplikasi.

Cara memanggil komponen pada *form.jsx*:

```

import { TextInput } from '@components/ui/forms'
...
return (
  <TextInput
    title="Book Title"
    name="bookTitle"
    value={formData.bookTitle}
    onChange={handleInputChange}
    required
  />

  <TextAreaInput
    title="Sinopsis"
    name="sinopsis"
    value={formData.sinopsis}
    onChange={handleInputChange}
    rows={2}
    required
  />
)

```

Author Name *

Sinopsis *

Gambar 6.7 Tampilan Komponen TextInput dan TextArea

ui/cards.jsx

```

import React from 'react'
import { Alert } from "@components/ui/alerts";

```

```

const Cards = ({ children }) => {
  let header, toolbar, body, footer;

  React.Children.forEach(children, child => {
    if (child.type === Cards.Header) {
      header = child;
    } else if (child.type === Cards.Toolbar) {
      toolbar = child;
    } else if (child.type === Cards.Body) {
      body = child;
    } else if (child.type === Cards.Footer) {
      footer = child;
    }
  });

  try {
    return (
      <div className="card card-xl-stretch mb-4 shadow-sm">
        {header && (
          <div className="card-header border-0 py-0 bg-
white">
            {header}
            {toolbar && (
              <div className="card-toolbar">
                {toolbar}
              </div>
            )}
          </div>
        )}

        {body}
        {footer && footer}
      </div>
    );
  } catch (error) {
    return <Alert messages={error.message} title={`Card`} />
  }
};

Cards.Header = ({ children }) => (
  <div className="d-flex align-items-start justify-content-between
mb-0">
    {children}
  </div>
);

Cards.Toolbar = ({ children }) => (
  <div>
    {children}
  </div>
);

```

```

Cards.Body = ({ children, className }) => {
  let class_name = `card-body ${className}`
  return (
    <div className={class_name}>
      {children}
    </div>
  )
};

Cards.Footer = ({ children, className }) => (
  <div className={`card-footer ${className}`}>
    {children}
  </div>
);

export {Cards}

```

Komponen Cards ini adalah komponen kartu modular yang menggunakan pola *Compound Component Pattern* untuk membuat card Bootstrap yang fleksibel dan terstruktur. Berikut cara menggunakan komponen Cards:

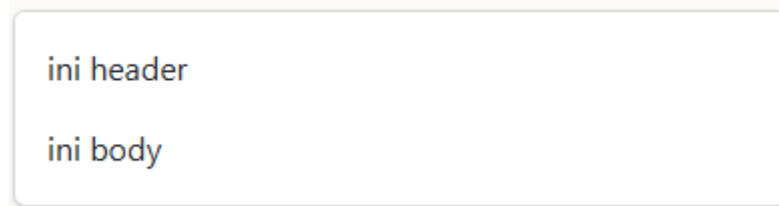
```

import { Cards } from "@components/ui/cards"
...
return (
  <Cards>
    <Cards.Header>ini header</Cards.Header>
    <Cards.Body>ini body</Cards.Body>
  </Cards>
)

```

Konsep Utama: Compound Component Pattern

Komponen ini menggunakan pola di mana komponen utama (*Cards*) memiliki sub-komponen (*Cards.Header*, *Cards.Body*, dll) yang bekerja bersama-sama.



Gambar 6.8 Tampilan Penggunaan komponen Card

📁 ui/modals.jsx

Komponen Modals ini adalah komponen modal atau popup global yang dapat dipanggil dari mana saja dalam aplikasi tanpa perlu merender komponen modal di setiap halaman.

Menggunakan pola **Singleton State Management**.

```
"use client"

import React, { useState } from "react";
import { Modal } from "react-bootstrap";
import { Button } from "@components/ui/buttons";

const states = {
  setState: null,
  changeState(data) {
    if (!this.setState) return;
    this.setState((prevData) => {
      return {
        ...prevData,
        ...data,
      };
    });
  },
};

const handleClose = () => {
  states.changeState({
    open: false,
  });
};

const Modals = () => {
  const [data, setData] = useState({
    open: false,
    header: "ini header",
    message: "ini message",
    size: "md",
    footer: "",
    onClose: handleClose,
    closable: true,
  });

  states.setState = setData;

  const styles = `
    .modal-backdrop-dark {
      opacity: 0.8 !important;
      background-color: #000;
    }
  `;

  return (
    <>
```

```

<style>{styles}</style>
<Modal
  show={data.open}
  onHide={data.closable ? data.onClose : undefined}
  size={data.size}
  backdrop="static"
  keyboard={false}
>
  {data.header && (
    <Modal.Header closeButton={false}>
      <h3 className="modal-title">{data.header}</h3>
      <button
        onClick={data.onClose}
        className="btn-close btn-close-white"
      >
        <i className="bi bi-x-lg"></i>
      </button>
    </Modal.Header>
  )}
  <Modal.Body>
    {!data.header && data.closable ? (
      <div className="">
        <button
          onClick={data.onClose}
          className="btn btn-sm btn-light btn-icon rounded-circle
position-absolute end-0 top-0 fw-bolder m-2 text-hover-danger"
        >
          <i className="bi bi-x-lg"></i>
        </button>
        {data.message}
      </div>
    ) : (
      data.message
    )}
  </Modal.Body>
  {data.footer && (
    <Modal.Footer>
      {data.closable && (
        <Button
          variant="secondary"
          outline
          className="px-5"
          onClick={() => handleEdit(book)}
          title="Edit"
        >
          No
        </Button>
      )}
      {data.footer}
    </Modal.Footer>
  )}
</Modal>

```

```

    </>
  );
};

const openModal = ({
  open = true,
  message,
  header,
  size,
  footer,
  onClose = () => { },
  closable = true,
}) => {
  states.changeState({
    message,
    header,
    size,
    open,
    footer,
    closable,
    onClose: closable
      ? () => {
        onClose();
        handleClose();
      }
      : undefined,
  });
};

export default Modals;
export { openModal };

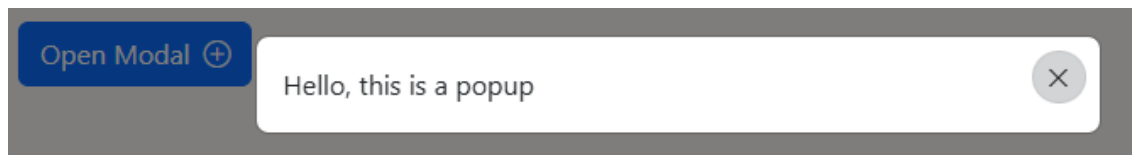
```

Penggunaan dari komponen modal seperti dibawah ini:

```

import { openModal } from "@components/ui/modals";
...
return (
  <Button
    variant="primary"
    onClick={()=>openModal({message:"Hello, this is a popup",
    size:"lg"})}
  >
    <i className="bi bi-plus-circle"></i>
    Open Modal
  </Button>
)

```



Gambar 6.9 Tampilan Modal Pop Up

ui/datatables.jsx

```
import React, { useState } from 'react'
import React, { useEffect, useMemo, useState } from 'react'
import { Pagination } from 'react-bootstrap';

const HeaderDatatables = ({ headers, onSorting, ...others }) => {
  const [sortingField, setSortingField] = useState("");
  const [sortingOrder, setSortingOrder] = useState("asc");
  const onSortingChange = (field) => {
    const order =
      field === sortingField && sortingOrder === "asc" ? "desc" :
      "asc";
    setSortingField(field);
    setSortingOrder(order);
    onSorting(field, order);
  };

  return (
    <thead>
      <tr className="text-start fw-bold text-uppercase gs-0"
{...others}>
        {headers.map(({ name, field, sortable }) => (
          <th
            className={"text-secondary fs-6 " +
              (sortable === true && "cursor-
                pointer")}
            key={name}
            onClick={() => (sortable ? onSortingChange(field) :
null)} >
              {name}
              {sortingField && sortingField === field && (
                <i className={"fs-6 ms-1 text-secondary "+
                  (sortingOrder === "asc"
                    ? "bi bi-sort-up"
                    : "bi bi-sort-down")}
                />
              )}
            )}
          </th>
        )}
      </tr>
    </thead>
  );
};

const SearchInput = ({ keyword, onAction }) => {
```

```

    return (
      <div className="input-group">
        <span className="input-group-text bg-white border-end-0">
          <i className="bi bi-search"></i>
        </span>
        <input
          type="text"
          className="form-control border-start-0"
          placeholder="Search here..."
          value={keyword}
          onChange={onAction}
        />
      </div>
    )
  };

const PaginationComponent = ({
  total = 0,
  itemsPerPage = 10,
  currentPage = 1,
  onPageChange,
  maxPageItems = 10
}) => {
  const [totalPages, setTotalPages] = useState(0);

  useEffect(() => {
    if (total > 0 && itemsPerPage > 0)
      setTotalPages(Math.ceil(total / itemsPerPage));
  }, [total, itemsPerPage]);

  const paginationItems = useMemo(() => {
    const pages = [];

    if (totalPages <= maxPageItems) {
      for (let i = 1; i <= totalPages; i++) {
        pages.push(
          <Pagination.Item
            key={i}
            active={i === currentPage}
            onClick={() => onPageChange(i)}
          >
            {i}
          </Pagination.Item>
        );
      }
    } else {
      let startPage = Math.max(1, currentPage - Math.floor(maxPageItems /
2));
      let endPage = Math.min(totalPages, startPage + maxPageItems - 1);

      if (endPage === totalPages) {
        startPage = Math.max(1, endPage - maxPageItems + 1);
      }

      if (startPage > 1) {

```

```

        pages.push(
            <Pagination.Item
                key={1}
                active={1 === currentPage}
                onClick={() => onPageChange(1)}
            >
                1
            </Pagination.Item>
        );
        if (startPage > 2) {
            pages.push(
                <Pagination.Ellipsis key="ellipsis-start" disabled />
            );
        }
    }
    for (let i = startPage; i <= endPage; i++) {
        pages.push(
            <Pagination.Item
                key={i}
                active={i === currentPage}
                onClick={() => onPageChange(i)}
            >
                {i}
            </Pagination.Item>
        );
    }
    if (endPage < totalPages) {
        if (endPage < totalPages - 1) {
            pages.push(
                <Pagination.Ellipsis key="ellipsis-end" disabled />
            );
        }

        pages.push(
            <Pagination.Item
                key={totalPages}
                active={totalPages === currentPage}
                onClick={() => onPageChange(totalPages)}
            >
                {totalPages}
            </Pagination.Item>
        );
    }
}

return pages;
}, [totalPages, currentPage, maxPageItems]);

if (totalPages === 0) return null;

return (
    <Pagination>
        <Pagination.First
            onClick={() => onPageChange(1)}
            disabled={currentPage === 1}

```

```

        />
        <Pagination.Prev
          onClick={() => onPageChange(currentPage - 1)}
          disabled={currentPage === 1}
        />
        {paginationItems}
        <Pagination.Next
          onClick={() => onPageChange(currentPage + 1)}
          disabled={currentPage === totalPages}
        />
        <Pagination.Last
          onClick={() => onPageChange(totalPages)}
          disabled={currentPage === totalPages}
        />
      </Pagination>
    );
  };
}

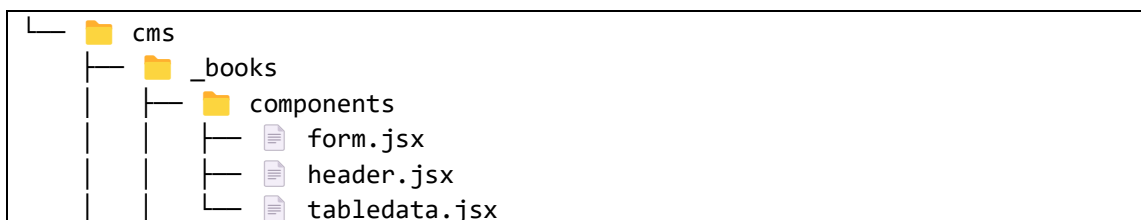
export { HeaderDatatables, SearchInput, PaginationComponent}

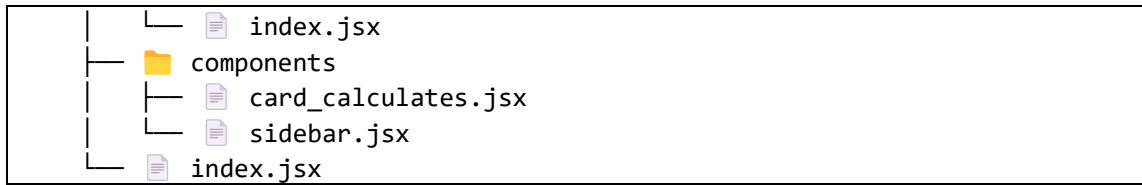
```

File *datatables.jsx* berisi komponen-komponen untuk membuat tabel data interaktif dengan fitur sorting, pencarian dan paging. Komponen ini dirancang untuk membuat tabel yang *user-friendly* dan fungsional.

Langkah 5: Membangun module management Buku

Pada tahap ini akan dibuat struktur folder komponen untuk fitur *Content Management System* (CMS) agar pengembangan aplikasi lebih terorganisir dan mudah dikelola. Dalam pengembangan menggunakan React, pengelompokan komponen berdasarkan fitur sangat penting untuk menjaga kerapian kode dan memudahkan proses pengembangan. Oleh karena itu, dibuat folder *cms* yang berisi seluruh komponen yang berkaitan dengan halaman CMS, termasuk navigasi sidebar dan komponen tampilan lainnya. Di dalam folder tersebut juga terdapat folder *_books* yang digunakan khusus untuk fitur manajemen buku, sehingga seluruh komponen yang berhubungan dengan pengelolaan data buku dapat dikelompokkan dalam satu modul yang sama.





Pada modul `_books`, dibuat folder `components` yang berisi komponen-komponen penyusun halaman seperti `form.jsx` untuk input data buku, `header.jsx` untuk bagian judul halaman, dan `tabledata.jsx` untuk menampilkan daftar buku dalam bentuk tabel. Selain itu, pada folder `cms` juga terdapat folder `components` yang menyimpan komponen umum seperti `sidebar.jsx` sebagai navigasi utama dan `card_calculates.jsx` untuk menampilkan informasi ringkasan data. Struktur ini juga didukung oleh file `index.jsx` pada beberapa folder yang berfungsi sebagai penghubung komponen agar dapat diimpor dengan lebih mudah pada halaman utama CMS yang dibangun menggunakan Next.js.

`card_calculates.jsx`

```

import { Cards } from "@components/ui/cards"

const CardCalculates = ({ title, value, icon }) => {
  return (
    <Cards>
      <Cards.Body>
        <div className="d-flex justify-content-between align-items-center">
          <div>
            <p className="text-muted mb-1 small">{title}</p>
            <h4
              className="mb-0"
              style={{ color: '#437059' }}>
              {value}
            </h4>
          </div>
          <div
            className="rounded-circle text-center"
            style={{
              backgroundColor: 'rgba(67, 112, 89, 0.1)',
              width: 40,
              height: 40 }
          >
            <i
              className={`bi bi-${icon} fs-4`}
              style={{ color: '#437059' }}></i>
          </div>
        </div>
      </Cards.Body>
    </Cards>
  )
}

```

```

        </div>
      </div>
    </Cards.Body>
  </Cards>
)
}
export { CardCalculates }

```

 header.jsx

```

import React from 'react'
import { Button } from "@components/ui/buttons";

export function Header({ handleAdd }) {
  return (
    <div className="row mb-4">
      <div className="col-12">
        <div className="d-flex justify-content-between align-items-center flex-wrap gap-3">
          <div>
            <h2 className="mb-1 text-green">
              <i className="bi bi-book me-2"></i>
              Books Management
            </h2>
            <p className="text-muted mb-0">Manage your book collection</p>
          </div>
          <Button
            className="d-flex align-items-center gap-2"
            onClick={handleAdd}
            style={{ backgroundColor: '#437059',
borderColor: '#437059' }}
            >
            <i className="bi bi-plus-circle"></i>
            Add New Book
          </Button>
        </div>
      </div>
    </div>
  )
}

```

 tabledata.jsx

Komponen *TableData* adalah sebuah komponen React yang menampilkan data buku dalam bentuk tabel interaktif dengan fitur pencarian, sorting, dan pagination. Komponen ini menggunakan Bootstrap untuk styling dan beberapa komponen UI custom.

```

"use client";

import React, { useMemo, useState } from 'react'
import { Cards } from '@components/ui/cards';
import { Button } from "@components/ui/buttons";
import { HeaderDatatables, SearchInput, PaginationComponent } from
'@/components/ui/datatables';

export default function Tabledata({ data }) {
  const [search, setSearch] = useState("");
  const [sorting, setSorting] = useState({ field: "", order: "" });
  const [totalitems, setTotalItems] = useState(0);
  const [currentPage, setCurrentPage] = useState(1);
  const ITEMS_PER_PAGE = 10;

  const table_headers = [
    { name: "No", field: "id", sortable: false },
    { name: "Title", field: "title", sortable: true },
    { name: "Author", field: "author", sortable: true },
    { name: "Language", field: "language", sortable: true },
    { name: "Rate/View", field: "rate", sortable: false },
    { name: "Subscribe", field: "is_free", sortable: true },
    { name: "Actions", field: "id", sortable: false },
  ];

  const ResultData = useMemo(() => {
    let computedData = data;

    if (search) {
      computedData = computedData.filter((listData) => {
        return Object.keys(listData).some((key) => {
          try {
            const value = listData[key];
            return value != null &&
String(value).toLowerCase().includes(search.toLowerCase());
          } catch (error) {
            console.error(`Error processing key "${key}":`,
error);
            return false;
          }
        });
      });
    }

    setTotalItems(computedData.length);

    if (sorting.field) {
      const reversed = sorting.order === "asc" ? 1 : -1;
      computedData = computedData.sort(
        (a, b) => reversed *
a[sorting.field].localeCompare(b[sorting.field])
      );
    }

    if (computedData.length > 0) {

```

```

        return computedData.slice(
            (currentPage - 1) * ITEMS_PER_PAGE,
            (currentPage - 1) * ITEMS_PER_PAGE + ITEMS_PER_PAGE
        );
    } else {
        return [];
    }
}, [data, search, sorting, currentPage]);

return (
    <Cards>
        <Cards.Header>
            <span className="card-label fw-bold fs-3">Book Lists</span>
            <div className="w-50">
                <SearchInput
                    keyword={search}
                    onAction={(e) => setSearch(e.target.value)}
                />
            </div>
        </Cards.Header>
        <Cards.Body className={`px-0 pb-0`}>
            <div className="table-responsive">
                <table className="table table-hover">
                    <HeaderDatatables
                        headers={table_headers}
                        onSorting={(field, order) => setSorting({
field, order })}
                    />
                    <tbody>
                        {ResultData.length > 0 ? (
                            ResultData.map((book, index) => (
                                <tr key={book.id}>
                                    <td className="text-center">
                                        {(currentPage - 1) *
ITEMS_PER_PAGE + index + 1}
                                    </td>
                                    <td>
                                        <strong>{book.title}</strong>
                                    </td>
                                    <td>{book.author}</td>
                                    <td>{book.language}</td>
                                    <td>
                                        <div className="d-flex">
                                            <div className="me-3">
                                                <i className="bi bi-
star-fill text-warning"></i>
                                            <span className="text-
dark ms-1">
                                                {book.rating}
                                            </span>
                                        </div>
                                        <div className="me-3">
                                            <i className="bi bi-eye
text-info"></i>

```

```

                                <span className="text-
dark ms-1">{book.views}</span>
                                </div>
                                </div>
                                </td>
                                <td>
                                    <span className="badge bg-
secondary">
                                        {book.is_free ? "Yes" :
                                        "No"}
                                    </span>
                                </td>
                                <td className="text-end">
                                    <Button
                                        variant="warning"
                                        outline
                                        className="btn-sm me-2"
                                        onClick={() =>
                                            handleEdit(book)}
                                        title="Edit"
                                    >
                                        <i className="bi bi-
pencil"></i>
                                    </Button>
                                    <Button
                                        variant="danger"
                                        outline
                                        className="btn-sm me-2"
                                        onClick={() =>
                                            handleDelete(book.id)}
                                        title="Delete"
                                    >
                                        <i className="bi bi-
trash"></i>
                                    </Button>
                                </td>
                            </tr>
                        ))
                    ) : (
                        <tr>
                            <td colspan="7" className="text-center
py-5">
                                <i className="bi bi-inbox fs-1
text-muted d-block mb-3"></i>
                                <p className="text-muted mb-0">No
books found</p>
                            </td>
                        </tr>
                    )
                </tbody>
            </table>

            {totalitems > 0 && (
                <div className="d-flex align-items-center justify-
content-center">

```

```

        <PaginationComponent
          total={totalitems}
          itemsPerPage={ITEMS_PER_PAGE}
          currentPage={currentPage}
          onPageChange={(page) =>
setCurrentPage(page)}
        />
      </div>
    )}
  </div>
</Cards.Body>
</Cards>
)
}

```

Penjelasan:

Props:

- ``data``: Array of objects yang berisi data buku yang akan ditampilkan

State Management:

```

const [search, setSearch] = useState("");
// Menyimpan keyword pencarian
const [sorting, setSorting] = useState({ field: "", order: "" });
// Menyimpan field dan urutan sorting
const [totalitems, setTotalItems] = useState(0);
// Total item setelah filtering
const [currentPage, setCurrentPage] = useState(1);
// Halaman aktif saat ini
const ITEMS_PER_PAGE = 10;
// Jumlah item per halaman (konstanta)

```

Struktur Kolom Tabel:

```

const table_headers = [
  { name: "No", field: "id", sortable: false },
  { name: "Title", field: "title", sortable: true },
  { name: "Author", field: "author", sortable: true },
  { name: "Language", field: "language", sortable: true },
  { name: "Rate/View", field: "rate", sortable: false },
  { name: "Subscribe", field: "is_free", sortable: true },
  { name: "Actions", field: "id", sortable: false },
];

```

Struktur tersebut digunakan pada komponen tabel dinamis agar header tabel dapat dibuat secara otomatis menggunakan proses mapping. Dengan pendekatan ini, pengembang tidak perlu menuliskan header tabel satu per satu secara manual. Selain membuat kode

lebih ringkas, metode ini juga memudahkan proses pengelolaan tabel ketika jumlah kolom bertambah atau berubah.

Tabel 6.4 Deskripsi Kolom Tabel Headers

No	Name (Judul Kolom)	Field (Key Data)	Sortable	Penjelasan
1	No	id	false	Menampilkan nomor atau identitas data buku pada tabel dan tidak dapat diurutkan.
2	Title	title	true	Menampilkan judul buku dan dapat diurutkan berdasarkan abjad judul.
3	Author	author	true	Menampilkan nama penulis buku dan dapat diurutkan.
4	Language	language	true	Menampilkan bahasa yang digunakan dalam buku dan dapat diurutkan.
5	Rate/View	rate	false	Menampilkan informasi rating atau jumlah pembaca buku dan tidak digunakan untuk sorting.
6	Subscribe	is_free	true	Menunjukkan apakah buku dapat diakses gratis atau membutuhkan langganan, serta dapat diurutkan.
7	Actions	id	false	Digunakan untuk menampilkan tombol aksi seperti edit atau delete pada setiap baris data dan tidak dapat diurutkan.

Logika Pemrosesan Data (useMemo):

1. Filtering

```

if (search) {
  computedData = computedData.filter((listData) => {
    return Object.keys(listData).some((key) => {
      try {
        const value = listData[key];
        return value != null &&
String(value).toLowerCase().includes(search.toLowerCase());
      } catch (error) {
        console.error(`Error processing key "${key}":`, error);
        return false;
      }
    });
  });
}

```

- Mencari keyword di semua field dalam setiap object
- Case-insensitive (tidak peduli huruf besar/kecil)

- Menangani error dengan try-catch untuk keamanan

2. Update total items

```
setTotalItems(computedData.length);
```

- Menghitung total data setelah filtering untuk pagination

3. Sorting

```
if (sorting.field) {  
  const reversed = sorting.order === "asc" ? 1 : -1;  
  computedData = computedData.sort(  
    (a, b) => reversed *  
    a[sorting.field].localeCompare(b[sorting.field])  
  );  
}
```

- Mengurutkan data berdasarkan field yang dipilih
- Mendukung ascending (A-Z) dan descending (Z-A)
- Menggunakan localeCompare untuk sorting string yang lebih baik

4. Pagination

```
if (computedData.length > 0) {  
  return computedData.slice(  
    (currentPage - 1) * ITEMS_PER_PAGE,  
    (currentPage - 1) * ITEMS_PER_PAGE + ITEMS_PER_PAGE  
  );  
} else {  
  return [];  
}
```

- Memotong data sesuai halaman aktif
- Menampilkan 10 item per halaman

 index.jsx

RFC MBooks adalah komponen utama (*parent component*) untuk halaman manajemen buku dalam CMS. Komponen ini berfungsi sebagai *container* yang mengatur dan menampilkan semua fitur terkait manajemen buku, termasuk statistik, form input, dan tabel data.

```

"use client";

import React, { useState } from "react";
import { ListBooks } from "@const/bookList";
import { CardCalculates } from "../components/card_calculates";
import { Header } from "../components/header";
import Tabledata from "../components/tabledata";

export function MBooks() {
  const [books, setBooks] = useState(ListBooks);

  return (
    <div className="container-fluid">
      <Header handleAdd={()=>alert('add new book')} />

      <div className="row">
        <div className="col-md-3">
          <CardCalculates
            title={`Total Books`}
            value={books.length}
            icon={`book`}
          />
        </div>
        <div className="col-md-3">
          <CardCalculates
            title={`Free Book`}
            value={books.filter((b) => !b.is_free).length}
            icon={`grid`}
          />
        </div>
        <div className="col-md-3">
          <CardCalculates
            title={`Subscribe`}
            value={books.filter((b) => b.is_free).length}
            icon={`calendar-event`}
          />
        </div>
        <div className="col-md-3">
          <CardCalculates
            title={`Subscribe`}
            value={books.filter((b) => b.author).length}
            icon={`people`}
          />
        </div>
      </div>

      <Tabledata data={books} />
    </div>
  );
}

export default MBooks;

```

MBooks adalah komponen orchestrator yang:

- Mengatur state management untuk data buku
- Menampilkan dashboard dengan statistik
- Menyediakan interface untuk CRUD operations
- Mengintegrasikan semua komponen child

Langkah 6: Memanggil komponen MBook pada root *cms/books*

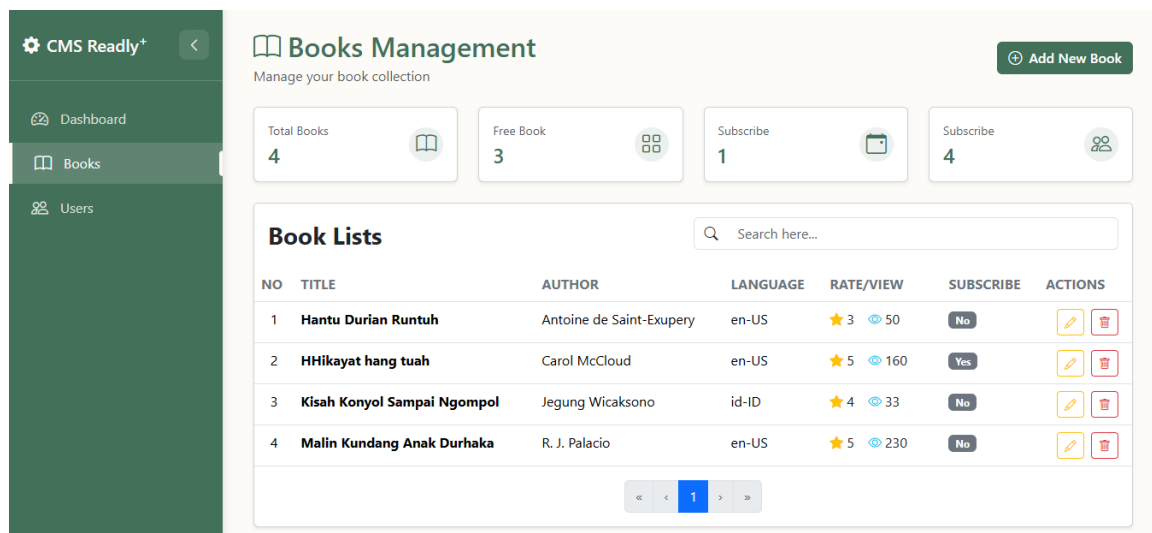
 cms/books/page.jsx

```
export { default } from "@components/cms/_books/index";
```

- Import default export dari file *@components/cms/_books/index.jsx* (yaitu komponen MBooks)
- Langsung export ulang sebagai default export dari file ini
- Membuat *page.jsx* sebagai *proxy/barrel* yang meneruskan export dari komponen sebenarnya

Langkah 7: Tampilan Halaman Books Management

Pada browser masukan address <http://localhost:3000/cms/books> dimana root ini akan mengarahkan pada komponen MBooks sesuai dengan Langkah 6.



Gambar 6.10 Halaman Books Management

Hasil Logika Pemrosesan Data (useMemo):

1. Pencarian : dapat mencari berdasarkan title, author ataupun attribute lainnya

Book Lists						
hantu						
NO	TITLE	AUTHOR	LANGUAGE	RATE/VIEW	SUBSCRIBE	ACTIONS
1	Hantu Durian Runtuh	Antoine de Saint-Exupery	en-US	★ 3 50	No	
<< < 1 > >>						

2. Sorting: Mengurutkan berdasarkan kolom HeaderTable yang telah diset nilai sorting

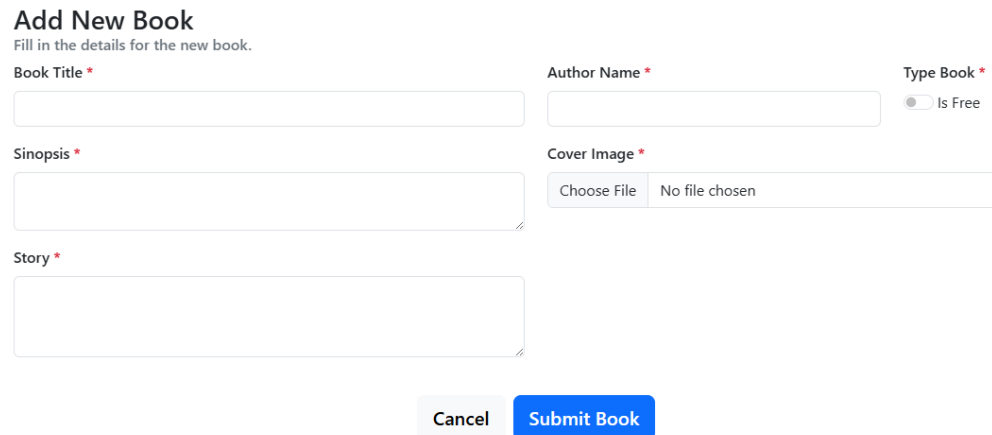
Book Lists						
Search here...						
NO	TITLE	AUTHOR	LANGUAGE	RATE/VIEW	SUBSCRIBE	ACTIONS
1	Malin Kundang Anak Durhaka	R. J. Palacio	en-US	★ 5 230	No	
2	Kisah Konyol Sampai Ngompol	Jegung Wicaksono	id-ID	★ 4 33	No	
3	HHikayat hang tuah	Carol McCloud	en-US	★ 5 160	Yes	
4	Hantu Durian Runtuh	Antoine de Saint-Exupery	en-US	★ 3 50	No	
<< < 1 > >>						

3. Pagination: Jika anda mengubah jumlah ITEMS_PER_PAGE atau menambahkan data array object ListBook menjadi lebih dari 10 maka akan menghasilkan bentuk number of page

Book Lists						
Search here...						
NO	TITLE	AUTHOR	LANGUAGE	RATE/VIEW	SUBSCRIBE	ACTIONS
3	HHikayat hang tuah	Carol McCloud	en-US	★ 5 160	Yes	
4	Hantu Durian Runtuh	Antoine de Saint-Exupery	en-US	★ 3 50	No	
<< < 1 2 > >>						

6.4 Tugas Pembelajaran

1. Buatlah halaman `form.jsx` yang berfungsi sebagai form input data buku pada modul CMS. Form tersebut harus memiliki beberapa field seperti *Book Title*, *Author Name*, *Type Book*, *Sinopsis*, *Story*, serta fitur *Upload Image*. Dalam proses pembuatannya, gunakan kembali komponen **UI Forms global** yang telah dibuat sebelumnya agar tampilan form tetap konsisten dan kode lebih modular.



2. Implementasikan sebuah tombol *Add New Book* pada halaman manajemen buku. Ketika tombol tersebut diklik, jalankan properti fungsi *handleAdd* untuk memanggil sebuah *popup modal* yang menampilkan komponen Form. Modal tersebut akan digunakan sebagai antarmuka untuk melakukan proses penambahan data buku baru (Add New Book) berdasarkan field yang telah dibuat pada komponen `form.jsx`.



3. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
4. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-MERN-STACK-NPM"** dengan nama commit **"Praktikum 6"**.
Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB VII

MEMBANGUN REST API DENGAN EXPRESS.JS

7.1 Tujuan Pembelajaran

1. Memahami konsep dasar REST API serta cara kerja komunikasi antara client dan server menggunakan protokol HTTP.
2. Memahami penggunaan Node.js sebagai lingkungan pengembangan backend aplikasi.
3. Mengimplementasikan framework Express.js untuk membangun endpoint REST API.
4. Memahami konsep ORM (*Object Relational Mapping*) dalam pengelolaan database menggunakan Sequelize.
5. Menggunakan fitur migration untuk membuat dan mengelola struktur tabel database secara terstruktur.
6. Mengembangkan endpoint API untuk melakukan operasi CRUD (*Create, Read, Update, Delete*) pada data aplikasi.
7. Mengorganisasi struktur proyek backend agar lebih modular dan mudah dipelihara selama proses pengembangan aplikasi.

7.2 Dasar Teori

7.2.1 REST API

REST (*Representational State Transfer*) adalah sebuah gaya arsitektur standar untuk membangun layanan web. Sedangkan API (*Application Programming Interface*) adalah perantara yang memungkinkan dua aplikasi untuk saling "berbicara". Singkatnya, REST API adalah cara bagi aplikasi untuk bertukar data secara terorganisir menggunakan protokol internet yang sudah ada, yaitu HTTP.

Dalam dunia REST API, hanya ada dua pemain utama yang selalu berkomunikasi:

- Client: Aplikasi yang meminta data (misalnya: browser di laptopmu, aplikasi mobile Instagram, atau script Python).
- Server: Komputer pusat yang menyimpan data dan memproses permintaan dari Client.

Proses ini bekerja melalui siklus *Request* (Permintaan) dan *Response* (Tanggapan).

A. *Request* (Dari Client ke Server)

Saat Client ingin sesuatu, ia mengirimkan paket data yang berisi:

1. Endpoint (URL): Alamat tujuan (Contoh: <https://api.ibik.com/student>).
2. HTTP Method: Kata kerja yang menunjukkan apa yang ingin dilakukan:
 - GET: Mengambil data.
 - POST: Mengirim data baru.
 - PUT: Memperbarui data yang sudah ada.
 - DELETE: Menghapus data.
3. Headers: Informasi tambahan seperti token keamanan (API Key) atau jenis data (JSON).
4. Body: Isi pesan (biasanya digunakan pada POST atau PUT untuk mengirim data detail).

B. *Response* (Dari Server ke Client)

Setelah menerima Request, Server akan memprosesnya dan memberikan jawaban berupa:

1. HTTP Status Code: Angka rahasia yang menunjukkan status permintaan:
 - 200 OK: Berhasil!
 - 201 Created: Data baru berhasil dibuat.
 - 404 Not Found: Alamat tidak ditemukan.
 - 500 Internal Server Error: Server sedang bermasalah.
2. Body (Data): Biasanya dalam format JSON.

7.2.2 Sequelize

Sequelize adalah salah satu ORM (*Object-Relational Mapping*) paling populer untuk Node.js. Jika sudah memahami konsep REST API, Sequelize adalah alat yang membantu "Server" berkomunikasi dengan "Database" tanpa harus menulis query SQL manual yang panjang.



Gambar 7.1 Logo Sequelize

Sequelize menjembatani kode JavaScript dengan database (seperti MySQL, PostgreSQL, atau SQLite).

- Tanpa Sequelize: Query sql ditulis seperti
SELECT * FROM users WHERE id = 1.
- Dengan Sequelize: Query sql cukup seperti User.findByPk(1).

A. Instalasi awal

Untuk memulai, perlu menginstal library Sequelize dan driver database yang digunakan.

```
npm install sequelize
```

B. Menghubungkan ke Database

Membuat instance Sequelize untuk mendefinisikan koneksi.

```
// Format: 'database', 'username', 'password'  
const sequelize = new Sequelize('nama_db', 'root', 'password', {  
  host: 'localhost',  
  dialect: 'mysql' // atau 'postgres', 'sqlite', dll.  
});
```

C. Konsep Model

Model adalah inti dari Sequelize. Model merepresentasikan sebuah tabel di database.

Jika memiliki tabel Users, maka buatlah model User.

```
const { DataTypes } = require('sequelize');  
  
const User = sequelize.define('User', {  
  // Kolom-kolom tabel  
  firstName: {  
    type: DataTypes.STRING,  
    allowNull: false  
  },  
  lastName: {  
    type: DataTypes.STRING
```

```
}  
});
```

D. Sinkronisasi Database

Agar model yang telah dibuat benar-benar menjadi tabel di database, diperlukan melakukan sinkronisasi:

```
await sequelize.sync({ force: true });  
console.log("Tabel berhasil dibuat!");
```

force: true akan menghapus tabel lama jika sudah ada. Gunakan dengan hati-hati!

E. Operasi Dasar (CRUD)

Setelah terhubung, selanjutnya mengelola data dengan sangat mudah:

- Create (Tambah Data): *User.create({ firstName: "Febry" })*
- Read (Ambil Data): *User.findAll()*
- Update (Ubah Data): *User.update({ lastName: "Fairuz" }, { where: { id: 1 } })*
- Delete (Hapus Data): *User.destroy({ where: { id: 1 } })*

7.2.2.1 Konsep Migration pada Database

Migration merupakan teknik untuk mengelola perubahan struktur database secara bertahap dan terkontrol selama proses pengembangan aplikasi. Konsep ini mirip dengan version control pada kode program, di mana setiap perubahan pada skema database dicatat dalam sebuah file migration sehingga perubahan tersebut dapat dilacak dan dikembalikan ke versi sebelumnya jika diperlukan.

Dengan migration, pengembang dapat memindahkan database dari satu kondisi ke kondisi lain melalui serangkaian perubahan yang tersimpan dalam file. Setiap migration menyimpan dua proses utama yaitu proses untuk menerapkan perubahan dan proses untuk membatalkan perubahan tersebut. Pendekatan ini sangat penting dalam pengembangan aplikasi backend karena struktur database sering mengalami perubahan seiring berkembangnya fitur aplikasi.

7.2.2.2 Sequelize CLI

Untuk mengelola migration pada proyek backend yang menggunakan Node.js, digunakan Sequelize CLI yang merupakan alat bantu berbasis command line untuk melakukan inisialisasi proyek, membuat model, migration, serta seed data.

Sequelize CLI dapat diinstal melalui perintah berikut:

```
npm install --save-dev sequelize-cli
```

Setelah terinstal, CLI dapat digunakan untuk melakukan inisialisasi struktur proyek menggunakan perintah:

```
npx sequelize-cli init
```

Perintah tersebut akan membuat beberapa folder utama dalam proyek seperti *config*, *models*, *migrations*, dan *seeders* yang digunakan untuk mengelola koneksi database, model data, perubahan struktur database, serta data awal aplikasi.

7.2.2.3 Struktur Folder Sequelize

Setelah menjalankan proses inisialisasi menggunakan Sequelize CLI, sistem akan membuat beberapa folder penting yang digunakan dalam pengembangan backend.

Beberapa folder tersebut antara lain:

-  **config**
Berisi file konfigurasi koneksi database seperti username, password, host, dan jenis database yang digunakan.
-  **models**
Berisi definisi model yang merepresentasikan tabel pada database dalam bentuk objek JavaScript.
-  **migrations**
Berisi file migration yang digunakan untuk membuat atau mengubah struktur tabel database.
-  **seeders**
Berisi file yang digunakan untuk mengisi data awal atau data contoh pada database.

Struktur folder ini membantu pengembang untuk memisahkan pengelolaan struktur database, model data, dan data awal aplikasi secara lebih terorganisir.

7.2.2.4 Struktur Folder Sequelize

Pada Sequelize, setiap file migration merupakan file JavaScript yang memiliki dua fungsi utama yaitu `up` dan `down`.

- `up` digunakan untuk menjalankan perubahan pada database seperti membuat tabel atau menambahkan kolom.
- `down` digunakan untuk mengembalikan perubahan tersebut ke kondisi sebelumnya.

Contoh struktur dasar file migration adalah sebagai berikut:

```
module.exports = {
  up: (queryInterface, Sequelize) => {

  },
  down: (queryInterface, Sequelize) => {

  },
};
```

Objek *queryInterface* digunakan untuk menjalankan perintah perubahan database seperti *createTable*, *addColumn*, atau *dropTable*, sedangkan objek *Sequelize* menyediakan berbagai tipe data seperti *STRING*, *INTEGER*, atau *BOOLEAN*.

7.2.2.5 Menjalankan Migration

Setelah file migration dibuat, perubahan tersebut belum diterapkan pada database. Untuk menjalankan migration digunakan perintah berikut:

```
npx sequelize-cli db:migrate
```

Ketika perintah ini dijalankan, Sequelize akan mencari semua file migration yang belum dijalankan dan mengeksekusinya secara berurutan. Sistem juga akan membuat tabel khusus bernama **SequelizeMeta** yang berfungsi untuk mencatat migration mana saja yang telah dijalankan pada database.

7.2.2.6 Membatalkan Migration

Jika terjadi kesalahan pada struktur database atau diperlukan pengembalian ke kondisi sebelumnya, migration dapat dibatalkan menggunakan perintah:

```
npx sequelize-cli db:migrate:undo
```

Perintah tersebut akan membatalkan migration terakhir yang telah dijalankan. Selain itu, pengembang juga dapat membatalkan seluruh migration menggunakan perintah:

```
npx sequelize-cli db:migrate:undo:all
```

Dengan fitur ini, perubahan struktur database dapat dikelola secara fleksibel selama proses pengembangan aplikasi backend.

7.2.2.7 Seeders pada Sequelize

Selain migration, Sequelize juga menyediakan fitur seeder yang digunakan untuk menambahkan data awal pada database. Seeder sering digunakan untuk mengisi data contoh seperti akun pengguna awal, kategori produk, atau data referensi lainnya.

Seeder dapat dibuat menggunakan perintah:

```
npx sequelize-cli seed:generate --name demo-data
```

Setelah file seeder dibuat, data dapat dimasukkan ke dalam database menggunakan perintah:

```
npx sequelize-cli db:seed:all
```

Seeder membantu proses pengujian aplikasi karena pengembang dapat dengan mudah menambahkan data awal tanpa harus memasukkan data secara manual

7.2.3 Mongoose

Mongoose merupakan ODM (*Object Data Modeling*) library untuk MongoDB yang digunakan pada lingkungan Node.js. Mongoose menyediakan mekanisme untuk mendefinisikan schema, melakukan validasi data, serta mempermudah proses interaksi antara aplikasi Node.js dengan database MongoDB.

Dengan menggunakan Mongoose, pengembang dapat mendefinisikan struktur dokumen database dalam bentuk schema JavaScript. Schema tersebut kemudian dikompilasi menjadi model yang dapat digunakan untuk melakukan berbagai operasi database seperti menyimpan data, membaca data, memperbarui data, dan menghapus data.



Gambar 7.2 Logo Mongoose

Selain itu, Mongoose juga menyediakan fitur seperti *middleware* (hooks), *validation*, *population*, dan *query builder* yang membantu pengembang dalam membangun aplikasi backend yang lebih terstruktur dan mudah dipelihara.

7.2.3.1 Struktur Dasar

Setiap dokumen di MongoDB harus memiliki skema. Skema ini menentukan "bentuk" data. Jika data yang masuk tidak sesuai, Mongoose akan menolak atau mencoba mengonversinya.

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: { type: String, required: true },
  isActive: { type: Boolean, default: true }
});
```

7.2.3.2 Validasi

Mongoose memungkinkan untuk menambahkan aturan ketat langsung di dalam skema tanpa perlu menulis logika validasi tambahan.

- *required*: Memastikan field harus diisi.
- *min / max*: Membatasi nilai angka.
- *enum*: Membatasi input agar hanya menerima nilai tertentu (misal: ['admin', 'user']).
- *match*: Menggunakan Regex untuk pola data (misal: format email).

```
const productSchema = new mongoose.Schema({
  status: {
    type: String,
    enum: ['draft', 'published'], // Hanya menerima salah satu dari
    ini
    default: 'draft'
  },
  price: {
    type: Number,
    min: 0 // Tidak boleh negatif
  }
});
```

7.2.3.3 Middleware (Hooks)

Mongoose mengizinkan untuk menyisipkan logika sebelum atau sesudah operasi database tertentu (seperti `.save()` atau `.find()`).

- *pre*: Menjalankan kode sebelum data disimpan (misal: hashing password).
- *post*: Menjalankan kode setelah data tersimpan (misal: mengirim notifikasi email).

7.2.3.4 Static and Methods

Menambahkan fungsi kustom ke dalam skema:

- *Methods*: Fungsi yang bisa dipanggil pada instance (dokumen tertentu). Contoh: `user.verifyPassword()`.
- *Statics*: Fungsi yang bisa dipanggil langsung dari Model. Contoh: `User.findByEmail()`.

7.2.3.5 Opsi Skema

Menambahkan konfigurasi tambahan di argumen kedua `new mongoose.Schema(definition, options)`. Salah satu yang paling sering dipakai adalah *timestamps*:

```
const blogSchema = new mongoose.Schema({
  title: String
}, {
  timestamps: true // Otomatis membuat field 'createdAt' dan
  'updatedAt'
});
```

7.3 Latihan Pembelajaran

Pada latihan pembelajaran ini, akan mempraktikkan proses pembangunan *backend* (BE) untuk mengelola data buku menggunakan database SQL. Pengembangan backend dilakukan menggunakan Node.js dengan framework Express.js, serta memanfaatkan *ORM Sequelize* untuk menghubungkan aplikasi dengan database. Penggunaan ORM bertujuan untuk mempermudah pengelolaan database tanpa harus menuliskan query SQL secara langsung.

Pada akhir latihan, diharapkan mampu membangun beberapa endpoint REST API seperti menampilkan daftar buku, menambahkan buku baru, memperbarui data buku, serta menghapus data buku. Dengan latihan ini dapat memahami alur lengkap pengembangan backend mulai dari pembuatan struktur database menggunakan migration, pembuatan model menggunakan Sequelize, hingga implementasi endpoint API untuk mengelola data buku.

Langkah 1: Inisialisasi dan Setup

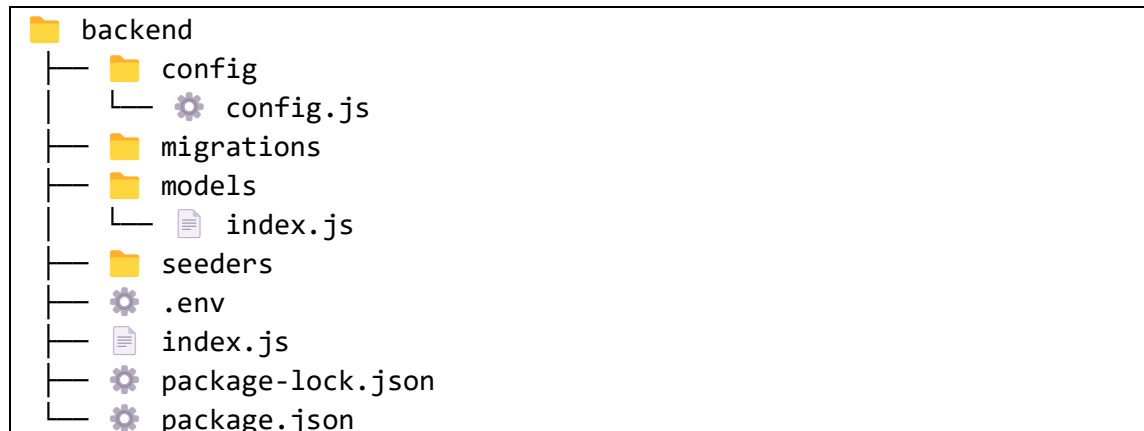
Pada project backend yang telah dibuat pada bab keempat, bukalah project tersebut dan masuk kedalam terminal dan tambahkan library ORM Sequelize dibawah ini:

```
npm install sequelize sequelize-cli
```

Setelah menginstall masih dalam terminal direktori project backend masukan syntax dibawah ini untuk inisialisasi sequelize CLI:

```
npx sequelize-cli init
```

Ini akan membuat struktur folder:



- config/ - Konfigurasi database
- models/ - Model Sequelize
- migrations/ - File migration
- seeders/ - File seeder

1. Setup file *environment* (.env)

Selanjutnya melakukan setup terhadap koneksi database pada project Express dengan Sequelize.

Buatlah file .env dan masukan beberapa key dibawah ini:

⚙️ .env

```
API_PORT=3001
NODE_ENV=production
EXPRESS_DB_HOST=localhost
EXPRESS_DB_USERNAME=root
EXPRESS_DB_PASSWORD=
EXPRESS_DB_NAME=pw_praktikum_db
EXPRESS_DB_PORT=3306
```

A. Konfigurasi Aplikasi & Keamanan

- API_PORT=3001: Menentukan di port berapa server Express akan berjalan. Biasanya diatur agar tidak bentrok dengan aplikasi lain.
- NODE_ENV=production: Memberitahu sistem (dan beberapa library) dalam mode apa aplikasi berjalan.
 - production: Mengaktifkan optimasi performa dan keamanan.
 - development: Biasanya lebih santai dan memberikan pesan error yang detail untuk membantu proses debugging.

B. Konfigurasi Database (Sequelize/MySQL)

Ini adalah kumpulan credentials yang digunakan oleh Sequelize (atau driver database) untuk membuka "pintu" ke database:

- EXPRESS_DB_HOST: Lokasi database. localhost berarti database berada di komputer/server yang sama dengan aplikasi.

- EXPRESS_DB_USERNAME: Username untuk masuk ke database (standar XAMPP/WAMP biasanya root).
- EXPRESS_DB_PASSWORD: Kata sandi database. Dalam contohmu kosong, yang berarti database tidak diproteksi password (umum untuk tahap belajar).
- EXPRESS_DB_NAME: Nama spesifik database yang akan digunakan (pw_praktikum_db).
- EXPRESS_DB_PORT: Port tempat server database mendengarkan koneksi. 3306 adalah standar untuk MySQL/MariaDB.

Tambahkan library *dotenv* kedalam project backend dengan sintak berikut:

```
npm install dotenv
```

2. Setup Sequelize CLI

Pada folder config terdapat file config.js, silakan sesuaikan dengan value yang telah disetup pada .env:

 config/config.jsx

```
require("dotenv").config();

module.exports = {
  development: {
    username: process.env.EXPRESS_DB_USERNAME,
    password: process.env.EXPRESS_DB_PASSWORD,
    database: process.env.EXPRESS_DB_NAME,
    host: process.env.EXPRESS_DB_HOST,
    dialect: "mysql",
    port: process.env.EXPRESS_DB_PORT,
    pool: {
      max: 10,
      min: 0,
      acquire: 30000,
      idle: 10000
    }
  },
  test: {
    username: process.env.EXPRESS_DB_USERNAME,
    password: process.env.EXPRESS_DB_PASSWORD,
    database: process.env.EXPRESS_DB_NAME,
    host: process.env.EXPRESS_DB_HOST,
    dialect: "mysql",
    port: process.env.EXPRESS_DB_PORT,
  },
}
```

```
production: {
  username: process.env.EXPRESS_DB_USERNAME,
  password: process.env.EXPRESS_DB_PASSWORD,
  database: process.env.EXPRESS_DB_NAME,
  host: process.env.EXPRESS_DB_HOST,
  dialect: "mysql",
  port: process.env.EXPRESS_DB_PORT,
  pool: {
    max: 10,
    min: 0,
    acquire: 30000,
    idle: 10000
  }
},
};
```

Penjelasan:

a) `require("dotenv").config();`

Ini adalah perintah untuk memuat variabel dari file `.env` ke dalam sistem aplikasi (`process.env`). Tanpa baris ini, `process.env.EXPRESS_DB_USERNAME` akan bernilai `undefined`, dan Sequelize tidak akan tahu password atau database mana yang harus diakses.

b) Pembagian Lingkungan (Development, Test, Production)

Sequelize memungkinkan memiliki konfigurasi yang berbeda untuk kebutuhan yang berbeda:

- *development*: Digunakan saat sedang menulis kode di laptop/pc sendiri. Biasanya menggunakan database lokal (seperti `localhost`).
- *test*: Digunakan saat menjalankan automated testing (misalnya untuk menjalankan unit test agar tidak merusak data asli).
- *production*: Digunakan saat aplikasi sudah di-deploy ke server sesungguhnya. Biasanya menggunakan database yang lebih aman atau server database terpisah.

c) SQL Connection Pooling

Di dalam konfigurasi `development` dan `production`, terdapat objek `pool`. Ini adalah fitur *Connection Pooling* dari Sequelize untuk menjaga performa:

- *max*: 10: Membatasi jumlah koneksi database maksimal yang dibuka secara bersamaan (agar server database tidak kelebihan beban).
- *min*: 0: Jumlah koneksi minimal yang tetap terbuka.

- *acquire*: 30000: Waktu (dalam milidetik) maksimal untuk mencoba mendapatkan koneksi sebelum menampilkan error.
- *idle*: 10000: Waktu (dalam milidetik) maksimal sebuah koneksi boleh tidak melakukan apa-apa sebelum diputus.

3. Setup server express agar terkoneksi Sequelize

Pada file index.js di dalam root project backend update lah dengan code dibawah ini:

 index.jsx

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const db = require("../models");
const app = express();

// Middleware
app.use(cors({
  origin: "http://localhost:3000",
  credentials: true
}));

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Test database connection
db.sequelize.authenticate()
  .then(() => {
    console.log("✓ Koneksi ke database MySQL berhasil!");
  })
  .catch((err) => {
    console.error("X Koneksi ke database gagal:", err.message);
    process.exit(1);
  });

// Basic Routes
app.get("/", (req, res) => {
  res.json({
    message: "Server berjalan dengan baik",
    status: "active",
    timestamp: new Date()
  });
});

app.get("/api/info", (req, res) => {
  res.json({
    message: "API MERN Stack Build by Express JS",
    version: "1.0.0",
  });
});
```

```

        status: "active",
        database: "Connected with Sequelize",
        timestamp: new Date()
    });
});

// 404 Handler
app.use((req, res) => {
    res.status(404).json({
        success: false,
        message: "Route not found"
    });
});

// Error Handler
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({
        success: false,
        message: "Internal server error",
        error: err.message
    });
});

// Start Server
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
    console.log(`✓ Server running on port ${PORT}`);
    console.log(`✓ API available at http://localhost:${PORT}`);
});

```

Penjelasan:

a) Inisialisasi dan Konfigurasi Dasar

- *require("dotenv").config()*: Memuat variabel dari file .env (seperti PORT atau DB_PASSWORD) ke dalam process.env.
- *cors*: Ini adalah "satpam" untuk API. Baris origin: "http://localhost:3000" memastikan bahwa hanya website yang berjalan di port 3000 (frontend React) yang diizinkan memanggil data dari server ini. Jika aplikasi lain mencoba mengakses, akses akan ditolak oleh browser.
- *express.json()* & *urlencoded*: Ini adalah parser. Tanpa ini, server tidak bisa membaca data JSON yang dikirimkan oleh frontend.

b) Koneksi ke Database

`db.sequelize.authenticate()`: Sebelum server benar-benar melayani user, dia melakukan "tes koneksi" ke database.

- Jika berhasil: Lanjut menjalankan server.
- Jika gagal (misal: MySQL mati atau salah password): Server akan berhenti (`process.exit(1)`) untuk mencegah aplikasi berjalan dalam kondisi "buta" data.

c) Routing

- Basic Routes (`/` dan `/api/info`): Ini adalah endpoint dasar untuk mengecek *Health Check* server.
- *404 Handler*: Ini adalah penangkap terakhir. Jika seseorang mencoba mengakses URL yang tidak terdaftar (misal `/api/buku/salah`), kode ini akan mengirimkan pesan 404 Not Found alih-alih membiarkan aplikasi hang.

d) Error Handler

- Error Handler: Ini adalah fungsi khusus yang dipicu jika terjadi kesalahan sistem di dalam kode (seperti syntax error atau database timeout).
- Daripada membiarkan user melihat pesan error teknis yang membingungkan, fungsi ini menangkapnya dan mengirimkan respon 500 Internal Server Error yang bersih dan rapi.

e) Start Server

`app.listen(PORT, ...)`: Mengaktifkan server pada port yang ditentukan. Baris `process.env.PORT || 3001` berarti: "Gunakan port dari file `.env` jika ada, jika tidak, pakai port 3001".

4. Menjalankan project backend

Jalankan server backend express untuk mengetahui apakah koneksi dengan Sequelize telah berhasil terkoneksi:

```
✓ Server running on port 3001
✓ API available at http://localhost:3001
Executing (default): SELECT 1+1 AS result
✓ Koneksi ke database MySQL berhasil!
```

Gambar 7.3 Koneksi Sequelize telah berhasil

Langkah 2: Membangun Model ORM

Model adalah inti dari Sequelize. Model merupakan sebuah abstraksi yang merepresentasikan (mewakili) sebuah tabel di database. Di dalam Sequelize, model adalah sebuah kelas (class) yang merupakan turunan (extends) dari kelas Model.

Model memberitahu Sequelize beberapa hal tentang entitas yang diwakilinya, seperti nama tabel di database dan kolom apa saja yang dimiliki (beserta tipe datanya).

Sebuah model di Sequelize memiliki nama. Nama ini tidak harus sama dengan nama tabel yang diwakilinya di database. Biasanya, model memiliki nama tunggal (seperti Book), sementara tabel memiliki nama jamak (seperti Books), meskipun hal ini dapat dikonfigurasi sepenuhnya.

a. Menjalankan perintah Generator

Masukan syntax dibawah ini untuk membuat file autogenerate untuk merepresentasikan table books dalam bentuk model bernama Book:

```
npx sequelize-cli model:generate --name Book --attributes
title:string,author:string,rating:decimal,views:integer,is_free:boole
an,language:string,sinopsis:text,story:text,image:string
```

Pemetaan tipe data untuk perintah di atas:

- string: Digunakan untuk teks pendek (seperti title, author, language, image).
- text: Digunakan untuk teks panjang (seperti sinopsis dan story).
- float: Digunakan untuk angka dengan desimal (seperti rating).
- integer: Digunakan untuk angka bulat (seperti views).
- boolean: Digunakan untuk nilai benar/salah (seperti is_free).

Setelah berhasil membuat file generator model book maka akan tercipta file tersebut pada folder models:

```
├─ models
│  ├── book.js
│  └── index.js
```

- b. Mengupdate model Book untuk validasi

 models/book.js

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Book extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize lifecycle.
     * The `models/index` file will call this method automatically.
     */
    static associate(models) {
      // define association here
    }
  }
  Book.init({
    id: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
      allowNull: false
    },
    title: {
      type: DataTypes.STRING(255),
      allowNull: false,
      validate: {
        notEmpty: {
          msg: 'Title cannot be empty'
        },
        len: {
          args: [3, 255],
          msg: 'Title must be between 3 and 255 characters'
        }
      }
    },
    author: {
      type: DataTypes.STRING(255),
      allowNull: false,
      validate: {
        notEmpty: {
          msg: 'Author cannot be empty'
        }
      }
    },
    rating: {
      type: DataTypes.DECIMAL(3, 2),
      allowNull: true,
      defaultValue: 0.00,
      validate: {
        min: {
          args: [0],
          msg: 'Rating must be at least 0'
        }
      }
    }
  }, {
    tableName: 'books'
  });
};
```

```

    },
    max: {
      args: [5],
      msg: 'Rating cannot exceed 5'
    }
  },
  },
  views: {
    type: DataTypes.INTEGER,
    allowNull: true,
    defaultValue: 0,
    validate: {
      min: {
        args: [0],
        msg: 'Views cannot be negative'
      }
    }
  },
  },
  is_free: {
    type: DataTypes.BOOLEAN,
    allowNull: false,
    defaultValue: false
  },
  language: {
    type: DataTypes.STRING(50),
    allowNull: false,
    defaultValue: 'English',
    validate: {
      notEmpty: {
        msg: 'Language cannot be empty'
      }
    }
  },
  },
  synopsis: {
    type: DataTypes.TEXT,
    allowNull: false,
    validate: {
      notEmpty: {
        msg: 'Sinopsis cannot be empty'
      },
      len: {
        args: [10, 1000],
        msg: 'Synopsis must be between 10 and 1000 characters'
      }
    }
  },
  },
  story: {
    type: DataTypes.TEXT('long'),
    allowNull: false,
    validate: {
      notEmpty: {
        msg: 'Story cannot be empty'
      },
      len: {
        args: [10],

```

```

        msg: 'Story must be at least 10 characters'
      }
    },
    image: {
      type: DataTypes.STRING(500),
      allowNull: true,
      validate: {
        isValidPath(value) {
          if (value && !value.startsWith('/uploads/')) {
            throw new Error('Image must be a valid file path starting
with /uploads/');
          }
        },
        isValidExtension(value) {
          if (value) {
            const validExtensions = ['.jpg', '.jpeg', '.png', '.gif',
'.webp'];
            const hasValidExtension = validExtensions.some(ext =>
value.toLowerCase().endsWith(ext)
);
            if (!hasValidExtension) {
              throw new Error('Image must have a valid extension (jpg,
jpeg, png, gif, webp)');
            }
          }
        }
      }
    },
    created_by: {
      type: DataTypes.STRING(255),
      allowNull: true
    },
    updated_by: {
      type: DataTypes.STRING(255),
      allowNull: true
    }
  }, {
    sequelize,
    modelName: 'Book',
    tableName: 'books',
    timestamps: true,
    underscored: true,
    createdAt: 'created_at',
    updatedAt: 'updated_at'
  });
  return Book;
};

```

Langkah 3: Membangun Migration

a. Membangun migration untuk table books

Pada saat melakukan generate model, periksa folder migrations apakah telah berhasil membuat file migration untuk create-book. Jika tidak ada file tersebut silakan masuk ke perintah dibawah ini. Jika sebaliknya abaikan tahapan ini.

Pada terminal project express masukan syntax dibawah ini untuk membuat table Books pada database.

```
npx sequelize-cli migration:generate --name create-books
```

```
D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend>npx sequelize-cli migration:generate --name create-books-table
Sequelize CLI [Node: 22.17.0, CLI: 6.6.5, ORM: 6.37.8]

migrations folder at "D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend\migrations" already exists.
New migration was created at D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend\migrations\20260308083544-create-books-table.js .
```

Gambar 7.4 Hasil Eksekusi Create Database Menggunakan Sequelize CLI

Setelah berhasil melakukan create migration maka pada terminal akan memberitahukan telah berhasil membuat sebuah file migration pada folder migrations. Silakan buka file tersebut dan lanjut ke step berikutnya.

b. Mengupdate file migration

```
├─ migrations
  └─ 20260308083544-create-books.js
```

Setelah file migrasi terbentuk, mendefinisikan atribut tabel secara mendalam di dalam fungsi *up*, yang mencakup kolom *id* sebagai *primary key*, serta *title*, *author*, *sinopsis*, *story*, *language*, dan *image* dengan tipe data STRING atau TEXT. Selanjutnya, akan menyematkan atribut angka dan status seperti *rating* dengan tipe FLOAT, *views* dengan tipe INTEGER, serta *is_free* dengan tipe BOOLEAN untuk memastikan data tersimpan dengan format yang tepat di database. Terakhir, jangan lupa untuk mendefinisikan fungsi *down* dengan perintah *queryInterface.dropTable('books')* agar memiliki kemampuan untuk melakukan *rollback* atau membatalkan migrasi tersebut jika terjadi kesalahan di kemudian hari.

```
'use strict';
```

```

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('books', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      title: {
        type: Sequelize.STRING(255),
        allowNull: false
      },
      author: {
        type: Sequelize.STRING(255),
        allowNull: false
      },
      rating: {
        type: Sequelize.DECIMAL(3, 2),
        allowNull: true,
        defaultValue: 0.00
      },
      views: {
        type: Sequelize.INTEGER,
        allowNull: true,
        defaultValue: 0
      },
      is_free: {
        type: Sequelize.BOOLEAN,
        allowNull: false,
        defaultValue: false
      },
      language: {
        type: Sequelize.STRING(50),
        allowNull: false,
        defaultValue: 'English'
      },
      synopsis: {
        type: Sequelize.TEXT,
        allowNull: false
      },
      story: {
        type: Sequelize.TEXT('long'),
        allowNull: false
      },
      image: {
        type: Sequelize.STRING(500),
        allowNull: true
      },
      created_at: {

```

```

        allowNull: false,
        type: Sequelize.DATE,
        defaultValue: Sequelize.literal('CURRENT_TIMESTAMP')
      },
      updated_at: {
        allowNull: false,
        type: Sequelize.DATE,
        defaultValue: Sequelize.literal('CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP')
      }
    });

    // Add indexes
    await queryInterface.addIndex('books', ['title']);
    await queryInterface.addIndex('books', ['author']);
    await queryInterface.addIndex('books', ['is_free']);
  },

  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('books');
  }
});

```

c. Menjalankan Migration

Setelah file migrasi selesai dikonfigurasi, langkah pertama adalah menjalankan perintah dibawah ini pada terminal untuk memicu eksekusi kode migrasi tersebut terhadap database yang telah terhubung.

```
npx sequelize-cli db:migrate
```

Setelah proses migrasi berjalan sukses dan terminal menampilkan pesan bahwa tabel books telah berhasil dibuat, langkah selanjutnya adalah memverifikasi hasil tersebut dengan mengakses database menggunakan database manager seperti MySQL Workbench, phpMyAdmin, atau melalui command line.

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>npx sequelize-cli db:migrate
```

```
Sequelize CLI [Node: 22.17.0, CLI: 6.6.5, ORM: 6.37.8]
```

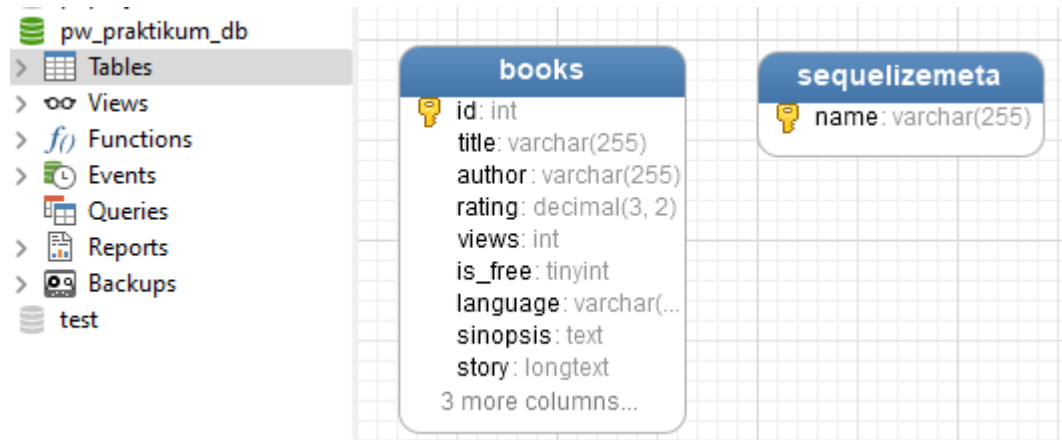
```

[dotenv@17.3.1] injecting env (8) from .env -- tip: enable debug logging with { debug: true }
Loaded configuration file "config\config.js".
Using environment "production".
== 20260308083544-create-books-table: migrating =====
== 20260308083544-create-books-table: migrated (0.079s)

```

Gambar 7.5 Hasil Eksekusi Migrasi Database Menggunakan Sequelize CLI

Di dalam database manager tersebut, arahkan pandangan ke daftar tabel pada database yang digunakan untuk memastikan bahwa tabel Books kini sudah terdaftar dengan seluruh atribut yang telah tentukan sebelumnya.



Gambar 7.6 Daftar Tabel Database pw_praktikum_db

Sebagai langkah akhir, dapat menjalankan perintah DESCRIBE Books; di dalam terminal database untuk memeriksa secara detail apakah tipe data dan struktur setiap kolom, seperti id, rating, views, hingga is_free, telah sesuai dengan rencana skema yang disusun dalam file migrasi.

Query Builder

Query Editor

1 DESCRIBE books

Message

Result1

Profile

Status

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	(Null)	auto_increment
title	varchar(255)	NO	MUL	(Null)	
author	varchar(255)	NO	MUL	(Null)	
rating	decimal(3,2)	YES		0.00	
views	int(11)	YES		0	
is_free	tinyint(1)	NO	MUL	0	
language	varchar(50)	NO		English	
sinopsis	text	NO		(Null)	
story	longtext	NO		(Null)	
image	varchar(500)	YES		(Null)	
created_at	datetime	NO		current_time	
updated_at	datetime	NO		current_time on update current_timestamp	

Gambar 7.7 Spesifikasi Tipe Data Tabel books

Langkah 4: Mengimplementasi CRUD (Controller)

a. Struktur Folder

```
├── backend
│   ├── config
│   ├── controllers
│   │   └── bookController.js
│   ├── migrations
│   ├── models
│   ├── routes
│   │   └── bookRoutes.js
│   ├── seeders
│   ├── .gitignore
│   ├── index.js
│   ├── package-lock.json
│   ├── package.json
│   └── yarn.lock
```

Tambahkan file *controllers* dan *routes* pada project backend sesuai dengan struktur diatas.

- *controllers/*: Berisi logika bisnis (mengolah data dari database).
- *routes/*: Berisi daftar endpoint (URL) yang mengarah ke controller.

b. Pembuatan Controller

📄 controllers/bookController.js

```
const db = require("../models");
const Book = db.Book;
const { Op } = db.Sequelize;
const path = require('path');
const fs = require('fs');

const deleteFile = (filePath) => {
  if (filePath && fs.existsSync(filePath)) {
    fs.unlinkSync(filePath);
  }
};

const renameBookImage = (oldPath, bookId, bookTitle) => {
  try {
    const ext = path.extname(oldPath);
    // Sanitize title: lowercase, replace spaces with hyphens,
    // remove special chars
    const sanitizedTitle = bookTitle
      .toLowerCase()
      .replace(/\s+/g, '-')
      .replace(/[^a-z0-9-]/g, '');
  }
};
```

```

        .substring(0, 50);

        const newFilename = `${sanitizedTitle}-${bookId}${ext}`;
        const newPath = path.join(path.dirname(oldPath), newFilename);

        fs.renameSync(oldPath, newPath);

        return `/uploads/books/${newFilename}`;
    } catch (error) {
        console.error('Error renaming file:', error);
        return oldPath; // Return old path if rename fails
    }
};

// Get all books
exports.getAllBooks = async (req, res) => {
    try {
        const { search, is_free, language, sort_by, order } = req.query;

        // Build where clause
        let whereClause = {};

        if (search) {
            whereClause = {
                ...whereClause,
                [Op.or]: [
                    { title: { [Op.like]: `%${search}%` } },
                    { author: { [Op.like]: `%${search}%` } },
                    { synopsis: { [Op.like]: `%${search}%` } }
                ]
            };
        }

        if (is_free !== undefined) {
            whereClause.is_free = is_free === 'true';
        }

        if (language) {
            whereClause.language = language;
        }

        // Build order clause
        let orderClause = [['created_at', 'DESC']];
        if (sort_by) {
            const sortOrder = order && order.toUpperCase() === 'ASC' ?
'ASC' : 'DESC';
            orderClause = [[sort_by, sortOrder]];
        }

        const books = await Book.findAll({
            where: whereClause,
            order: orderClause
        });

        res.json({

```

```

        success: true,
        count: books.length,
        data: books
    });
} catch (error) {
    console.error("Error fetching books:", error);
    res.status(500).json({
        success: false,
        message: "Failed to fetch books",
        error: error.message
    });
}
};

// Get single book by ID
exports.getBookById = async (req, res) => {
    try {
        const book = await Book.findPk(req.params.id);

        if (!book) {
            return res.status(404).json({
                success: false,
                message: "Book not found"
            });
        }

        // Increment views
        await book.increment('views');
        await book.reload();

        res.json({
            success: true,
            data: book
        });
    } catch (error) {
        console.error("Error fetching book:", error);
        res.status(500).json({
            success: false,
            message: "Failed to fetch book",
            error: error.message
        });
    }
};

// Create new book
exports.createBook = async (req, res) => {
    try {
        console.log('Request body:', req.body);
        console.log('Request file:', req.file);
        console.log('User from token:', req.user);

        const { title, author, rating, views, is_free, language,
sinopsis, story } = req.body;

        // Validation

```

```

        if (!title || !author || !sinopsis || !story) {
            if (req.file) {
                deleteFile(req.file.path);
            }

            return res.status(400).json({
                success: false,
                message: "Title, author, sinopsis, and story are
required",
                received: {
                    title: title || 'missing',
                    author: author || 'missing',
                    sinopsis: sinopsis || 'missing',
                    story: story || 'missing'
                }
            });
        }

        // Prepare book data (without image first)
        const bookData = {
            title: title.trim(),
            author: author.trim(),
            rating: rating ? parseFloat(rating) : 0.00,
            views: views ? parseInt(views) : 0,
            is_free: is_free === 'true' || is_free === true || is_free
=== 1,
            language: language || 'English',
            sinopsis: sinopsis.trim(),
            story: story.trim(),
            image: null,
            created_by: req.user?.email || 'system',
        };

        console.log('Book data to create:', bookData);

        // Create book
        const book = await Book.create(bookData);

        // Rename and update image if file was uploaded
        if (req.file) {
            const newImagePath = renameBookImage(req.file.path, book.id,
book.title);
            await book.update({ image: newImagePath });
        }

        res.status(201).json({
            success: true,
            message: "Book created successfully",
            data: book
        });
    } catch (error) {
        console.error("Error creating book:", error);

        if (req.file) {
            deleteFile(req.file.path);
        }
    }
}

```

```

    }

    if (error.name === 'SequelizeValidationError') {
      return res.status(400).json({
        success: false,
        message: "Validation error",
        errors: error.errors.map(e => ({
          field: e.path,
          message: e.message
        })))
    });
  }

  res.status(500).json({
    success: false,
    message: "Failed to create book",
    error: error.message
  });
}

};

// Update book (full update)
exports.updateBook = async (req, res) => {
  try {
    const book = await Book.findByPk(req.params.id);

    if (!book) {
      if (req.file) {
        deleteFile(req.file.path);
      }

      return res.status(404).json({
        success: false,
        message: "Book not found"
      });
    }

    const { title, author, rating, views, is_free, language,
    sinopsis, story } = req.body;

    // Prepare update data
    const updateData = {
      title: title || book.title,
      author: author || book.author,
      rating: rating ? parseFloat(rating) : book.rating,
      views: views ? parseInt(views) : book.views,
      is_free: is_free !== undefined ? (is_free === 'true' ||
is_free === true) : book.is_free,
      language: language || book.language,
      sinopsis: sinopsis || book.sinopsis,
      story: story || book.story,
      updated_by: req.user?.email || 'system'
    };

    // Handle image update

```

```

        if (req.file) {
            if (book.image) {
                const oldImagePath = path.join(__dirname, '..',
book.image);
                deleteFile(oldImagePath);
            }

            const newImagePath = renameBookImage(req.file.path, book.id,
updateData.title);
            updateData.image = newImagePath;
        }

        await book.update(updateData);

        res.json({
            success: true,
            message: "Book updated successfully",
            data: book
        });
    } catch (error) {
        console.error("Error updating book:", error);

        if (req.file) {
            deleteFile(req.file.path);
        }

        if (error.name === 'SequelizeValidationError') {
            return res.status(400).json({
                success: false,
                message: "Validation error",
                errors: error.errors.map(e => ({
                    field: e.path,
                    message: e.message
                })))
        });
    }

    res.status(500).json({
        success: false,
        message: "Failed to update book",
        error: error.message
    });
}
};

// Patch book (partial update)
exports.patchBook = async (req, res) => {
    try {
        const book = await Book.findByPk(req.params.id);

        if (!book) {
            if (req.file) {
                deleteFile(req.file.path);
            }

```

```

        return res.status(404).json({
            success: false,
            message: "Book not found"
        });
    }

    const updateData = { ...req.body };

    // Convert is_free to boolean if present
    if (updateData.is_free !== undefined) {
        updateData.is_free = updateData.is_free === 'true' ||
updateData.is_free === true;
    }

    // Convert rating to float if present
    if (updateData.rating !== undefined) {
        updateData.rating = parseFloat(updateData.rating);
    }

    // Convert views to integer if present
    if (updateData.views !== undefined) {
        updateData.views = parseInt(updateData.views);
    }

    // Add updated_by
    updateData.updated_by = req.user?.email;

    // Handle image update
    if (req.file) {
        if (book.image) {
            const oldImagePath = path.join(__dirname, '..',
book.image);
            deleteFile(oldImagePath);
        }

        const titleForFilename = updateData.title || book.title;
        const newImagePath = renameBookImage(req.file.path, book.id,
titleForFilename);
        updateData.image = newImagePath;
    }

    await book.update(updateData);

    res.json({
        success: true,
        message: "Book updated successfully",
        data: book
    });
} catch (error) {
    console.error("Error updating book:", error);

    if (req.file) {
        deleteFile(req.file.path);
    }
}

```

```

        if (error.name === 'SequelizeValidationError') {
            return res.status(400).json({
                success: false,
                message: "Validation error",
                errors: error.errors.map(e => ({
                    field: e.path,
                    message: e.message
                })))
        });
    }

    res.status(500).json({
        success: false,
        message: "Failed to update book",
        error: error.message
    });
}

};

// Delete book
exports.deleteBook = async (req, res) => {
    try {
        const book = await Book.findByPk(req.params.id);

        if (!book) {
            return res.status(404).json({
                success: false,
                message: "Book not found"
            });
        }

        // Delete image file if exists
        if (book.image) {
            const imagePath = path.join(__dirname, '..', book.image);
            deleteFile(imagePath);
        }

        await book.destroy();

        res.json({
            success: true,
            message: "Book deleted successfully"
        });
    } catch (error) {
        console.error("Error deleting book:", error);
        res.status(500).json({
            success: false,
            message: "Failed to delete book",
            error: error.message
        });
    }
};

// Get books statistics

```

```

exports.getStatistics = async (req, res) => {
  try {
    const totalBooks = await Book.count();
    const freeBooks = await Book.count({ where: { is_free: true } });
    const paidBooks = await Book.count({ where: { is_free: false } });

    const languages = await Book.findAll({
      attributes: [
        'language',
        [db.Sequelize.fn('COUNT', db.Sequelize.col('language')),
        'count']
      ],
      group: ['language']
    });

    const topRated = await Book.findAll({
      order: [['rating', 'DESC']],
      limit: 5
    });

    const mostViewed = await Book.findAll({
      order: [['views', 'DESC']],
      limit: 5
    });

    res.json({
      success: true,
      data: {
        total_books: totalBooks,
        free_books: freeBooks,
        paid_books: paidBooks,
        languages: languages,
        top_rated: topRated,
        most_viewed: mostViewed
      }
    });
  } catch (error) {
    console.error("Error fetching statistics:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch statistics",
      error: error.message
    });
  }
};

```

Penjelasan:

1) Struktur Utama

- Impor Model: Kode mengambil model Book dari db (index.js).

- Operator Sequelize (Op): Digunakan untuk membuat query yang lebih kompleks, seperti pencarian teks (LIKE) atau logika OR.

2) Fungsi-Fungsi Utama (CRUD & Statistics)

A. Pencarian & Filter (*getAllBooks*)

Fungsi ini sangat dinamis karena mendukung:

- Pencarian: Mencari kata kunci di title, author, atau sinopsis secara bersamaan menggunakan [*Op.or*] dan [*Op.like*].
- Filter & Sorting: Bisa memfilter buku berdasarkan status (*is_free*) atau bahasa, serta mengurutkan hasil (*sort_by*) dari ASC atau DESC.

B. Logika View Counter (*getBookById*)

Ini fitur yang menarik: setiap kali seseorang mengakses detail buku dengan ID tertentu, server akan otomatis menjalankan:

- *book.increment('views')*: Menambah nilai kolom views di database sebesar 1.
- *book.reload()*: Mengambil data terbaru agar views yang ditampilkan ke user adalah angka yang sudah di-update.

C. Validasi & Update (*createBook*, *updateBook*, *patchBook*)

- Validasi: Sebelum menyimpan ke database, kode mengecek apakah field wajib (seperti title dan author) sudah diisi.
- Error Handling: Jika data tidak valid menurut skema Sequelize, fungsi catch akan menangkap *SequelizeValidationError* dan mengirimkan pesan error yang rapi ke frontend.

c. Pembuatan Routes

 routes/bookRoutes.js

```
const express = require("express");
const router = express.Router();
const bookController = require("../controllers/bookController");

// CRUD routes
router.get("/", bookController.getAllBooks);
```

```

router.get("/:id", bookController.getBookById);
router.post("/", bookController.createBook);
router.put("/:id", bookController.updateBook);
router.patch("/:id", bookController.patchBook);
router.delete("/:id", bookController.deleteBook);

module.exports = router;

```

Penjelasan:

1. Struktur Pemetaan (Routing)

Setiap baris kode menghubungkan sebuah HTTP Method + URL dengan sebuah fungsi di bookController:

Tabel 7.1 Routing Rest API Book

Method	Endpoint	Controller Action	Keterangan
GET	/	getAllBooks	Mengambil daftar semua buku.
GET	/:id	getBookById	Mengambil detail buku berdasarkan ID unik.
POST	/	createBook	Menambah data buku baru.
PUT	/:id	updateBook	Memperbarui seluruh data buku (full update).
PATCH	/:id	patchBook	Memperbarui sebagian data buku (partial update).
DELETE	/:id	deleteBook	Menghapus buku berdasarkan ID.

2. Konsep Parameter

Tanda titik dua (:id) adalah *Route Parameter*. Ini artinya, bagian tersebut bersifat dinamis.

Contoh: Jika memanggil /123, maka angka 123 akan ditangkap oleh Express dan disimpan ke dalam *req.params.id*, yang kemudian digunakan oleh controller untuk mencari data di database.

3. Modularitas

Di akhir kode terdapat *module.exports = router;*. Ini memungkinkan file index.js utama (server) untuk mengimpor kumpulan route ini dengan mudah. Biasanya di index.js akan melihat kode seperti ini:

```

const bookRoutes = require("./routes/bookRoutes");
app.use("/api/books", bookRoutes);

```

Dengan cara ini, semua route di file ini akan otomatis memiliki prefix `/api/books`, sehingga URL lengkapnya menjadi `/api/books`, `/api/books/1`, dan seterusnya.

Langkah 5: Testing Rest API CRUD Books

Pada tahap ini akan dilakukan pengujian terhadap Rest API yang telah dibuat sesuai dengan table 7.1. Tools yang dipakai ialah PostMan (<https://www.postman.com/>)

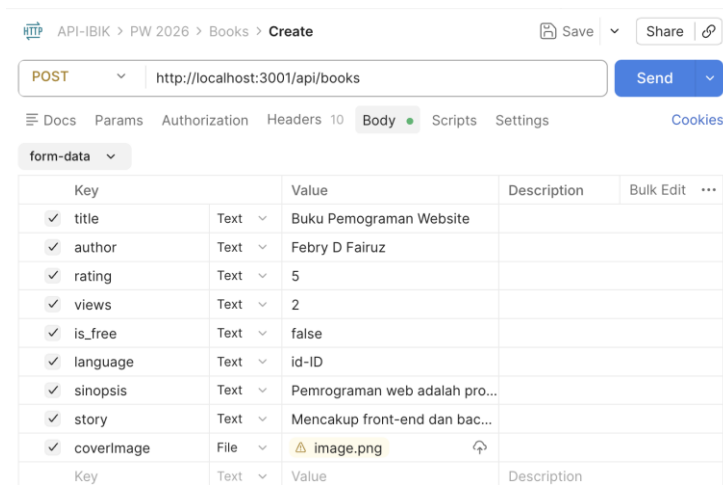
1) Create Book

URL: OST <http://localhost:3001/api/books>

Raw:

```
{
  "title": "Buku Pemograman Website",
  "author": "Febry D Fairuz",
  "rating": 5,
  "views": 2,
  "is_free": false,
  "language": "id-ID",
  "sinopsis": "Pemrograman web adalah proses pengembangan aplikasi atau situs yang berjalan di browser, mengubah ide menjadi antarmuka interaktif menggunakan HTML, CSS, dan JavaScript",
  "story": "Mencakup front-end dan back-end (PHP, Python, SQL), bidang ini berfokus pada logika, fungsionalitas, serta manajemen database untuk menciptakan website yang dinamis.",
  "image": "file_path_image/image-file.png"
}
```

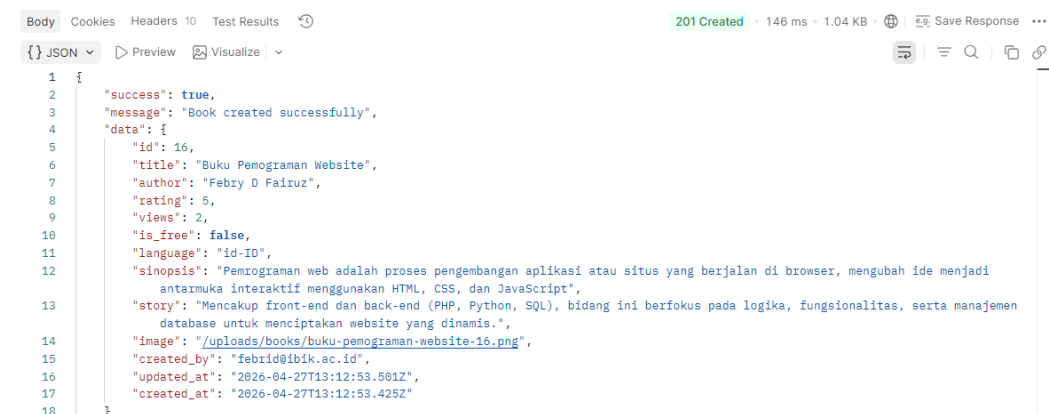
Body:



Gambar 7.8 Pengujian API Create Data Books Menggunakan Postman

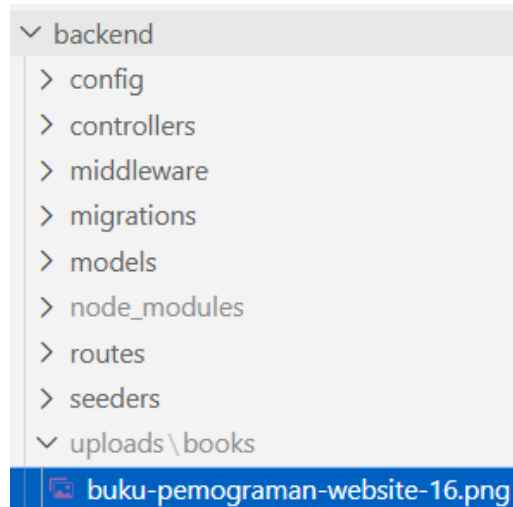
Response:

```
{
  "success": true,
  "message": "Book created successfully",
  "data": {
    "id": 16,
    "title": "Buku Pemograman Website",
    "author": "Febry D Fairuz",
    "rating": 5,
    "views": 2,
    "is_free": false,
    "language": "id-ID",
    "sinopsis": "Pemrograman web adalah proses pengembangan
aplikasi atau situs yang berjalan di browser, mengubah ide
menjadi antarmuka interaktif menggunakan HTML, CSS, dan
JavaScript",
    "story": "Mencakup front-end dan back-end (PHP, Python,
SQL), bidang ini berfokus pada logika, fungsionalitas, serta
manajemen database untuk menciptakan website yang dinamis.",
    "image": "/uploads/books/buku-pemograman-website-
16.png",
    "created_by": "febrid@ibik.ac.id",
    "updated_at": "2026-04-27T13:12:53.501Z",
    "created_at": "2026-04-27T13:12:53.425Z"
  }
}
```



Gambar 7.9 Response API Create Data Books pada Postman

Untuk mengecek apakah file gambar yang telah diupload telah berhasil masuk kedalam server-side. Buka project anda dan cari folder `/uploads/books/`, cari filename berdasarkan hasil return dari key `'image'`.



Gambar 7.10 Hasil pencarian filename return dari key 'image'.

2) View Book by ID

URL: GET <http://localhost:3001/api/books/6>

Response:

```
{
  "success": true,
  "data": {
    "id": 6,
    "title": "Buku Pemograman Website",
    "author": "Febry D Fairuz",
    "rating": "5.00",
    "views": 3,
    "is_free": false,
    "language": "id-ID",
    "sinopsis": "Pemrograman web adalah proses pengembangan aplikasi atau situs yang berjalan di browser, mengubah ide menjadi antarmuka interaktif menggunakan HTML, CSS, dan JavaScript",
    "story": "Mencakup front-end dan back-end (PHP, Python, SQL), bidang ini berfokus pada logika, fungsionalitas, serta manajemen database untuk menciptakan website yang dinamis.",
    "image": "buku-pemrograman-website-6.png",
    "created_at": "2026-03-08T10:07:05.000Z",
    "updated_at": "2026-03-08T10:10:14.000Z"
  }
}
```

3) Update Book

URL: PUT <http://localhost:3001/api/books/6>

RAW:

```
{
  "title": "Pemograman Website II"
}
```

Response:

```
{
  "success": true,
  "message": "Book updated successfully",
  "data": {
    "id": 6,
    "title": "Pemograman Website II",
    "author": "Febry D Fairuz",
    "rating": "5.00",
    "views": 3,
    "is_free": false,
    "language": "id-ID",
    "sinopsis": "Pemrograman web adalah proses pengembangan aplikasi atau situs yang berjalan di browser, mengubah ide menjadi antarmuka interaktif menggunakan HTML, CSS, dan JavaScript",
    "story": "Mencakup front-end dan back-end (PHP, Python, SQL), bidang ini berfokus pada logika, fungsionalitas, serta manajemen database untuk menciptakan website yang dinamis.",
    "image": "buku-pemrograman-website-6.png",
    "created_at": "2026-03-08T10:07:05.000Z",
    "updated_at": "2026-03-08T10:13:01.043Z"
  }
}
```

4) Delete Book

URL: DELETE <http://localhost:3001/api/books/6>

Response:

```
{
  "success": true,
  "message": "Book deleted successfully"
}
```

5) View All Book

URL: GET <http://localhost:3001/api/books>

Response:

```
{
  "success": true,
  "count": 6,
  "data": [
    {
      "id": 16,
      "title": "Buku Pemrograman Website",
      "author": "Febry D Fairuz",
      "rating": "5.00",
      "views": 2,
      "is_free": false,
      "language": "id-ID",
      "sinopsis": "Pemrograman web adalah proses pengembangan aplikasi
atau situs yang berjalan di browser, mengubah ide menjadi antarmuka
interaktif menggunakan HTML, CSS, dan JavaScript",
      "story": "Mencakup front-end dan back-end (PHP, Python, SQL),
bidang ini berfokus pada logika, fungsionalitas, serta manajemen database
untuk menciptakan website yang dinamis.",
      "image": "/uploads/books/buku-pemrograman-website-16.png",
      "created_by": "febrid@ibik.ac.id",
      "updated_by": null,
      "created_at": "2026-04-27T13:12:53.000Z",
      "updated_at": "2026-04-27T13:12:53.000Z"
    },
    {
      "id": 5,
      "title": "Malin Kundang Anak Durhaka",
      "author": "Titis Asmarandana",
      "rating": "4.50",
      "views": 252,
      "is_free": false,
      "language": "id-ID",
      "sinopsis": "Lorem ipsum dolor sit amet consectetur adipisicing
elit. Distinctio ad reprehenderit eius consequatur ea! Enim, tempore.
Dolores unde ipsam fugiat, repellendus laborum numquam sequi impedit porro
dolorem eveniet laudantium ex.",
      "story": "Lorem ipsum dolor sit amet consectetur adipisicing
elit. Distinctio ad reprehenderit eius consequatur ea! Enim, tempore.
Dolores unde ipsam fugiat, repellendus laborum numquam sequi impedit porro
dolorem eveniet laudantium ex.",
      "image": "/uploads/books/malin-kundang-anak-durhaka-5.png",
      "created_by": "febrid@ibik.ac.id",
      "updated_by": null,
      "created_at": "2026-03-08T10:04:21.000Z",
      "updated_at": "2026-03-25T19:47:20.000Z"
    }
  ],
}
```

```

{
  "id": 4,
  "title": "Kisah Konyol Sampai Ngompol",
  "author": "Jegung Wicaksono",
  "rating": "4.50",
  "views": 26,
  "is_free": false,
  "language": "id-ID",
  "sinopsis": "Lorem ipsum dolor sit amet consectetur adipisicing
elit. Distinctio ad reprehenderit eius consequatur ea! Enim, tempore.
Dolores unde ipsam fugiat, repellendus laborum numquam sequi impedit porro
dolorem eveniet laudantium ex.",
  "story": "Sebagai bakti gue pada tanah air dan tanggung jawab
gue sebagai mahasiswa. Kini saatnya gue berangkat untuk menunaikan ibadah
KKN. Jadi gue harus segera menyelesaikan Kuliah Kerja Nyata ini. Gue pikir
setelah KKN gue bakal dapet hadiah gadis cantik dan manis dari desa. Gue
pikir setelah KKN gue bakal rajin bersih-bersih rumah. Gue pikir setelah KKN
gue bakal cepet lulus kuliah dengan nilai cemerlang.",
  "image": "/uploads/books/kisah-konyol-sampai-ngompol-4.png",
  "created_by": "febrid@ibik.ac.id",
  "updated_by": null,
  "created_at": "2026-03-08T10:02:27.000Z",
  "updated_at": "2026-03-25T19:47:20.000Z"
},
{
  "id": 3,
  "title": "Hikayat hang tuah",
  "author": "Carol McCloud",
  "rating": "5.00",
  "views": 184,
  "is_free": true,
  "language": "en-US",
  "sinopsis": "This heartwarming book introduces the concept of
'bucket filling' as a metaphor for expressing kindness and appreciation
towards others. Through simple yet profound illustrations and language,
readers learn the power of empathy, compassion, and positivity in filling
not only others' buckets but also their own.",
  "story": "In a world where everyone carries an invisible bucket,
young Felix learns the importance of filling others' buckets with acts of
kindness and positivity. Inspired by his teacher's metaphorical lesson,
Felix sets out on a mission to spread joy and happiness wherever he goes.
From complimenting a classmate to helping a friend in need, Felix discovers
that small gestures can have a big impact on others' well-being. As he fills
the buckets of those around him, Felix realizes that his own bucket is
overflowing with happiness, proving that kindness is contagious and that the
more you give, the more you receive.",
  "image": "/uploads/books/hikayat-hang-tuah-3.jpeg",

```

```

    "created_by": "febrid@ibik.ac.id",
    "updated_by": null,
    "created_at": "2026-03-08T10:01:00.000Z",
    "updated_at": "2026-03-25T19:47:20.000Z"
  },
  {
    "id": 2,
    "title": "Hantu Durian Runtuh",
    "author": "Mohd Nizam bin Abd Razak",
    "rating": "3.00",
    "views": 52,
    "is_free": false,
    "language": "en-US",
    "sinopsis": "Antoine de Saint-Exupéry's timeless classic tells
the story of a young prince who travels from planet to planet, meeting a
variety of characters and learning important life lessons along the way.
Through his encounters, he discovers the value of love, friendship, and the
beauty of innocence.",
    "story": "In the vast expanse of the universe, a young prince
embarks on a journey from his tiny planet to explore the mysteries of the
cosmos. Along the way, he encounters quirky inhabitants of various planets,
each with their own peculiarities and wisdom. Through his adventures, the
little prince learns profound truths about human nature, the importance of
love and friendship, and the beauty of seeing the world through the eyes of
innocence. Ultimately, he discovers that what truly matters cannot be seen
with the eyes but felt with the heart.",
    "image": "/uploads/books/hantu-durian-runtuh-2.jpg",
    "created_by": "febrid@ibik.ac.id",
    "updated_by": null,
    "created_at": "2026-03-08T09:59:29.000Z",
    "updated_at": "2026-03-25T19:47:20.000Z"
  },
  {
    "id": 1,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "rating": "4.50",
    "views": 2,
    "is_free": false,
    "language": "English",
    "sinopsis": "A classic American novel set in the Jazz Age,
exploring themes of wealth, love, and the American Dream through the eyes of
narrator Nick Carraway.",
    "story": "The story primarily concerns the young and mysterious
millionaire Jay Gatsby and his quixotic passion and obsession for the
beautiful former debutante Daisy Buchanan. Considered to be Fitzgerald's
magnum opus, The Great Gatsby explores themes of decadence, idealism,

```

```

resistance to change, social upheaval, and excess, creating a portrait of
the Roaring Twenties that has been described as a cautionary tale regarding
the American Dream.",
    "image": "/uploads/books/the-great-gatsby-1.jpeg",
    "created_by": "febrid@ibik.ac.id",
    "updated_by": null,
    "created_at": "2026-03-08T09:44:30.000Z",
    "updated_at": "2026-03-25T19:47:20.000Z"
  }
]
}

```

7.4 Tugas Pembelajaran

1. Lengkapi sistem backend dengan modul manajemen pengguna (*User Management*). Modul ini akan menangani data pengguna yang nantinya akan berinteraksi dengan koleksi buku. Dengan ketentuan seperti dibawah ini:

- a. Buatlah model yang merepresentasikan table users dengan attribute seperti table users dibawah ini:

```
1 DESCRIBE users
```

Message	Result1	Profile	Status		
Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	(Null)	auto_increment
email	varchar(100)	YES			
username	varchar(50)	YES			
password	varchar(255)	YES		(Null)	
is_active	tinyint(1)	YES		(Null)	
createdAt	datetime	NO		(Null)	
updatedAt	datetime	NO		(Null)	

- b. Jalankan migrasi untuk memastikan table users terbentuk dengan benar di database
2. Buatkan routing rest api untuk user management seperti table dibawah ini:

Method	Endpoint	Controller Action	Keterangan
GET	/	getAllUsers	Mengambil daftar semua user.
GET	/:id	getUserById	Mengambil detail user berdasarkan ID unik.
POST	/	createUser	Menambah data user baru.

PUT	/:id	updateUser	Memperbarui seluruh data user.
DELETE	/:id	deleteUser	Menghapus user berdasarkan ID.

3. Lakukan pengujian menggunakan aplikasi penguji API (seperti Postman atau Insomnia) untuk memastikan bahwa setiap endpoint mengembalikan status kode HTTP yang benar (Contoh: 200 untuk sukses, 201 untuk data baru, 404 jika user tidak ditemukan).
4. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran soal nomor 1-3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
5. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-MERN-STACK-NPM"** dengan nama commit **"Praktikum 7"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

BAB VIII

AUTHENTICATION AND INTEGRATING

8.1 Tujuan Pembelajaran

1. Memahami konsep dasar authentication dan perbedaannya dengan authorization.
2. Memahami cara kerja login dan registrasi pada aplikasi berbasis web.
3. Mengimplementasikan sistem autentikasi menggunakan token (JWT).
4. Mengamankan endpoint API pada backend menggunakan middleware.
5. Mengintegrasikan API backend dengan frontend menggunakan HTTP request.
6. Mengelola state autentikasi pada frontend (login, logout, session).
7. Menangani proteksi halaman (protected routes) pada frontend berbasis Next.js.

8.2 Dasar Teori

8.2.1 Authentication vs Authorization

Dalam pengembangan aplikasi web, khususnya menggunakan React dan Express.js, terdapat dua konsep penting dalam keamanan sistem, yaitu authentication dan authorization. Keduanya sering digunakan bersama, tetapi memiliki fungsi yang berbeda.

1. Authentication (Autentikasi)

Authentication adalah proses untuk memverifikasi identitas pengguna. Tujuannya adalah memastikan bahwa orang yang mencoba mengakses sistem memang benar orang yang ia klaim.

- Proses: Terjadi di awal saat Anda pertama kali membuka aplikasi.
- Data yang digunakan: Kredensial pengguna.
- Hasil: Jika berhasil, sistem akan memberikan "identitas" (seperti ID pengguna atau Token).

Cara Kerja Utama (Metode):

- *Something You Know*: Password, PIN, atau jawaban pertanyaan keamanan.
- *Something You Have*: Kode OTP di HP, kunci fisik (USB Token), atau kartu pintar.
- *Something You Are*: Biometrik seperti sidik jari, pengenalan wajah (Face ID), atau scan retina.

2. Authorization (Otorasi)

Authorization adalah proses menentukan hak akses atau izin yang dimiliki oleh pengguna yang sudah terautentikasi. Setelah sistem tahu siapa Anda, sistem akan mengecek daftar izin Anda.

- Proses: Terjadi setelah Authentication berhasil.
- Data yang digunakan: Peran (Role), kebijakan (Policy), atau grup pengguna.
- Hasil: Menentukan apakah Anda bisa melihat, mengedit, atau menghapus data tertentu.

Model Otorisasi yang Umum:

- RBAC (*Role-Based Access Control*): Izin diberikan berdasarkan jabatan. Contoh: "Admin" bisa menghapus user, sementara "Editor" hanya bisa menulis artikel.
- ABAC (*Attribute-Based Access Control*): Izin berdasarkan kondisi tertentu (waktu, lokasi, departemen). Contoh: "Hanya karyawan departemen HR yang bisa akses data gaji di jam kerja."
- ACL (*Access Control List*): Daftar spesifik siapa saja yang boleh mengakses satu file tertentu.

Perbedaan antara Authentication dengan Authorization:

Tabel Perbandingan Utama

Tabel 8.1 Perbandingan Authentication dan Authorization

Aspek	Authentication	Authorization
Tujuan	Verifikasi identitas	Menentukan hak akses
Waktu proses	Saat login	Setelah login
Pertanyaan utama	Siapa kamu?	Apa yang boleh kamu lakukan?
Contoh	Login email & password	Akses halaman admin
Output	User terverifikasi	Hak akses diberikan

Hubungan Keduanya

Authentication dan authorization bekerja secara berurutan:

Login → Authentication → Authorization → Akses Sistem

Artinya:

- User login (authentication)
- Sistem mengenali user
- Sistem menentukan hak akses (authorization)
- User hanya bisa mengakses fitur tertentu

Misalnya pada sistem CMS:

- Authentication
User login menggunakan email dan password
- Authorization
 - Admin → bisa CRUD
 - User → hanya bisa melihat

8.2.2 JSON Web Token (JWT)

JSON Web Token (JWT) adalah sebuah standar keamanan berbasis token yang digunakan untuk proses *authentication* dan *authorization* dalam aplikasi web. JWT biasanya digunakan pada aplikasi berbasis API untuk mengamankan komunikasi antara *client* (*frontend*) dan server (*backend*), misalnya pada aplikasi yang dibangun dengan React dan Express.js.

JWT memungkinkan server untuk memberikan sebuah token unik kepada pengguna setelah berhasil login. Token ini kemudian digunakan oleh client untuk mengakses endpoint tertentu tanpa perlu mengirimkan username dan password berulang kali.

Struktur JWT

JWT terdiri dari tiga bagian utama yang dipisahkan oleh tanda titik (.):

Header.Payload.Signature

Contoh:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. .
--

```
eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pbjI9
.
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

1. Header

Header berisi informasi tentang tipe token dan algoritma enkripsi yang digunakan.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Payload berisi data atau informasi pengguna (disebut claims).

```
{
  "userId": 1,
  "role": "admin"
}
```

3. Signature

Signature digunakan untuk memastikan bahwa token tidak diubah oleh pihak lain.

Dibuat dari:

- Header
- Payload
- Secret key dari server

Cara Kerja JWT

Alur penggunaan JWT dalam Aplikasi:

```
Login → Server verifikasi → Generate JWT → Kirim ke client → Client kirim token →
Server verifikasi token
```

Penjelasan:

- User login dengan email dan password
- Server memverifikasi data
- Jika valid → server membuat JWT
- Token dikirim ke client

- Client menyimpan token (localStorage / cookie)
- Setiap request ke server → token dikirim
- Server memverifikasi token sebelum memberikan akses

Contoh Penggunaan JWT pada API

Saat mengakses API menambahkan key bernama Authorization dengan value diawali dengan Bearer <token>. Contoh:

Authorization: Bearer
 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pbjJ9.Sf1KxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

Fungsi JWT:

JWT digunakan untuk:

- Authentication (login system)
- Authorization (akses berdasarkan role)
- Secure data exchange antar sistem
- Stateless session (tanpa session server)

Keuntungan dan Kekurangan JWT

Tabel 8.2 Keuntungan dan Kekurangan JWT

Keuntungan	Kekurangan
Tidak perlu menyimpan session di server (stateless)	Token tidak bisa dibatalkan sebelum expired
Mudah digunakan pada REST API	Jika token bocor, maka bisa disalahgunakan
Ringan dan Cepat	Payload tidak terenkripsi
Dapat digunakan lintas platform (web, mobile, dll)	Ukuran token bisa menjadi besar jika payload terlalu banyak

8.2.3 Hasing Password

Hashing password adalah proses mengubah password asli menjadi bentuk string acak (hash) menggunakan algoritma tertentu, sehingga password tidak disimpan dalam bentuk aslinya di database. Teknik ini digunakan untuk meningkatkan keamanan data

pengguna dalam aplikasi web, terutama pada backend yang dibangun dengan Node.js dan Express.js.

Berbeda dengan enkripsi, hashing bersifat satu arah (*one-way*), artinya hasil hash tidak dapat dikembalikan ke bentuk password asli. Oleh karena itu, saat proses login, sistem tidak membandingkan password secara langsung, tetapi membandingkan hasil hash dari input user dengan hash yang tersimpan di database.

Perbandingan

Tabel 8.3 Perbandingan Plaintext vs Hash

Penyimpanan	Contoh	Keamanan
Plaintext	123456	Sangat berbahaya
Hash	\$2b\$10\$abc...	Aman

Hashing password adalah teknik keamanan penting yang:

- Mengubah password menjadi bentuk tidak terbaca
- Melindungi data pengguna dari kebocoran
- Digunakan dalam proses login modern
- Menjadi standar dalam pengembangan backend

Tanpa hashing, sistem login menjadi sangat rentan terhadap serangan.

8.2.4 Middleware pada Backend

Dalam pengembangan backend menggunakan Express.js di atas Node.js, middleware merupakan fungsi yang berjalan di antara request (permintaan dari client) dan response (balasan dari server). Secara sederhana, middleware ialah, fungsi yang memproses request sebelum diteruskan ke endpoint utama.

Middleware memiliki akses ke:

- req (request)
- res (response)
- next (fungsi untuk melanjutkan ke proses berikutnya)

Cara kerja Middleware:

Client → Middleware → Route → Response

Artinya:

- Client mengirim request
- Middleware memproses request
- Jika diizinkan → lanjut ke route
- Server mengirim response

8.2.5 REST API Integration

REST API Integration adalah proses menghubungkan frontend dengan backend API agar aplikasi dapat saling bertukar data secara real-time melalui protokol HTTP. Dalam pengembangan modern, integrasi ini biasanya terjadi antara frontend berbasis React atau Next.js dengan backend yang dibangun menggunakan Express.js. Dengan adanya REST API Integration, frontend tidak lagi mengelola data secara langsung, melainkan mengambil dan mengirim data melalui endpoint API yang disediakan oleh backend.

Konsep Dasar REST API

Seperti yang telah dijelaskan pada bab sebelumnya di mana REST API menggunakan metode HTTP untuk melakukan operasi terhadap data:

Tabel 8.4 Method REST API

Method	Fungsi
GET	Mengambil data
POST	Menambahkan data
PUT	Mengupdate data
DELETE	Menghapus data

Contoh Integrasi di Frontend

A. Menggunakan Fetch API

```
const requestOptions = {  
  method: "GET",
```

```

    body: raw,
    redirect: "follow"
  });

  fetch("http://localhost:3001/api/books", requestOptions)
    .then((response) => response.text())
    .then((result) => console.log(result))
    .catch((error) => console.error(error));

```

B. Menggunakan AXIOS

```

const axios = require('axios');
let data = '';

let config = {
  method: 'get',
  maxLength: Infinity,
  url: 'http://localhost:3001/api/books',
  data : data
};

axios.request(config)
  .then((response) => {
    console.log(JSON.stringify(response.data));
  })
  .catch((error) => {
    console.log(error);
  });

```

C. Integrasi dengan Authentication (JWT)

```

const axios = require('axios');
let data = '';

let config = {
  method: 'get',
  maxLength: Infinity,
  url: 'http://localhost:3001/api/books',
  headers: {
    'Authorization': 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJmZWJyaWRaawJpay5hYy5pZCI6InVzZXJuYW1lIjoieZmVicmlkIiwiaWF0IjoxNzc0MTYyODc0fQ.6375qHn1tsuiR6jcaT4T3IuHhXcbE8ZKxnZOhs4jRUs'
  },
  data : data
};

axios.request(config)
  .then((response) => {
    console.log(JSON.stringify(response.data));
  });

```

```
})  
.catch((error) => {  
  console.log(error);  
});
```

REST API Integration adalah proses penting dalam pengembangan aplikasi modern yang:

- Menghubungkan frontend dan backend
- Menggunakan HTTP request untuk komunikasi data
- Menggunakan format JSON sebagai standar pertukaran data
- Mendukung arsitektur aplikasi yang modular dan scalable

Tanpa integrasi API, aplikasi frontend tidak dapat berinteraksi dengan data secara dinamis.

8.2.6 State Management Authentication

State Management Authentication adalah proses mengelola status login pengguna pada aplikasi frontend. Dalam aplikasi berbasis React atau Next.js, state digunakan untuk menyimpan informasi apakah pengguna sudah login atau belum, serta data pengguna seperti token, role, atau profil. Tujuan utama dari state management authentication ialah, menjaga agar aplikasi mengetahui kondisi user secara konsisten selama aplikasi digunakan.

Saat user login, biasanya backend akan memberikan token (JWT). Token ini kemudian disimpan di frontend dan digunakan untuk:

- Menandai bahwa user sudah login
- Mengakses API yang dilindungi
- Menentukan hak akses (role)

Implementasi Authentication

1. Menyimpan Token

Dalam pengembangan aplikasi web menggunakan JavaScript, browser menyediakan fitur penyimpanan data di sisi client yang disebut Web Storage API, yaitu `localStorage` dan `sessionStorage`. Keduanya digunakan untuk menyimpan data secara sementara di browser tanpa perlu menggunakan database atau server.

Fitur ini sering dimanfaatkan dalam aplikasi berbasis React atau Next.js, terutama untuk menyimpan data seperti token authentication, preferensi pengguna, atau data sementara lainnya.

A. LocalStorage

LocalStorage adalah penyimpanan data di browser yang bersifat persisten (tetap tersimpan) meskipun browser ditutup atau komputer dimatikan.

Karakteristik:

- Data tetap ada sampai dihapus manual
- Kapasitas sekitar 5MB
- Berbasis key-value (string)
- Tidak memiliki batas waktu (no expiration)

Contoh penggunaan *localStorage*:

Menyimpan data:

```
localStorage.setItem("token", "abc123");
```

Mengambil data:

```
const token = localStorage.getItem("token");
```

Menghapus data:

```
localStorage.removeItem("token");
```

B. SessionStorage

SessionStorage adalah penyimpanan data yang hanya berlaku selama session browser aktif.

Karakteristik:

- Data akan hilang saat tab/browser ditutup
- Kapasitas sekitar 5MB
- Berbasis key-value (string)
- Bersifat sementara

Contoh penggunaan *sessionStorage*:

Menyimpan data:

```
sessionStorage.setItem("token", "abc123");
```

Memanggil data:

```
const token = sessionStorage.getItem("token");
```

Menghapus data:

```
sessionStorage.removeItem("token");
```

2. Menggunakan Context (Global State)

Dalam pengembangan aplikasi menggunakan React, Context adalah fitur yang digunakan untuk mengelola dan membagikan state secara global ke seluruh komponen tanpa harus mengirim data melalui props secara berulang (*props drilling*). Dengan kata lain, context memungkinkan data dapat diakses langsung oleh banyak komponen tanpa harus melewati setiap level komponen.

Tanpa Context, data harus dikirim melalui props dari parent ke *child* secara berantai:

Jika setiap komponen harus menerima dan meneruskan props, maka kode menjadi:

- Lebih panjang
- Sulit dipelihara
- Tidak efisien

Masalah ini disebut props drilling.

Context memungkinkan kita menyimpan data di satu tempat (global), lalu komponen mana pun bisa mengaksesnya langsung.

Contoh data yang sering menggunakan Context:

- Data user (*authentication*)
- Tema aplikasi (*dark/light mode*)
- Bahasa (*multi-language*)

Contoh menggunakan context:

❖ Membuat Context

```
import { createContext } from "react";

export const AuthContext = createContext();
```

❖ Membuat Provider

```
import { useState } from "react";
import { AuthContext } from "../AuthContext";

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
}
```

❖ Membungkus Aplikasi

```
<AuthProvider>
  <App />
</AuthProvider>
```

❖ Menggunakan Context

```
import { useContext } from "react";
import { AuthContext } from "../AuthContext";

const { user } = useContext(AuthContext);
```

Context adalah fitur pada React yang digunakan untuk:

- ❖ Mengelola state global
- ❖ Menghindari props drilling
- ❖ Mempermudah distribusi data ke banyak komponen

Context sangat penting dalam implementasi fitur seperti authentication, karena memungkinkan seluruh aplikasi mengetahui status login user secara konsisten.

8.2.7 Protected Route

Protected Route adalah teknik dalam pengembangan aplikasi web untuk membatasi akses ke halaman tertentu hanya untuk pengguna yang telah terautentikasi (login). Konsep ini umum digunakan pada aplikasi berbasis React maupun Next.js, terutama pada halaman seperti dashboard, CMS, atau halaman admin. Dengan kata lain, *Protected Route* memastikan bahwa hanya pengguna yang memiliki akses (login/role tertentu) yang dapat membuka halaman tertentu.

Tujuan Protected Route

Protected Route digunakan untuk:

- Mengamankan halaman sensitif (dashboard, admin, CMS)
- Mencegah akses tanpa login
- Mengontrol hak akses berdasarkan role (authorization)

Contoh implementasi:

1. Protected Route Sederhana

```
function ProtectedRoute({ user, children }) {  
  if (!user) {  
    return <p>Silakan login terlebih dahulu</p>;  
  }  
  return children;  
}
```

Penggunaan:

```
<ProtectedRoute user={user}>  
  <Dashboard />  
</ProtectedRoute>
```

2. Redirect ke Halaman Login

```
import { Navigate } from "react-router-dom";  
  
function ProtectedRoute({ user, children }) {  
  if (!user) {  
    return <Navigate to="/login" />;  
  }  
  return children;  
}
```

Protected Route adalah mekanisme untuk:

- ✓ Membatasi akses halaman tertentu
- ✓ Mengontrol user berdasarkan login atau role
- ✓ Meningkatkan keamanan aplikasi

Protected Route biasanya dikombinasikan dengan:

- ✓ Authentication (login)
- ✓ State management
- ✓ JWT token
- ✓ Middleware backend

8.3 Latihan Pembelajaran

Pada tahap sebelumnya, aplikasi CMS yang telah dibangun masih menggunakan data statis berupa array objek untuk menampilkan daftar buku. Pendekatan ini hanya bersifat sementara dan tidak mencerminkan kondisi aplikasi nyata, karena data tidak tersimpan secara dinamis di database dan tidak dapat dikelola secara terpusat.

Pada bab ini, akan melakukan transformasi sistem menjadi aplikasi *fullstack* yang terintegrasi. Data buku yang sebelumnya bersifat statis akan digantikan dengan data yang diambil dari REST API yang telah dibangun pada backend menggunakan Express.js dan Sequelize. Selain itu, akan diimplementasikan sistem authentication menggunakan token (JWT), sehingga hanya pengguna yang telah login yang dapat mengakses dan mengelola data buku.

Melalui proses ini, memahami bagaimana *frontend* berbasis Next.js terhubung dengan *backend*, serta bagaimana mengelola autentikasi pengguna untuk membangun aplikasi yang lebih aman dan realistis.

Langkah 1: Persiapan Backend Authentication

- Pada tahap ini, backend harus menyediakan endpoint POST */login* sebagai proses autentikasi pengguna menggunakan Express.js.
- Endpoint ini menerima input email dan password, kemudian melakukan verifikasi ke database.

- Password akan dibandingkan menggunakan hashing (bcrypt) untuk memastikan keamanan.
- Jika data valid, sistem akan menghasilkan token (JWT) sebagai identitas pengguna.
- Token tersebut akan digunakan untuk mengakses endpoint API yang bersifat protected.

1. Pada project backend express tambahkan library baru untuk JWT dan Hasing dengan perintah dibawah ini:

```
npm install jsonwebtoken bcrypt
```

Lalu tambahkan library atau import tersebut pada userController:

```
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
```

2. Tambahkan key JWT_SECRET pada .env, penamaan key tersebut bisa bebas namun harus menggunakan huruf kapital pada setiap hurufnya. Dan value yang terhadap key tersebut harus berisi nilai unik, contoh:

 .env

```
JWT_SECRET=1bik_b3st
```

Nilai value dari JWT_SECRET setidaknya memiliki format yang unik, terdiri dari alfabet, angka ataupun symbol.

3. Tambahkan fungsi baru bernama loginUser pada userController dengan method POST:

 userController.js

```
exports.loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        success: false,
        message: "Email and password are required"
      });
    }
  }
}
```

```

// Find user by email
const user = await User.findOne({ where: { email } });

if (!user) {
  return res.status(401).json({
    success: false,
    message: "Invalid email or password"
  });
}

// Check if user is active
if (!user.is_active) {
  return res.status(403).json({
    success: false,
    message: "Account is inactive. Please contact
administrator"
  });
}

// Verify password
const isPasswordValid = await bcrypt.compare(password,
user.password);

if (!isPasswordValid) {
  return res.status(401).json({
    success: false,
    message: "Invalid email or password"
  });
}

const userResponse = {
  id: user.id,
  email: user.email,
  username: user.username
}

const expirationTime = Math.floor(Date.now() / 1000) + (6 * 60 *
60);
const accessToken = jwt.sign(
  userResponse,
  process.env.JWT_SECRET,
  { expiresIn: '6h' }
);

res.json({

```

```

        success: true,
        message: "Login successful",
        data: userResponse,
        accessToken,
        expiresIn: expirationTime
    });

} catch (error) {
    console.error("Error during login:", error);
    res.status(500).json({
        success: false,
        message: "Failed to login",
        error: error.message
    });
}
};

```

Fungsi loginUser adalah controller untuk menangani proses *autentikasi* (login) pengguna. Berikut penjelasan detail dari setiap bagiannya:

A. Validasi Input

```

if (!email || !password) {
    return res.status(400).json({
        success: false,
        message: "Email and password are required"
    });
}

```

- Memastikan email dan password dikirim dalam request body
- Mengembalikan error 400 (Bad Request) jika salah satu field kosong

B. Pencarian User di Database

```

const user = await User.findOne({ where: { email } });

if (!user) {
    return res.status(401).json({
        success: false,
        message: "Invalid email or password"
    });
}

```

- Mencari user berdasarkan email menggunakan Sequelize ORM
- Mengembalikan error 401 (Unauthorized) jika user tidak ditemukan

- Menggunakan pesan generic "Invalid email or password" untuk keamanan (tidak membocorkan apakah email terdaftar atau tidak)

C. Pengecekan Status Akun

```
if (!user.is_active) {
  return res.status(403).json({
    success: false,
    message: "Account is inactive. Please contact administrator"
  });
}
```

- Memverifikasi apakah akun user aktif
- Mengembalikan error 403 (Forbidden) jika akun tidak aktif
- Mencegah user yang di-nonaktifkan untuk login

D. Verifikasi Password

```
const isPasswordValid = await bcrypt.compare(password,
user.password);

if (!isPasswordValid) {
  return res.status(401).json({
    success: false,
    message: "Invalid email or password"
  });
}
```

- Membandingkan password yang dikirim dengan password ter-hash di database menggunakan bcrypt
- Mengembalikan error 401 jika password tidak cocok
- Menggunakan pesan yang sama dengan error email tidak ditemukan untuk keamanan

E. Pembuatan Response Data

```
const userResponse = {
  id: user.id,
  email: user.email,
  username: user.username
}
```

- Membuat objek yang berisi data user yang aman untuk dikirim ke client

- Tidak menyertakan password untuk keamanan

F. Generate JWT Token

```
const expirationTime = Math.floor(Date.now() / 1000) + (6 * 60 * 60);
const accessToken = jwt.sign(
  userResponse,
  process.env.JWT_SECRET,
  { expiresIn: '6h' }
);
```

- Menghitung waktu kadaluarsa token (6 jam dari sekarang dalam format Unix timestamp)
- Membuat JWT token dengan payload berisi data user
- Menggunakan secret key dari environment variable untuk signing token
- Note: Token ini tidak memiliki expiration time di dalam JWT payload, hanya dikirim sebagai informasi terpisah

G. Response Sukses

```
res.json({
  success: true,
  message: "Login successful",
  data: userResponse,
  accessToken,
  expiresIn: expirationTime
});
```

Mengembalikan response sukses dengan:

- Status flag success: true
- Pesan konfirmasi
- Data user (tanpa password)
- Access token untuk autentikasi request selanjutnya
- Waktu kadaluarsa token

H. Error Handling

```
try{
  ...
} catch (error) {
```

```
console.error("Error during login:", error);
res.status(500).json({
  success: false,
  message: "Failed to login",
  error: error.message
});
}
```

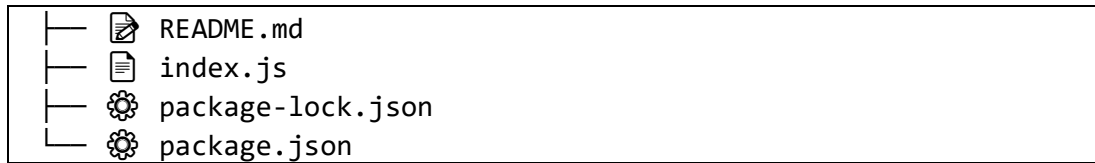
- Menangkap error yang tidak terduga
- Log error ke console untuk debugging
- Mengembalikan error 500 (Internal Server Error)

Langkah 2: Membuat Middleware pada Backend

- Pada tahap ini, backend perlu dibuatkan middleware untuk mengamankan endpoint API menggunakan Express.js.
- Middleware berfungsi untuk memeriksa keberadaan dan validitas token (JWT) pada setiap request dari client.
- Token biasanya dikirim melalui header *Authorization* dengan format *Bearer Token*.
- Jika *token valid*, maka request akan diteruskan ke endpoint berikutnya menggunakan *next()*.
- Jika token tidak valid atau tidak ada, maka sistem akan menolak akses dengan status *Unauthorized*.

1. Membuat folder dan file middleware pada root project backend seperti struktur dibawah ini:





2. Isilah code pada file auth.js seperti dibawah ini:

```
const jwt = require('jsonwebtoken');

exports.verifyToken = (req, res, next) => {
  try {
    // Get token from header
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        message: "Access token is required"
      });
    }

    // Verify token
    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
      if (err) {
        if (err.name === 'TokenExpiredError') {
          return res.status(401).json({
            success: false,
            message: "Token has expired"
          });
        }
        return res.status(403).json({
          success: false,
          message: "Invalid token"
        });
      }

      // Save user info to request
      req.user = decoded;
      next();
    });
  } catch (error) {
    console.error("Error verifying token:", error);
    res.status(500).json({
      success: false,
```

```

        message: "Failed to authenticate token",
        error: error.message
    });
    }
};

exports.checkUserActive = async (req, res, next) => {
    try {
        const db = require("../models");
        const User = db.User;

        const user = await User.findByPk(req.user.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: "User not found"
            });
        }

        if (!user.is_active) {
            return res.status(403).json({
                success: false,
                message: "Account is inactive"
            });
        }

        next();
    } catch (error) {
        console.error("Error checking user status:", error);
        res.status(500).json({
            success: false,
            message: "Failed to verify user status",
            error: error.message
        });
    }
};

```

File ini berisi middleware untuk autentikasi dan otorisasi menggunakan JWT (JSON Web Token). Terdapat 2 fungsi utama:

A. Function verifyToken pada middleware

Middleware ini berfungsi untuk memverifikasi JWT token yang dikirim oleh client.

1) Ekstraksi Token dari Header

```
const authHeader = req.headers['authorization'];
const token = authHeader && authHeader.split(' ')[1];
```

- Mengambil header Authorization dari request
- Format yang diharapkan: *Bearer <token>*
- Memisahkan string dengan *split(' ')* untuk mengambil token (index ke-1)
- Contoh: *"Bearer eyJhbGc..."*

2) Validasi Keberadaan Token

```
if (!token) {
  return res.status(401).json({
    success: false,
    message: "Access token is required"
  });
}
```

- Memeriksa apakah token ada
- Mengembalikan error 401 Unauthorized jika token tidak ditemukan
- Menghentikan request dan tidak melanjutkan ke handler berikutnya

3) Verifikasi Token dengan JWT

```
jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
  if (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        message: "Token has expired"
      });
    }
    return res.status(403).json({
      success: false,
      message: "Invalid token"
    });
  }

  req.user = decoded;
  next();
});
```

Proses Verifikasi:

- `jwt.verify()` memvalidasi token dengan secret key
- Jika token valid, akan men-decode payload dan mengembalikannya
- Jika ada error, akan menangani 2 jenis error:
 - `TokenExpiredError`: Token sudah kadaluarsa → Error 401
 - `Error lainnya`: Token tidak valid/rusak → Error 403 Forbidden

Jika Berhasil:

- Data user dari token (decoded payload) disimpan di `req.user`
- Memanggil `next()` untuk melanjutkan ke middleware/handler berikutnya
- Data di `req.user` bisa diakses di controller selanjutnya

4) Error Handling Global

```
try{
  ...
} catch (error) {
  console.error("Error verifying token:", error);
  res.status(500).json({
    success: false,
    message: "Failed to authenticate token",
    error: error.message
  });
}
```

- Menangkap error yang tidak terduga
- Mengembalikan error 500 Internal Server Error

B. Function `checkUserActive` pada middleware

Middleware tambahan untuk memverifikasi apakah user masih aktif di database.

1) Import Model User

```
const db = require("../models");
const User = db.User;
```

- Import dilakukan di dalam fungsi (bukan di top-level)
- Mencegah circular dependency issues

2) Query User dari Database

```
const user = await User.findByPk(req.user.id);

if (!user) {
  return res.status(404).json({
    success: false,
    message: "User not found"
  });
}
```

- Mengambil data user dari database berdasarkan ID yang ada di req.user
- req.user.id berasal dari decoded JWT token (diset oleh verifyToken)
- Mengembalikan error 404 Not Found jika user tidak ada

3) Pengecekan Status Aktif

```
if (!user.is_active) {
  return res.status(403).json({
    success: false,
    message: "Account is inactive"
  });
}

next();
```

- Memeriksa field is_active di database
- Mengembalikan error 403 Forbidden jika akun tidak aktif
- Memanggil next() jika user aktif

4) Error Handling

```
try {
  ...
} catch (error) {
  console.error("Error checking user status:", error);
  res.status(500).json({
    success: false,
    message: "Failed to verify user status",
    error: error.message
  });
}
```

- Menangani error database atau error lainnya
- Mengembalikan error 500 Internal Server Error

3. Berikan *Authorization* pada file route untuk mengamankan endpoint API books dan users, dimana end point tersebut hanya bisa diakses jika telah login atau telah memiliki Token.

Tambahkan protected route pada file bookRoutes.js

 *bookRoutes.js*

```
const express = require('express');
const router = express.Router();
const bookController = require('../controllers/bookController');
const { verifyToken } = require('../middleware/auth');

// Apply verifyToken middleware to all routes
router.use(verifyToken);

// rest of code
...
```

Baris `router.use(verifyToken);` menerapkan middleware *verifyToken* ke SEMUA route yang ada di bawahnya. Artinya, semua endpoint books memerlukan autentikasi JWT

Melakukan customize terhadap end point mana saja yang perlu di protected dan public routes, dalam contoh disini akan memisahkan end point login dan register sebagai PUBLIC Routes (tidak memerlukan autentikasi), sedangkan end point lainnya akan diberlakukan PROTECTED Routes (memerlukan autentikasi).

 *userRoutes.js*

```
const express = require("express");
const router = express.Router();
const userController = require("../controllers/userController");
const { verifyToken } = require("../middleware/auth");

// PUBLIC ROUTES (Tidak perlu autentikasi)
router.post("/login", userController.loginUser);
router.post("/register", userController.createUser);
// END PUBLIC ROUTES

// PROTECTED ROUTES (Perlu autentikasi)
router.use(verifyToken);
// CRUD routes
```

```

router.get("/", userController.getAllUsers);
router.get("/:id", userController.getUserById);
router.post("/", userController.createUser);
router.put("/:id", userController.updateUser);
router.patch("/:id", userController.patchUser);
router.delete("/:id", userController.deleteUser);
// END PROTECTED ROUTES

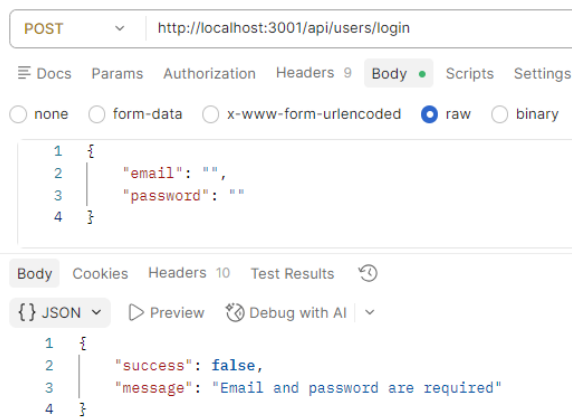
module.exports = router;

```

Langkah 3: Testing REST API

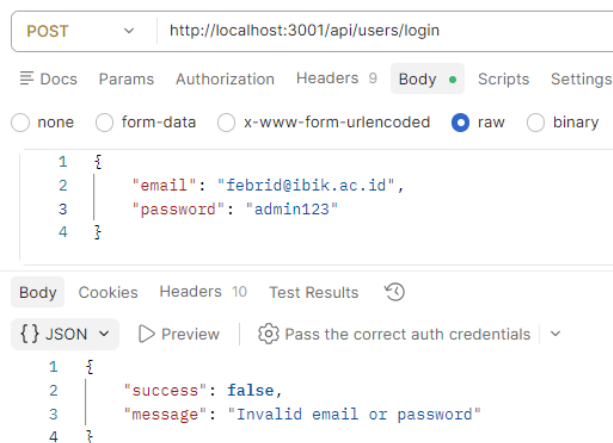
1. Testing API Login

a) Kondisi username atau password tidak diisi



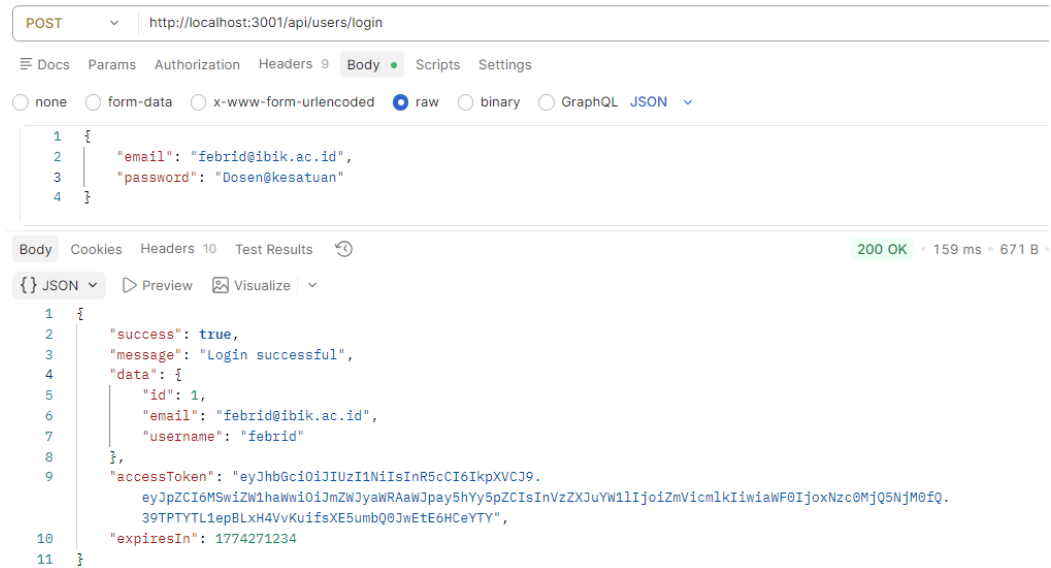
Gambar 8.1 Output Uji Coba `/api/users/login` tanpa Username dan Password

b) Kondisi akun ada namun password salah



Gambar 8.2 Output Output Uji Coba `/api/users/login` dengan Password Salah

c) Kondisi akun berhasil terverifikasi



Gambar 8.3 Output Uji Coba Endpoint `/api/users/login` Akun terverifikasi

Response Login

Status Code: 200 OK → Request berhasil diproses.

Penjelasan Response Body

Tabel 8.5 Deskripsi Penjelasan Response Body

Field	Tipe	Penjelasan
<code>success</code>	boolean	Menandakan apakah proses login berhasil. Nilai true berarti login sukses.
<code>message</code>	string	Pesan konfirmasi dari server. Di sini bernilai "Login successful".
<code>data</code>	object	Informasi dasar user yang berhasil login.
<code>data.id</code>	number	ID unik user di database, bernilai 1.
<code>data.email</code>	string	Email yang digunakan untuk login, yaitu febrid@ibik.ac.id.
<code>data.username</code>	string	Username dari akun yang login, yaitu febrid.
<code>accessToken</code>	string	JWT Token yang digunakan untuk autentikasi request-request berikutnya. Token ini harus disertakan di header Authorization: Bearer <token>.

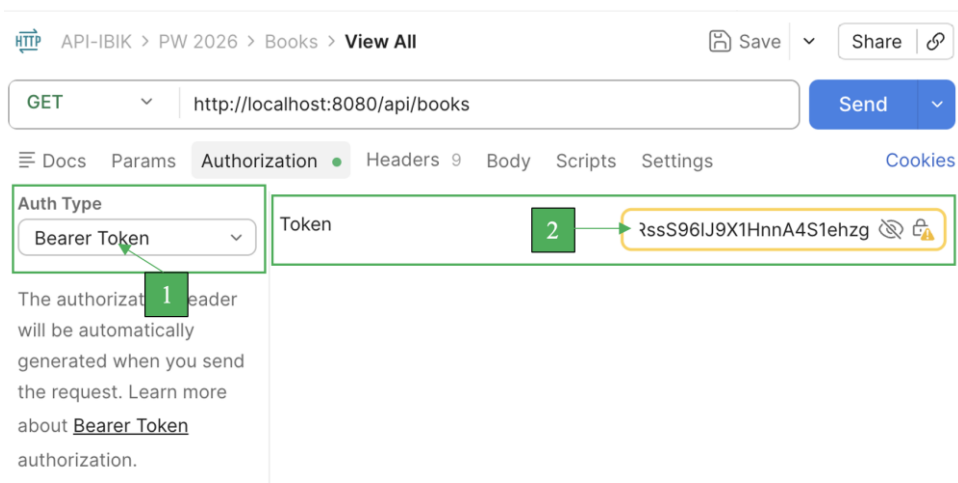
<i>expiresIn</i>	number	Waktu kedaluwarsa token dalam format Unix Timestamp (1774271234), yang menunjukkan kapan accessToken tidak lagi valid.
------------------	--------	--

- *accessToken* menggunakan format JWT (JSON Web Token) dengan algoritma HS256 (terlihat dari header token eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9). Token ini perlu disimpan dan digunakan sebagai Bearer Token untuk endpoint-endpoint yang memerlukan autentikasi.
- *expiresIn* adalah Unix timestamp absolut, bukan durasi dalam detik. Artinya token akan expired pada waktu tersebut.

2. Testing API Book

Pada tahap pengujian API Books, setiap request yang dilakukan memerlukan proses autentikasi menggunakan token. Token tersebut diperoleh dari hasil login user melalui endpoint authentication sebelumnya. Token kemudian dikirim melalui header Authorization dengan format Bearer Token pada setiap request ke API Books. Jika token valid, maka API akan merespon data buku sesuai permintaan, sedangkan jika tidak valid maka akses akan ditolak.

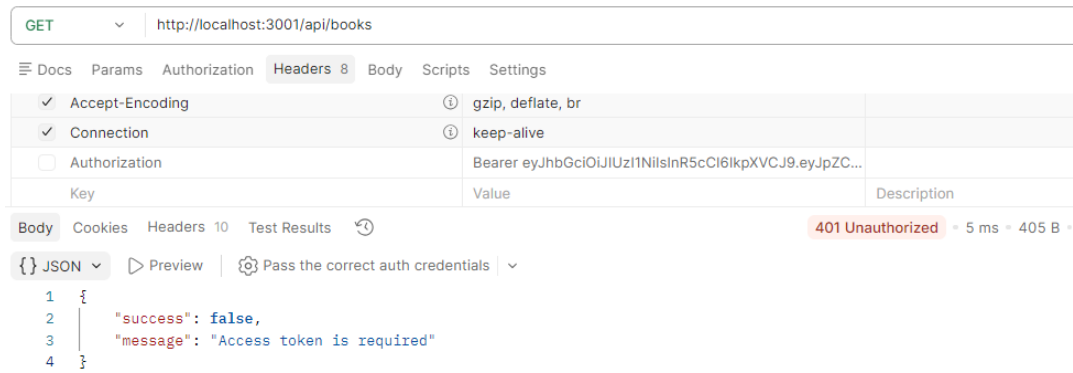
Untuk mengirimkan token autentikasi menggunakan Postman, buka tab Authorization, kemudian mengisikan Auth Type dan pilih Bearer Token pada bagian Token masukan token yang diperoleh dari proses login.



Gambar 8.4 Konfigurasi Autentikasi Bearer Token pada Postman

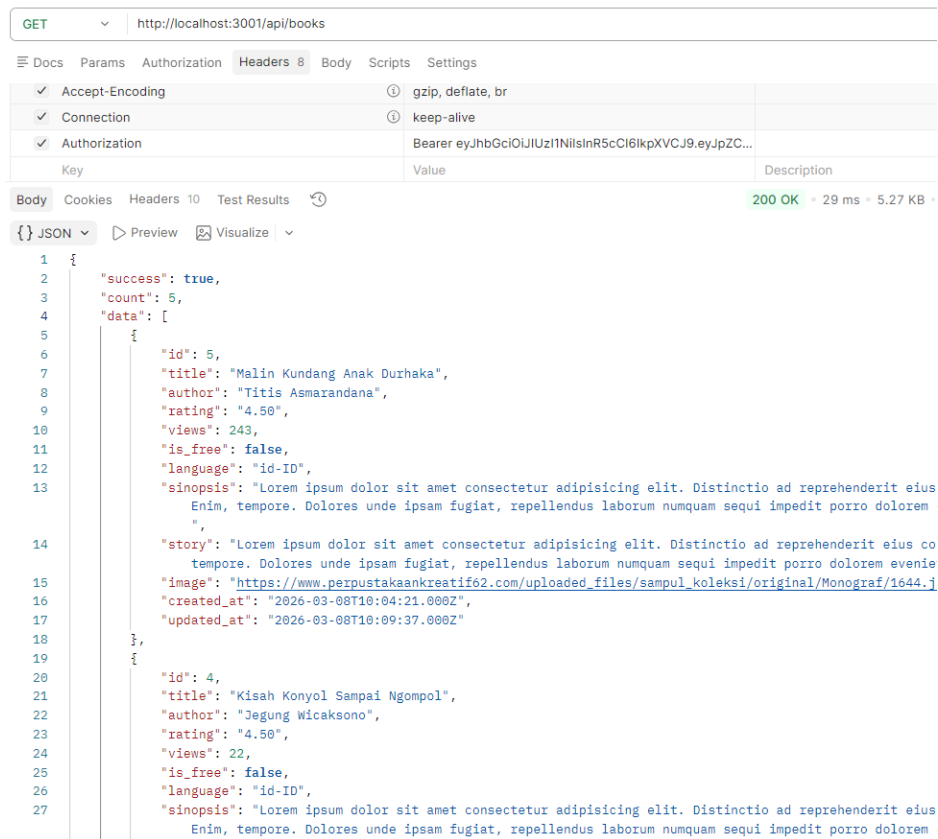
Token : didapatkan dari hasil response Login dengan mengambil nilai dari key `accessToken`

a) Kondisi request pada end-point book tanpa autentikasi



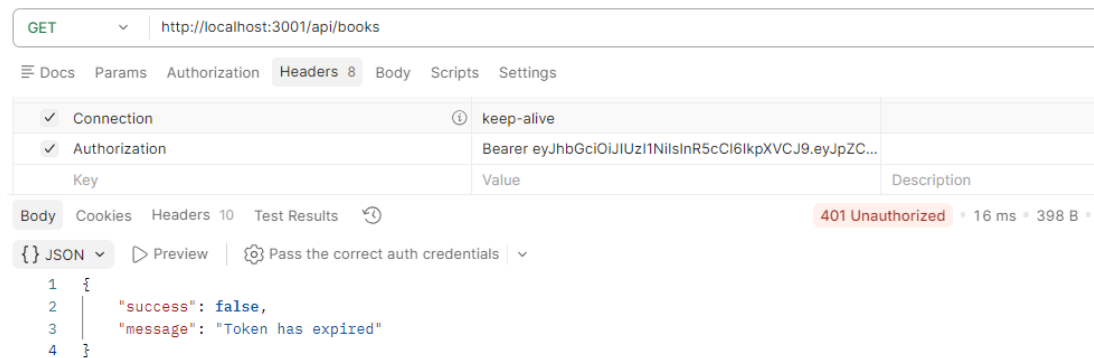
Gambar 8.5 Hasil Pengujian Endpoint Books tanpa Access Token

b) Kondisi request pada end-point book dengan autentikasi



Gambar 8.6 Hasil Pengujian Endpoint Books dengan Access Token

c) Kondisi token expired ketika request pada end-point book

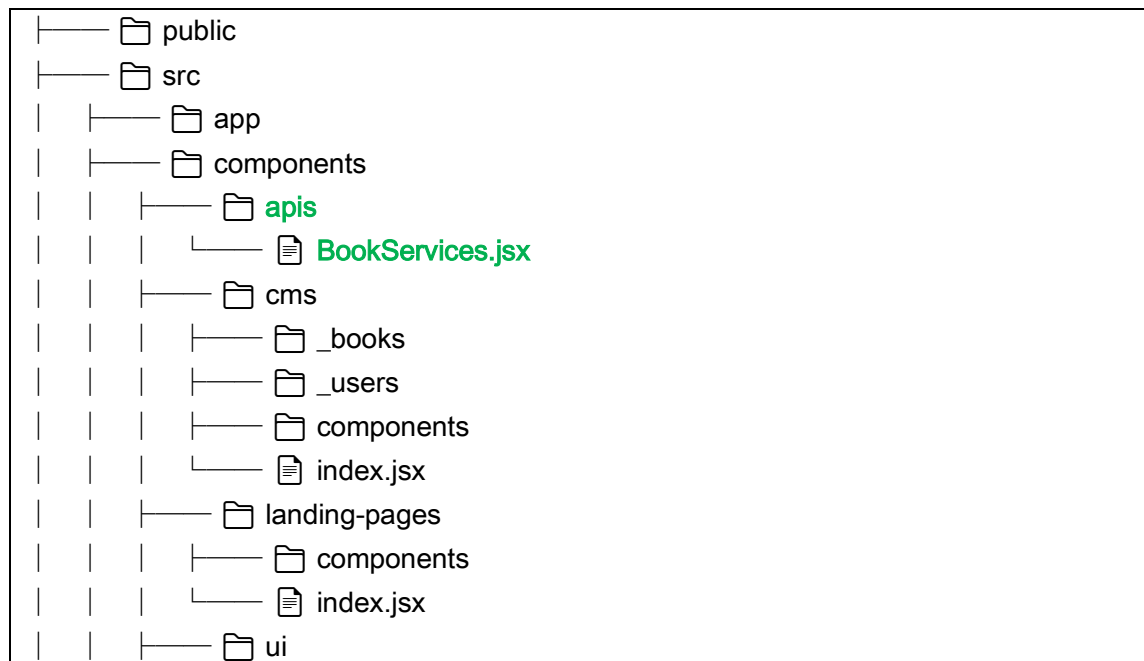


Gambar 8.7 Hasil Pengujian Endpoint Books dengan Acces Token Expired

Langkah 4: Mengubah Data Statis ke API (Frontend Integration)

Pada BAB 6 kita telah membuat halaman management buku secara static, dimana data dalam bentuk JSON Object yang disimpan dalam variable ListBooks. Tahap ini kita akan integrasikan management buku dan user menjadi data dinamik berdasarkan Rest API yang telah dibuat pada tahap sebelumnya. Berikut adalah tahapan perubahan untuk management buku:

1. Menambahkan folder dan file untuk handle library API





Untuk memenuhi kebutuhan handler API buat folder dan file dalam folder `/src` sesuai gambar struktur project diatas:

- `hooks/RequestAPI.jsx`
- `context/AuthContext.jsx`
- `components/apis/BookServices.jsx`

Menambahkan file konfigurasi yang digunakan untuk menyimpan **Environment Variables** pada file `.env` di root project anda:

```
API_PORT=3000
NODE_ENV=production
REACT_APP_BASE_URL=http://localhost:3000
NEXT_PUBLIC_BACKEND_URI=http://localhost:3001
JWT_SECRET=1bik_b3st
```

2. Membuat library Request API menggunakan Axios.

Fungsi RequestAPI adalah sebuah *wrapper* (pembungkus) untuk library axios. Tujuannya adalah untuk membuat proses pemanggilan API menjadi lebih konsisten dan rapi.

`hooks/RequestAPI.jsx`

```
import React from "react";
import axios from "axios";

const RequestAPI = async (method, url, headers, data) => {
  try {
    let config = {
      method,
      url,
      maxLength: Infinity,
      headers,
      data,
    };
    let response = await axios(config)
```

```

.then(function (response) {
  let result = response.data;
  if (result.success && result.message) {
    return {
      loading: false,
      message: "",
      data: data || [],
      success: result.success,
    };
  } else if (
    result.success &&
    result.data &&
    Object.values(result.data).length > 0
  ) {
    return {
      loading: false,
      message: "",
      data: result.data,
      success: result.success,
    };
  } else {
    return {
      loading: false,
      message: result.message || "No record found",
      data: [],
      success: result.success,
    };
  }
})
.catch(function (error) {
  return {
    loading: false,
    message: error.message,
    data: [],
    success: false,
  };
});
return response;
} catch (error) {
  return { loading: false, message: error.message, data: [] };
}
};

export { RequestAPI };

```

A. Struktur Parameter

Fungsi ini menerima empat argumen utama untuk mengonfigurasi request HTTP:

- *method*: Metode HTTP (seperti 'GET', 'POST', 'PUT', 'DELETE').
- *url*: Alamat endpoint API yang dituju.
- *headers*: Objek yang berisi informasi tambahan (seperti Token Autentikasi atau Content-Type).
- *data*: Payload atau body data yang akan dikirim ke server (biasanya untuk POST atau PUT).

B. Konfigurasi Axios

Di dalam blok try, kode membuat objek config:

```
let config = {  
  method,  
  url,  
  maxLength: Infinity,  
  headers,  
  data,  
};
```

Baris `maxLength: Infinity` berguna jika kamu berencana mengirim file besar atau data JSON yang sangat masif agar tidak terkena limit default dari axios.

C. Alur Penanganan Respon

Bagian yang paling menarik adalah cara kode ini mengolah hasil dari API. Kode ini mencoba menstandarisasi output agar selalu mengembalikan objek dengan format: `{ loading, message, data, success }`.

Skenario Berhasil (*.then*)

Ada tiga kondisi logika yang diperiksa setelah API merespons:

- **Jika Sukses & Hanya Ada Pesan:**
Jika API mengembalikan `success: true` dan ada `message`, fungsi menganggap operasi berhasil (misal: *"Data berhasil dihapus"*) tapi tidak ada data baru untuk ditampilkan.
- **Jika Sukses & Ada Data:**
Jika API mengembalikan data yang isinya tidak kosong, data tersebut langsung dikembalikan ke komponen React kamu.

- Kondisi Lainnya (Data Kosong/Gagal Logis):

Jika API merespons tapi isinya tidak sesuai kriteria di atas, ia akan mengembalikan pesan *"No record found"*.

Skenario Gagal (.catch)

Jika terjadi error pada jaringan atau server (misal: 404 atau 500), blok *.catch* akan menangkapnya dan mengembalikan status *success: false* beserta pesan error-nya.

3. Membuat function library API List berdasarkan module operator Book */api/books*.

Pada BAB 7 kita telah memangun Rest API untuk manajemen buku dengan list routing seperti pada table 7.1. Dari table tersebut dibuat dalam bentuk library API yang tersimpan pada folder */apis* dengan nama file *BookServices.jsx*.

Kode ini adalah kumpulan fungsi Service API atau API Client. Fungsinya adalah untuk mengumpulkan semua logika pemanggilan database buku ke dalam satu tempat agar komponen UI kamu tetap bersih. Kode ini menggunakan fungsi *RequestAPI* yang tadi kita bahas dan variabel *.env* yang sudah disiapkan.

```
import React from 'react'
import { RequestAPI } from '@/hooks/RequestAPI'

const API_URL = process.env.NEXT_PUBLIC_BACKEND_URI;
const TOKEN = localStorage.getItem('accessToken');

const GET_ALL_BOOK = () => {
  const HEADERS = {
    'Authorization': `Bearer ${TOKEN}`,
    'Content-Type': 'application/json'
  }
  return RequestAPI('GET', `${API_URL}/api/books`, HEADERS)
}

const CREATE_BOOK = (payload) => {
  const HEADERS = {
    'Authorization': `Bearer ${TOKEN}`,
    'Content-Type': 'multipart/form-data',
  }
  return RequestAPI('POST', `${API_URL}/api/books`, HEADERS, payload)
}
```

```

const GET_BOOK_BY_ID = (book_id) => {
  const HEADERS = {
    'Authorization': `Bearer ${TOKEN}`,
    'Content-Type': 'application/json'
  }
  return RequestAPI('GET', `${API_URL}/api/books/${book_id}`, HEADERS)
}

const UPDATE_BOOK = (book_id, payload) => {
  const HEADERS = {
    'Authorization': `Bearer ${TOKEN}`,
    'Content-Type': 'multipart/form-data',
  }
  return RequestAPI('PUT', `${API_URL}/api/books/${book_id}`, HEADERS,
payload)
}

const DELETE_BOOK = (book_id) => {
  const HEADERS = {
    'Authorization': `Bearer ${TOKEN}`,
    'Content-Type': 'application/json'
  }
  return RequestAPI('DELETE', `${API_URL}/api/books/${book_id}`,
HEADERS)
}

export { GET_ALL_BOOK, GET_BOOK_BY_ID, CREATE_BOOK, UPDATE_BOOK,
DELETE_BOOK }

```

A. Inisialisasi & Keamanan

- *API_URL*: Mengambil alamat backend dari file *.env* kamu (*NEXT_PUBLIC_BACKEND_URI*).
- *TOKEN*: Mengambil token akses dari *localStorage*. Token ini digunakan untuk membuktikan kepada server bahwa user adalah pengguna yang sah (terautentikasi).

B. Headers Authorization

Semua fungsi menyertakan *Bearer \${TOKEN}*. Ini adalah standar keamanan JWT. Tanpa ini, biasanya server akan menolak akses kamu dengan **error 401 Unauthorized**.

Content-Type yang Berbeda

Perhatikan bahwa kode kamu menggunakan dua jenis header yang berbeda:

- *application/json*: Digunakan saat kamu hanya mengirim teks biasa (untuk GET dan DELETE).
- *multipart/form-data*: Digunakan pada fungsi CREATE dan UPDATE. Ini menandakan bahwa payload yang dikirim bisa berupa objek FormData yang berisi file (seperti gambar) atau input form yang kompleks.

Langkah 5: Mengupdate data static Management Books menjadi dinamik dengan Rest API

A. Mengupdate tampilan view table management buku

Updatelah code anda pada komponen react MBooks dengan menambahkan Logic di Komponen React untuk menghubungkan fungsi API yang tadi telah dibuat ke tampilan aplikasi.

RFC MBooks

```
"use client";

import React, { useEffect, useState } from "react";
import { openModal } from "@components/ui/modals";
import { Alert } from "@components/ui/alerts";
import { Skeleton } from "@components/ui/loading";
import { GET_ALL_BOOK } from "@components/apis/BookServices";
import { CardCalculates } from "../components/card_calculates";
import { Header } from "../components/header";
import Form from "../components/form";
import Tabledata from "../components/tabledata";

export function MBooks() {
  const [books, setBooks] = useState({ loading: false, data: [], message: "" });

  const ReloadBook = async () => {
    setBooks({ loading: true, data: [], message: "" });
    const results = await GET_ALL_BOOK();
    setBooks(results)
  }

  useEffect(() => {
    ReloadBook();
  })
}
```

```

    }, []);

    return (
      <div className="container-fluid">
        <Header handleAdd={() => openModal({ message: <Form
ReloadBook={ReloadBook} />, size: "xl" })} />

        <div className="row">
          <div className="col-md-3">
            <CardCalculates
              title={`Total Books`}
              value={books?.data.length}
              icon={`book`}
            />
          </div>
          <div className="col-md-3">
            <CardCalculates
              title={`Free Book`}
              value={books?.data.filter((b) => !b.is_free).length || 0}
              icon={`grid`}
            />
          </div>
          <div className="col-md-3">
            <CardCalculates
              title={`Subscribe`}
              value={books?.data.filter((b) => b.is_free).length || 0}
              icon={`calendar-event`}
            />
          </div>
          <div className="col-md-3">
            <CardCalculates
              title={`Authors`}
              value={books?.data.filter((b) => b.author).length || 0}
              icon={`people`}
            />
          </div>
        </div>

        {books.loading ? <Skeleton /> :
        books.message ? <Alert message={books.message} variant="danger" />
        :
        (books.data && books.data.length > 0)? (
          <Tabledata data={books.data} ReloadData={ReloadBook} />
        ) : ""}
      </div>
    );
  }
}

```

```

    );
  }

  export default MBooks;

```

Inisialisasi State

state bernama *books* untuk menyimpan data dari API, dimana nilai state *books* sebelumnya ialah *ListBooks*.

- *loading*: Memberi tahu UI apakah data sedang diproses (untuk menampilkan spinner/loading).
- *data*: Tempat menyimpan daftar buku yang berhasil diambil.
- *message*: Menyimpan pesan (misal: "Gagal memuat data").

Fungsi Eksekusi (ReloadBook)

Fungsi ini bersifat *async* (asinkron) karena harus menunggu jawaban dari server.

- Tahap 1: *setBooks({ loading: true, ... })* : Segera ubah status menjadi loading. Di layar, user mungkin akan melihat tulisan "*Loading...*" atau icon *Loading*.
- Tahap 2: *await GET_ALL_BOOK()*: Memanggil fungsi API yang tadi telah dibahas.
- Tahap 3: *setBooks(results)*: Setelah server merespons, simpan hasilnya (baik itu sukses maupun gagal) ke dalam *state books*. Karena *RequestAPI* sudah kita buat mengembalikan format yang sama, maka *state* akan langsung terisi dengan benar.

Otomatisasi dengan *useEffect*

useEffect dengan array kosong `[]` di akhir berarti: "*Jalankan fungsi ini tepat satu kali, segera setelah komponen muncul di layar.*" Tanpa ini, data buku tidak akan pernah muncul kecuali user menekan tombol secara manual.

Conditional Rendering

```

{books.loading ? <Skeleton /> :
books.message ? <Alert message={books.message} variant="danger" /> :
(books.data && books.data.length > 0)? (
  <Tabledata data={books.data} ReloadData={ReloadBook} />
) : ""}

```

Kode diatas merupakan teknik Conditional Rendering di React menggunakan Ternary Operator. Yang secara sederhana, kode ini sedang mengatur *"Apa yang harus dilihat user?"* berdasarkan status data buku saat ini.

Kondisi Loading

- Logika: *"Jika books.loading bernilai true"*
- Tampilan: Tampilkan komponen `<Skeleton />`.

Kondisi Error/Gagal

- Logika: *"Jika loading sudah selesai, tapi ada pesan error (books.message tidak kosong)..."*
- Tampilan: Tampilkan komponen `<Alert />` berwarna merah (danger) yang berisi pesan error tersebut.

Kondisi Data Berhasil Dimuat

- Logika: *"Jika tidak loading dan tidak ada error, periksa apakah ada isinya (length > 0)..."*
- Tampilan: Tampilkan komponen `<TableData />`. Dan juga mengirimkan *prop* data (isi bukunya) dan *ReloadData* agar tabel bisa memicu refresh data sendiri.

B. Mengupdate Form tambah buku dengan mengintegrasikan Rest API

Pada komponen form di `/_books/components/form.jsx` dengan menambahkan library `CREATE_BOOK`, `UPDATE_BOOK` dan `GET_BOOK_ID`. Jika sebelumnya untuk melakukan tambah buku, update buku, dan detail buku menggunakan data statik berupa array object yang disimpan pada `ListBooks`, sekarang diubah menjadi dinamis agar terlihat real-time data. Dimana pada komponen *form* ini sebagai formulir untuk menambah buku baru maupun untuk mengubah data buku yang sudah ada (edit).

```
"use client";

import React, { useEffect, useState } from 'react'
import { TextAreaInput, TextInput, InputImage, InputCheckbox } from
'@/components/ui/forms'
```

```

import { Button } from "@components/ui/buttons";
import { openModal, ModalResponse } from '@components/ui/modals';
import { GET_BOOK_BY_ID } from '@components/apis/BookServices';
import { Alert } from '@components//ui/alerts';
import { CREATE_BOOK, UPDATE_BOOK } from '@components/apis/BookServices';

export default function Form({ book_id, ReloadBook }) {
  const obj_book = {
    title: '',
    author: '',
    synopsis: '',
    story: '',
    is_free: false,
    image: null
  }
  const [formData, setFormData] = useState(obj_book)
  const [imagePreview, setImagePreview] = useState(null)
  const [error, setError] = useState("");

  const ReloadBookByID = async () => {
    const result = await GET_BOOK_BY_ID(book_id);
    console.log('result', result);
    if (result.data && Object.values(result.data).length > 0) {
      setFormData(result.data);
      if (result.data.image) {
        const imageUrl = `
        ${process.env.NEXT_PUBLIC_BACKEND_URI}${result.data.image}`;
        setImagePreview(imageUrl);
      }
    } else {
      openModal({ message: <ModalResponse message={result.message}
title={"No record found"} /> })
      setFormData({ obj_book })
    }
  }

  useEffect(() => {
    if (book_id) {
      ReloadBookByID();
    }
  }, [book_id])

  const handleInputChange = (e) => {
    const { name, value, type, checked } = e.target

```

```

    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    })))
  }

const handleImageChange = (e) => {
  const file = e.target.files[0]
  if (file) {
    // Validate file type
    const validTypes = ['image/jpeg', 'image/jpg', 'image/png']
    if (!validTypes.includes(file.type)) {
      setError('Please select a valid image file (JPEG, or PNG)')
      return
    }

    // Validate file size (5MB max)
    if (file.size > 5 * 1024 * 1024) {
      setError('Image size must be less than 5MB')
      return
    }

    setFormData(prev => ({ ...prev, image: file })))
    setError(null)

    // Create preview
    const reader = new FileReader()
    reader.onloadend = () => {
      setImagePreview(reader.result)
    }
    reader.readAsDataURL(file)
  }
}

const handleSubmit = async (e) => {
  e.preventDefault()
  console.log('Form Data:', formData)
  try {
    const formDataToSend = new FormData()

    // Append all form fields to FormData
    formDataToSend.append('title', formData.title)
    formDataToSend.append('author', formData.author)
    formDataToSend.append('sinopsis', formData.sinopsis)
    formDataToSend.append('story', formData.story)
  }
}

```

```

formDataToSend.append('is_free', formData.is_free)
// Append image file if exists
if (formData.image instanceof File) {
  formDataToSend.append('coverImage', formData.image)
}
if (book_id) {
  formDataToSend.append('id', book_id)
  const result = await UPDATE_BOOK(book_id, formDataToSend)
  console.log(result);
  if (result.success) {
    openModal({
      message: <ModalResponse
        message="Book has been successfully updated!"
        title="Success"
      />
    })
    ReloadBook();
    // Reset form
    setFormData(obj_book)
    setImagePreview(null)
  } else {
    setError(result.message || 'Failed to create book')
    setImagePreview(null)
  }
} else {
  const result = await CREATE_BOOK(formDataToSend)
  if (result.success) {
    openModal({
      message: <ModalResponse
        message="Book has been successfully created!"
        title="Success"
      />
    })
    ReloadBook();
    // Reset form
    setFormData(obj_book)
    setImagePreview(null)
  } else {
    setError(result.message || 'Failed to create book')
    setImagePreview(null)
  }
}
} catch (error) {
  setError(error.message)
}

```

```

    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <h3 className='d-flex align-items-start flex-column'>
        <span className="">Add New Book</span>
        <span className="text-secondary fs-6">Fill in the details for the
new book.</span>
      </h3>

      <div className="row">
        <div className="col-lg-6">
          <TextInput
            title="Book Title"
            name="title"
            value={formData.title}
            onChange={handleInputChange}
            required
          />

          <TextAreaInput
            title="Sinopsis"
            name="sinopsis"
            value={formData.sinopsis}
            onChange={handleInputChange}
            rows={2}
            required
          />

          <TextAreaInput
            title="Story"
            name="story"
            value={formData.story}
            onChange={handleInputChange}
            rows={3}
            required
          />
        </div>
        <div className="col-lg-6">
          <div className="row">
            <div className="col-8">
              <TextInput
                title="Author Name"
                name="author"

```

```

        value={formData.author}
        onChange={handleInputChange}
        required
      />
    </div>
    <div className="col-4">
      <InputCheckbox
        title="Type Book"
        value="Is Free"
        name="is_free"
        is_switch={true}
        required={true}
        checked={formData.is_free}
        onChange={handleInputChange}
      />
    </div>
  </div>

  <InputImage
    title="Cover Image"
    onChange={handleImageChange}
    required
    imagePreview={imagePreview}
  />
</div>
</div>

{error && (
  <div className="mt-4">
    <Alert message={error} variant='danger' />
  </div>
)}

<div className="mt-4 text-center">
  <Button type="submit" variant="light" className="me-2 btn-lg"
onClick={() => openModal({ open: false })}>
    Cancel
  </Button>
  <Button type="submit" variant="primary" className="btn-lg">
    Submit Book
  </Button>
</div>
</form>
)
}

```

Inisialisasi dan State

- `obj_book`: Berfungsi sebagai blueprint atau kerangka data kosong untuk buku.
- `formData`: State utama yang menyimpan data inputan user.
- `imagePreview`: Digunakan untuk menampilkan gambar sementara (preview) sebelum benar-benar diunggah ke server.
- `error`: Menampung pesan validasi atau pesan gagal dari server.

Logika Mode Edit (*ReloadBookByID*)

- Komponen ini menerima props bernama *book_id*.
- **Pengecekan**: Di dalam *useEffect*, jika *book_id* ada, maka fungsi *ReloadBookByID* dijalankan.
- **Fungsi**: Ia mengambil data buku dari API berdasarkan ID, lalu mengisi *formData* dengan data tersebut. Ini memungkinkan formulir terisi otomatis saat user ingin mengedit buku.

Penanganan Input & Gambar

- **handleInputChange**: Fungsi universal untuk menangani perubahan teks maupun *checkbox*. Untuk *checkbox*, ia mengambil nilai dari *e.target.checked*.
- **handleImageChange**: Fungsi khusus untuk mengolah file gambar:
- **Validasi**: Mengecek tipe file (harus JPEG/PNG) dan ukuran (maksimal 5MB).
- **FileReader**: Mengubah file gambar menjadi URL string (*readAsDataURL*) agar bisa ditampilkan di layar sebagai preview sebelum di-submit.

Proses Submit (Create vs Update)

Bagian paling krusial ada di fungsi *handleSubmit*. Karena ada pengiriman gambar, kode ini menggunakan objek *FormData()* (bukan JSON biasa).

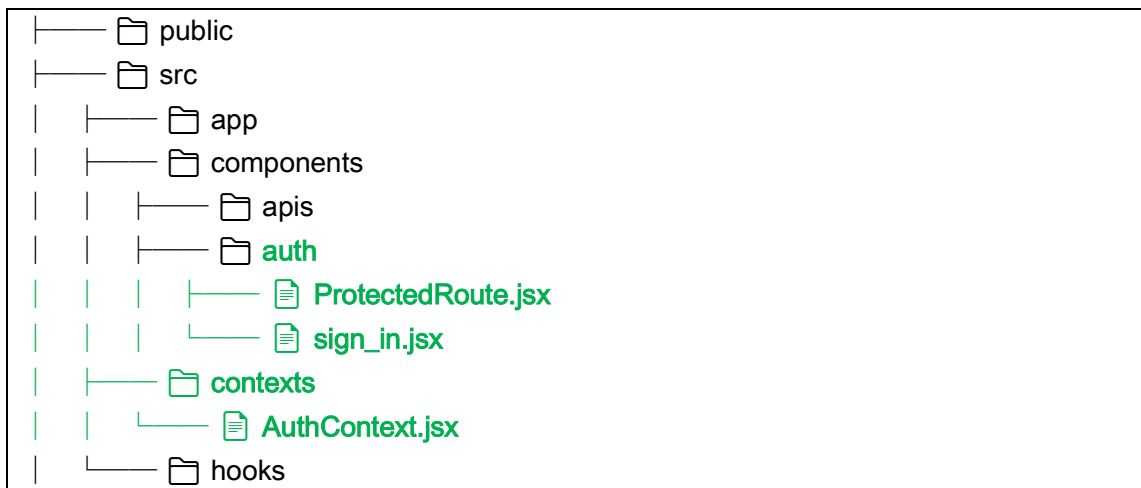
- **multipart/form-data**: Karena menggunakan *new FormData()*, Axios akan mengirim data dengan format *multipart*, yang memang diperlukan untuk unggah file.
- **Pembeda**:
 - Jika *book_id* ada: Maka menjalankan fungsi `UPDATE_BOOK`.

- Jika *book_id* kosong: Maka menjalankan fungsi `CREATE_BOOK`.
- **Append Data:** Semua data dimasukkan ke dalam objek *formDataToSend*, termasuk file gambar jika user memilih file baru.
- **Feedback:** Jika sukses, akan muncul Modal sukses, state di-reset, dan fungsi *ReloadBook()* dipanggil untuk memperbarui daftar buku di halaman utama.

Langkah 6: Membuat Halaman Sign In dan mengatur *State Management Authentication*

Buatlah halaman authentication untuk melakukan sign-in kedalam aplikasi dengan isian form berupa email dan password berdasarkan Rest API User yang telah dibuat pada langkah 1 dan 2. Dengan request API di endpoint `/api/users/login`.

Berikut adalah susunan project unntuk menghandle halaman sign-in:



contexts/AuthContext.jsx

```

"use client";

import { createContext, useContext, useState, useEffect } from "react";
import { useRouter } from "next/navigation";

const AuthContext = createContext({});

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

```

```

useEffect(() => {
    checkAuth();
}, []);

const checkAuth = () => {
    try {
        const token = localStorage.getItem("accessToken");
        const userData = localStorage.getItem("user");
        const expiresIn = localStorage.getItem("expiresIn");

        if (token && userData && expiresIn) {
            // Cek apakah token sudah expired
            const currentTime = Math.floor(Date.now() / 1000);
            const expirationTime = parseInt(expiresIn);

            if (currentTime >= expirationTime) {
                // Token sudah expired
                console.log("Token has expired, logging out...");
                logout();
                return;
            }
            setUser(JSON.parse(userData));
        } else {
            setUser(null);
        }
    } catch (error) {
        console.error("Error checking auth:", error);
        setUser(null);
    } finally {
        setLoading(false);
    }
};

const login = (userData, token, expiresIn) => {
    localStorage.setItem("accessToken", token);
    localStorage.setItem("expiresIn", expiresIn);
    localStorage.setItem("user", JSON.stringify(userData));
    setUser(userData);

    return Promise.resolve();
};

const logout = () => {
    localStorage.removeItem("accessToken");

```

```

        localStorage.removeItem("expiresIn");
        localStorage.removeItem("user");
        setUser(null);
        router.push("/sign-in");
    };

    return (
        <AuthContext.Provider value={{ user, loading, login, logout,
checkAuth }}>
            {children}
        </AuthContext.Provider>
    );
};

export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error("useAuth must be used within AuthProvider");
    }
    return context;
};

```

"use client" & Context Setup

- "use client": Wajib ada karena kode ini menggunakan Hooks (*useState*, *useEffect*) yang hanya bisa berjalan di browser (Client Side), bukan server.
- *createContext*: Membuat sebuah "wadah data global" bernama *AuthContext*. Dengan ini, data user bisa diakses oleh komponen mana pun tanpa perlu operan props.

State Utama

Di dalam *AuthProvider*, ada dua state penting:

- *user*: Menyimpan data profil user yang sedang login (seperti nama, email, role).
- *loading*: Menandakan apakah aplikasi sedang sibuk mengecek token di storage atau tidak.

Logika Keamanan (*checkAuth*) Pemeriksaan Token

- *checkAuth* Fungsi ini berjalan otomatis saat halaman pertama kali dibuka (*useEffect*).

- Mengambil *accessToken* dan *expiresIn* dari *localStorage*.
- Validasi Waktu: Ia membandingkan waktu sekarang dengan waktu kadaluarsa token.
- Jika sudah expired atau data tidak lengkap, sistem otomatis memanggil fungsi *logout()*.

Function login & logout

- *login*: Bertugas mencatat data dari backend ke dalam *localStorage* dan mengupdate state user agar UI berubah menjadi mode "*Sudah Login*".
- *logout*: Kebalikannya, ia menghapus semua jejak di *localStorage*, mengosongkan state user, dan mengarahkan paksa (*router.push*) user kembali ke halaman login.

AuthContext.Provider

```
<AuthContext.Provider value={{ user, loading, login, logout, checkAuth
  }}>
  {children}
</AuthContext.Provider>
```

Baris ini menyebarkan semua fungsi dan data di atas ke seluruh aplikasi (*{children}*) atau keseluruhan nested komponen dengan memanfaatkan Hook *useContext*.

The Hook (*useAuth*)

```
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within AuthProvider");
  }
  return context;
};
```

Ini adalah *Custom Hook* agar bisa memanggil data *auth* dengan sangat mudah di komponen lain.

Selanjutnya ialah membangun tampilan User Interface untuk halaman sign-in:

 components/auth/sign-in.jsx

```
"use client";
```

```

import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import axios from "axios";
import { useAuth } from "@/contexts/AuthContext";
import { TextInput, TextInputPassword } from "@/components/ui/forms";

export default function SignInPage() {
  const router = useRouter();
  const { login } = useAuth();
  const [formData, setFormData] = useState({
    email: "",
    password: "",
  });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
    setError("");
  };

  const togglePasswordVisibility = () => {
    setShowPassword(!showPassword);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");

    if (!formData.email || !formData.password) {
      setError("Email and password are required");
      return;
    }

    setLoading(true);

    try {
      const response = await axios.post(
        `${process.env.NEXT_PUBLIC_BACKEND_URI}/api/users/login`,
        {
          email: formData.email,
          password: formData.password,
        },
      );

      if (response.data.success) {

```

```

        await login(
            response.data.data,
            response.data.accessToken,
            response.data.expiresIn,
        );

        await new Promise((resolve) => setTimeout(resolve, 100));

        const                                redirectToUrl                                =
        sessionStorage.getItem("redirectToLogin");
        if (redirectToUrl) {
            sessionStorage.removeItem("redirectToLogin");
            router.push(redirectToUrl);
        } else {
            router.push("/cms");
        }
    }
} catch (error) {
    if (error.response) {
        setError(error.response.data.message || "Login failed");
    } else if (error.request) {
        setError("No response from server. Please try again.");
    } else {
        setError("An error occurred. Please try again.");
    }
} finally {
    setLoading(false);
}
};

return (
    <div className="signin-container">
        <div className="signin-card">
            <h1 className="signin-title">Welcome Back</h1>
            <p className="signin-subtitle">Sign in to your account</p>

            {error && <div className="error-message">{error}</div>}

            <form onSubmit={handleSubmit} className="signin-form">
                <TextInput
                    title={`Email`}
                    required={true}
                    id="email"
                    name="email"
                    value={formData.email}
                    onChange={handleChange}
                    placeholder="Enter your email"
                />

                <TextInputPassword
                    required={true}
                    title={"Password"}

```

```

        id="password"
        name="password"
        value={formData.password}
        onChange={handleChange}
        placeholder="Enter your password"
        required
      />

      <button          type="submit"          className="signin-button"
disabled={loading}>
        {loading ? "Signing in..." : "Sign In"}
      </button>
    </form>

    <div className="signin-footer">
      <p>
        Do not have an account?{" "}
        <Link href="/sign-up" className="signup-link">
          Sign up here
        </Link>
      </p>
    </div>
  </div>
</div>
);
}

```

Refactoring dengan Custom UI Components

Alih-alih menulis elemen `<input>` dan `<label>` secara manual, kode ini mengimpor:

- `TextInput`: Komponen kustom untuk input teks (email).
- `TextInputPassword`: Komponen khusus password yang kemungkinan besar sudah memiliki logika internal untuk toggle "lihat password".

Alur Logic Flow

- *State Management*: `formData` menyimpan email dan password. Saat kamu mengetik, fungsi `handleChange` memperbarui state tersebut dan sekaligus menghapus pesan error yang mungkin muncul sebelumnya (memberikan pengalaman user yang mulus).
- *Validasi Client-Side*: Sebelum mengirim data ke internet, kode mengecek apakah kolom sudah terisi. Ini menghemat *resource server*.
- *Proses Login (Backend Request)*:
 - Menggunakan `axios.post` ke URL backend kamu.
 - Jika sukses, memanggil `login()` (dari `AuthContext`) untuk menyimpan data user dan token.

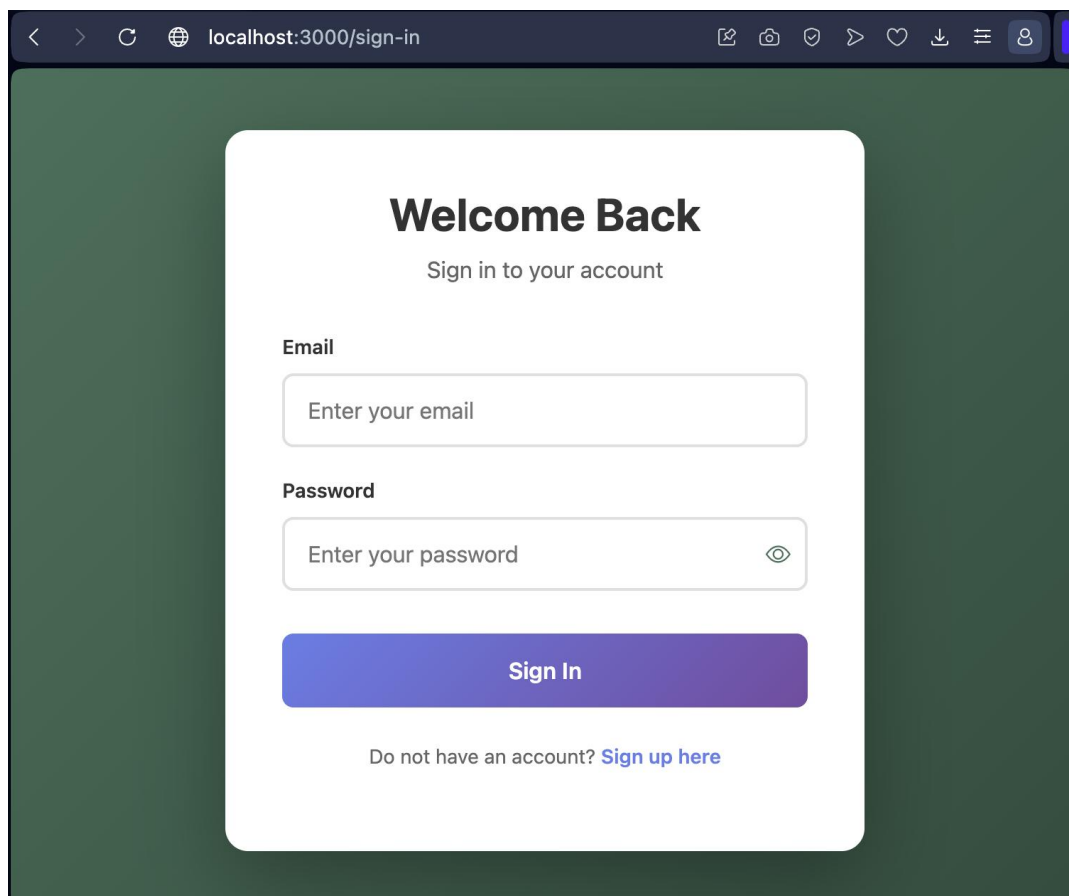
- *Sistem Redirect*: Kode mengecek `sessionStorage` untuk kunci `redirectAfterLogin`.

Penanganan Error & Loading

- `setLoading(true)`: Tombol dinonaktifkan (disabled) agar user tidak menekan login berkali-kali jika koneksi sedang lambat.
- *Try-Catch Block*: Memisahkan error menjadi 3 jenis:
 - Error dari Server (misal: "User tidak ditemukan" atau "Password salah").
 - Error Jaringan (Server mati atau tidak merespon).
 - Error Tak Terduga (Bug di kode).

Keamanan

Penggunaan `new Promise(resolve => setTimeout(resolve, 100))` memberikan jeda singkat agar state `AuthContext` benar-benar terupdate di storage sebelum `router.push` dijalankan. Ini mencegah "*race condition*" di mana halaman berpindah sebelum token siap.



Gambar 8.8 Tampilan Halaman Sign-in

Langkah 7: Implementasi Protected Route

Protected Route adalah sebuah mekanisme keamanan di aplikasi React untuk membatasi akses ke halaman tertentu. Jika user belum login dan mencoba masuk ke halaman terproteksi (seperti */dashboard* atau */profile*), sistem akan secara otomatis menendang (redirect) mereka kembali ke halaman */login*.

A. Membuat sebuah *Wrapper Component* (komponen pembungkus) yang bertugas memeriksa status autentikasi dari *AuthContext* yang telah dibuat sebelumnya.

Buatlah file Wrapper Component yang berada pada:

/components/auth/ProtectedRoute.jsx:

```
"use client";

import { useEffect, useState } from "react";
import { useRouter, usePathname } from "next/navigation";
import { useAuth } from "@/contexts/AuthContext";
import { Skeleton } from "@/components/ui/loading";

export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  const router = useRouter();
  const pathname = usePathname();
  const [shouldRedirect, setShouldRedirect] = useState(false);

  useEffect(() => {
    if (!loading && !user) {
      // Tambahkan delay kecil sebelum redirect
      const timer = setTimeout(() => {
        setShouldRedirect(true);
      }, 100);

      return () => clearTimeout(timer);
    }
  }, [user, loading]);

  useEffect(() => {
    if (shouldRedirect) {
      sessionStorage.setItem("redirectAfterLogin", pathname);
      router.push("/sign-in");
    }
  }, [shouldRedirect, router, pathname]);
}
```

```

    if (loading) {
      return (
        <Skeleton />
      );
    }

    if (!user) {
      return null;
    }

    return <>{children}</>;
  }
}

```

State dan Hooks Utama

- *useAuth()*: Mengambil status user dan loading dari Context yang kita bahas sebelumnya.
- *usePathname()*: Mencatat URL halaman yang sedang berusaha dikunjungi user (misalnya /cms/edit-buku).
- *shouldRedirect*: Sebuah state tambahan untuk menandai kapan aplikasi harus benar-benar melakukan pengalihan halaman ke menu login.

Logika Auth Persistence State (sessionStorage)

Ini adalah bagian yang sangat cerdas:

```
sessionStorage.setItem("redirectAfterLogin", pathname);
```

Sebelum user diusir ke halaman /sign-in, kode ini menyimpan alamat halaman yang gagal diakses ke dalam *sessionStorage*.

Layout Wrapper Component:

Implementasikan Wrapper Component kedalam Layout, yang bertugas sebagai pembungkus seluruh halaman yang diproteksi. Pada layout.jsx milik CMSLayout, tambahkanlah code Wrapper Component yang telah dibuat sebelumnya, /app/cms/layout.jsx:

```

import Sidebar from "@components/cms/components/sidebar";
import "./cms.css";
import Modals from "@components/ui/modals";
import ProtectedRoute from "@components/auth/ProtectedRoute";

```

```

export const metadata = {
  title: "CMS - Content Management System",
  description: "Content Management System for managing application content",
};

export default function CMSLayout({ children }) {
  return (
    <ProtectedRoute>
      <div className="cms-container">
        <Sidebar />
        <main className="main-content p-4">{children}</main>
      </div>
      <Modals />
    </ProtectedRoute>
  );
}

```

Langkah 8: Implementasi Sign Out

Kita akan pasang tombol Sign Out pada komponen Sidebar yang telah dibuat pada BAB 6. Updatelah komponen tersebut dengan memanggil logout pada Wrapper Komponen userAuth seperti dibawah ini:

```

//... rest of your code...

import { useAuth } from '@contexts/AuthContext';

export default function Sidebar() {
  const pathname = usePathname();
  const { user, logout } = useAuth();

  //... rest of your code...

  return (
    <aside
      className={`sidebar ${!isExpanded ? 'collapsed' : ''}`}
      onMouseEnter={handleMouseEnter}
      onMouseLeave={handleMouseLeave}
    >
      //... rest of your code...
      <div className="sidebar-content">...</div>

      <div className="account-user">

```

```

        {isExpanded ? (
            <>
                <div className="user-info">
                    <div className="user-avatar">
                        <i className="bi bi-person-circle"></i>
                    </div>
                    <div className="user-details">
                        <p className="user-name">{user?.username
|| 'User'}</p>
                        <p className="user-email">{user?.email
|| 'user@example.com'}</p>
                    </div>
                </div>
                <button
                    className="logout-btn"
                    onClick={logout}
                    title="Sign Out"
                >
                    <i className="bi bi-box-arrow-right"></i>
                    <span className="menu-text">Sign Out</span>
                </button>
            </>
        ) : (
            <button
                className="logout-btn-icon"
                onClick={logout}
                title="Sign Out"
            >
                <i className="bi bi-box-arrow-right"></i>
            </button>
        )}
    </div>
</aside>
);
}

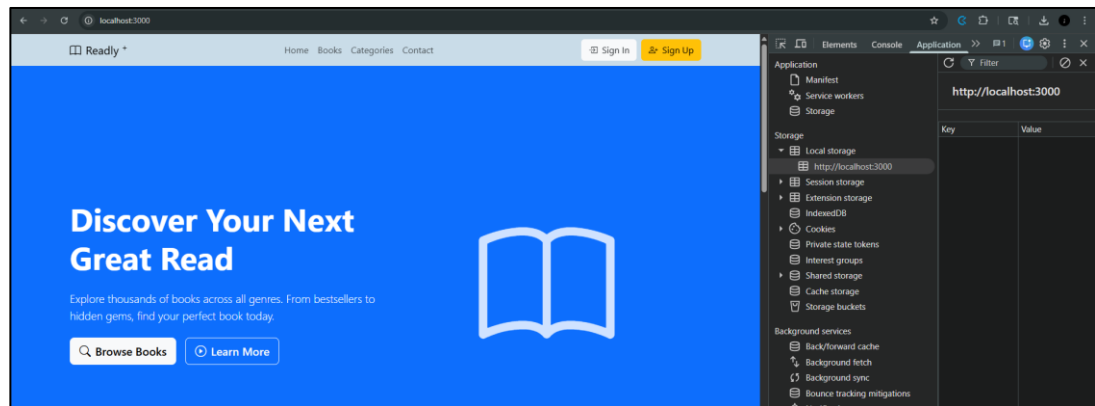
```

Langkah 9: Testing Integrasi

Untuk memulai pengujian aplikasi web tersebut, lakukan beberapa tahapan dibawah ini agar skenario pengujian bisa dikerjakan dengan baik.

1: Inisialisasi Environment (Backend & Frontend)

- **Jalankan Backend:** Buka terminal, masuk ke direktori server, dan jalankan perintah untuk menyalakan Backend Express. Pastikan server berjalan stabil di *port 3001*.
- **Jalankan Frontend:** Buka terminal baru, masuk ke direktori aplikasi React/Next.js, dan jalankan perintah *npm run dev*. Pastikan frontend berjalan di *port 3000*.



Gambar 8.9 Tampilan Halaman utama frontend

Pada gambar diatas, merupakan halaman utama dalam project frontend dengan kondisi belum melakukan proses sign-in. Hal ini dapat terlihat dalam Developer Tools pada tab Application belum tersimpan token pada Local Storage.

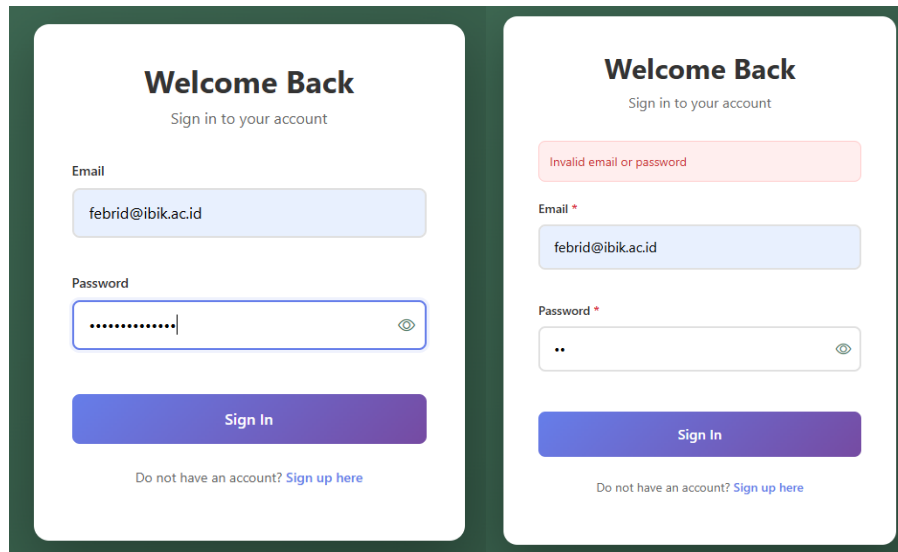
2: Pengujian Proteksi Rute (Protected Route)

- **Akses Jalur Terproteksi:** Buka browser dan langsung ketikkan URL *http://localhost:3000/cms* pada bar pencarian.
- **Verifikasi Redirect:** Sistem harus secara otomatis mendeteksi bahwa anda belum memiliki token akses, lalu mengalihkan (*redirect*) ke halaman */sign-in*.

3: Registrasi dan Autentikasi

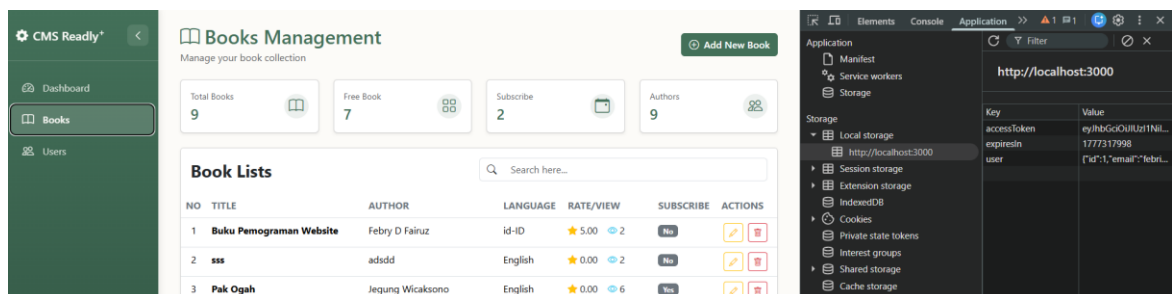
- **Pendaftaran Akun:** Jika belum memiliki akun, klik tautan ke halaman *sign-up*, isi formulir pendaftaran, dan simpan.
- **Proses Sign-In:** Masukkan *email* dan *password* yang telah didaftarkan pada halaman login.

- **Verifikasi Data Profil:** Setelah berhasil masuk, periksa bagian sidebar. Pastikan nama dan email anda muncul dengan benar sebagai tanda bahwa data user dari API telah berhasil disimpan ke dalam *Auth Context*.



Gambar 8.10 Halaman Pengisian Sign In
(kondisi valid dan invalid akun)

Setelah berhasil melakukan sign-in pada system maka akan masuk kehalaman dashboard aplikasi dan pada development tools di tab Application, telah memiliki beberapa key yang telah degenerate sebagai bagain dari Local Storage. Hal ini digambarkan pada gambar dibawah ini.



Gambar 8.11 Halaman setelah melakukan sign-in

4: Pengujian Fitur Utama (CRUD Management Books)

- **Sinkronisasi Data (Read):** Masuk ke menu Management Books dan pastikan daftar buku muncul (ini menandakan fungsi GET_ALL_BOOK bekerja).
- **Operasi Data (Create, Update, Delete):**

- Coba tambahkan buku baru (Create).
- Ubah salah satu data buku (Update).
- Hapus buku yang telah dibuat (Delete).
- Pastikan setiap aksi memberikan respon yang sesuai di tabel data.

5: Pengujian Keamanan dan Sesi (Sign-Out)

- **Proses Sign-Out:** Klik tombol keluar (*sign-out*). Sistem harus menghapus token dari *localStorage* dan mengembalikan ke halaman login.
- **Uji Akses Ilegal:** Coba ketik kembali URL */cms* secara manual. Jika sistem aman, Anda tidak akan bisa masuk dan akan terus dikembalikan ke halaman login selama sesi belum dibuat kembali.

8.4 Tugas Pembelajaran

1. Berdasarkan latihan pembelajaran diatas, integrasikan Rest API untuk management user pada frontend apss kalian dengan nama menu *Users*, berdasarkan api users yang telah dikerjakan.
2. Buatlah validasi keamanan dengan kondisi jika user telah melakukan sign-in kedalam aplikasi, maka url path */sign-in* atau */sign-up* tidak bisa kembali diakses melainkan di redirect ke */cms*.
3. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
4. Upload project tersebut pada GITHUB dengan menggunakan repository bernama "**PW-MERN-STACK-NPM**" dengan nama commit "**Praktikum 8**".
5. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

DAFTAR PUSTAKA

- Abdulloh, R. (2020). *Menguasai React JS untuk pemula: Panduan belajar JavaScript dari dasar hingga membuat aplikasi web modern*. Elex Media Komputindo.
- Abdulloh, R. (2023). *Pemrograman web modern: MongoDB & Express.js*. Elex Media Komputindo.
- Axza, F., Sofi'ie, F., & Qoiriah, A. (2023). Analisis perbandingan framework front-end JavaScript React dan Vue pada pengembangan website. *Journal of Informatics and Computer Science*, 5(1), 12-20.
- Calloran, B. N. (2025). *Complete React, Next.js & TypeScript projects course 2025: Hands-on guide to building 25+ real-world apps*. Martin Chavez.
- Express. (2026). *Express - Node.js web application framework*. <https://expressjs.com>
- Haverbeke, M. (2024). *Eloquent JavaScript: A modern introduction to programming* (4th ed.). No Starch Press.
- Next.js. (2026). *Next.js documentation*. <https://nextjs.org/docs>
- React. (2026). *React reference*. <https://react.dev>
- Riva, M. (2022). *Real-world Next.js: Build scalable, high-performance, and modern web applications using Next.js*. Packt Publishing.
- SantriKoding. (2024). *eBook fullstack Next.js developer*. <https://santrikoding.com>
- Siahaan, & Kenidy. (2023). Analisis komparatif kinerja Next.js, Nuxt.js, dan Remix.js dalam implementasi server side rendering website berita. *Jurnal Teknik Informatika (TIN)*, 4(2), 145-152.
- Widarti, D. W., & Akbar, M. K. (2024). Peningkatan performa aplikasi web SIAKAD menggunakan Next.js di Puskordat Stimata. *Journal of Information System and Management (JOISM)*, 6(1), 88-95.



Pemrograman Website

Program Studi Teknologi Informasi
Fakultas Informatika dan Pariwisata

INSTITUT BISNIS & INFORMATIKA KESATUAN
Jl. Rangka Gading No.1, RT.02/RW.09, Gudang,
Kecamatan Bogor Tengah, Kota Bogor, Jawa Barat 16123